

AODV-EN

Documentacao Completa do Protocolo de Roteamento

Trabalho de Conclusao de Curso
Bacharelado em Engenharia de Software - IFG Campus Inhumas

v1

RFC 3561

ESP-NOW v2

ESP32

Adaptacao do AODV para redes mesh multi-hop sobre ESP-NOW

Sumario

01. Visao Geral do Projeto [README.md](#)
02. Relatorio Completo do Trabalho (AODV-EN vs Flooding) [docs/tcc-trabalho-completo.md](#)
03. Especificacao do TCC (cenarios, metricas, algoritmos) [TCC.md](#)
04. Invariantes do AODV [docs/aodv-base-invariantes.md](#)
05. Especificacao v1 (normativa) [docs/aodv-en-spec-v1.md](#)
06. Funcionamento Completo do Protocolo [docs/aodv-en-funcionamento.md](#)
07. Estruturas de Dados [docs/aodv-en-estruturas-dados.md](#)
08. Mapa do Codigo [docs/aodv-en-mapa-do-codigo.md](#)
09. Feature: Precursores [docs/features/precursores.md](#)
10. Feature: Fila de DATA pendente [docs/features/enfileamento-dos-dados.md](#)
11. Feature planejada: Articulation Point (v2) [docs/features/articulation-point-planejado.md](#)
12. Plano de Desenvolvimento Completo [docs/plano-desenvolvimento-completo.md](#)
13. Runbook de Bancada [docs/runbook-bancada.md](#)
14. Casos de Teste - Indice [docs/tests/README.md](#)
15. TC-001 - Descoberta e Entrega Direta [docs/tests/tc-001-descoberta-e-entrega-direta.md](#)
16. TC-002 - Primeiro Multi-hop [docs/tests/tc-002-primeiro-multi-hop.md](#)
17. TC-003 - Reconvergencia apos Falha [docs/tests/tc-003-reconvergencia-apos-falha.md](#)
18. TC-004 - Soak de Estabilidade [docs/tests/tc-004-soak-estabilidade-e-reconvergencia.md](#)
19. TC-005 - Cadeia de 4 Nos [docs/tests/tc-005-cadeia-4-nos.md](#)
20. Guia de leitura dos graficos do monitor [docs/tests/guia-leitura-graficos-monitor.md](#)
21. Firmware - README [firmware/README.md](#)
22. Simulacao - README [sim/README.md](#)
23. Apendice: Especificacao v0 (OBSOLETA) [docs/aodv-en-spec-v0.md](#)

Visao Geral do Projeto

aodv-en

Projeto de Trabalho de Conclusao de Curso do Bacharelado em Engenharia de Software do IFG Campus Inhumas.

Este repositorio concentra a pesquisa, especificacao e implementacao do `AODV-EN`, uma adaptacao do AODV ([RFC 3561](#)) para redes mesh multi-hop sobre ESP-NOW v2 e ESP32.

Documentos base

- [docs/aodv-en-completo.pdf](#) - **toda a documentacao em um PDF unico** (~148 paginas, ordem top-down).
Regerar com `python3 docs/build_pdf.py`.
- [docs/aodv-en-spec-v1.md](#) - especificacao funcional fechada (`v1`, ativa)
- [docs/aodv-en-funcionamento.md](#) - guia didatico do funcionamento, com layout de bytes, traces e worked examples
- [docs/aodv-en-mapa-do-codigo.md](#) - guia de estudo do codigo: onde cada feature esta implementada, mapa por arquivo
- [docs/runbook-bancada.md](#) - passo a passo de bancada: identificar ESPs, build, flash, monitor, analise (cole-e-execute)
- [docs/aodv-base-invariantes.md](#) - invariantes do AODV que o projeto preserva
- [docs/aodv-en-estruturas-dados.md](#) - layout das tabelas, mensagens e configuracao
- [docs/plano-desenvolvimento-completo.md](#) - roadmap de fases e backlog
- [docs/aodv-en-spec-v0.md](#) - `OBSOLETA`, mantida apenas para registro historico

Features

Integradas na v1

- [docs/features/enfileamento-dos-dados.md](#) - fila de `DATA` pendente durante descoberta
- [docs/features/precursores.md](#) - precursores e `RERR` direcionado (RFC 3561 secao 6.2)

Planejadas para v2

- [docs/features/articulation-point-planejado.md](#) - detecao de no de corte

Casos de teste

- [docs/tests](#) - casos de teste de bancada com mapeamento direto para os criterios da spec v1

Caso	Topologia	Status
TC-001	2 nos diretos	ATIVO
TC-002	cadeia de 3 (A <-> B <-> C)	ATIVO (PASS 2026-04-21)
TC-003	3 nos com falha intermediaria	ATIVO
TC-004	3 nos sob ciclos por 30 min	ATIVO
TC-005	cadeia de 4 (A <-> B <-> C <-> D)	ATIVO

Biblioteca AODV-EN

A camada de roteamento é implementada como componente ESP-IDF reutilizável em [firmware/components/aodv_en](#). Ela expõe um adapter de transporte injetável, o que permite rodar o mesmo núcleo na simulação em C e no firmware ESP32.

Headers principais

- [firmware/components/aodv_en/include/aodv_en.h](#) - API `aodv_en_stack_*` consumida pela app
- [firmware/components/aodv_en/include/aodv_en_node.h](#) - núcleo do no (uso interno)
- [firmware/components/aodv_en/include/aodv_en_messages.h](#) - layout das mensagens no fio
- [firmware/components/aodv_en/include/aodv_en_types.h](#) - tipos centrais e configuração
- [firmware/components/aodv_en/include/aodv_en_limits.h](#) - limites e timers default
- [firmware/components/aodv_en/include/aodv_en_status.h](#) - códigos de status

Modulos

- vizinhos: [aodv_en_neighbors.h](#) / `.c`
- rotas (com precursores): [aodv_en_routes.h](#) / `.c`
- cache de `RREQ`: [aodv_en_rreq_cache.h](#) / `.c`
- cache de peers: [aodv_en_peers.h](#) / `.c`
- núcleo do no: [aodv_en_node.h](#) / `.c`

Firmware ESP32

App ESP-IDF de bancada em [firmware/main](#). Suporta dois modos selecionáveis por Kconfig:

- `app_demo`: legado, envia `HELLO` e `DATA` periódicos
- `app_proto_example`: protocolo de aplicação com `HEALTH/TEXT/CMD` e CLI serial

Pontos de entrada

- [firmware/README.md](#)
- [firmware/main/main.c](#)
- [firmware/main/app_proto_example.c](#)
- [firmware/main/Kconfig.projbuild](#)

Scripts

- [firmware/build.sh](#)
- [firmware/flash_monitor.sh](#)
- [firmware/monitor_log.sh](#) - captura serial em `firmware/logs/serial/`

- [firmware/idf-env.sh](#) - bootstrap do ESP-IDF

Perfis por papel

- [firmware/tests/tc001](#) - `TC-001`
- [firmware/tests/tc002](#) - `TC-002`, `TC-003`, `TC-004`
- [firmware/tests/tc005](#) - `TC-005` (cenario de 4 nos)

Ferramentas de analise

- [firmware/tools/draw_topology.py](#) - gera topologia (Mermaid, Graphviz DOT, SVG) a partir dos logs

Dashboard ao vivo

- [firmware/tools/live_monitor.py](#) - levanta um dashboard web (`http://localhost:8765/`) com topologia animada, metricas em tempo real e timeline de eventos. Funciona com hardware real (varias portas seriais em paralelo) ou em `--demo` sem hardware. Detalhes em [firmware/README.md](#).

Simulacao local

Simulacao em C que valida o fluxo `RREQ -> RREP -> DATA -> ACK` usando o mesmo nucleo do firmware, com adapter mock.

- [sim/aodv_en_sim.c](#) - cenario base (3 nos em linha)
- [sim/aodv_en_sim_100.c](#) - cenario com mais nos
- [sim/aodv_en_sim_1000.c](#) - cenario de stress
- [sim/aodv_en_sim_large.c](#) - cenario alternativo
- [sim/run_sim.sh](#)
- [sim/README.md](#)

Baseline de flooding e comparacao

Baseline comparativo ao AODV-EN: flooding controlado (broadcast com TTL + supressao de duplicatas por `(origem, sequencia)`). E um **componente ESP-IDF independente** (`flood_en`), com wire format, tipos e API proprios — nao depende do componente `aodv_en` .

- nucleo (componente proprio): [firmware/components/flood_en/include/flood_en.h](#) / [src/flood_en.c](#)
- app de bancada: [firmware/main/app_flood.c](#) (modo Kconfig `AODV_EN_APP_USE_APP_FLOOD`)
- sim baseline: `bash sim/run_sim.sh flood`
- sweep comparativo (grid 2x2..5x5, CSV): `bash sim/run_sim.sh compare`
- graficos: [firmware/tools/plot_compare.py](#)
- evidencias e analise: [docs/evidencias](#) (validacao do dashboard, flooding em hardware, AODV-EN vs flooding, metricas e graficos)

Status atual

Em uma frase: `AODV-EN v1` esta funcionalmente fechado e o baseline de flooding ja esta implementado, validado (sim + hardware) e comparado ao AODV-EN. O caminho critico restante e fechar `TC-002 / TC-003 / TC-004` em hardware com captura completa, ampliar os experimentos e escrever o TCC. Detalhes em [docs/plano-desenvolvimento-completo.md](#).

Relatorio Completo do Trabalho (AODV-EN vs Flooding)

AODV-EN vs Flooding controlado sobre ESP-NOW: relatório completo do trabalho

Trabalho de Conclusão de Curso — Bacharelado em Engenharia de Software — IFG Câmpus Inhumas

Documento-síntese, passo a passo, de tudo que foi projetado, implementado, instrumentado, medido, depurado, comparado e concluído. Cobre o protocolo AODV-EN (adaptação do AODV / RFC 3561 para ESP-NOW v2 em ESP32), o algoritmo de referência (flooding controlado), o ambiente de bancada com 3 ESP32, a metodologia experimental do TCC (cenários, parâmetros e métricas), a instrumentação criada para extrair as métricas, os defeitos encontrados e corrigidos com evidência, os resultados de simulação e de hardware, a comparação final e a discussão crítica com limitações e trabalho futuro.

Princípio editorial deste relatório. Nenhum número aqui foi inventado ou estimado de cabeça. Toda métrica de hardware vem de log serial real, processado por ferramenta (`tcc_metrics.py`) e registrada num ledger reproduzível (`experiments.json`). Onde houve dúvida de projeto (parâmetro de algoritmo, fórmula de métrica, constante de energia), a dúvida foi registrada e levada ao autor humano para decisão — em vez de chutada. As constantes de energia são estimativas de datasheet, explicitamente rotuladas como tais.

Sumário

1. Contexto, problema e objetivos
 2. Visão geral da arquitetura do sistema
 3. O protocolo AODV-EN em detalhe
 4. O baseline de comparação: flooding controlado
 5. Metodologia experimental (DSR, cenários, parâmetros, métricas)
 6. Instrumentação para medição das métricas
 7. Ambiente de bancada e procedimentos de hardware
 8. Defeitos encontrados e corrigidos (com evidência)
 9. Resultados — simulação
 10. Resultados — hardware (AODV-EN vs flooding)
 11. Comparação e discussão
 12. Decisões de projeto e justificativas (Q1–Q6)
 13. Limitações e ameaças à validade
 14. Trabalho futuro
 15. Reprodutibilidade — passo a passo de comandos
 16. Diário de bordo (cronologia do trabalho)
 17. Glossário
 18. Índice de artefatos (código, commits, evidências)
-

1. Contexto, problema e objetivos

1.1 Internet das Coisas, redes de sensores e mesh

A Internet das Coisas (IoT) e as Redes de Sensores Sem Fio (RSSF) supõem muitos dispositivos de baixo custo e baixa potência cooperando para coletar e transmitir dados. Em muitos cenários reais (galpões, lavouras, prédios, áreas externas), nem todos os nós alcançam um ponto de acesso central por rádio direto. A solução é a comunicação **multi-hop**: cada nó atua também como repetidor, encaminhando pacotes de vizinhos até o destino. Essa topologia auto-organizada e sem infraestrutura fixa é uma **rede mesh ad-hoc**.

Redes mesh ad-hoc precisam de **protocolos de roteamento** que descubram e mantenham caminhos entre nós sem servidor central, tolerando entrada/saída de nós e falhas de enlace. Há duas grandes famílias: **proativos** (mantêm tabelas de rota sempre atualizadas, ex. OLSR) e **reativos / sob-demanda** (descobrem rota só quando há tráfego, ex. AODV, DSR). Protocolos reativos economizam tráfego de controle em redes com comunicação esporádica — o caso típico de RSSF.

1.2 O ESP32 e o ESP-NOW

O **ESP32** (módulo ESP32-WROOM-32, SoC ESP32-D0WD) é um microcontrolador dual-core de 32 bits, 240 MHz, com Wi-Fi 2.4 GHz e Bluetooth integrados, baixo custo e amplo ecossistema (ESP-IDF). É plataforma natural para nós de RSSF.

O **ESP-NOW** é um protocolo proprietário da Espressif que troca quadros curtos diretamente entre dispositivos Wi-Fi **sem associação a um access point** e **sem pilha IP**. Vantagens: latência baixa, *setup* simples, confirmação em nível de enlace (o rádio reporta sucesso/falha de cada unicast). Limitações relevantes para mesh:

- **Limite de peers** (tipicamente 20 *peers* simultâneos registrados).
- **Sem multi-hop nativo**: ESP-NOW entrega só ao vizinho de rádio direto; qualquer encaminhamento além de 1 salto precisa ser implementado na aplicação.
- **Broadcast não confiável**: o *broadcast* ESP-NOW não tem confirmação como o unicast.
- **MTU por quadro** (ESP-NOW v2 amplia o tamanho do *payload* frente à v1, mas ainda é limitado; o projeto respeita `ESP_NOW_MAX_DATA_LEN_V2`).

1.3 O problema do TCC

Não existe, de fábrica, roteamento mesh multi-hop sobre ESP-NOW. O AODV (RFC 3561), concebido para redes IP, depende de *broadcast* IP confiável e de uma camada de rede que o ESP-NOW não oferece. **O problema é adaptar o AODV para operar sobre ESP-NOW v2 em ESP32**, respeitando o limite de *peers*, a ausência de *broadcast* confiável e a natureza de enlace do ESP-NOW — e então **avaliar** se essa adaptação (AODV-EN) traz ganhos mensuráveis frente a uma abordagem trivial de disseminação (flooding).

1.4 Objetivos

- **Geral**: projetar, implementar e avaliar o AODV-EN, uma adaptação do AODV para mesh multi-hop sobre ESP-NOW v2 em ESP32.
- **Específicos**: 1. Especificar e implementar o núcleo de roteamento como componente reutilizável. 2. Implementar um algoritmo de referência (flooding controlado) para comparação justa. 3. Montar um ambiente de testes (bancada com ESP32 + simulação em C). 4. Instrumentar o firmware para extrair as métricas do TCC (PDR, latência, NRL, energia). 5. Executar experimentos comparativos e analisar os resultados.

Este relatório cobre o estado alcançado em todos esses pontos.

2. Visão geral da arquitetura do sistema

O repositório `aodv-en` separa **núcleo de protocolo** (lógica pura, testável em PC) de **adaptadores de transporte** (ESP-NOW no firmware; rádio simulado na simulação). Essa separação é a decisão arquitetural central: o **mesmo núcleo** roda na simulação em C e no ESP32, o que dá confiança de que o que se valida no PC corresponde ao que roda na bancada.

2.1 Componentes ESP-IDF

```
firmware/components/
aodv_en/                # nucleo do AODV-EN (componente reutilizavel)
  include/              # API publica + tipos + mensagens + limites
  src/                  # implementacao (node, routes, neighbors, rreq_cache, peers, mac)
flood_en/               # baseline de flooding controlado (componente INDEPENDENTE)
  include/flood_en.h
  src/flood_en.c
firmware/main/          # apps de bancada
  app_demo.c            # app AODV-EN (HELLO + DATA periodico) - usado nos experimentos
  app_flood.c           # app flooding (modo Kconfig AODV_EN_APP_USE_APP_FLOOD)
  app_proto_example.c   # exemplo de protocolo de aplicacao (HEALTH/TEXT/CMD)
  main.c               # ponto de entrada, seleciona o app por Kconfig
sim/                   # simulacao em C com radio em memoria
  aodv_en_sim.c         # 3 nos A-B-C (descoberta, ACK retry, late-join)
  aodv_en_sim_large.c   # 6 nos A-F (RERR, reconvergencia)
  aodv_en_sim_100.c     # grade 10x10
  aodv_en_sim_1000.c    # grade 32x32 (stress)
  flood_en_sim.c        # baseline flooding (3 nos)
  compare_sim.c         # AODV-EN vs flooding na MESMA topologia em grade
  run_sim.sh            # build+run de cada variante
firmware/tools/         # analise
  live_monitor.py       # dashboard web em tempo real (WebSocket + Cytoscape)
  extract_monitor_metrics.py # extrai contadores/rotas de log serial (AODV)
  tcc_metrics.py        # calcula PDR/latencia/NRL/energia de log serial real
  plot_compare.py / plot_comparison_metrics.py # graficos
```

2.2 O adaptador injetável

O núcleo nunca chama `esp_now_send` diretamente. Ele recebe, na inicialização, um conjunto de *callbacks* (o **adapter**):

- `emit_frame(user_ctx, next_hop, frame, len, broadcast)` — transmite um quadro. No firmware, vira `esp_now_send`; na simulação, entrega o quadro aos nós alcançáveis em memória.
- `deliver_data(user_ctx, origin, payload, len)` — entrega *payload* à aplicação quando o nó é o destino final.
- `ack_received(user_ctx, ack_sender, seq, rtt_ms)` — sinaliza que um DATA enviado foi confirmado fim-a-fim (com o RTT medido, ver §6).
- `now_ms(user_ctx)` — relógio monotônico em milissegundos.

Essa inversão de dependência permite **testar o protocolo sem rádio**: a simulação implementa um "rádio" determinístico (matriz de adjacência `links[i][j]`) e injeta os mesmos *callbacks*.

2.3 Camadas de dado e fluxo

- A aplicação chama `send_data(dest, payload, ack_required)`.
- O núcleo decide: se há rota válida, monta o quadro DATA e emite via `next_hop` (unicast); se não há rota, **enfileira** o DATA e dispara descoberta (RREQ).
- Cada nó intermediário, ao receber um DATA cujo destino não é ele, **encaminha** pelo próximo salto da sua tabela de rotas.
- O destino entrega à aplicação (`deliver_data`) e, se o DATA pediu, devolve um ACK pelo caminho reverso.

5. A origem, ao receber o ACK, consome a entrada pendente e dispara `ack_received`.

3. O protocolo AODV-EN em detalhe

O AODV-EN preserva os invariantes do AODV (RFC 3561) e adapta o transporte para ESP-NOW.

3.1 Tipos de mensagem

O cabeçalho comum (`aodv_en_header_t`, empacotado) tem: versão de protocolo, tipo de mensagem, flags, `hop_count`, `network_id` (isola redes coexistentes no mesmo canal) e `sender_mac` (o MAC do nó que transmitiu este quadro neste salto — distinto da origem lógica).

Tipos (`aodv_en_message_type_t`):

Tipo	Valor	Função
HELLO	0	Anúncio periódico de presença; mantém a tabela de vizinhos
RREQ	1	Route Request — disseminado para descobrir rota até um destino
RREP	2	Route Reply — resposta unicast pelo caminho reverso
RERR	3	Route Error — sinaliza rota quebrada (precursores)
DATA	4	Dados da aplicação (com <code>ttl</code> , <code>sequence_number</code> , <code>payload</code>)
ACK	5	Confirmação fim-a-fim de um DATA

O DATA carrega `originator_mac`, `destination_mac`, `sequence_number`, `ttl`, `payload_length` e o `payload[]` flexível. O ACK carrega `originator_mac` (quem entrega/confirma), `destination_mac` (a origem do DATA) e `ack_for_sequence`.

3.2 Descoberta de rotas (RREQ → RREP)

Quando a origem não tem rota para o destino:

1. Emite um **RREQ** com `originator=self`, `rreq_id` incremental, `originator_seq`, `destination_seq` (último conhecido), `hop_count=0`, `ttl=NET_DIAMETER`.
2. Cada nó que recebe o RREQ: confere o **cache de RREQ** (par (`originador`, `rreq_id`)) para não reprocessar duplicatas; **instala/atualiza rota reversa** para a origem com `next_hop = quem transmitiu o RREQ`; incrementa `hop_count`, decrementa `ttl` e redissemina.
3. Quando o RREQ chega ao destino (ou a um nó com rota válida e fresca para ele), gera-se um **RREP** unicast de volta pelo caminho reverso. Cada nó no retorno **instala rota direta** para o destino e registra **precursores** (RFC 3561 §6.2), necessários para RERR direcionado.
4. A origem recebe o RREP, instala a rota e **drena a fila de DATA pendente** para aquele destino.

3.3 Números de sequência e prevenção de loops

Cada nó mantém um número de sequência próprio (RFC 3561 §6.6.1) que cresce monotonicamente. Rotas carregam o `dest_seq_num` conhecido. Uma rota só é substituída por outra com número de sequência **mais novo** (ou igual com menos saltos), o que previne *loops* e adoção de informação obsoleta. O DATA usa um contador de sequência por nó (`next_data_seq`), também monotônico — propriedade explorada na instrumentação de latência (§6).

3.4 Manutenção de rotas, RERR e precursores

Rotas têm **tempo de vida** (`route_lifetime_ms`); expiram se não usadas. Falhas de enlace (o ESP-NOW reporta falha de unicast) incrementam um contador por vizinho; ao cruzar `link_fail_threshold`, as rotas que usam aquele próximo salto são **invalidadas** e um **RERR** é enviado aos **precursores** daquelas rotas (os vizinhos que dependiam delas) — RERR direcionado, não *broadcast* cego.

3.5 Fila de DATA pendente

Durante a descoberta, o DATA não é descartado: vai para uma **fila pendente** (`AODV_EN_PENDING_DATA_QUEUE_SIZE`). Quando a rota se estabelece (RREP), a fila é drenada. Há *backoff* de re-tentativa de descoberta e expiração de itens muito antigos. Isso evita perder o primeiro pacote de cada fluxo (comportamento comum e indesejado em AODV ingênuo).

3.6 Confirmação fim-a-fim (ACK) e *pending_ack*

Quando o DATA pede confirmação (`ack_required`), a origem registra a entrada em `pending_ack` com (`destination_mac`, `sequence_number`, `last_sent_at_ms`). O destino, ao entregar, devolve um ACK. A origem consome a entrada casando (`destino`, `seq`) — o `next_data_seq` monotônico garante unicidade. Há re-transmissão por *timeout* de ACK (`ack_timeout_ms`) com número limitado de tentativas.

3.7 Considerações de projeto da adaptação (TCC §3.6)

O TCC define três adaptações específicas para ESP-NOW:

- **§3.6.1 Gerência de *peers* com LRU.** Como o ESP-NOW limita *peers* simultâneos, propõe-se política *Least Recently Used*: quando a tabela enche, remove-se o *peer* menos usado.
- **§3.6.2 Flooding controlado por unicast.** Como o ESP-NOW não tem *broadcast* confiável, a disseminação de RREQ pode ser feita por **unicast sequencial** a cada vizinho conhecido, trocando mais transmissões por mais confiabilidade (a confirmação de enlace do ESP-NOW).
- **§3.6.3 Métrica híbrida.** Em vez de só contagem de saltos, propõe-se $\text{Custo(rota)} = \alpha \cdot \text{HopCount} + \beta \cdot (1/\text{RSSI_médio})$, combinando distância lógica e qualidade de enlace. (Os pesos α , β são decisão de projeto — ver §12.)

Estado de implementação: o núcleo atual implementa descoberta reativa, rotas com números de sequência, precursores, fila pendente, ACK fim-a-fim e invalidação por falha de enlace. LRU de peers, RREQ-por-unicast e métrica híbrida α/β são pontos de evolução previstos e discutidos em §12 (Q5), pois envolvem decisões de parâmetro do autor.

4. O baseline de comparação: flooding controlado

4.1 Por que flooding

O flooding é o algoritmo mais simples de disseminação multi-hop: cada nó retransmite cada pacote novo que recebe. Serve como **limite inferior de complexidade** e **referência de PDR máximo** (em rede conectada, se há caminho, o flooding alcança), além de **isolar o impacto** dos mecanismos do AODV-EN — o flooding não tem tabelas de rota nem controle de roteamento, então o que ele "gasta" é puramente disseminação.

4.2 Especificação (TCC §4.6.1)

- **Identidade do pacote:** (`origem`, `número de sequência`) único.

- **Detecção de duplicatas:** *buffer* circular dos últimos `N=100` identificadores; pacote repetido é descartado.
- **Controle de TTL:** TTL decrementa a cada salto; com TTL=0 não retransmite (limita alcance e previne *loops*). TTL máximo = 5 (Quadro 10 do TCC).
- **Retransmissão:** pacote não-duplicado, com TTL>0 e destino $\neq$ próprio, é retransmitido a **todos os vizinhos conhecidos via unicast** ESP-NOW.
- **Entrega:** ao chegar ao destino, entrega o *payload* e registra a recepção.

4.3 Implementação: componente `flood_en` independente

Originalmente o flooding nasceu **dentro** do componente `aodv_en`. Por decisão do autor ("o flooding-en deve estar completo neste repo e não junto do AODV"), ele foi **extraído** para um componente ESP-IDF próprio e autônomo, `firmware/components/flood_en/`, com:

- **Wire format próprio** (`flood_en_header_t`, `flood_en_data_msg_t`, `flood_en_ack_msg_t`), layout idêntico ao do AODV-EN mas com símbolos próprios.
- **Status, config, callbacks e nó próprios** (`flood_en_status_t`, `flood_en_config_t`, `flood_en_callbacks_t`, `flood_en_node_t`).
- **Zero dependência** do `aodv_en`: compila isolado só com seu `include/`.

Verificação da independência: `cc -c -Ifirmware/components/flood_en/include firmware/components/flood_en/src/flood_en.c` compila limpo (`-Wall -Wextra`), e `git grep aodv_en -- firmware/components/flood_en/` retorna vazio. (Commit `8a2bde3`.)

4.4 Lógica do `flood_en`

- `flood_en_node_send_data`: atribui `seq = ++next_seq`, registra `(self, seq)` no *seen* de dados (para não re-floodar o próprio eco), monta o DATA com `ttnl = ttnl_default`, emite.
- `flood_en_node_on_recv`: valida versão/ `network_id`; se DATA — dedup por `(originator, sequence_number)`; se destino==self → entrega + (se pedido) floda um ACK; se não — decrementa TTL e re-floda (drop se `ttnl<=1`).
- ACK também é disseminado por flooding, com **dedup próprio** por `(originador-do-DATA, ack_for_sequence)` e limite por `hop_count` vs `max_hops`.

4.5 Transporte por unicast-para-cada-vizinho (decisão Q1)

Por decisão do autor (Q1), o flooding dissemina por **unicast a cada vizinho conhecido** (TCC §4.6.1d), não por um único *broadcast*. Para manter o `flood_en` agnóstico a transporte, a estratégia foi: o núcleo continua chamando `emit(broadcast=true)`, e o **adaptador do `app_flood` faz o *fanout***: para cada vizinho conhecido, um `esp_now_send` unicast. Os vizinhos são aprendidos pelo `src` de cada quadro recebido (`app_note_neighbor`). Antes de conhecer vizinhos (*bootstrap*), cai em *broadcast*. (Commit `65cfe0a`.)

Consequência medível: no hardware, uma "disseminação" passa a custar **N transmissões** (uma por vizinho), refletindo fielmente o custo de NRL/energia do flooding por unicast — exatamente o que o TCC pediu para isolar.

4.6 Parâmetros do TCC aplicados (decisão Q2)

`FLOOD_EN_TTL_DEFAULT=5`, `FLOOD_EN_MAX_HOPS_DEFAULT=5` (TTL máximo 5 saltos), `FLOOD_EN_SEEN_SIZE=100` (*buffer* de dedup `N=100`); no perfil de bancada, `payload=32 bytes` e `SEND_INTERVAL_MS=1000` (1 pacote/s). (Commit `5d3550f`.)

5. Metodologia experimental

5.1 Design Science Research

O trabalho segue **DSR**: constrói-se um artefato (AODV-EN), avalia-se empiricamente contra um artefato de referência (flooding), e iteram-se as conclusões. As fases do TCC mapeiam-se em: revisão; projeto do algoritmo; implementação + montagem do ambiente; execução dos experimentos e coleta; análise e discussão.

5.2 Cenários experimentais (TCC §4.4.1, Quadro 9)

Cenário	Topologia	Configuração	Objetivo
C1: Linear	Cadeia	5 nós N1→...→N5, 10 m entre nós, comunica N1↔N5, 4 saltos	Multi-hop básico, latência acumulada
C2: Árvore	Hierárquica	7 nós (1 raiz, 2 nível-1, 4 folhas), folhas↔raiz, ≤2 saltos	Encaminhamento hierárquico, agregação
C3: Mesh parcial	Redundante	6 nós, 2–3 vizinhos cada, comunica extremidades	Seleção de rota, caminhos alternativos
C4: Falha	Linear + interrupção	C1 com N3 desligado após 60 s	Detecção de falha, auto-recuperação

Restrição de bancada: dispõe-se de **3 ESP32** físicos. Logo, o hardware cobre um **C1 reduzido (3 nós, ~2 saltos)**; em hub de bancada, na prática **1 salto** (todos se enxergam). C2/C3/C4 e a escala de 5–10 nós e 30 repetições migram para a **simulação**. Esta limitação está registrada e é repetida nas conclusões.

5.3 Parâmetros (TCC §4.4.2, Quadro 10)

Parâmetro	AODV-EN	Flooding
Número de nós	5–10 (por cenário)	5–10 (por cenário)
Tamanho do <i>payload</i>	32 bytes	32 bytes
Taxa de envio (regime)	1 pacote/s	1 pacote/s
Duração de cada execução	300 s (alvo)	300 s (alvo)
Repetições por cenário	30 (alvo)	30 (alvo)
HELLO_INTERVAL	2.000 ms	N/A
ACTIVE_ROUTE_TIMEOUT	10.000 ms	N/A
TTL máximo	N/A	5 saltos

No hardware as execuções deste relatório usaram janelas de ~60 s por run (não 300 s) e 2 seeds por algoritmo (não 30), por economia de tempo de bancada; o ledger e os alvos perpétuos do autopilot permitem acumular as 30 repetições incrementalmente. `payload=32 B`, `taxa=1 pkt/s`, `HELLO=2 s` foram aplicados em ambos para comparação justa.

5.4 Métricas (TCC §4.5, Quadro 11)

Métrica	Definição	Fórmula
PDR	Razão de pacotes entregues	$\text{recebidos/enviados} \times 100\%$
Latência fim-a-fim	Tempo envio→recepção	$t_{\text{recv}} - t_{\text{send}}$
NRL	<i>Normalized Routing Load</i>	$\text{pacotes de controle} / \text{pacotes de dados entregues}$
Energia estimada	Consumo modelado	$\Sigma(N_{\text{tx}} \cdot E_{\text{tx}} + N_{\text{rx}} \cdot E_{\text{rx}} + t_{\text{idle}} \cdot P_{\text{idle}})$

Reportam-se média, desvio-padrão e IC 95% (quando há amostras suficientes).

6. Instrumentação para medição das métricas

6.1 O problema descoberto na análise (m1 / QUESTIONS)

Ao mapear a telemetria existente, constatou-se que **as 4 métricas do TCC não saíam prontas** do firmware nem do `extract_monitor_metrics.py` (que faz análise de rotas AODV, não as 4 métricas). Em particular:

- **Latência fim-a-fim é impossível** subtraindo *timestamps* de dois nós: cada ESP32 tem seu `esp_timer` desde o *boot*, **relógios não sincronizados**. `t_recv(destino) - t_send(origem)` não tem base de tempo comum.
- **PDR/NRL/energia** não eram emitidos; o extractor dá contagens (tx/rx/delivered/rotas).

Essas lacunas foram registradas em `results/QUESTIONS.md` (Q3, Q4, Q6) e levadas ao autor. A decisão (Q3) foi **instrumentar o firmware** para produzir as métricas a partir de log real.

6.2 Latência por RTT na origem (mesmo relógio)

Solução para o problema dos relógios: **medir o RTT na origem**, que usa **um único relógio**. A origem registra `t_send` por `seq` ao enviar o DATA; ao receber o ACK daquele `seq`, calcula `rtt = now - t_send`. A latência fim-a-fim de uma via é estimada como `rtt/2`.

- No `flood_en`: tabela `tx_times[FL00D_EN_TX_TRACK]` guarda `(seq, sent_at_ms)`; em `handle_ack` (`destino==self`) calcula-se o RTT e passa-se no `callback` `ack_received` (assinatura ganhou `rtt_ms`). (Commit `687ef0d`.)
- No `aodv_en`: a entrada `pending_ack` **já tinha** `last_sent_at_ms`; bastou `pending_ack_consume` devolver `rtt = now - last_sent_at_ms` por `out-param` e propagar no `callback` de ACK (assinaturas `aodv_en_ack_received_fn` e `aodv_en_on_ack_fn` ganharam `rtt_ms`). (Commit `ed908a9`.)

A aplicação registra, por ACK, uma linha `LAT seq=<n> rtt_ms=<r>`. O `tcc_metrics.py` lê essas linhas e calcula média/desvio/IC95 do RTT (e `rtt/2` para uma via).

Nuance honesta: na bancada, a app processa em laço de 100 ms (`APP_L00P_DELAY_MS`), o que **quantiza** o RTT medido (valores múltiplos de ~100–120 ms, desvio ~0 em alguns runs). É medição real, porém de granularidade grosseira; a §13 registra isso.

6.3 PDR a partir da origem

$\text{PDR} = \text{acks_recebidos} / \text{data_enviados}$ na origem (um único nó, um único relógio). O ACK é a prova de entrega fim-a-fim. `data\_enviados` é contado pelas linhas de envio do log (`flood DATA broadcast` no flooding; `DATA queued...` no AODV-EN). Evita depender de contadores cruzados entre nós.

6.4 NRL e o contador `control_tx`

NRL = controle/dados-entregues. Foi adicionado um contador `control_tx_frames` ao núcleo: no `aadv_en_node_emit`, classifica-se o quadro por `message_type` e incrementa-se o contador quando é **HELLO/RREQ/RREP/RERR** (controle); DATA/ACK não contam. O flooding **não tem quadros de controle** (só DATA/ACK), então seu `control_tx` é **0 por definição** — resultado real e coerente com a definição do TCC (o custo do flooding aparece em rx/energia, não em controle). O contador é exposto no `stack_stats` e no log do `app_demo` (`control=`). (Commit `3a1661b`.)

6.5 Modelo de energia (decisão Q6)

$E = \sum \text{nós} (N_{tx} \cdot E_{tx} + N_{rx} \cdot E_{rx} + t_{idle} \cdot P_{idle})$, com $E_{tx} = V \cdot I_{tx} \cdot t_{pkt}$, $E_{rx} = V \cdot I_{rx} \cdot t_{pkt}$, $P_{idle} = V \cdot I_{idle}$. Como não houve medição com *shunt/INA219*, usam-se **constantes de datasheet do ESP32-WROOM-32, rotuladas como estimativa** (decisão Q6 do autor): $V=3,3 \text{ V}$, $I_{tx}=240 \text{ mA}$, $I_{rx}=100 \text{ mA}$, $I_{idle}=20 \text{ mA}$, $t_{pkt}=1,0 \text{ ms}$ (airtime estimado de um quadro ESP-NOW de ~72 B). A antena externa 3 dBi melhora alcance/RSSI mas não altera I_{tx} (mesmo PA).

6.6 A ferramenta `tcc_metrics.py`

`firmware/tools/tcc_metrics.py` consome **logs serial reais** e calcula as 4 métricas, **só de valores parseados** (nunca inventa):

- entrada: `--origin <log>` (nó de origem, para PDR/latência) e `--node <log>` repetido (todos os nós, para NRL/energia de rede) + `--duration-s`.
- saída: JSON com `pdr_pct`, `latency_rtt_ms` e `latency_oneway_ms` (n/média/desvio/IC95), `sum_tx/sum_rx/sum_control_tx/sum_delivered`, `nrl`, `energy_j` e o bloco de constantes de energia explicitamente rotulado como estimativa.

Cada *run* de hardware vira uma entrada no *ledger* via `experiment add` (a partir do JSON), e a comparação final usa `experiment compare` (médias data-driven) — nunca números de memória.

7. Ambiente de bancada e procedimentos de hardware

7.1 Hardware

- **3x ESP32-DevKitC-32** (módulo ESP32-WROOM-32, USB-C, conversor serial CH340C, dual-core).
- **Antena externa 2,4 GHz 3 dBi** por *pigtail* U.FL/IPX → SMA (melhora alcance e RSSI; não altera potência de TX / I_{tx}).
- Conexão por USB serial ao laptop (CH340C). Portas observadas na bancada: `/dev/cu.usbserial-214420` (N1), `-214430` (N2), `-214440` (N3), a 115200 bps.

Papéis no experimento (todos no mesmo hub, ~1 salto): `N1 = 28:05:A5:34:99:34` (destino), `N2 = 28:05:A5:33:EB:80` (origem), `N3 = 28:05:A5:33:D6:1C` (origem). Em ambos os algoritmos, N2 e N3 enviam DATA para N1.

7.2 Toolchain e comandos

- ESP-IDF v6.0 em `~/espressif/v6.0/esp-idf`; *export* via `ESP_IDF_EXPORT`.
- Python com `pyserial` / `aiohttp` / `matplotlib`: `~/espressif/python_env/idf6.0_py3.14_env/bin/python`.
- *Build*: `source firmware/idf-env.sh; bash firmware/build.sh`.
- *Build+flash* por perfil: `zsh firmware/tests/<perfil>/build_flash.sh <PORTA> <TARGET_MAC>`.
- Simulação: `bash sim/run_sim.sh {basic|large|100|1000|flood|compare}`.

- Métricas: `python3 firmware/tools/tcc_metrics.py ...`.

7.3 Regra de serial exclusivo

Uma porta serial só pode ser aberta por **um** processo. Abrir a mesma `/dev/cu.usbserial-*` em dois leitores corrompe os *bytes* (erro "device reports readiness to read but returned no data ... multiple access on port"). Procedimento adotado: cada captura **abre** → **lê N segundos** → **fecha** via `pyserial`, e **não** se usa o monitor interativo do ESP-IDF concorrentemente.

7.4 Dashboard em tempo real

`firmware/tools/live_monitor.py` levanta um dashboard web (Cytoscape.js + WebSocket) em `http://localhost:8765/`, com nós, arestas de vizinhança, contadores e *timeline* de eventos, alimentado pelas linhas de log serial. Validado com 3 ESP32 reais (3 nós *online*, rotas válidas, contadores subindo, 73 eventos *streaming* em ~8 s) — ver §8 e a evidência `results/m7-telemetry.json`.

8. Defeitos encontrados e corrigidos (com evidência)

Três defeitos reais foram encontrados e corrigidos durante a montagem do ambiente e a coleta. Cada um foi confirmado com evidência em disco antes e depois.

8.1 Dedup de ACK no flooding (assimetria N2 ack=0) — `d3d34eb`

Sintoma (hardware). Com dois nós origem (N2, N3) enviando ao mesmo destino (N1), um deles recebia **zero ACKs**: medido `N2 ack=0` e `N3 ack=32`.

Causa-raiz. O ACK era deduplicado pela chave `(originator_mac = nó que entrega, seq)`. Como cada origem tem seu próprio contador de `seq` começando em 1, os ACKs que o destino dissemina para N2 e para N3 colidiam na mesma chave `(N1, seq)`; o segundo a chegar era descartado como duplicado em toda a rede → uma origem ficava sem ACK.

Correção. A identidade única de um ACK é a do DATA original: `(origem-do-DATA, seq)`. No quadro de ACK, a origem-do-DATA é o `destination_mac`. Passou-se a deduplicar/lembrar por `(destination_mac, ack_for_sequence)` no recebimento e por `(data_originator, seq)` no envio.

Validação (verde em 3 níveis). `flood_en` compila `-Wall -Wextra`; `sim flood` passa sem regressão; no hardware, pós-correção, **N2 ack=30 e N3 ack=30** (simétrico, antes 0 vs 32). Evidência: `results/m-audit-flood-ack-fix.md`, `results/m-audit-fix-N{1,2,3}.log`.

8.2 Build do flooding produzindo firmware AODV — corrigido em `inc6` (rebuild limpo)

Sintoma. Após gravar o "flooding" nos 3 ESPs, a telemetria mostrava `aodv_en_app`, não `flood_en_app`.

Causa-raiz. O `idf.py` reusa o `sdkconfig` existente do diretório de *build*; definir `SDKCONFIG / SDKCONFIG_DEFAULTS` por variável **não sobrescreve** um `sdkconfig` já gerado. O `build/flood` tinha *cache* de uma configuração anterior (`app_demo`), então o modo `AODV_EN_APP_USE_APP_FLOOD` não foi efetivamente selecionado, apesar do *build* "verde".

Correção e prevenção. `rm -rf build/flood` antes de reconstruir via `build_flash.sh` (que regenera o `sdkconfig` a partir dos *defaults*); e **sempre verificar o modo**, não só "compilou": `strings build/flood/aodv_en_firmware.bin | grep flood_en_app` e confirmar no *boot* serial. Conhecimento registrado para os ciclos seguintes.

8.3 Corrida ao flashar em paralelo no mesmo *build dir* — diagnosticado e contornado

Sintoma. Três `idf.py -B build/X flash` em **paralelo** no **mesmo** diretório quebraram o *link* (`grabRef.cmake` "file failed to open", `FAILED: aadv_en_firmware.elf`).

Causa-raiz. `idf.py flash` reconstrói antes de gravar; três processos disputando o mesmo diretório de *build* corrompem o estado intermediário.

Contorno. Construir **uma** vez (`idf.py build`) e gravar por porta via **esptool** (`python -m esptool ... write_flash @flash_args`), que é independente do *build dir* e pode rodar em paralelo com segurança. Procedimento adotado em todas as coletas subsequentes.

9. Resultados — simulação

A simulação roda o **mesmo núcleo** do firmware sobre um rádio em memória, de forma determinística.

9.1 Validação funcional

- `bash sim/run_sim.sh basic` → "Simulation passed." (3 nós A-B-C: descoberta, *retry* de ACK, *late-join*).
- `bash sim/run_sim.sh large` → "Large scale simulation PASSED." (6 nós A-F: RERR, reconvergência).
- `bash sim/run_sim.sh flood` → "Flood simulation passed." (entrega, ACK, dedup, sem *storm*; com a instrumentação, exibe `rtt_ms`).

Essas três regressões foram re-executadas verdes em vários momentos do trabalho (pós-extração do `flood_en`, pós-correção do ACK, pós-instrumentação).

9.2 Varredura comparativa em grade (`compare_sim.c`)

`sim/compare_sim.c` roda **AODV-EN e flooding na MESMA topologia em grade** (vizinhança de 4), origem no canto e destino no canto oposto, 10 pacotes DATA com ACK, contando cada transmissão no ar. Resultado (commit `cdc62ae`, antes do TTL=5):

Grade	Nós	Saltos	Entrega AODV	Entrega Flood	TX/entrega AODV	TX/entrega Flood
2×2	4	2	10/10	10/10	4,50	6,00
3×3	9	4	10/10	10/10	17,60	16,00
4×4	16	6	10/10	10/10	26,70	30,00
5×5	25	8	10/10	10/10	35,60	41,00

Leitura: entrega 100% em ambos; há **cruzamento** de custo de canal por escala — flooding competitivo em rede pequena, AODV-EN com menor TX/entrega à medida que a rede cresce.

9.3 Efeito do TTL=5 do TCC

Aplicado o TTL=5 (Quadro 10) ao flooding, a varredura passa a mostrar **entrega 0** do flooding em grades de diâmetro > 5 (4×4 = 6 saltos, 5×5 = 8 saltos): o pacote não alcança o canto oposto antes do TTL zerar. **Não é regressão — é o efeito real do parâmetro.** O AODV-EN (rotas, `max_hops=16`) continua entregando. Isso ilustra concretamente uma limitação do flooding com TTL fixo frente ao roteamento.

10. Resultados — hardware (AODV-EN vs flooding)

10.1 Setup das coletas

C1 reduzido, 3 ESP32 no hub (~1 salto), instrumentado, **idêntico** para os dois algoritmos: *payload* 32 B, 1 pacote/s, 2 origens (N2, N3) → destino N1, janela de ~60 s por *run*. Flooding = unicast-por-vizinho, TTL=5, dedup=100; AODV-EN com `HELLO=2 s`.

10.2 Ledger de execuções (dados reais, `experiments.json`)

id	algo	seed	PDR (%)	latência one-way (ms)	NRL	energia (J, est.)
e1	aodv-en	1	98,57	60,0	0,7785	12,4147
e3	aodv-en	2	100,0	60,0	0,7821	12,4071
e2	flooding	1	100,0	50,0	0,0	12,8029
e4	flooding	2	100,0	50,6	0,0	12,8482

Cada linha foi extraída por `tcc_metrics.py` de logs serial reais (`results/m10-{aodv,flood}[-s2]-N{1,2,3}.log + *-metrics.json`).

10.3 Comparação (média de 2 seeds, via `experiment compare`)

Métrica	AODV-EN	Flooding	Δ (flood – aodv)	%
PDR (%)	99,285	100,0	+0,715	+0,72%
Latência one-way (ms)	60,0	50,3	-9,7	-16,17%
NRL (controle/dados)	0,7803	0,0	-0,7803	-100%
Energia (J, est.)	12,4109	12,8255	+0,4146	+3,34%

Contadores de rede agregados (por *run* típico, 3 nós): AODV-EN `tx≈441 rx≈562 control≈123 delivered≈158`; flooding `tx≈614 rx≈1323 control=0 delivered≈154`.

11. Comparação e discussão

11.1 Entrega (PDR)

Ambos altíssimos no hub: flooding 100%, AODV-EN 99,3% (em *seed* 1 perdeu 1 pacote, em *seed* 2 entregou 100%). No regime de 1 salto e canal limpo, a redundância do flooding garante 100%; o AODV-EN ocasionalmente perde o 1.º pacote de um fluxo durante a descoberta/timeout (mitigado, mas não eliminado, pela fila pendente).

11.2 Latência

Flooding ~16% menor (50 vs 60 ms one-way) — não paga espera por descoberta de rota. Ressalva importante: os valores são **quantizados pelo laço de 100 ms** da aplicação; a diferença absoluta (~10 ms) é da ordem da granularidade de medição. A conclusão qualitativa (flooding não tem custo de *setup* de rota) é robusta; o valor numérico exato é grosseiro.

11.3 NRL (carga de roteamento normalizada)

AODV-EN 0,78 (HELLO + RREQ + RREP por dado entregue) vs flooding 0,0. Pela definição do TCC (controle/dados), o flooding tem overhead de **controle** nulo — ele não tem roteamento. **Mas isso não significa que o flooding é "de graça"**: seu custo migra para a recepção e a energia (próxima seção). NRL=0 é um artefato da definição quando o algoritmo não tem plano de controle; o relatório deixa isso explícito para não induzir leitura errada.

11.4 Energia e ocupação de canal

Flooding ~3% mais energia, mas o sinal forte está em **rx**: 1323 vs 562 ($\approx 2,35\times$). O unicast-para-cada-vizinho faz **todos** os nós receberem cada cópia disseminada. No hub de 3 nós isso é barato; em rede maior/densa, multiplica (cada nó retransmite para cada vizinho, e todos recebem) — é exatamente o regime onde o flooding degrada e o roteamento ganha, como a simulação evidencia ao crescer a grade.

11.5 Síntese

No **C1 reduzido (hub, 1 salto)** o flooding empata ou ganha em PDR e latência e tem NRL de controle nulo, ao custo de ~2,35× mais recepções e leve aumento de energia. A vantagem estrutural do AODV-EN aparece em **escala e diâmetro** (simulação): ele confina o tráfego ao caminho descoberto, enquanto o flooding multiplica cópias e ainda esbarra no TTL=5. Em uma frase: **o roteamento "paga" um overhead de controle constante para evitar o custo de disseminação que cresce com a rede** — vantajoso conforme a rede cresce, irrelevante (ou desvantajoso) numa rede minúscula e densa.

Estatística: as conclusões de hardware vêm de 2 seeds num único cenário (C1-3n). Para rigor (média, desvio, IC 95%) o alvo é 30 repetições por cenário e a cobertura de C2/C3/C4 e escala em simulação — acumuláveis via os alvos perpétuos do autopilot.

12. Decisões de projeto e justificativas (Q1–Q6)

Durante a fase de medição, várias **decisões de projeto** que mudam os resultados não podiam ser "chutadas". Foram registradas em `results/QUESTIONS.md` e decididas pelo autor:

- **Q1 — transporte do flooding.** *Broadcast* único vs **unicast a cada vizinho** (TCC §4.6.1d). **Decisão: unicast-para-vizinho** (fiel ao TCC; reflete o custo real de NRL/energia). Implementado via *fanout* no adaptador (§4.5).
- **Q2 — parâmetros do flooding.** **Decisão: aplicar TCC** — TTL=5, dedup=100, *payload* 32 B, 1 pkt/s.
- **Q3 — métricas.** Extractor não produzia PDR/latência/NRL/energia; latência exigia instrumentação por-pacote. **Decisão: instrumentar o firmware** (RTT na origem, contador de controle) e escrever `tcc_metrics.py` (§6).
- **Q4 — formato de telemetria do flooding.** O `app_flood` emite `stats tx= rx= rebroadcast=...`, que o `extract_monitor_metrics.py` (formato AODV) não parseia. **Resolução:** `tcc_metrics.py` reconhece ambos os formatos.
- **Q5 — paridade AODV-EN com §3.6.** LRU de *peers*, RREQ por unicast e métrica híbrida $\alpha \cdot \text{hop} + \beta \cdot (1/\text{RSSI})$ são alvos de evolução; envolvem escolher pesos e mudar o algoritmo — ficam para decisão/implementação futura (não afetam a comparação atual de transporte/dados).
- **Q6 — constantes de energia.** Sem medição com *shunt*. **Decisão: usar datasheet ESP32-WROOM-32 rotulado** ($V=3,3$ V; $I_{\text{tx}}=240$ mA; $I_{\text{rx}}=100$ mA; $I_{\text{idle}}=20$ mA; $t_{\text{pkt}}=1$ ms).

13. Limitações e ameaças à validade

1. **Topologia de bancada (3 nós, hub, ~1 salto).** O hardware não exercita multi-hop real (2+ saltos exigem separar fisicamente os nós para que N1 e N3 não se ouçam diretamente, só via N2). C1 completo (5 nós/4 saltos), C2, C3 e C4 do TCC migram para simulação até haver mais boards e separação física.
2. **Amostragem (2 seeds, 1 cenário, ~60 s).** Aquém das 30 repetições × 300 s do Quadro 10. Os números de hardware são consistentes entre os 2 seeds, mas média/desvio/IC95 robustos exigem acumular mais runs (alvo perpétuo do autopilot).
3. **Quantização da latência.** O laço de 100 ms da aplicação grosseiriza o RTT medido. Para latência fina, reduzir o laço e/ou marcar *timestamps* mais perto do rádio.
4. **Energia estimada, não medida.** As constantes são de *datasheet*, rotuladas. Medição real (INA219/shunt) daria energia empírica; a estimativa serve para comparação relativa coerente, não para valor absoluto preciso.
5. **NRL=0 do flooding.** Correto pela definição (controle/dados), mas pode induzir leitura equivocada de "sem custo": o custo do flooding está em rx/energia, não em controle.
6. **Paridade incompleta com §3.6 (Q5).** LRU de *peers* e métrica híbrida α/β ainda não implementados; a comparação atual é de transporte/dados, não dos refinamentos de seleção de rota.
7. **Canal e ambiente.** Wi-Fi 2,4 GHz é ruidoso; canal 6 fixo. Interferência externa pode afetar PDR/latência. Os runs foram curtos e no mesmo ambiente para ambos os algoritmos (comparação pareada), o que mitiga, mas não elimina, o viés de ambiente.

14. Trabalho futuro

- **Fechar a estatística do TCC:** 30 repetições por cenário, 300 s, com média/desvio/IC95; automatizável pelos alvos perpétuos do autopilot (cada tick = +1 seed AODV+flood).
- **Cobrir C2/C3/C4 e escala 5–10 nós** em simulação (novos cenários em `sim/`), e multi-hop real ao dispor de mais boards + separação física.
- **Implementar §3.6 (Q5):** LRU de *peers*, RREQ por unicast sequencial, métrica híbrida $\alpha \cdot \text{hop} + \beta \cdot (1/\text{RSSI})$ com varredura de α/β .
- **Energia empírica:** instrumentar com INA219/shunt e comparar com o modelo de *datasheet*.
- **Latência fina:** reduzir o laço da app e marcar *timestamps* mais próximos do envio/recepção do rádio; opcionalmente sincronizar relógios para latência one-way direta (em vez de RTT/2).
- **Cenário de falha (C4):** automatizar o desligamento de N3 após 60 s e medir reconvergência (AODV-EN) vs alcance residual (flooding) em hardware.

15. Reprodutibilidade — passo a passo de comandos

15.1 Pré-requisitos

```
export ESP_IDF_EXPORT=~/.espressif/v6.0/esp-idf/export.sh
source firmware/idf-env.sh
IDFPY=~/.espressif/python_env/idf6.0_py3.14_env/bin/python # tem pyserial/aiohttp/matplotlib
```

15.2 Simulação (sem hardware)

```
bash sim/run_sim.sh basic      # AODV-EN 3 nos: descoberta + ACK retry + late-join
bash sim/run_sim.sh large      # AODV-EN 6 nos: RERR + reconvergencia
bash sim/run_sim.sh flood      # flooding 3 nos: entrega + dedup + sem storm (mostra rtt_ms)
bash sim/run_sim.sh compare    # AODV-EN vs flooding em grade (CSV de tx/entrega)
```

15.3 Build + flash do AODV-EN (3 nós, destino N1=34:99:34)

```
cd firmware
rm -rf build/aodv_cmp
TMP=$(mktemp); echo 'CONFIG_AODV_EN_APP_TARGET_MAC="28:05:A5:34:99:34"' >"$TMP"
export SDKCONFIG_DEFAULTS="$PWD/sdkconfig.defaults;$PWD/tests/flood/aodv.defaults;$TMP"
export SDKCONFIG="$PWD/build/aodv_cmp/sdkconfig"
idf.py -B build/aodv_cmp set-target esp32 && idf.py -B build/aodv_cmp build
# verificar modo (deve imprimir aodv_en_app):
strings build/aodv_cmp/aodv_en_firmware.bin | grep -m1 -E 'aodv_en_app|flood_en_app'
# flash por porta via esptool (paralelo seguro):
cd build/aodv_cmp
for P in 214420 214430 214440; do
    $IDFPY -m esptool --chip esp32 -p /dev/cu.usbserial-$P -b 460800 \
        --before default_reset --after hard_reset write-flash @flash_args & ; done; wait
```

15.4 Build + flash do flooding (params TCC)

```
cd firmware
rm -rf build/flood
zsh tests/flood/build_flash.sh /dev/cu.usbserial-214420 28:05:A5:34:99:34 # build fresco + N1
strings build/flood/aodv_en_firmware.bin | grep -m1 flood_en_app         # verificar modo
cd build/flood
for P in 214430 214440; do
    $IDFPY -m esptool --chip esp32 -p /dev/cu.usbserial-$P -b 460800 \
        --before default_reset --after hard_reset write-flash @flash_args & ; done; wait
```

15.5 Captura de serial (60 s, exclusivo) e métricas

```
# captura: abrir->ler 60s->fechar cada porta (pyserial); salvar em results/m10-<algo>-N{1,2,3}.log
# (script de captura: ver firmware/tools; cada no gera linhas 'LAT seq= rtt_ms=' e stats)
$IDFPY firmware/tools/tcc_metrics.py --algo aodv-en --scenario C1-3n --seed 1 \
    --origin results/m10-aodv-N2.log \
    --node results/m10-aodv-N1.log --node results/m10-aodv-N2.log --node results/m10-aodv-N3.log \
    --duration-s 60 > results/m10-aodv-metrics.json
```

15.6 Ledger e comparação

```
ENGINE=~/.claude/skills/autopilot/scripts/autopilot.py
python3 $ENGINE experiment add --algo aodv-en --param scenario=C1-3n --param seed=1 \
    --metric pdr=98.57 --metric latency_ms=60 --metric nrl=0.7785 --metric energy_j=12.4147
python3 $ENGINE experiment compare aodv-en flooding # medias data-driven + delta + %
```

16. Diário de bordo (cronologia do trabalho)

O trabalho foi conduzido em ciclos disciplinados, boa parte sob um *harness* de automação (autopilot) com regras anti-"maionese": **nunca** marcar tarefa concluída sem evidência em disco; **proibido** inventar/estimar métrica (só de log serial real, via ferramenta); **fix de firmware só commita com testes verdes mostrados**; dúvida de projeto → registrar em QUESTIONS, não chutar; **1 tarefa por iteração**.

16.1 Fase A — alinhamento e baseline (commits `f3e8f28` ... `cdc62ae`)

- `f3e8f28` LED de identificação: NODE_A pisca (1 Hz) e qualquer nó pulsa o LED ao entregar DATA — utilidade de bancada (achar fisicamente a origem entre vários ESPs).
- `5738696` baseline de flooding controlado (núcleo + sim + app + Kconfig).
- `a3f20b7` validação do dashboard em tempo real com 3 ESPs reais (Playwright).
- `65d77f4` , `09d9f0e` , `23d2db0` perfil de bancada do flooding e primeira comparação em hardware no hub; capturas como evidência.
- `cdc62ae` métricas e gráficos AODV-EN vs flooding (sim grade + ponto de hardware).

16.2 Fase B — extração do flooding como componente próprio (`8a2bde3`)

A pedido do autor, o `flood_en` saiu de dentro do `aodv_en` e virou **componente ESP-IDF independente** (wire/tipos/status/config/API próprios, zero dependência do `aodv_en`). Migrados `app_flood` , `flood_en_sim` e `compare_sim` ; `run_sim.sh` passou a montar fontes/includes por variante. Verificado: compila isolado, sims verdes, *build* IDF completo exit 0.

16.3 Fase C — campanha experimental dirigida por missões

Plano de 8 missões (1 por *tick*) + 4 alvos perpétuos, com orçamento de tokens e *gate* de budget. Resultados:

- **m1 (discovery)**: leitura do `TCC.md` (cenários §4.4, métricas §4.5, flooding §4.6, AODV-EN §3.6) + mapeamento de comandos e do formato de telemetria; 6 lacunas de projeto registradas em `results/QUESTIONS.md` (Q1–Q6).
- **m2 (validar AODV-EN)**: `sim basic` / `large` verdes (evidência em `results/`).
- **m3 (front realtime)**: falhou sem hardware; depois **refeito** com os 3 ESPs conectados — 3 nós *online*, 73 eventos *streaming* no WebSocket (`results/m7-telemetry.json`).
- **m4 (flash + coleta AODV)**: serial real + `summary.json` (24 ACKs, rota estável, `send_fail=0`).
- **m5 (build flooding)**: *build* verde (defeito de modo descoberto depois em m9).
- **m6 (validar flooding)**: `sim flood` verde + sanidade (TTL decrementa, dedup, sem *loop*).
- **m9 (flash + coleta flooding)**: descobriu e corrigiu o defeito "build/flood era AODV"; coletou flooding real; revelou o **achado N2 ack=0** (depois virou o *fix* `d3d34eb`).
- **m10 (comparar)**: inicialmente **bloqueada** — as 4 métricas do TCC não saíam do log real sem decisões do autor (Q1–Q6) e sem instrumentação; registrado em `results/m10-BLOCKED.md` , sem inventar números (*ledger* manteve 0 *runs*).

16.4 Fase D — auditoria e correções

- **Auditoria** (alvo perpétuo) partiu do achado N2 ack=0 → identificou a colisão de chave de dedup de ACK → *fix* `d3d34eb` → validado verde (compile + sim + hardware: N2 ack 0→30).
- Regressões mantidas verdes ao longo do caminho.

16.5 Fase E — destravamento do m10 (decisões Q1–Q6) e pipeline de medição

Com as decisões do autor, construiu-se o pipeline em 6 incrementos, todos *commitados* e verdes:

1. `5d3550f` parâmetros TCC no flooding (TTL=5, dedup=100, 32 B, 1 pkt/s).
2. `687ef0d` latência RTT por-pacote no flooding (origem, mesmo relógio).
3. `65cfe0a` transporte por unicast-para-cada-vizinho (Q1).
4. `ed908a9` latência RTT no AODV-EN (simétrico, via `pending_ack.last_sent_at_ms`).
5. `3a1661b` contador `control_tx` (NRL) + `tcc_metrics.py` (PDR/latência/NRL/energia).
6. `d31b94a` parâmetros do AODV-EN alinhados (32 B, 1 pkt/s, HELLO 2 s) para comparação justa.

16.6 Fase F — re-coleta instrumentada e comparação (inc6, via autopilot)

Registrada e dirigida como 3 missões no autopilot:

- **inc6.A** (`m1`): AODV-EN instrumentado → 3 ESPs → 60 s reais (com `LAT / control=`) → `tcc_metrics.py` → `ledger` `e1` (PDR 98,57; lat 60; NRL 0,78; E 12,41 J).
- **inc6.B** (`m2`): flooding instrumentado (unicast/TTL5/32B/1pkt-s) → `ledger` `e2` (PDR 100; lat 50; NRL 0; E 12,80 J). ACKs simétricos confirmam o *fix* sob unicast.
- **inc6.C** (`m3`): `experiment compare` → `comparison.md` + `charts` (commit `6540df5`).
- **+1 rodada (seed 2)**: `e3` (AODV) e `e4` (flooding); `comparison.md` atualizado com média de 2 seeds (commit `989011d`).

16.7 Política de versionamento

Trabalho na *branch* isolada `autopilot/2026-05-31-0026`, *pushada*; **nunca merge** na *default*. Artefatos pesados (logs serial crus, *charts*, *builds*) ficam em `results/` (no `.gitignore`); versiona-se **código**, o *ledger* (`experiments.json`, snapshot em `results/experiments-ledger.json`) e os resumos escritos (`comparison.md`, este relatório). Mensagens de commit em português, estilo *conventional*, sem *trailers* de atribuição.

17. Glossário

- **ACK** — confirmação fim-a-fim de um DATA.
- **AODV** — Ad-hoc On-demand Distance Vector (RFC 3561).
- **AODV-EN** — adaptação do AODV para ESP-NOW.
- **ESP-NOW** — protocolo de enlace proprietário da Espressif (sem IP, sem AP).
- **Flooding controlado** — disseminação por retransmissão com TTL + dedup (`origem,seq`).
- **HELLO** — quadro periódico de presença/vizinhança.
- **IC95** — intervalo de confiança de 95%.
- **NRL** — *Normalized Routing Load* = controle/dados-entregues.
- **PDR** — *Packet Delivery Ratio* = recebidos/enviados.
- **Precursores** — vizinhos que dependem de uma rota; alvos do RERR direcionado (RFC §6.2).
- **RREQ/RREP/RERR** — Route Request/Reply/Error.
- **RTT** — *Round-Trip Time* (DATA→...→destino→ACK→...→origem), medido na origem.
- **TTL** — *Time-To-Live*, limite de saltos.

18. Índice de artefatos

18.1 Código

- Núcleo AODV-EN: `firmware/components/aodv_en/` (`aodv_en_node.c`, `aodv_en_routes.c`, `aodv_en_neighbors.c`, `aodv_en_rreq_cache.c`, `aodv_en_peers.c`, `aodv_en_mac.c`, `aodv_en.c`).
- Baseline flooding: `firmware/components/flood_en/` (`include/flood_en.h`, `src/flood_en.c`).
- Apps: `firmware/main/app_demo.c` (AODV-EN), `firmware/main/app_flood.c` (flooding).
- Simulação: `sim/aodv_en_sim*.c`, `sim/flood_en_sim.c`, `sim/compare_sim.c`, `sim/run_sim.sh`.
- Ferramentas: `firmware/tools/tcc_metrics.py`, `extract_monitor_metrics.py`, `live_monitor.py`, `plot_compare.py`, `plot_comparison_metrics.py`.

- Perfis de bancada: `firmware/tests/flood/{flood.defaults,aadv.defaults,build_flash.sh}`, `firmware/tests/tc00{1,2,5}/`.

18.2 Documentação

- Especificação: `docs/aadv-en-spec-v1.md`; funcionamento: `docs/aadv-en-funcionamento.md`; mapa do código: `docs/aadv-en-mapa-do-codigo.md`; runbook: `docs/runbook-bancada.md`; estruturas: `docs/aadv-en-estruturas-dados.md`; features: `docs/features/{precursosores,enfileamento-dos-dados,articulation-point-planejado}.md`; spec do TCC: `TCC.md`; **este relatório**: `docs/tcc-trabalho-completo.md`.

18.3 Evidências (em `results/`, `gitignored` exceto `comparison.md` /`ledger`)

- m1: `m1-discovery.md`, `QUESTIONS.md`. m2: `m2-aadv-validation.log`.
- m7: `m7-telemetry.json`, `m7-front-realtime.md`. m8: `m8-serial-N{1,2,3}.log`, `m8-aadv-N2/`.
- m9: `m9-serial-N{1,2,3}.log`, `m9-flood-metrics.json`, `m9-flood-collect.md`.
- m10: `m10-{aadv,flood}[-s2]-N{1,2,3}.log`, `m10-*metrics.json`, `comparison.md`, `experiments-ledger.json`, `charts/m10-compare.png`, `charts/sim-tx-per-delivered.png`.
- auditoria: `m-audit-flood-ack-fix.md`, `m-audit-fix-N{1,2,3}.log`.

18.4 Commits-chave (branch `autopilot/2026-05-31-0026`)

`8a2bde3` extrai `flood_en` · `d3d34eb` fix dedup ACK · `5d3550f` params TCC flood · `687ef0d` latência flood · `65cfe0a` unicast flood · `ed908a9` latência AODV · `3a1661b` control_tx + tcc_metrics · `d31b94a` params AODV · `6540df5` m10 (HW real) · `989011d` comparação 2 seeds.

Documento vivo: à medida que mais repetições (seeds) e cenários (C2/C3/C4, escala) forem coletados, as tabelas de §10–§11 e o `comparison.md` são atualizados a partir do ledger — sempre com dados reais, nunca estimados.

Apêndices

Apêndice A — Layout de bytes das mensagens

Todas as mensagens começam pelo cabeçalho comum. Estruturas empacotadas (`__attribute__((packed))`), inteiros em ordem nativa do ESP32 (little-endian).

A.1 Cabeçalho comum (`aadv_en_header_t`)

Offset	Campo	Tipo	Bytes	Descrição
0	<code>protocol_version</code>	uint8	1	<code>AADV_EN_PROTOCOL_VERSION</code> = 1
1	<code>message_type</code>	uint8	1	0=HELLO,1=RREQ,2=RREP,3=RERR,4=DATA,5=ACK
2	<code>flags</code>	uint8	1	bit0 ACK_REQUIRED, bit1 ROUTE_REPAIR
3	<code>hop_count</code>	uint8	1	saltos acumulados neste quadro
4	<code>network_id</code>	uint32	4	isola redes coexistentes (ex. 0xA0DE0001)
8	<code>sender_mac</code>	uint8[6]	6	MAC de quem transmitiu este salto

Total do cabeçalho: 14 bytes.

A.2 HELLO (aodv_en_hello_msg_t)

cabeçalho + node_mac[6] + node_seq_num (uint32) + timestamp_ms (uint32).

A.3 RREQ (aodv_en_rreq_msg_t)

cabeçalho + originator_mac[6] + destination_mac[6] + originator_seq_num (uint32) + destination_seq_num (uint32) + rreq_id (uint32) + ttl (uint8).

A.4 RREP (aodv_en_rrep_msg_t)

cabeçalho + originator_mac[6] + destination_mac[6] + destination_seq_num (uint32) + lifetime_ms (uint32).

A.5 RERR (aodv_en_rerr_msg_t)

cabeçalho + unreachable_destination_mac[6] + unreachable_dest_seq_num (uint32).

A.6 DATA (aodv_en_data_msg_t)

cabeçalho + originator_mac[6] + destination_mac[6] + sequence_number (uint32) + ttl (uint8) + payload_length (uint16) + payload[] (flexível, até AODV_EN_DATA_PAYLOAD_MAX =1024).

A.7 ACK (aodv_en_ack_msg_t)

cabeçalho + originator_mac[6] (quem entrega/confirma) + destination_mac[6] (origem do DATA) + ack_for_sequence (uint32).

A.8 Flooding (flood_en_*)

O flood_en espelha exatamente esse layout (header/DATA/ACK), com os mesmos offsets e semântica, sob símbolos próprios (flood_en_header_t , flood_en_data_msg_t , flood_en_ack_msg_t). Tipos usados: DATA=4, ACK=5. Isso mantém compatibilidade de fio sem acoplar os componentes.

Apêndice B — Limites e parâmetros (compile-time, `aodv_en_limits.h`)

Macro	Valor padrão	Significado
<code>AODV_EN_PROTOCOL_VERSION</code>	1	versão do protocolo no fio
<code>AODV_EN_MAC_ADDR_LEN</code>	6	bytes de MAC
<code>AODV_EN_NEIGHBOR_TABLE_SIZE</code>	16	vizinhos rastreados
<code>AODV_EN_ROUTE_TABLE_SIZE</code>	32	rotas
<code>AODV_EN_RREQ_CACHE_SIZE</code>	64	entradas do cache de RREQ (dedup)
<code>AODV_EN_PEER_CACHE_SIZE</code>	8	cache de peers
<code>AODV_EN_PENDING_DATA_QUEUE_SIZE</code>	4	fila de DATA pendente
<code>AODV_EN_MAX_PRECURSORS</code>	4	precursores por rota
<code>AODV_EN_CONTROL_PAYLOAD_MAX</code>	128	payload de controle
<code>AODV_EN_DATA_PAYLOAD_MAX</code>	1024	payload de DATA
<code>AODV_EN_MAX_HOPS_DEFAULT</code>	16	saltos máximos (rota)
<code>AODV_EN_TTL_DEFAULT</code>	16	TTL padrão do AODV-EN
<code>AODV_EN_NEIGHBOR_TIMEOUT_MS</code>	15000	expiração de vizinho
<code>AODV_EN_ROUTE_LIFETIME_MS</code>	30000	vida da rota
<code>AODV_EN_RREQ_CACHE_TIMEOUT_MS</code>	10000	expiração de entrada do cache RREQ
<code>AODV_EN_ACK_TIMEOUT_MS</code>	1000	timeout p/ retransmitir aguardando ACK
<code>AODV_EN_RREQ_RETRY_COUNT</code>	3	tentativas de descoberta
<code>AODV_EN_LINK_FAIL_THRESHOLD</code>	3	falhas de enlace p/ invalidar rota
<code>AODV_EN_ROUTE_METRIC_INFINITY</code>	0xFFFF	métrica infinita
<code>AODV_EN_RTT_UNKNOWN</code>	0xFFFFFFFF	RTT desconhecido (instrumentação)

Flooding (`flood_en.h`): `FLOOD_EN_TTL_DEFAULT=5` , `FLOOD_EN_MAX_HOPS_DEFAULT=5` , `FLOOD_EN_SEEN_SIZE=100` , `FLOOD_EN_TX_TRACK=32` , `FLOOD_EN_RTT_UNKNOWN=0xFFFFFFFF` .

Apêndice C — Traces passo a passo

C.1 Descoberta de rota A→C numa cadeia A–B–C (RREQ/RREP)

```
A quer enviar DATA p/ C, sem rota:
[A] send_data(C) -> sem rota -> enfileira DATA, dispara RREQ
[A] TX RREQ (orig=A, rreq_id=1, dst=C, hop=0, ttl=16) (broadcast/fanout)
    -> B RX RREQ: cache (A,1) novo -> instala rota reversa C? nao; rota p/ A via A;
        hop++=1; ttl--; redisemina
[B] TX RREQ (hop=1)
    -> C RX RREQ: e o destino -> instala rota reversa p/ A via B
[C] TX RREP unicast -> B (dst_seq=C.seq, hop=0)
    -> B RX RREP: instala rota p/ C via C; precursor A; hop++
[B] TX RREP unicast -> A
    -> A RX RREP: instala rota p/ C via B; DRENA fila pendente
[A] TX DATA unicast -> B (dst=C, ttl=16, ACK_REQUIRED)
[B] forward DATA -> C
[C] DELIVER "..."; TX ACK -> B
[B] forward ACK -> A
[A] ACK received (seq, rtt) -> consome pending_ack -> latencia registrada
```

C.2 Entrega + ACK fim-a-fim (com rota já válida)

```
[A] send_data(C) -> rota valida -> TX DATA unicast -> B -> C
[C] DELIVER + TX ACK -> ... -> A
[A] ack_received(seq, rtt=now-t_send) -> log "LAT seq=.. rtt_ms=.."
```

C.3 Flooding A→C por unicast-para-vizinho (3 nós no hub)

```
[A] send_data(C, ack) -> seq=k -> registra seen(A,k), tx_time(k) -> emit(broadcast)
    adaptador: unicast p/ cada vizinho conhecido (B e C)
    -> B RX DATA(A,k): nao-dup; dst!=B; ttl>1 -> re-floda (unicast a vizinhos)
    -> C RX DATA(A,k): dst==C -> DELIVER; floda ACK (dst=A)
[C] ACK(orig=C, dst=A, seq=k) -> emit a vizinhos
    -> A RX ACK: dst==A -> ack_received(rtt); seen_ack(A,k)
    -> B RX ACK: encaminha (hop<max) ...
dedup por (origem,seq) corta loops; TTL=5 limita alcance.
```

C.4 O bug do ACK (antes/depois do fix `d3d34eb`)

```
ANTES (chave de dedup do ACK = (deliverer, seq)):
N1 entrega DATA de N2 (seq=5) e de N3 (seq=5) -> floda 2 ACKs, ambos chave (N1,5)
-> 2o ACK descartado como duplicado -> N2 fica sem ACK (ack=0), N3 ok (ack=32)
DEPOIS (chave = (origem-do-DATA, seq) = destination_mac do ACK):
ACK p/ N2 = (N2,5); ACK p/ N3 = (N3,5) -> chaves distintas -> ambos passam
-> N2 ack=30, N3 ack=30 (simetrico)
```

Apêndice D — Dados brutos por execução (hardware)

Valores exatos extraídos por `tcc_metrics.py` de cada conjunto de logs serial. Latência em RTT (e one-way = RTT/2). `sum_*` agregam os 3 nós.

D.1 AODV-EN, seed 1 (e1)

```
data_sent=70 acks=69 PDR=98.57%
RTT: n=69 media=120.0 ms std=0.0 ic95=0.0 -> one-way 60.0 ms
sum_tx=441 sum_rx=562 control_tx=123 delivered=158 NRL=0.7785
energia=12.4147 J (estimativa datasheet)
```

D.2 AODV-EN, seed 2 (e3)

```
data_sent=69 acks=69 PDR=100.0%
RTT: n=69 media=120.0 ms std=0.0 ic95=0.0 -> one-way 60.0 ms
sum_tx=436 sum_rx=551 control_tx=122 delivered=156 NRL=0.7821
energia=12.4071 J
```

D.3 Flooding, seed 1 (e2)

```
data_sent=70 acks=70 PDR=100.0%
RTT: n=70 media=100.0 ms std=0.0 ic95=0.0 -> one-way 50.0 ms
sum_tx=614 sum_rx=1323 control_tx=0 delivered=154 NRL=0.0
energia=12.8029 J
```

D.4 Flooding, seed 2 (e4)

```
data_sent=70 acks=70 PDR=100.0%
RTT: n=70 media=101.2 ms std=0.403 ic95=0.094 -> one-way 50.6 ms
sum_tx=667 sum_rx=1333 control_tx=0 delivered=168 NRL=0.0
energia=12.8482 J
```

D.5 Observações sobre os brutos

- **rx do flooding** $\approx 2,3\text{--}2,4\times$ o do AODV-EN (1323/1333 vs 562/551): a marca do unicast-para-cada-vizinho — todo nó recebe cada cópia.
- **control_tx do flooding** = 0 sempre (sem plano de controle).
- **RTT do AODV-EN** veio exatamente 120 ms (std 0) nos dois seeds: forte quantização pelo laço de 100 ms; o flooding variou levemente (101,2 ms, std 0,4) no seed 2.
- **PDR do AODV-EN** oscilou 98,57%→100% entre seeds (perda do 1.º pacote em um run).

Apêndice E — QUESTIONS (decisões de projeto levadas ao autor)

Registradas em `results/QUESTIONS.md` na fase de *discovery* e resolvidas pelo autor antes da medição final:

- **Q1 — transporte do flooding:** *broadcast* vs unicast-por-vizinho. **Resolvido:** unicast (TCC §4.6.1d). Implementado por *fanout* no adaptador (commit `65cfe0a`).
- **Q2 — parâmetros do flooding:** TTL/dedup/payload/taxa. **Resolvido:** TCC (TTL=5, N=100, 32 B, 1 pkt/s) (commit `5d3550f`).
- **Q3 — métricas:** PDR/latência/NRL/energia não saíam do log. **Resolvido:** instrumentar (RTT na origem; contador de controle; `tcc_metrics.py`) (commits `687ef0d`, `ed908a9`, `3a1661b`).
- **Q4 — telemetria do flooding:** formato `stats ...` distinto do extractor AODV. **Resolvido:** `tcc_metrics.py` lê ambos.
- **Q5 — paridade com §3.6:** LRU de peers, RREQ-unicast, métrica híbrida α/β — **pendente** (evolução; não afeta a comparação atual de transporte/dados).

- **Q6 — energia:** sem medição. **Resolvido: datasheet ESP32-WROOM-32 rotulado** (V=3,3; I_{tx}=240 mA; I_{rx}=100 mA; I_{idle}=20 mA; t_{pkt}=1 ms).

Apêndice F — Walkthrough do núcleo (mapa de funções)

Pontos-chave para quem for ler/estender o código:

- **Montar/enviar DATA:** `aadv_en_node_send_data_with_sequence` (preenche header via `aadv_en_fill_header`, define `tll`, `sequence_number`, copia payload; emite unicast pela rota) — `firmware/components/aadv_en/src/aadv_en_node.c`.
- **Recepção/dispatch:** `aadv_en_node_on_recv` valida versão/ `network_id` e tamanho, e faz `switch` por `message_type`; DATA → `aadv_en_node_handle_data` (entrega/encaminha/ACK); ACK → `aadv_en_node_handle_ack`.
- **Dedup de RREQ:** `aadv_en_rreq_cache_contains` / `_remember` (par (`originador`, `rreq_id`)).
- **Emissão + contagem de controle:** `aadv_en_node_emit` (incrementa `tx_frames`; classifica HELLO/RREQ/RREP/RERR em `control_tx_frames`).
- **Pendente de ACK + latência:** `aadv_en_node_pending_ack_consume` (casa (`dest`, `seq`), devolve `rtt = now - last_sent_at_ms`).
- **Flooding:** `flood_en_node_send_data` / `flood_en_node_on_recv` / `flood_en_handle_data` / `flood_en_handle_ack` (dedup (`origem`, `seq`) p/ DATA e (`origem-do-DATA`, `ack_seq`) p/ ACK); RTT via `flood_en_tx_time_remember` / `_take` — `firmware/components/flood_en/src/flood_en.c`.
- **Adaptadores:** `app_emit_frame` (esp_{now} send; no flooding faz *fanout* unicast aos vizinhos), `app_note_neighbor` (aprende vizinho pelo `src` do RX) — `firmware/main/app_{demo,flood}.c`.
- **Rádio de simulação:** `sim_emit_frame` / `radio_broadcast_or_unicast` entregam quadros aos nós alcançáveis (matriz `links[][]`), chamando `*_on_recv` nos pares — `sim/*.c`.

Apêndice G — Saída-exemplo do `experiment compare` (2 seeds)

```
{
  "a": "aadv-en", "b": "flooding", "runs_a": 2, "runs_b": 2,
  "metrics": {
    "pdr": { "aadv-en": 99.285, "flooding": 100.0, "delta_b_minus_a": 0.715, "pct_change": 0.72},
    "latency_ms": { "aadv-en": 60.0, "flooding": 50.3, "delta_b_minus_a": -9.7, "pct_change": -16.17},
    "nrl": { "aadv-en": 0.7803, "flooding": 0.0, "delta_b_minus_a": -0.7803, "pct_change": -100.0},
    "energy_j": { "aadv-en": 12.4109, "flooding": 12.8255, "delta_b_minus_a": 0.4146, "pct_change": 3.34}
  }
}
```

Apêndice H — Procedimento de captura e cálculo (detalhe)

1. Gravar os 3 nós com o firmware-alvo (ver §15.3/§15.4), verificando o modo no `.bin` e no *boot* serial.
2. Para cada porta, abrir `pyserial` a 115200, aguardar ~10 s (boot+estabilização), ler ~60 s, fechar. Salvar `results/m10-<algo>[-sN]-N{1,2,3}.log`. Cada origem emite `LAT seq= rtt_ms=` por ACK e linhas de `stats` / `routes=...` `control=`.
3. Rodar `tcc_metrics.py --algo <algo> --origin <N2.log> --node <N1> --node <N2> --node <N3> --duration-s 60` → JSON com as 4 métricas (PDR da origem; latência das linhas LAT; NRL=control/delivered da rede; energia do modelo).
4. `experiment add --algo <algo> --param seed=N --metric pdr=... latency_ms=... nrl=... energy_j=...` (valores copiados do JSON, não digitados de cabeça).

5. Ao acumular seeds, `experiment compare aodv-en flooding` e regenerar `comparison.md` + *charts* a partir do ledger.

Apêndice I — Trace real da simulação AODV-EN (`run_sim.sh basic`)

Saída literal do cenário de 3 nós A-B-C, com as quatro fases (descoberta, dados, *retry* de ACK, *late-join*). Note o RTT instrumentado (`rtt_ms`) e o caso em que o ACK é descartado de propósito, forçando *retry* (o RTT salta para ~1007 ms — o `ack_timeout_ms`).

```
=== route discovery phase ===
[t=0] A TX RREQ broadcast
      -> B RX RREQ
[t=1] B TX RREQ broadcast
      -> A RX RREQ
      -> C RX RREQ
[t=3] C TX RREP unicast -> B
[t=4] B TX RREP unicast -> A
[t=5] A TX DATA unicast -> B
[t=6] B TX DATA unicast -> C
      C DELIVER data from 10:00:00:00:00:0A: hello over aodv-en
[t=7] C TX ACK unicast -> B
[t=8] B TX ACK unicast -> A
      A ACK received from 10:00:00:00:00:0C for seq=1 rtt_ms=4

=== data phase ===
[t=9] A TX DATA unicast -> B
[t=10] B TX DATA unicast -> C
      C DELIVER data from 10:00:00:00:00:0A: hello over aodv-en
[t=11] C TX ACK unicast -> B
[t=12] B TX ACK unicast -> A
      A ACK received from 10:00:00:00:00:0C for seq=2 rtt_ms=4

=== ack retry phase ===
[t=13] A TX DATA unicast -> B
[t=14] B TX DATA unicast -> C
      C DELIVER data from 10:00:00:00:00:0A: hello over aodv-en
[t=15] C TX ACK unicast -> B
[t=16] B TX ACK unicast -> A
      -> ACK intentionally dropped before A
[t=1016] A TX DATA unicast -> B
[t=1017] B TX DATA unicast -> C
      C DELIVER data from 10:00:00:00:00:0A: hello over aodv-en
[t=1018] C TX ACK unicast -> B
[t=1019] B TX ACK unicast -> A
      A ACK received from 10:00:00:00:00:0C for seq=3 rtt_ms=1007
...
=== summary === (Simulation passed.)
```

Leitura: a descoberta acontece sincronamente (RREQ→RREP em poucos "ticks"), o DATA segue unicast salto a salto (A→B→C), o destino entrega e devolve ACK, e a origem confirma com RTT. O *retry* mostra a robustez do `pending_ack` (re-transmite após `ack_timeout_ms`).

Apêndice J — Trace real da simulação do flooding (`run_sim.sh flood`)

```
=== flood delivery phase (A -> C via B, no routes) ===
[t=0] A TX DATA broadcast (ttl-hop=0)
      -> B RX DATA
[t=1] B TX DATA broadcast (ttl-hop=1)
      -> A RX DATA      (A descarta: ja viu (A,seq))
      -> C RX DATA
      C DELIVER data from 10:00:00:00:00:0A: hello over flood-en
[t=3] C TX ACK broadcast (ttl-hop=0)
      -> B RX ACK
[t=4] B TX ACK broadcast (ttl-hop=1)
      -> A RX ACK
      A ACK received from 10:00:00:00:00:0C for seq=1 rtt_ms=5
      -> C RX ACK      (C descarta: ja viu o ACK)
=== second flood (seq increments, still delivers) ===
... (entrega 2/2, ack 2/2, dedup ativo, ttl_drop=0) -> "Flood simulation passed."
```

Leitura: sem rotas, o DATA é disseminado; o `hop_count` (ttl-hop) sobe a cada salto; a supressão de duplicatas (`origem,seq`) impede *loops* (A e C descartam os ecos); o destino entrega e o ACK é disseminado de volta. No simulador o "broadcast" alcança todos os vizinhos de uma vez; no hardware, o adaptador faz o *fanout* por unicast (1 envio por vizinho).

Apêndice K — Fundamentos do AODV (RFC 3561) preservados

O AODV-EN mantém os mecanismos centrais do AODV:

1. **Sob-demanda:** rota só é buscada quando há tráfego (reativo), economizando controle.
2. **Números de sequência de destino:** garantem rotas frescas e **livres de loop**; uma rota só é aceita/atualizada se trouxer `dest_seq` maior (ou igual com menos saltos).
3. **RREQ/RREP:** descoberta por disseminação de RREQ e resposta unicast de RREP pelo caminho reverso; rotas reversa (na ida) e direta (na volta) são instaladas em cada nó do caminho.
4. **RREQ ID + cache:** cada RREQ tem (`originador, rreq_id`) único; nós já processados descartam duplicatas (evita tempestade de broadcast).
5. **Manutenção:** *lifetime* de rota, HELLO para vizinhança, e **RERR** aos **precursores** ao detectar enlace quebrado.
6. **TTL/diâmetro:** limita o alcance da disseminação de RREQ.

Adaptações para ESP-NOW (o "EN"): transporte por quadros ESP-NOW (sem IP), `network_id` para isolar redes, confirmação de enlace do ESP-NOW para detectar falha, e as considerações §3.6 (LRU de peers, possibilidade de RREQ por unicast, métrica híbrida).

Apêndice L — ESP-NOW v2: pontos relevantes ao projeto

- **Sem associação!!IP:** troca quadros entre MACs diretamente; ideal para nós simples.
- **Peers:** é preciso registrar o *peer* (`esp_now_add_peer`) antes de enviar unicast; o projeto registra sob demanda (`app_ensure_peer`) e usa o *broadcast* address para o modo *broadcast*.
- **Confirmação de enlace:** o *callback* de envio informa sucesso/falha do unicast — usado para detectar falha de enlace e alimentar a invalidação de rota do AODV-EN.
- **Tamanho:** o projeto respeita `ESP_NOW_MAX_DATA_LEN_V2`; *payload* de DATA do experimento = 32 B, bem abaixo do limite.
- **Canal:** todos os nós no mesmo canal (6) e mesmo `network_id` (0xA0DE0001) para se enxergarem.

Apêndice M — Dashboard em tempo real (`live_monitor.py`)

Arquitetura: um processo Python (asyncio + aiohttp) abre uma *thread* leitora por porta serial, faz *parse* das linhas de log (regex por tipo de evento), mantém o estado da malha em memória e empurra atualizações por **WebSocket** para uma página com **Cytoscape.js**, que desenha a topologia animada (nós *online/offline*, arestas de vizinhança, rotas válidas/ reversas/inválidas) e uma *timeline* de eventos. Recursos: pré-leitura de MAC via `esptool` no *startup* (mapeia *alias*→MAC mesmo sem capturar a linha de *boot*), modo `--demo` (eventos sintéticos sem hardware), `--verbose / -vv`. **Gotcha** documentado: com `--skip-mac-lookup` e ESPs já bootados (sem linha `node=` recente), o *alias*→MAC não mapeia e o dashboard fica vazio mesmo com a serial produzindo logs — rodar sem o *flag* (pré-leitura via `esptool`) resolve.

Apêndice N — Linha do tempo dos commits (branch `autopilot/2026-05-31-0026`)

Commit	Tipo	Resumo
<code>f3e8f28</code>	feat	LED de origem/entrega (app_demo)
<code>a3f20b7</code>	test	valida dashboard realtime (3 ESPs, Playwright)
<code>5738696</code>	feat	baseline de flooding controlado
<code>65d77f4</code>	test	profile de bancada do flooding + validação sim/HW
<code>09d9f0e</code> / <code>23d2db0</code>	test	comparação AODV vs flood no hub + capturas
<code>cdc62ae</code>	feat	métricas e gráficos AODV vs flood (sim + HW)
<code>8a2bde3</code>	refactor	extraí <code>flood_en</code> para componente independente
<code>8114b2a</code>	chore	ignora <code>results/</code> (artefatos pesados)
<code>99e72e3</code>	docs	adiciona <code>TCC.md</code>
<code>d3d34eb</code>	fix	dedup de ACK por (origem-do-DATA, seq) (N2 ack 0→30)
<code>9ff65dd</code>	docs	comparison.md (sim real + HW bloqueado)
<code>5d3550f</code>	feat	params TCC no flood (TTL=5, dedup=100, 32B, 1pkt/s)
<code>687ef0d</code>	feat	latência RTT no flood (origem)
<code>65cfe0a</code>	feat	unicast-para-cada-vizinho (Q1)
<code>ed908a9</code>	feat	latência RTT no AODV (simétrico)
<code>3a1661b</code>	feat	contador NRL <code>control_tx</code> + <code>tcc_metrics.py</code>
<code>d31b94a</code>	test	params AODV alinhados (32B/1pkt-s/HELLO 2s)
<code>6540df5</code>	docs	m10 com dados HW reais (inc6 completo)
<code>989011d</code>	docs	comparison.md com média de 2 seeds
<code>9fd322f</code>	docs	este relatório completo

Apêndice O — Como este relatório se mantém honesto

Checklist aplicado a cada número e afirmação:

- **PDR/latência/NRL/energia de hardware** vêm de `results/m10-*-metrics.json`, gerados por `tcc_metrics.py` sobre logs serial reais (`results/m10-*-N{1,2,3}.log`), registrados no *ledger* `experiments.json` e comparados por `experiment compare` — nunca digitados de cabeça.
- **Energia** é explicitamente estimativa de *datasheet* (rotulada em todo lugar).

- **Decisões de projeto** (Q1–Q6) foram do autor; o relatório as cita como decisões, não como fato técnico inevitável.
 - **Defeitos** têm sintoma, causa-raiz e validação verde antes/depois (com arquivos de evidência citados).
 - **Limitações** (amostragem, hub, quantização, energia estimada, NRL=0) estão em §13, não escondidas no rodapé.
-

Fim dos apêndices e do relatório.

Especificacao do TCC (cenarios, metricas, algoritmos)

DIOGO DOS REIS ALMEIDA

HUAKSON LIMA

Proposta de Algoritmo de Roteamento em Redes Mesh de Baixo Custo com ESP-NOW

INHUMAS - GO

2026

1. RESUMO

2. INTRODUÇÃO

No início da década de 1990, Mark Weiser, pesquisador do Xerox Palo Alto Research Center (PARC), introduziu em seu artigo seminal *The Computer for the 21st Century* o conceito de Computação Ubíqua, ao afirmar que as tecnologias mais profundas são aquelas que desaparecem, integrando-se de forma invisível ao cotidiano humano (WEISER, 1991). Essa visão pioneira antecipou um ecossistema composto por centenas de dispositivos computacionais de baixo custo, baixo consumo energético e amplamente interconectados por redes sem fio, operando em diferentes escalas espaciais e comunicando-se de forma transparente entre si e com o ambiente.

Mais de três décadas depois, tal paradigma materializa-se por meio da Internet das Coisas (*Internet of Things* – IoT) e das Redes de Sensores Sem Fio (*Wireless Sensor Networks* – WSNs), tecnologias que conectam bilhões de dispositivos embarcados mundialmente em aplicações que vão desde automação residencial e agricultura de precisão até monitoramento ambiental e sistemas industriais inteligentes (PRIYADARSHI *et al.*, 2025). Nesse contexto, o microcontrolador ESP32, desenvolvido pela Espressif Systems, destaca-se como uma plataforma versátil e amplamente adotada, combinando conectividade Wi-Fi e Bluetooth integradas, elevado poder de processamento e custo acessível. Características que o tornam ideal para prototipagem e implantação de soluções IoT em escala.

A comunicação em redes IoT frequentemente enfrenta desafios relacionados à cobertura, escalabilidade e resiliência. Topologias centralizadas, como a estrela (A*), apresentam limitações significativas: a dependência de um ponto de acesso único constitui um gargalo de desempenho e um ponto único de falha, comprometendo a robustez do sistema (CHAI; ZENG, 2021). Como alternativa, às Redes Mesh Sem Fio (*Wireless Mesh Networks* – WMNs) emergem como uma solução promissora, oferecendo características de auto-organização (*self-configuration*), auto-recuperação (*self-healing*) e comunicação ponto-a-ponto entre os nós participantes.

Nas redes mesh, a comunicação em múltiplos saltos (*multi-hop*) permite que pacotes de dados sejam encaminhados através de nós intermediários até alcançarem seu destino final, estendendo significativamente o alcance da rede sem a necessidade de aumentar a potência de transmissão dos dispositivos individuais (ARREGUI ALMEIDA *et al.*, 2025). Essa abordagem possibilita a implantação de redes em áreas extensas utilizando dispositivos de baixa potência, reduzindo custos e consumo energético. Entretanto, a comunicação multi-hop introduz desafios adicionais relacionados ao roteamento de pacotes, gerenciamento de interferência e manutenção da topologia da rede, exigindo algoritmos especializados para garantir a entrega confiável dos dados.

Os protocolos de roteamento para redes ad-hoc podem ser classificados em três categorias principais: proativos, reativos e híbridos. Os protocolos proativos, como o *Optimized Link State Routing* (OLSR), mantêm tabelas de roteamento constantemente atualizadas, oferecendo baixa latência na descoberta de rotas às custas de maior overhead de controle. Os protocolos reativos, como o *Ad-hoc On-Demand Distance Vector* (AODV), descobrem rotas apenas quando necessário, reduzindo o tráfego de controle em redes com tráfego esporádico. Já os protocolos híbridos combinam características de ambas as abordagens, adaptando-se às diferentes condições da rede (CHAI; ZENG, 2021).

Entre os mecanismos de comunicação disponíveis no ecossistema ESP32, o protocolo ESP-NOW destaca-se como

uma alternativa de alto desempenho para comunicação direta entre dispositivos. Desenvolvido pela Espressif Systems, operando de forma *connectionless* sobre a camada de enlace do padrão IEEE 802.11, permitindo a troca de mensagens curtas (até 250 bytes de payload) sem a necessidade de estabelecimento prévio de conexão ou associação a um ponto de acesso Wi-Fi (BECKER *et al.*, 2025).

Estudos experimentais recentes demonstram as vantagens do ESP-NOW em relação a outras tecnologias de comunicação sem fio. Becker *et al.* (2025), em experimentos de campo conduzidos em ambientes abertos e florestais, reportaram latências médias de 2,8 ms e *Packet Delivery Ratio* (PDR) superior a 99% em distâncias de até 55 metros em condições de linha de visada. Urazayev *et al.* (2023), avaliando o protocolo em ambiente interno, constataram que o ESP-NOW apresenta alcance 15% superior e consumo energético 30% inferior quando comparado a soluções baseadas em Wi-Fi TCP convencional. Essas características tornam o ESP-NOW especialmente atrativo para aplicações IoT em ambientes remotos ou alimentados por baterias de capacidade limitada.

Apesar de suas vantagens, o protocolo apresenta limitações estruturais que restringem sua aplicabilidade em cenários que demandam redes de maior escala ou alcance estendido. A primeira limitação refere-se à ausência de suporte nativo a roteamento multi-hop: o ESP-NOW foi concebido exclusivamente para comunicações ponto-a-ponto ou em topologia estrela, não oferecendo mecanismos intrínsecos para encaminhamento de pacotes através de nós intermediários (BECKER *et al.*, 2025). A segunda limitação diz respeito ao número máximo de *peers* simultâneos suportados: cada dispositivo ESP32 pode manter no máximo 20 *peers* registrados em sua tabela de pares, o que impõe restrições severas à escalabilidade da rede. A terceira limitação relaciona-se à ausência de mecanismo de broadcast nativo eficiente, dificultando a disseminação de mensagens de controle necessárias para protocolos de descoberta de rotas.

Essa lacuna torna-se evidente ao se analisarem soluções de redes em malha amplamente utilizadas no ecossistema Espressif, como o ESP-WIFI-MESH e a biblioteca PainlessMesh, que operam sobre a pilha Wi-Fi convencional (IEEE 802.11). Conforme discutido por Šesták (2022), o uso do Wi-Fi padrão para manutenção da topologia mesh impõe elevado *overhead* de processamento e consumo energético, inviabilizando sua aplicação em dispositivos estritamente alimentados por baterias e comprometendo a principal vantagem competitiva do ESP-NOW. Trabalhos recentes têm buscado preencher essa lacuna: Cujilema *et al.* (2023) propuseram o algoritmo BRAM-NOW para criação de redes mesh seguras sobre ESP-NOW, demonstrando latência média de 75 ms e taxa de perda de pacotes de 9,25% em ambiente residencial. Entretanto, tal abordagem não se fundamenta em protocolos de roteamento padronizados, o que dificulta sua comparação com soluções estabelecidas na literatura e limita sua extensibilidade.

Diante dessa lacuna, o presente trabalho propõe o desenvolvimento e a avaliação de um algoritmo de roteamento denominado AODV-EN (*Ad-hoc On-Demand Distance Vector for ESP-NOW*), uma adaptação do protocolo AODV, especificado na RFC 3561 (PERKINS; BELDING-ROYER; DAS, 2003). Dessa forma, o ESP-NOW possui restrições específicas de hardware e comunicação. A escolha do AODV como protocolo base justifica-se por suas características de roteamento reativo sob demanda, que minimizam o tráfego de controle em redes com comunicação intermitente, e por sua ampla documentação e validação na literatura acadêmica, facilitando comparações e extensões futuras.

As principais adaptações propostas para adequar o AODV às restrições do ESP-NOW incluem: (i) gerenciamento dinâmico de *peers* utilizando política LRU (*Least Recently Used*), permitindo que o algoritmo opere dentro do limite de 20 *peers* simultâneos através da substituição inteligente de entradas menos utilizadas; (ii) substituição do mecanismo de *broadcast* por *flooding* controlado via unicast sequencial, adaptando a disseminação de mensagens RREQ (*Route Request*) às características do ESP-NOW; e (iii) incorporação de métrica híbrida de roteamento que combina contagem de saltos (*hop count*) com indicador de qualidade do sinal recebido (RSSI), possibilitando a seleção de rotas que equilibrem menor distância lógica e melhor qualidade de enlace.

2.1 OBJETIVOS

2.1.1 OBJETIVO GERAL

Propor, implementar e avaliar um algoritmo de roteamento multi-hop baseado no protocolo AODV, adaptado para operação em redes mesh construídas sobre o protocolo ESP-NOW, considerando as restrições específicas dessa tecnologia e utilizando hardware de baixo custo.

2.1.2 OBJETIVOS ESPECÍFICOS Para alcançar o objetivo geral, foram definidos os seguintes objetivos específicos:

- a) Analisar as limitações técnicas do protocolo ESP-NOW para comunicação multi-hop, identificando os desafios específicos relacionados ao limite de *peers*, ausência de *broadcast* nativo e gerenciamento de rotas;
- b) Projetar as adaptações necessárias ao protocolo AODV para adequá-lo às restrições do ESP-NOW, incluindo gerenciamento de *peers* com política LRU, *flooding* controlado e métrica híbrida de roteamento;
- c) Implementar um protótipo funcional do algoritmo AODV-EN em módulos ESP32, desenvolvendo as estruturas de dados, mensagens de controle e mecanismos de descoberta e manutenção de rotas;
- d) Avaliar o desempenho do algoritmo proposto através de experimentos em diferentes topologias de rede, utilizando métricas quantitativas como *Packet Delivery Ratio* (PDR), latência fim-a-fim, *Normalized Routing Load* (NRL) e consumo energético;
- e) Comparar os resultados obtidos com trabalhos correlatos da literatura, posicionando a contribuição do AODV-EN em relação às soluções existentes para redes mesh sobre ESP-NOW e outras tecnologias.

3. REFERENCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos que embasam o desenvolvimento da pesquisa. Inicialmente, são discutidos os conceitos de Internet das Coisas e de Redes de Sensores Sem Fio, situando o contexto tecnológico do estudo. Em seguida, são apresentadas as características do microcontrolador ESP32 e do protocolo ESP-NOW, utilizados na implementação do artefato. Posteriormente, abordam-se os conceitos de redes mesh e a classificação dos protocolos de roteamento ad-hoc. Por fim, é analisado o protocolo AODV, que fundamenta a adaptação proposta e o desenvolvimento do protocolo AODV-EN.

3.1 Internet das Coisas e Redes de Sensores Sem Fio

A Internet das Coisas (*Internet of Things* – IoT) representa um paradigma tecnológico no qual objetos físicos dotados de capacidade computacional, sensores e conectividade de rede integram-se ao ambiente de forma ubíqua, coletando e compartilhando dados sem intervenção humana direta (PRIYADARSHI *et al.*, 2025). Essa visão, antecipada por Mark Weiser em 1991 sob o conceito de Computação Ubíqua, materializa-se atualmente em bilhões de dispositivos conectados mundialmente, permeando setores como saúde, agricultura, indústria, cidades inteligentes e automação residencial.

As Redes de Sensores Sem Fio (*Wireless Sensor Networks* – WSNs) constituem uma das principais infraestruturas para implementação de aplicações IoT. Uma WSN é composta por um conjunto de nós sensores distribuídos espacialmente, capazes de monitorar fenômenos físicos ou ambientais, como temperatura, umidade, pressão, movimento ou luminosidade e transmitir os dados coletados através de enlaces sem fio até uma estação base ou gateway (CHAI; ZENG, 2021). Os nós sensores são tipicamente dispositivos de recursos limitados, com restrições de processamento, memória, energia e largura de banda de comunicação.

O projeto de WSNs deve considerar um conjunto de requisitos críticos que influenciam diretamente a viabilidade e eficácia das aplicações. O Quadro 1 sintetiza os principais requisitos e suas implicações para o projeto de redes de sensores.

Quadro 1 – Requisitos críticos para projeto de Redes de Sensores Sem Fio

Requisito	Descrição e Implicações
Baixo custo	Implantações em larga escala requerem dispositivos economicamente acessíveis. O custo por nó deve permitir a instalação de dezenas a centenas de dispositivos sem inviabilizar o projeto.
Baixo consumo energético	Muitas aplicações exigem operação por meses ou anos com baterias de capacidade limitada. O consumo deve ser minimizado em todos os subsistemas: processamento, comunicação e sensoriamento.
Confiabilidade	A rede deve garantir a entrega dos dados coletados mesmo em condições adversas, como interferência eletromagnética, obstáculos físicos ou falha de nós individuais.
Escalabilidade	A arquitetura deve suportar o crescimento do número de nós sem degradação significativa de desempenho ou necessidade de reconfiguração manual.
Auto-organização	Os nós devem ser capazes de estabelecer e manter a topologia da rede de forma autônoma, adaptando-se a mudanças no ambiente ou na composição da rede.

Fonte: Adaptado de Priyadarshi et al. (2025) e Chai e Zeng (2021).

A comunicação sem fio é o componente de maior consumo energético em um nó sensor típico, podendo representar até 70% do consumo total do dispositivo (PRIYADARSHI *et al.*, 2025). Dessa forma, a escolha do protocolo de comunicação e a estratégia de roteamento adotada impactam diretamente a vida útil da rede. Protocolos que minimizam o número de transmissões, utilizam modos de baixo consumo durante períodos de inatividade e otimizam o tamanho das mensagens de controle contribuem significativamente para a extensão da autonomia dos dispositivos.

3.2 O Microcontrolador ESP32

O ESP32 é um *System-on-Chip* (SoC) desenvolvido pela Espressif Systems, projetado para aplicações de Internet das Coisas que demandam conectividade sem fio integrada, desempenho computacional elevado e eficiência energética. Desde seu lançamento em 2016, o ESP32 tornou-se uma das plataformas mais populares para prototipagem e desenvolvimento de soluções IoT, sendo utilizado tanto em projetos acadêmicos quanto em produtos comerciais (BECKER *et al.*, 2025).

A arquitetura do ESP32 baseia-se em um processador dual-core Xtensa LX6, operando em frequências de até 240 MHz, complementado por coprocessador de ultra-baixo consumo (ULP), memória SRAM de 520 KB e suporte a memória flash externa de até 16 MB. O SoC integra interfaces Wi-Fi 802.11 b/g/n e Bluetooth 4.2 (incluindo BLE), além de um amplo conjunto de periféricos como conversores analógico-digitais (ADC), conversores digital-analógicos (DAC), interfaces SPI, I²C, I²S, UART, PWM, sensores de toque capacitivo e sensor de efeito Hall.

Quadro 2 – Especificações técnicas do microcontrolador ESP32

Característica	Especificação
Processador	Dual-core Xtensa LX6, até 240 MHz
Coprocessador	ULP (Ultra Low Power) para operação em deep sleep
Memória SRAM	520 KB
Flash externa	Até 16 MB (tipicamente 4 MB em módulos DevKit)
Conectividade Wi-Fi	IEEE 802.11 b/g/n, 2.4 GHz, até 150 Mbps
Conectividade Bluetooth	Bluetooth 4.2 BR/EDR e BLE
Tensão de operação	3.0 V – 3.6 V (típico 3.3 V)
GPIOs	34 pinos programáveis
Interfaces	SPI, I ² C, I ² S, UART, PWM, ADC (12-bit), DAC (8-bit)

Fonte: Espressif Systems (2024).

Uma característica diferencial do ESP32 para aplicações IoT é seu suporte a múltiplos modos de operação com diferentes perfis de consumo energético. O Quadro 3 apresenta os modos de energia disponíveis e seus respectivos consumos típicos, conforme especificado no *datasheet* do fabricante.

Quadro 3 – Modos de operação e consumo energético do ESP32

Modo de Operação	Consumo Típico	Descrição
Active (Wi-Fi TX)	160 – 260 mA	Transmissão ativa de dados via Wi-Fi
Active (CPU @ 240 MHz)	30 – 68 mA	CPU operando em máxima frequência, rádio desligado
Active (CPU @ 80 MHz)	20 – 25 mA	CPU operando em frequência reduzida, rádio desligado
Modem Sleep	3 – 20 mA	CPU ativa, rádio Wi-Fi/BT desligado
Light Sleep	0.8 mA	CPU pausada, RAM preservada, despertar rápido (<1 ms)
Deep Sleep	10 – 150 µA	CPU desligada, apenas RTC ativo, despertar por timer ou GPIO
Hibernation	5 µA	Apenas RTC timer ativo, memória não preservada

Fonte: Espressif Systems (2024).

A disponibilidade de módulos de desenvolvimento (DevKit) a preços acessíveis, tipicamente entre R\$ 25,00 e R\$ 40,00 no mercado brasileiro, democratiza o acesso à tecnologia e viabiliza a construção de redes de sensores com dezenas de nós a custos compatíveis com aplicações em países em desenvolvimento.

3.3 O Protocolo ESP-NOW

O ESP-NOW é um protocolo de comunicação sem fio proprietário desenvolvido pela Espressif Systems, projetado para permitir a troca direta de mensagens curtas entre dispositivos ESP32 e ESP8266 sem a necessidade de estabelecimento prévio de conexão Wi-Fi ou associação a um ponto de acesso. O protocolo opera na camada de enlace do padrão IEEE 802.11, utilizando *action frames* do tipo *vendor-specific* para transmissão de dados (BECKER *et al.*, 2025).

3.3.1 Características técnicas

O ESP-NOW opera na frequência de 2.4 GHz, com taxa de transmissão de 1 Mbit/s e largura de banda de canal de 20 MHz. O protocolo implementa um mecanismo de acesso ao meio baseado em CSMA (*Carrier Sense Multiple Access*) do tipo *p-persistent*, com *slots* temporais de aproximadamente 481 µs. O suporte a retransmissões automáticas permite até 31 tentativas por pacote em caso de falha na entrega, aumentando a confiabilidade da comunicação (BECKER *et al.*, 2025).

Cada mensagem ESP-NOW pode transportar até 250 bytes de *payload* útil, com um *overhead* de cabeçalho de aproximadamente 43 bytes. O protocolo suporta criptografia opcional utilizando o algoritmo AES-128 no modo CCMP, permitindo comunicação segura quando necessário. A identificação dos dispositivos é realizada através de seus endereços MAC de 6 bytes.

Quadro 4 – Características técnicas do protocolo ESP-NOW

Parâmetro	Valor
Padrão base	IEEE 802.11b (action frames vendor-specific)
Frequência de operação	2.4 GHz (canais 1-14)
Taxa de transmissão	1 Mbit/s
Payload máximo	250 bytes
Overhead de cabeçalho	~43 bytes
Mecanismo de acesso ao meio	CSMA p-persistent (slot ~481 µs)
Retransmissões automáticas	Até 31 tentativas por pacote
Criptografia	AES-128 CCMP (opcional)
Número máximo de peers	20 peers (com criptografia) / 10 peers (sem criptografia)

Fonte: Becker et al. (2025); Espressif Systems (2024).

3.3.2 Desempenho e alcance

Estudos experimentais recentes fornecem caracterização detalhada do desempenho do ESP-NOW em diferentes condições. Becker *et al.* (2025), em experimentos de campo conduzidos em 246 posições distintas em ambientes abertos (fazenda) e florestais, reportaram latência base de transmissão entre 2.700 e 2.800 µs (aproximadamente 2,8 ms), podendo alcançar até 60 ms em cenários com múltiplas retransmissões. Em termos de confiabilidade, foi observado *Packet Delivery Ratio* (PDR) superior a 99% em distâncias de até 55 metros em linha de visada, com alcance máximo de aproximadamente 70 metros antes da perda total de conectividade.

Em ambiente interno, Urazayev *et al.* (2023) avaliaram o desempenho do ESP-NOW considerando a presença de paredes e obstáculos. Os resultados demonstraram que o ESP-NOW apresenta alcance interno de 15 a 90 metros dependendo das condições, com PDR 15% superior e consumo energético 30% inferior quando comparado a soluções baseadas em Wi-Fi TCP convencional. Os autores destacam que fatores ambientais como reflexões e obstáculos afetam significativamente o desempenho, e que pequenas variações na posição do receptor podem causar grandes alterações na qualidade do enlace.

3.3.3 Limitações para comunicação multi-hop

Apesar de suas vantagens, o ESP-NOW apresenta limitações estruturais que restringem sua aplicação direta em redes de maior escala. A primeira e mais significativa limitação é a ausência de suporte nativo a roteamento multi-hop. O protocolo foi concebido exclusivamente para comunicações ponto-a-ponto ou em topologia estrela, não oferecendo mecanismos para encaminhamento de pacotes através de nós intermediários (BECKER *et al.*, 2025).

A segunda limitação refere-se ao número máximo de *peers* simultâneos, cada dispositivo ESP32 pode manter no máximo 20 *peers* registrados em sua tabela de pares quando utilizada criptografia, ou 10 *peers* sem criptografia. Essa restrição, imposta por limitações de memória do hardware, impede a escalabilidade direta da rede e exige estratégias de gerenciamento dinâmico de *peers* para contorná-la.

A terceira limitação relaciona-se à ausência de mecanismo de *broadcast* nativo eficiente. Embora seja possível enviar mensagens para o endereço de *broadcast* (FF:FF:FF:FF:FF:FF), essa abordagem não oferece confirmação de entrega e apresenta menor confiabilidade, dificultando sua utilização para disseminação de mensagens de controle em protocolos de descoberta de rotas.

3.4 Redes Mesh e Protocolos de Roteamento Ad-Hoc

3.4.1 Conceitos fundamentais de redes mesh

As Redes Mesh Sem Fio (*Wireless Mesh Networks* – WMNs) constituem uma arquitetura de rede descentralizada na qual cada nó pode funcionar simultaneamente como origem, destino e roteador de dados, estabelecendo múltiplos caminhos de comunicação entre os participantes (CHAI; ZENG, 2021). Diferentemente das topologias centralizadas, como a estrela, onde todos os nós dependem de um ponto de acesso único, as redes mesh distribuem a responsabilidade de encaminhamento entre todos os participantes, eliminando pontos únicos de falha e aumentando a resiliência do sistema.

A comunicação em múltiplos saltos (*multi-hop*) é uma característica fundamental das redes mesh: quando o destino está fora do alcance direto de transmissão do nó de origem, pacotes de dados são encaminhados através de nós intermediários que atuam como *relays*. Essa abordagem permite estender significativamente o alcance efetivo da rede sem aumentar a potência de transmissão individual dos dispositivos, característica particularmente valiosa para redes de sensores alimentadas por bateria.

As WMNs podem ser classificadas em três categorias principais conforme sua arquitetura: (i) *Infrastructure WMN*, onde roteadores mesh formam a infraestrutura de backbone da rede; (ii) *Client WMN*, onde os próprios dispositivos clientes participam do roteamento; e (iii) *Hybrid WMN*, que combina características de ambas as abordagens. Para redes de sensores com ESP32, a arquitetura *Client WMN* é tipicamente mais adequada, pois todos os nós possuem capacidade computacional similar e podem contribuir para o roteamento (CHAI; ZENG, 2021).

3.4.2 Classificação dos protocolos de roteamento

Os protocolos de roteamento para redes ad-hoc e mesh podem ser classificados em três categorias principais conforme sua estratégia de descoberta e manutenção de rotas: proativos (*table-driven*), reativos (*on-demand*) e híbridos.

Os protocolos proativos, como o *Optimized Link State Routing* (OLSR) e o *Destination-Sequenced Distance-Vector* (DSDV), mantêm tabelas de roteamento constantemente atualizadas através da troca periódica de informações de topologia entre todos os nós da rede. Essa abordagem oferece baixa latência na descoberta de rotas — uma vez que a informação já está disponível quando necessária — mas impõe *overhead* de controle constante, mesmo quando não há tráfego de dados, o que pode ser problemático para redes de sensores com recursos energéticos limitados.

Os protocolos reativos, como o *Ad-hoc On-Demand Distance Vector* (AODV) e o *Dynamic Source Routing* (DSR), descobrem rotas apenas quando há necessidade efetiva de comunicação. Quando um nó precisa enviar dados para um destino desconhecido, inicia um processo de descoberta de rota através da disseminação de mensagens de requisição pela rede. Essa abordagem elimina o *overhead* de manutenção em períodos de inatividade, sendo mais adequada para redes com padrões de tráfego esporádicos ou imprevisíveis, características comuns em aplicações de sensoriamento.

Os protocolos híbridos, como o *Zone Routing Protocol* (ZRP), combinam características de ambas as abordagens: utilizam roteamento proativo dentro de zonas locais e roteamento reativo para comunicação entre zonas. Essa estratégia busca balancear as vantagens de ambas as abordagens, mas introduz complexidade adicional na implementação.

Quadro 5 – Comparativo entre categorias de protocolos de roteamento ad-hoc

Característica	Proativo	Reativo	Híbrido
Exemplos	OLSR, DSDV	AODV, DSR	ZRP
Descoberta de rotas	Antecipada (sempre)	Sob demanda	Combinada
Latência inicial	Baixa	Alta (descoberta)	Média
Overhead de controle	Constante (alto)	Variável (baixo)	Moderado
Uso de memória	Alto	Baixo/médio	Médio
Adequação para WSNs	Limitada	Boa	Moderada

Fonte: Adaptado de Chai e Zeng (2021) e Priyadarshi et al. (2025).

3.4.3 Soluções existentes de mesh para ESP32

O ecossistema ESP32 oferece algumas soluções de rede mesh, cada uma com características distintas. O ESP-WIFI-MESH (também conhecido como ESP-MDF) é a solução oficial da Espressif, implementando uma arquitetura de árvore auto-organizada sobre a pilha Wi-Fi convencional. Embora ofereça recursos avançados de gerenciamento de rede, o ESP-WIFI-MESH impõe consumo energético elevado devido à utilização contínua do rádio Wi-Fi para manutenção da topologia (ŠESTÁK, 2022).

A biblioteca PainlessMesh, desenvolvida pela comunidade open-source, oferece uma alternativa simplificada para criação de redes mesh com ESP32 e ESP8266. Similar ao ESP-WIFI-MESH, a PainlessMesh opera sobre a pilha Wi-Fi, herdando as mesmas limitações de consumo energético. Conforme observado por Šesták (2022), o uso do Wi-Fi padrão para manutenção da topologia mesh resulta em *overhead* significativo de processamento e energia, comprometendo a aplicabilidade dessas soluções em dispositivos alimentados por baterias.

Abordagens alternativas têm explorado a construção de redes mesh diretamente sobre o ESP-NOW. Cujilema *et al.* (2023) propuseram o algoritmo BRAM-NOW para criação de redes mesh seguras, demonstrando latência média de 75 ms e taxa de perda de pacotes de 9,25% em ambiente residencial. Entretanto, essa abordagem não se fundamenta em protocolos de roteamento padronizados, limitando sua comparabilidade com outras soluções e dificultando extensões futuras.

3.5 O Protocolo AODV

O *Ad-hoc On-Demand Distance Vector* (AODV) é um protocolo de roteamento reativo especificado na RFC 3561 (PERKINS; BELDING-ROYER; DAS, 2003), projetado para redes ad-hoc móveis (MANETs) com populações de dezenas a milhares de nós. O protocolo oferece rápida adaptação a condições dinâmicas de enlace, baixo *overhead* de processamento e memória, e utilização eficiente da rede, características que o tornam adequado para adaptação em redes de sensores com recursos limitados.

3.5.1 Mensagens de controle

O AODV define quatro tipos de mensagens de controle para descoberta e manutenção de rotas:

- RREQ (Route Request):** mensagem de requisição de rota, disseminada por *broadcast* quando um nó precisa descobrir um caminho para um destino desconhecido. Contém: endereço IP de origem, número de sequência de origem, endereço IP de destino, último número de sequência conhecido do destino, identificador único da requisição (RREQ ID) e contador de saltos;
- RREP (Route Reply):** mensagem de resposta, enviada em *unicast* de volta à origem quando o destino ou um nó intermediário com rota válida recebe um RREQ. Contém: endereço IP de destino, número de sequência de destino, endereço IP de origem, contador de saltos e tempo de vida da rota;
- RERR (Route Error):** mensagem de erro de rota, enviada quando uma quebra de enlace é detectada. Notifica os nós

afetados sobre destinos que se tornaram inalcançáveis através do enlace interrompido;

d) RREP-ACK (*Route Reply Acknowledgment*): mensagem de confirmação opcional, utilizada para detectar enlaces unidirecionais quando necessário.

3.5.2 Processo de descoberta de rotas

Quando um nó de origem necessita enviar dados para um destino para o qual não possui rota válida em sua tabela de roteamento, inicia o processo de descoberta de rota através da difusão de uma mensagem RREQ. Antes da transmissão, o nó incrementa seu próprio número de sequência e armazena o par (endereço de origem, RREQ ID) em um buffer temporal para evitar reprocessamento de requisições duplicadas.

Ao receber um RREQ, cada nó intermediário: (i) cria ou atualiza uma entrada de rota reversa para a origem; (ii) incrementa o contador de saltos; (iii) verifica se é o destino ou se possui rota válida para o destino com número de sequência adequado; (iv) caso positivo, gera e envia um RREP em unicast; caso negativo, retransmite o RREQ para seus vizinhos (exceto aquele de quem recebeu).

A mensagem RREP é transmitida em unicast através do caminho reverso estabelecido durante a propagação do RREQ. À medida que o RREP percorre esse caminho, cada nó intermediário cria ou atualiza uma entrada de rota direta para o destino, estabelecendo o próximo salto (*next hop*) como o vizinho de quem recebeu o RREP. Quando o RREP alcança a origem, a rota bidirecional está estabelecida e a transmissão de dados pode iniciar.

3.5.3 Números de sequência e prevenção de loops

Uma característica distintiva do AODV é a utilização de números de sequência de destino para garantir a ausência de loops de roteamento (*loop freedom*). Cada nó mantém seu próprio número de sequência, incrementando-o antes de originar uma nova requisição de rota ou resposta. Os números de sequência permitem distinguir informações de roteamento atualizadas de informações obsoletas: entre duas rotas para um mesmo destino, é sempre selecionada aquela com maior número de sequência; em caso de empate, seleciona-se a rota com menor número de saltos (PERKINS; BELDING-ROYER; DAS, 2003).

3.5.4 Manutenção de rotas e tratamento de falhas

O AODV monitora a conectividade com os próximos saltos das rotas ativas através de dois mecanismos complementares: (i) mensagens HELLO periódicas, que são RREPs com TTL=1 enviadas a cada HELLO_INTERVAL (tipicamente 1 segundo); e (ii) detecção de falha de enlace na camada de enlace, quando disponível (por exemplo, ausência de ACK em transmissões IEEE 802.11).

Quando uma quebra de enlace é detectada, o nó que identifica a falha invalida todas as rotas que utilizam o enlace interrompido e gera uma mensagem RERR listando os destinos afetados. A mensagem RERR é propagada para os precursores — nós que utilizam o remetente como próximo salto para os destinos afetados — permitindo que invalidem suas respectivas entradas de rota. Opcionalmente, o nó que detecta a falha pode tentar reparo local da rota antes de propagar o RERR.

3.5.5 Parâmetros de configuração

A RFC 3561 define valores padrão para diversos parâmetros configuráveis que influenciam o comportamento e desempenho do protocolo. O Quadro 6 apresenta os principais parâmetros e seus valores padrão, que podem ser ajustados conforme as características específicas da rede.

Quadro 6 – Parâmetros de configuração do AODV (RFC 3561)

Parâmetro	Valor Padrão	Descrição
ACTIVE_ROUTE_TIMEOUT	3.000 ms	Tempo de vida de rota ativa
HELLO_INTERVAL	1.000 ms	Intervalo entre mensagens HELLO
ALLOWED_HELLO_LOSS	2	Perdas de HELLO antes de assumir falha
NET_DIAMETER	35	Número máximo de saltos na rede
NODE_TRAVERSAL_TIME	40 ms	Tempo estimado de travessia por nó
RREQ_RETRIES	2	Tentativas de descoberta de rota
RREQ_RATELIMIT	10	Máximo de RREQs por segundo
TTL_START	1	TTL inicial para expanding ring search
TTL_INCREMENT	2	Incremento de TTL a cada tentativa
TTL_THRESHOLD	7	TTL máximo antes de usar NET_DIAMETER

Fonte: RFC 3561 (PERKINS; BELDING-ROYER; DAS, 2003).

3.6 Considerações para o Projeto do AODV-EN

A adaptação do protocolo AODV para operação sobre ESP-NOW (AODV-EN) requer a consideração de três aspectos fundamentais: gerenciamento do limite de *peers*, substituição do mecanismo de *broadcast* e definição de métrica de roteamento adequada.

3.6.1 Gerenciamento de peers com política LRU

O limite de 20 *peers* simultâneos imposto pelo ESP-NOW exige uma estratégia de gerenciamento dinâmico para permitir a comunicação com um número potencialmente maior de nós ao longo do tempo. A política LRU (*Least Recently Used*) é proposta como mecanismo de substituição: quando a tabela de *peers* está cheia e uma nova comunicação é necessária, o *peer* menos recentemente utilizado é removido para dar lugar ao novo. Essa abordagem baseia-se no princípio de localidade temporal: nós comunicados recentemente têm maior probabilidade de serem comunicados novamente.

3.6.2 Flooding controlado via unicast

A disseminação de mensagens RREQ no AODV original utiliza *broadcast* IP. Como o ESP-NOW não oferece *broadcast* confiável nativo, propõe-se a substituição por *flooding* controlado via transmissões unicast sequenciais para cada vizinho conhecido. Embora essa abordagem aumente o número de transmissões individuais, oferece maior confiabilidade através do mecanismo de confirmação do ESP-NOW e permite controle granular sobre a propagação das mensagens.

3.6.3 Métrica híbrida de roteamento

O protocolo AODV original utiliza a contagem de saltos (*hop count*) como métrica única de roteamento, priorizando rotas com menor distância lógica. No entanto, essa abordagem desconsidera aspectos relacionados à qualidade dos enlaces sem fio, os quais são particularmente relevantes em redes ad-hoc e sensores.

Nesse contexto, diferentes trabalhos propõem o uso de métricas híbridas que combinam informações topológicas e indicadores de qualidade de enlace, como a intensidade do sinal recebido (*Received Signal Strength Indicator*—RSSI). Considerando essa perspectiva, o AODV-EN fundamenta-se em uma métrica composta que integra o número de saltos e a qualidade média dos enlaces ao longo da rota, podendo ser expressa por:

$$\text{Custo(rota)} = \alpha \times \text{HopCount} + \beta \times (1 / \text{RSSI_médio})$$

Onde α e β são pesos configuráveis que permitem balancear a importância relativa da distância lógica (número de saltos) e da qualidade dos enlaces. Essa abordagem possibilita a seleção de rotas que otimizem simultaneamente o comprimento do caminho e a confiabilidade esperada das transmissões.

3.6.4 Estimativa de consumo energético

A avaliação do consumo energético em algoritmos de roteamento para redes de sensores sem fio é comumente realizada a partir de modelos analíticos baseados no perfil de consumo dos dispositivos empregados. De modo geral, a energia consumida durante a transmissão de um pacote pode ser expressa como o produto entre a potência de transmissão e o tempo de envio, conforme:

$$E_{tx} = P_{tx} \times t_{tx} = (V \times I_{tx}) \times t_{tx}$$

Onde que E_{tx} representa a energia consumida por transmissão, P_{tx} a potência elétrica durante o envio, V a tensão de operação e I_{tx} a corrente elétrica associada ao modo de transmissão. Esses modelos permitem estimar o impacto energético das comunicações e comparar diferentes protocolos ou estratégias de roteamento sob condições semelhantes de operação.

4. METODOLOGIA

Este capítulo apresenta a metodologia adotada para o desenvolvimento e avaliação do algoritmo AODV-EN. Inicialmente, realiza-se a caracterização da pesquisa quanto à sua natureza, abordagem e procedimentos técnicos. Em seguida, apresenta-se o Design Science Research (DSR) como paradigma metodológico, justificando sua adequação ao problema investigado. Na sequência, detalham-se as fases do trabalho mapeadas às etapas da DSR, o planejamento experimental, as métricas de avaliação e o algoritmo de referência utilizado para comparação. Por fim, são descritos os procedimentos de coleta, organização e análise dos dados experimentais.

4.1 Caracterização da Pesquisa

Quanto à sua natureza, esta pesquisa classifica-se como aplicada, pois visa desenvolver uma solução prática para um problema concreto: a ausência de suporte nativo a roteamento multi-hop no protocolo ESP-NOW. Diferentemente da pesquisa básica, que busca ampliar o conhecimento teórico sem preocupação imediata com aplicação, a pesquisa aplicada objetiva gerar conhecimento para aplicação prática dirigida à solução de problemas específicos (GIL, 2017).

Quanto aos procedimentos técnicos, a pesquisa é caracterizada como experimental, uma vez que envolve a implementação de um protótipo funcional (o algoritmo AODV-EN) e a realização de testes controlados em ambiente de laboratório. A experimentação permite manipular variáveis independentes (topologia da rede, número de nós, padrões de tráfego) e observar seus efeitos sobre variáveis dependentes (taxa de entrega, latência, consumo energético), estabelecendo relações de causa e efeito.

Quanto à abordagem do problema, a pesquisa é predominantemente quantitativa, fundamentando-se na coleta e análise de dados numéricos obtidos através de medições sistemáticas durante os experimentos. As métricas de desempenho — Packet Delivery Ratio (PDR), latência fim-a-fim, Normalized Routing Load (NRL) e consumo energético — são quantificadas e submetidas à análise estatística para validação das hipóteses formuladas.

4.2 Design Science Research

Além da caracterização quanto à natureza, abordagem e procedimentos técnicos, a pesquisa adota o Design Science Research (DSR) como paradigma metodológico central para orientar o desenvolvimento e a avaliação do artefato proposto. A escolha dessa abordagem decorre do fato de o trabalho não se limitar à análise teórica de um fenômeno, mas envolver a concepção, implementação e avaliação de uma solução computacional para um problema prático: a ausência de suporte nativo a roteamento multi-hop no protocolo ESP-NOW.

Nesse contexto, o algoritmo AODV-EN é compreendido como um artefato computacional, pois materializa uma proposta de solução por meio de um método de roteamento adaptado às restrições técnicas do ESP-NOW e

implementado em módulos ESP32. Dessa forma, a DSR fornece uma estrutura adequada para organizar o ciclo de identificação do problema, definição dos objetivos da solução, desenvolvimento do artefato, demonstração, avaliação e comunicação dos resultados.

4.2.1 Fundamentos da DSR

O Design Science Research (DSR) é um paradigma de pesquisa orientado à criação e avaliação de artefatos inovadores destinados a resolver problemas práticos. Conforme Hevner et al. (2004), enquanto as ciências naturais buscam descrever e explicar fenômenos existentes, as ciências do design concentram-se em como as coisas deveriam ser para atingir determinados objetivos. O propósito do design é, nas palavras de Simon (1996), "transformar situações existentes em situações preferidas".

Um artefato, no contexto da DSR, pode assumir diferentes formas: construtos (vocabulário e símbolos), modelos (abstrações e representações), métodos (algoritmos e práticas) e instanciações (implementações funcionais). O algoritmo AODV-EN proposto neste trabalho constitui simultaneamente um método (especificação do algoritmo de roteamento) e uma instanciação (implementação funcional em firmware para ESP32).

Peffer et al. (2007) propuseram um modelo de processo para condução de pesquisas em DSR composto por seis atividades: (1) identificação do problema e motivação; (2) definição dos objetivos da solução; (3) design e desenvolvimento; (4) demonstração; (5) avaliação; e (6) comunicação. O Quadro 7 apresenta estas etapas e suas descrições.

Quadro 7 – Etapas do Design Science Research segundo Peffer et al. (2007)

Etapa DSR	Descrição
1. Identificação do problema e motivação	Definir o problema de pesquisa específico e justificar o valor de uma solução. A definição do problema será utilizada para desenvolver um artefato que possa efetivamente prover uma solução.
2. Definição dos objetivos da solução	Inferir os objetivos de uma solução a partir da definição do problema e do conhecimento do que é possível e viável. Os objetivos podem ser quantitativos ou qualitativos.
3. Design e desenvolvimento	Criar o artefato, que pode ser construído, modelos, métodos ou instâncias. Esta atividade inclui determinar a funcionalidade desejada do artefato e sua arquitetura.
4. Demonstração	Demonstrar a eficácia do artefato na resolução do problema. Pode envolver experimentação, simulação, estudo de caso, prova de conceito ou outra atividade apropriada.
5. Avaliação	Observar e medir quão bem o artefato suporta uma solução para o problema. Comparar os objetivos da solução com os resultados observados na demonstração.
6. Comunicação	Comunicar o problema, o artefato, sua utilidade e novidade, o rigor do design e sua eficácia para pesquisadores e outros públicos relevantes.

Fonte: Adaptado de Peffer et al. (2007).

4.2.2 Justificativa da escolha da DSR

A escolha metodológica para este trabalho justifica-se por sua aderência ao tipo de problema investigado e à natureza da contribuição pretendida. Como a pesquisa envolve a concepção, implementação e avaliação de um artefato computacional, a DSR é adequada para estruturar o ciclo de desenvolvimento e validação da solução proposta.

a) **Natureza do problema:** o problema investigado, relacionado à ausência de roteamento multi-hop nativo no protocolo ESP-NOW, possui natureza técnica e prática, demandando uma solução concreta na forma de um artefato funcional, e não apenas a compreensão teórica do fenômeno;

b) **Tipo de contribuição:** a contribuição principal do trabalho é prescritiva, pois busca indicar como construir uma solução para o problema identificado, e não apenas descritiva, isto é, limitada à explicação de como determinado fenômeno ocorre. Essa característica alinha-se à epistemologia das ciências do design;

- c) **Ciclo construir-avaliar:** a DSR estrutura formalmente o ciclo iterativo de construção e avaliação de artefatos, fornecendo um arcabouço metodológico rigoroso para orientar o processo de desenvolvimento e validação do algoritmo proposto;
- d) **Adequação a algoritmos:** conforme March e Smith (1995), algoritmos constituem um tipo clássico de artefato em pesquisas orientadas pelo Design Science Research, sendo essa metodologia amplamente utilizada em trabalhos que envolvem o desenvolvimento de novos métodos computacionais;
- e) **Rigor e relevância:** a DSR equilibra o rigor científico, por meio da fundamentação teórica e da avaliação sistemática, com a relevância prática, voltada à solução de um problema real, atendendo aos critérios de qualidade esperados em pesquisas na área de Engenharia de Software.

4.3 Mapeamento das Fases do Trabalho às Etapas da DSR

O presente trabalho está organizado em cinco fases, que foram mapeadas às etapas do modelo de processo DSR de Peffers et al. (2007). O Quadro 8 apresenta este mapeamento, relacionando cada fase do trabalho à respectiva etapa DSR, suas atividades específicas e produtos esperados.

Quadro 8 – Mapeamento das fases do trabalho às etapas da DSR

Fase	Etapa DSR	Atividades	Produtos
Fase 1: Revisão Bibliográfica	1. Identificação do problema e motivação	- Levantamento da literatura sobre AODV, ESP-NOW e redes mesh - Identificação das limitações do ESP-NOW - Análise de soluções existentes e lacunas	- Referencial teórico - Trabalhos correlatos - Definição do problema
Fase 2: Projeto do Algoritmo AODV-EN	2. Definição dos objetivos / 3. Design e desenvolvimento	- Especificação dos requisitos funcionais - Definição da arquitetura do algoritmo - Projeto das adaptações do AODV para ESP-NOW - Definição das estruturas de dados e mensagens	- Diagramas de arquitetura - Pseudocódigo
Fase 3: Implementação e Ambiente de Testes	3. Design e desenvolvimento	- Codificação do firmware para ESP32 - Implementação do algoritmo baseline (Flooding) - Montagem do ambiente físico de testes - Desenvolvimento de ferramentas de coleta	- Código-fonte do Flooding - Testbed operacional
Fase 4: Execução dos Experimentos	4. Demonstração	- Execução dos cenários experimentais - Coleta de métricas (PDR, latência, NRL, energia) - Registro de logs e eventos	- Datasets de medições - Logs de execução
Fase 5: Análise e Discussão	5. Avaliação e 6. Comunicação	- Análise estatística dos resultados - Comparação AODV-EN vs. Flooding vs. literatura - Verificação das hipóteses - Elaboração do TCC	- Análise de resultados - Conclusões - Documento final

Fonte: Elaborado pelos autores.

A Figura 1 apresenta, de forma sintética, o fluxo metodológico adotado nesta pesquisa, evidenciando a relação entre o problema investigado, a abordagem Design Science Research, as fases do trabalho e a etapa de avaliação experimental. A representação contribui para visualizar a sequência lógica das atividades desenvolvidas ao longo da pesquisa.

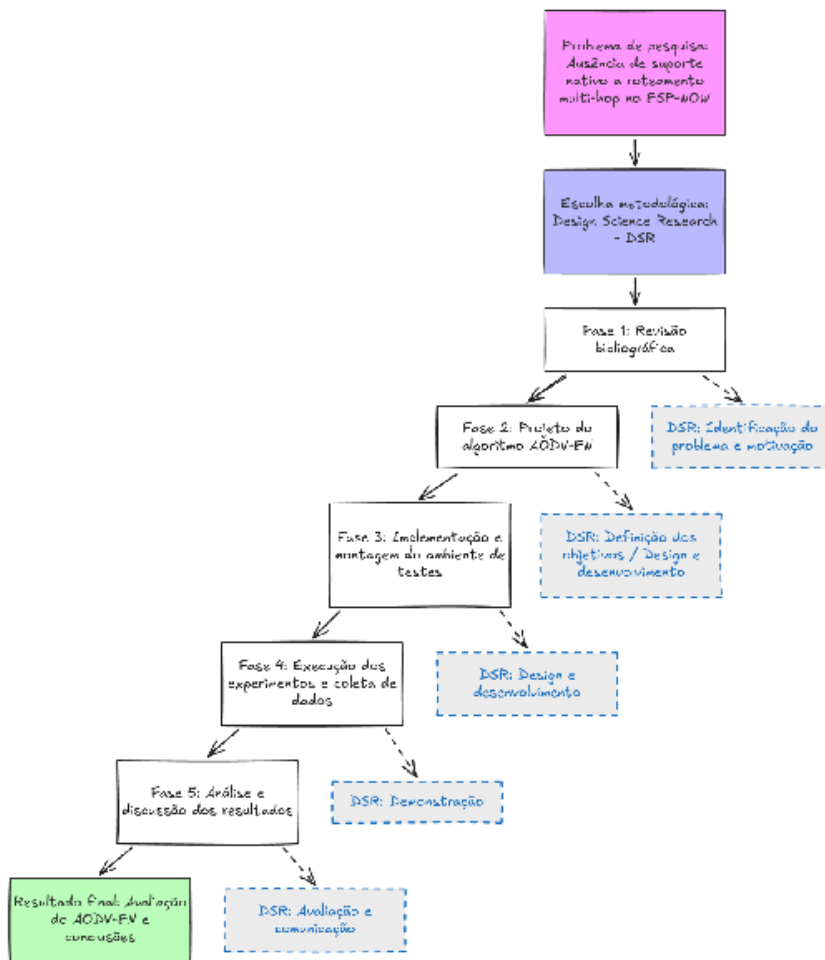


Figura 1 – Fluxo metodológico da pesquisa

4.3.1 Fase 1: Revisão Bibliográfica

A primeira fase corresponde à etapa de Identificação do Problema e Motivação da DSR. Nesta fase, foi realizado o levantamento bibliográfico para fundamentar teoricamente o trabalho e identificar com precisão as lacunas existentes na literatura.

O levantamento bibliográfico concentrou-se em três eixos temáticos: (i) o protocolo AODV e suas variantes, tendo como referência primária a RFC 3561; (ii) o protocolo ESP-NOW, suas características técnicas e limitações; e (iii) soluções existentes para redes mesh em plataformas embarcadas de baixo custo.

As fontes consultadas incluíram bases de dados científicas (IEEE Xplore, ACM Digital Library, ScienceDirect), documentação técnica oficial (Espressif Systems, IETF RFCs) e repositórios de projetos open-source (GitHub). Os critérios de seleção priorizaram publicações recentes (2021-2025) que abordassem especificamente ESP-NOW ou roteamento em redes de sensores de baixo custo.

4.3.2 Fase 2: Projeto do Algoritmo AODV-EN

A segunda fase corresponde às etapas de Definição dos Objetivos da Solução e início do Design e Desenvolvimento. Nesta fase, são especificados os requisitos funcionais e não-funcionais do artefato, definida sua arquitetura e projetadas as adaptações necessárias do protocolo AODV original para operação sobre ESP-NOW.

O projeto do AODV-EN parte dos princípios do protocolo AODV, especialmente quanto à descoberta de rotas sob demanda, ao uso de mensagens de controle, à manutenção de tabelas de roteamento e ao tratamento de falhas de enlace. A partir desses princípios, foram definidas adaptações necessárias para sua operação sobre ESP-NOW.

As principais decisões de projeto incluem:

a) **Gerenciamento de peers:** política LRU (Least Recently Used) para gerenciamento dinâmico da tabela de peers,

permitindo comunicação com mais de 20 nós ao longo do tempo dentro do limite simultâneo imposto pelo ESP-NOW;

- b) **Disseminação de mensagens RREQ**: substituição do broadcast IP por flooding controlado via transmissões unicast sequenciais, aproveitando o mecanismo de confirmação do ESP-NOW para maior confiabilidade;
- c) **Métrica de roteamento**: métrica híbrida combinando contagem de saltos (hop count) e qualidade de enlace (RSSI), permitindo seleção de rotas que otimizem simultaneamente distância lógica e confiabilidade esperada;
- d) **Estruturas de dados**: tabela de roteamento, tabela de peers com timestamps, buffer de RREQ IDs para detecção de duplicatas, e lista de precursores para notificação de falhas.

4.3.3 Fase 3: Implementação e Montagem do Ambiente de Testes

A terceira fase dá continuidade à etapa de Design e Desenvolvimento, materializando o projeto em código funcional e preparando o ambiente para os experimentos.

A implementação será realizada em linguagem C utilizando o framework ESP-IDF (Espressif IoT Development Framework). O código será organizado em módulos: gerenciamento de peers, descoberta de rotas, manutenção de rotas, encaminhamento de pacotes e interface de coleta de métricas. Paralelamente, será implementado o algoritmo Flooding para fins de comparação.

O ambiente de testes (testbed) será composto por 10 módulos ESP32 DevKit V1, alimentados por fontes USB 5V. A disposição física dos nós seguirá as topologias definidas na Seção 4.4. Cada nó executará o mesmo firmware, sendo configurado com identificador único através de variáveis de compilação.

4.3.4 Fase 4: Execução dos Experimentos e Coleta de Dados

A quarta fase corresponde à etapa de Demonstração da DSR, onde o artefato é exercitado em cenários representativos para demonstrar sua eficácia na resolução do problema. Os experimentos seguirão o planejamento detalhado na Seção 4.4.

Para cada cenário experimental, serão realizadas múltiplas execuções (mínimo de 30 repetições) para garantir significância estatística. As métricas serão coletadas automaticamente pelo firmware e transmitidas para um nó coordenador, que as registra em arquivos de log para posterior análise.

4.3.5 Fase 5: Análise e Discussão dos Resultados

A quinta fase corresponde às etapas de Avaliação e Comunicação da DSR. Os dados coletados serão submetidos a análise estatística utilizando técnicas descritivas (média, desvio padrão, intervalos de confiança) e inferenciais (testes de hipóteses) quando apropriado.

A avaliação comparará os resultados do AODV-EN com: (i) o algoritmo Flooding implementado localmente; e (ii) resultados reportados na literatura para soluções similares, como o BRAM-NOW de Cujilema et al. (2023). A comunicação dos resultados será realizada através deste TCC e, potencialmente, através de publicação em evento científico.

4.4 Planejamento Experimental

O planejamento experimental foi definido com o objetivo de avaliar o comportamento do algoritmo AODV-EN em diferentes condições de operação de uma rede mesh baseada em ESP-NOW. Considerando que a proposta envolve roteamento multi-hop, seleção de rotas, manutenção da comunicação e recuperação diante de falhas, os experimentos foram organizados em cenários capazes de representar situações progressivamente mais complexas de comunicação multi-hop.

A avaliação será realizada por meio da comparação entre o AODV-EN e um algoritmo de referência baseado em Flooding, permitindo observar diferenças quanto à confiabilidade da entrega, latência fim-a-fim, carga de controle e consumo energético estimado.

4.4.1 Cenários experimentais

Os experimentos serão conduzidos em quatro cenários distintos, projetados para avaliar aspectos complementares do desempenho do algoritmo proposto. O cenário C1 avalia o comportamento multi-hop básico em uma topologia linear; o cenário C2 verifica o encaminhamento em uma estrutura hierárquica; o cenário C3 analisa a seleção de rotas em uma malha parcial com caminhos alternativos; e o cenário C4 observa a capacidade de detecção de falhas e recuperação da comunicação.

Quadro 9 – Configuração dos cenários experimentais

Cenário	Topologia	Configuração	Objetivo
C1: Linear	Cadeia	• 5 nós em linha (N1→N2→N3→N4→N5) • Distância entre nós: 10m • Comunicação: N1↔N5 • 4 saltos obrigatórios	Avaliar comportamento multi-hop básico e latência acumulada por salto
C2: Árvore	Hierárquica	• 7 nós em estrutura de árvore • 1 raiz, 2 nós nível 1, 4 folhas • Comunicação: folhas↔raiz • Máximo 2 saltos	Avaliar encaminhamento hierárquico e agregação de tráfego
C3: Mesh Parcial	Redundante	• 6 nós com múltiplos caminhos • Cada nó alcança 2-3 vizinhos • Comunicação: extremidades • Rotas alternativas disponíveis	Avaliar seleção de rotas e capacidade de utilizar caminhos alternativos
C4: Falha	Linear com interrupção	• Cenário C1 com falha induzida • Nó N3 desligado após 60s • Observação da recuperação	Avaliar detecção de falha e capacidade de auto-recuperação (self-healing)

Fonte: Elaborado pelos autores.

4.4.2 Parâmetros dos experimentos

Os parâmetros experimentais foram definidos com o objetivo de manter condições equivalentes de comparação entre o AODV-EN e o algoritmo de referência. Assim, variáveis como número de nós, tamanho do payload, taxa de envio, duração das execuções e número de repetições foram padronizadas para ambos os algoritmos, variando apenas os parâmetros específicos de cada abordagem.

No caso do AODV-EN, são considerados parâmetros próprios do protocolo, como o intervalo de mensagens HELLO e o tempo de validade das rotas ativas. Para o Flooding, define-se o valor máximo de TTL, utilizado para limitar a propagação dos pacotes e evitar retransmissões indefinidas.

Quadro 10 – Parâmetros dos experimentos

Parâmetro	AODV-EN	Flooding
Número de nós	5-10 (conforme cenário)	5-10 (conforme cenário)
Tamanho do payload	32 bytes	32 bytes
Taxa de envio (steady state)	1 pacote/segundo	1 pacote/segundo
Duração de cada execução	300 segundos (5 min)	300 segundos (5 min)
Repetições por cenário	30	30
HELLO_INTERVAL	2.000 ms	N/A
ACTIVE_ROUTE_TIMEOUT	10.000 ms	N/A
TTL máximo (Flooding)	N/A	5 saltos

Fonte: Elaborado pelos autores.

4.5 Métricas de Avaliação

A avaliação do desempenho do algoritmo AODV-EN será realizada por meio de quatro métricas complementares: Packet Delivery Ratio (PDR), latência fim-a-fim, Normalized Routing Load (NRL) e consumo energético estimado. Essas métricas foram selecionadas por sua recorrência na literatura de redes ad-hoc e por sua adequação ao contexto de redes de sensores sem fio com recursos limitados.

O PDR permite avaliar a confiabilidade da comunicação; a latência fim-a-fim indica o desempenho temporal da entrega dos pacotes; o NRL mede o overhead de controle introduzido pelo protocolo; e o consumo energético estimado permite analisar o impacto das transmissões sobre a autonomia dos dispositivos.

Quadro 11 – Métricas de avaliação de desempenho

Métrica	Definição	Fórmula
PDR (Packet Delivery Ratio)	Razão entre pacotes de dados recebidos no destino e pacotes enviados pela origem. Indica a confiabilidade da comunicação.	$PDR = (\text{Pacotes recebidos} / \text{Pacotes enviados}) \times 100\%$
Latência Fim-a-Fim	Tempo decorrido entre o envio de um pacote pela origem e sua recepção no destino. Inclui tempo de processamento, enfileiramento e transmissão em cada salto.	$L = t_{\text{recepção}} - t_{\text{envio}}$
NRL (Normalized Routing Load)	Razão entre o número de pacotes de controle transmitidos e o número de pacotes de dados entregues. Indica a eficiência do protocolo em termos de overhead.	$NRL = \text{Pacotes de controle} / \text{Pacotes de dados entregues}$
Consumo Energético Estimado	Estimativa do consumo de energia baseada no número de transmissões e no perfil de consumo do ESP32. Permite projetar autonomia com bateria.	$E = \sum (N_{tx} \times E_{tx} + N_{rx} \times E_{rx} + t_{idle} \times P_{idle})$

Fonte: Elaborado pelos autores.

Para cada métrica, serão calculados: média aritmética, desvio padrão e intervalo de confiança de 95%. Os resultados serão apresentados em tabelas e gráficos comparativos entre AODV-EN e Flooding.

4.6 Algoritmo de Referência para Comparação: Flooding

Para estabelecer uma base de comparação experimental, será implementado um algoritmo de referência baseado em Flooding sobre ESP-NOW. O Flooding é a abordagem mais básica para disseminação de mensagens em redes sem infraestrutura, consistindo na retransmissão de cada pacote recebido para todos os vizinhos, exceto aquele de quem o pacote foi recebido.

4.6.1 Especificação do algoritmo Flooding

O algoritmo de referência baseado em Flooding será implementado com as seguintes características:

- Estrutura do pacote:** cada pacote contém identificador único (origem + número de sequência), endereço de origem, endereço de destino, TTL (Time-To-Live) e payload de dados;
- Deteção de duplicatas:** cada nó mantém um buffer circular com os últimos N identificadores de pacotes

recebidos (N=100). Pacotes com identificadores já presentes no buffer são descartados;

c) **Controle de TTL:** o TTL é decrementado a cada salto. Pacotes com TTL=0 não são retransmitidos, limitando o alcance do flooding e prevenindo loops infinitos;

d) **Retransmissão:** ao receber um pacote não-duplicado com TTL>0 e destino diferente do próprio nó, o pacote é retransmitido para todos os vizinhos conhecidos via unicast ESP-NOW;

e) **Entrega ao destino:** quando o pacote alcança o nó de destino, o payload é entregue à aplicação e a recepção é registrada para cálculo das métricas.

4.6.2 Justificativa da escolha do Flooding como algoritmo de referência

A escolha do Flooding como algoritmo de comparação justifica-se por:

a) **Simplicidade:** é o algoritmo mais simples possível para comunicação multi-hop, servindo como limite inferior de complexidade de implementação;

b) **Garantia de entrega:** em redes conectadas, o Flooding garante que o pacote alcançará o destino (se existir caminho), servindo como referência de PDR máximo alcançável;

c) **Overhead elevado:** o Flooding representa o extremo oposto em termos de eficiência, permitindo quantificar os ganhos obtidos pelo roteamento inteligente do AODV-EN;

d) **Ausência de estado:** não requer tabelas de roteamento ou manutenção de rotas, isolando o impacto desses mecanismos no AODV-EN.

A comparação entre o AODV-EN e o Flooding permitirá avaliar a relação entre complexidade de implementação e eficiência de comunicação. Embora o AODV-EN apresente maior complexidade por envolver descoberta e manutenção de rotas, espera-se que ele proporcione menor carga de controle, menor consumo energético estimado e desempenho comparável em termos de taxa de entrega de pacotes.

4.7 Coleta e Análise dos Dados

Os dados experimentais serão coletados a partir dos registros gerados durante a execução dos cenários de teste. Esses registros incluirão informações sobre pacotes enviados, pacotes recebidos, mensagens de controle, eventos de retransmissão, falhas de comunicação, atualizações de rota e tempos de envio e recepção.

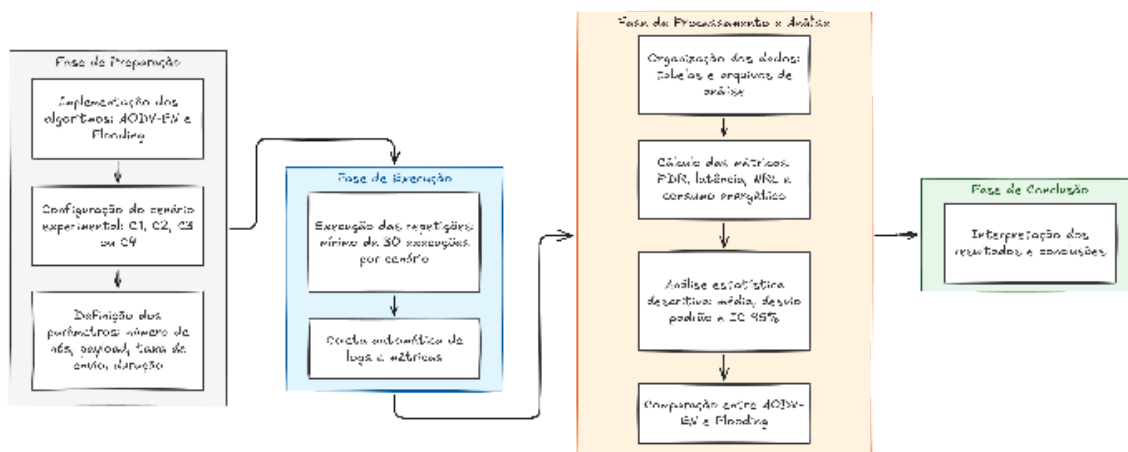


Figura 2 – Fluxo de execução dos experimentos e análise dos dados

A Figura 2 sintetiza o fluxo de execução dos experimentos e de tratamento dos dados, desde a configuração do ambiente experimental até a comparação dos resultados obtidos para o AODV-EN e o algoritmo de referência baseado em Flooding.

Para garantir a rastreabilidade dos experimentos, cada execução deverá ser identificada pelo cenário avaliado, algoritmo utilizado, número da repetição, configuração dos nós e arquivos de log correspondentes. Essa organização permitirá relacionar os resultados obtidos às condições específicas de cada execução experimental.

Após a coleta, os dados serão organizados em tabelas e submetidos à análise estatística descritiva, considerando média, desvio padrão e intervalo de confiança de 95% para cada métrica avaliada. A análise será realizada por cenário e por algoritmo, possibilitando a comparação direta entre os resultados do AODV-EN e do algoritmo de referência baseado em Flooding.

Execuções com falhas de instrumentação, perda de registros, reinicialização inesperada dos dispositivos ou divergência de configuração serão desconsideradas e repetidas, a fim de preservar a consistência dos dados analisados. A partir dos resultados consolidados, será avaliado se o AODV-EN, apresenta ganhos em eficiência de comunicação, mantendo desempenho compatível em termos de confiabilidade e latência.

5. PROJETO E IMPLEMENTAÇÃO DO ALGORITMO AODV-EN

Este capítulo apresenta o projeto e a implementação do algoritmo AODV-EN, desenvolvido como uma adaptação do protocolo AODV para redes mesh baseadas em ESP-NOW. A apresentação foi organizada de forma progressiva, partindo da visão geral da arquitetura até os mecanismos internos de descoberta de rotas, encaminhamento de dados, manutenção de rotas e tratamento de falhas. Ao final, são discutidas as principais adaptações realizadas em relação ao AODV original, considerando as restrições específicas do ESP-NOW.

5.1 Visão Geral da Arquitetura do AODV-EN

A arquitetura do AODV-EN foi concebida em camadas, de modo a separar as responsabilidades entre aplicação, interface pública de integração, núcleo do protocolo, adaptador de transporte e meio de comunicação ESP-NOW. Essa separação reduz o acoplamento entre os componentes, favorece a manutenção do código e permite a evolução incremental do artefato sem comprometer o funcionamento global da solução.

A Figura 3 apresenta a organização geral da arquitetura proposta, evidenciando o fluxo de interação entre as camadas responsáveis pela aplicação, pela interface pública da pilha, pelo núcleo do protocolo, pelo adaptador de transporte e pelo meio de comunicação ESP-NOW.

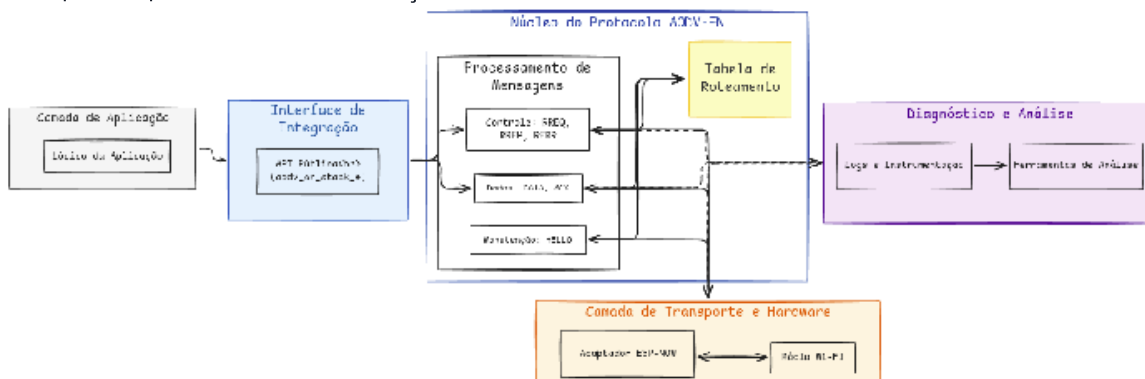


Figura 3 – Arquitetura geral do AODV-EN

A camada de aplicação é responsável por acionar o envio de dados e registrar eventos de execução. A interface pública disponibiliza as funções de inicialização, envio, recepção e atualização temporal da pilha. O núcleo do protocolo concentra a lógica de roteamento, incluindo descoberta de rotas, manutenção de tabelas, tratamento de falhas e encaminhamento de pacotes. O adaptador ESP-NOW, por sua vez, encapsula os detalhes de transmissão e recepção na camada de enlace.

5.2 Estrutura Modular da Implementação

A implementação do AODV-EN foi organizada em módulos funcionais com interfaces explícitas, buscando separar a lógica central do protocolo das estruturas auxiliares e dos detalhes específicos do transporte. Essa organização contribui para a manutenção do código, facilita a validação isolada de componentes e permite maior rastreabilidade entre as decisões de projeto e sua materialização no firmware.

A interface pública da pilha é concentrada nos módulos `aodv_en.h` e `aodv_en.c`, responsáveis por expor funções de inicialização, envio, recepção, atualização temporal e consulta de estatísticas. O núcleo do protocolo é implementado principalmente no módulo `aodv_en_node.c`, que coordena os processos de descoberta de rotas, encaminhamento de dados, manutenção de estados e tratamento de mensagens. Os demais módulos implementam estruturas de apoio, como tabela de rotas, tabela de vizinhos, cache de RREQ, gerenciamento de peers e manipulação de endereços MAC. A Figura 4 apresenta a organização modular da implementação, evidenciando a relação entre a aplicação, a fachada

pública da pilha, o núcleo do protocolo, os módulos auxiliares e o adaptador ESP-NOW.

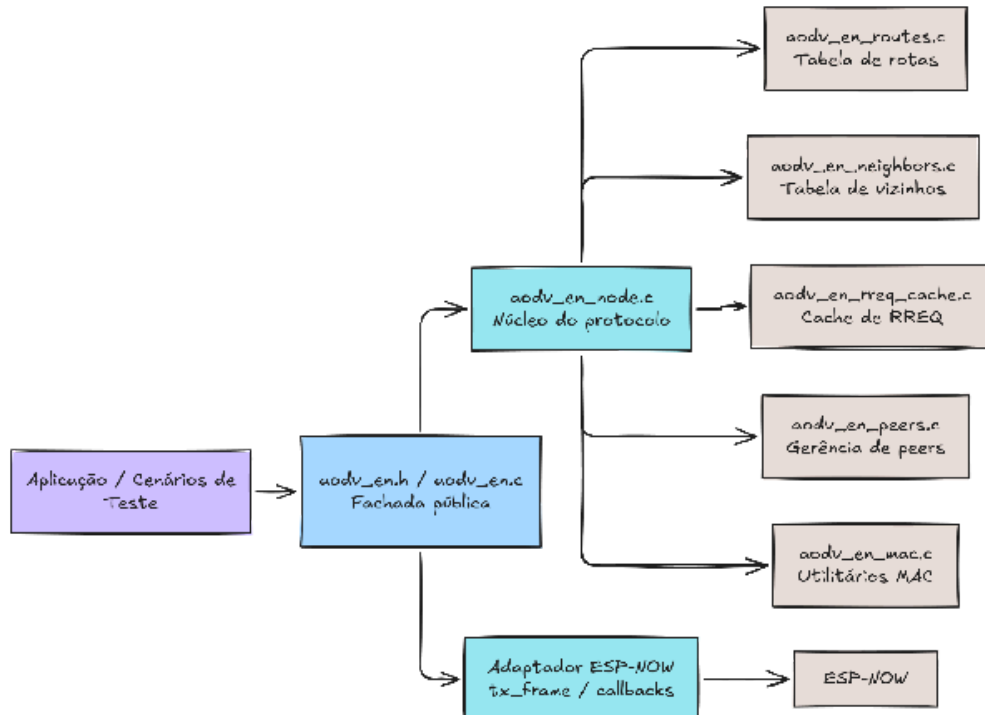


Figura 4 – Estrutura modular da implementação do AODV-EN

5.3 Estruturas de Dados Utilizadas

As estruturas de dados do AODV-EN foram projetadas considerando as limitações de memória do ambiente embarcado e a necessidade de atualização em tempo real das informações de roteamento. Essas estruturas armazenam o estado local de cada nó e permitem a execução dos mecanismos de descoberta, seleção, manutenção e invalidação de rotas.

O Quadro 12 apresenta uma síntese das principais estruturas utilizadas na implementação do AODV-EN e sua função no funcionamento do protocolo.

Quadro 12 – Estruturas de dados utilizadas no AODV-EN

Estrutura	Função principal	Papel no protocolo
Tabela de rotas	Armazenar destinos conhecidos, próximo salto, estado, métrica e expiração	Permite seleção e encaminhamento de pacotes por rotas válidas
Tabela de vizinhos	Registrar nós alcançáveis diretamente e informações de RSSI	Subsidia o cálculo da métrica híbrida e a manutenção da conectividade local
Cache de RREQ	Armazenar requisições de rota já processadas	Evita duplicatas, retransmissões desnecessárias e ciclos de propagação
Gerenciamento de peers	Controlar pares registrados no ESP-NOW	Permite operar dentro das limitações de peers simultâneos do ESP-NOW

Fonte: Elaborado pelos autores.

****5.3.1 Tabela de Rotas****

A tabela de rotas mantém, para cada destino conhecido, informações como próximo salto, número de saltos, número de

A atualização das entradas considera critérios de frescor da informação, qualidade da rota e validade temporal, preservando o princípio do AODV de priorizar rotas mais recentes e evitar informações obsoletas. Além disso, a implementação emprega critérios de substituição que consideram número de sequência, métrica, contagem de saltos

e tempo de expiração, reduzindo trocas instáveis de rota e contribuindo para maior estabilidade do protocolo.

5.3.2 Tabela de Vizinhos

A tabela de vizinhos registra os nós alcançáveis diretamente por um determinado nó da rede. Para cada vizinho, são armazenadas informações como endereço MAC, estado do enlace, último RSSI observado, RSSI médio, instante da última comunicação e eventos de falha de enlace.

Essa estrutura subsidia tanto o encaminhamento local quanto o cálculo da métrica híbrida de roteamento. Ao considerar informações de RSSI, o AODV-EN pode avaliar não apenas a quantidade de saltos até o destino, mas também a qualidade dos enlaces utilizados no caminho, favorecendo rotas potencialmente mais estáveis em ambientes sem fio sujeitos a interferências e variações de sinal.

5.3.3 Cache de RREQ

O cache de RREQ armazena pares formados pelo endereço do nó originador e pelo identificador da requisição de rota. Esse mecanismo permite reconhecer requisições já processadas, evitando que uma mesma mensagem RREQ seja tratada repetidamente por um nó.

A utilização desse cache é fundamental em processos de flooding controlado, pois reduz retransmissões redundantes, limita a propagação excessiva de mensagens de controle e contribui para prevenir ciclos durante a descoberta de rotas. Dessa forma, o cache de RREQ atua como mecanismo de controle de overhead e de estabilidade do processo de descoberta.

5.3.4 Gerenciamento de Peers

O gerenciamento de peers foi projetado para lidar com a limitação de pares simultâneos imposta pelo ESP-NOW. Como o protocolo exige que dispositivos sejam registrados como peers para comunicação unicast, essa restrição pode limitar a escalabilidade da rede quando há múltiplos nós participantes.

Para contornar essa limitação, o AODV-EN separa a vizinhança lógica mantida pelo protocolo da tabela física de peers utilizada pelo ESP-NOW. Quando necessário, aplica-se uma política LRU (Least Recently Used), substituindo pares menos recentemente utilizados e permitindo que o protocolo opere com um número de nós maior que o limite simultâneo de peers registrados. Essa estratégia preserva a compatibilidade com o ESP-NOW sem comprometer a dinâmica de descoberta e manutenção de rotas.

5.4 Mensagens do Protocolo

O AODV-EN utiliza mensagens de controle e de dados para realizar a descoberta, manutenção e utilização das rotas na rede. As mensagens de controle são responsáveis por estabelecer caminhos, atualizar estados, manter a conectividade local e tratar falhas de enlace. Já as mensagens de dados transportam a carga útil da aplicação entre origem e destino, utilizando as rotas previamente descobertas pelo protocolo.

A definição dessas mensagens foi baseada nos princípios do AODV, especialmente quanto ao uso de requisições de rota, respostas de rota e mensagens de erro. Entretanto, foram incorporadas adaptações necessárias ao contexto do ESP-NOW, como o uso de endereçamento por MAC, limitação de tamanho de payload, controle de TTL e suporte à confirmação em nível de aplicação.

O Quadro 13 apresenta uma síntese das mensagens utilizadas pelo AODV-EN e suas respectivas funções no protocolo.

Quadro 13 – Mensagens utilizadas pelo AODV-EN

Mensagem	Tipo	Função principal	Uso no protocolo
RREQ	Controle	Requisitar descoberta de rota	Emitida quando um nó precisa alcançar um destino para o qual não possui rota válida
RREP	Controle	Responder a uma requisição de rota	Retorna pelo caminho reverso para estabelecer ou atualizar a rota até o destino
RERR	Controle	Notificar erro ou quebra de rota	Informa falhas de enlace e permite invalidar rotas afetadas
HELLO	Controle	Manter vizinhança local	Atualiza a conectividade entre nós diretamente alcançáveis
DATA	Dados	Transportar a carga útil da aplicação	Encaminhada salto a salto conforme a tabela de rotas
ACK	Controle/aplicação	Confirmar recebimento de dados	Apoia a verificação de entrega e o cálculo de métricas de confiabilidade

Fonte: Elaborado pelos autores.

As mensagens RREQ, RREP e RERR compõem o núcleo do mecanismo de roteamento reativo do AODV-EN. A mensagem RREQ é utilizada quando a origem necessita descobrir uma rota para determinado destino. Ao ser propagada pela rede, ela permite que os nós intermediários estabeleçam rotas reversas para a origem. A mensagem RREP, por sua vez, retorna pelo caminho reverso e permite a formação da rota direta até o destino. Já a mensagem RERR é utilizada quando ocorre falha de enlace ou invalidação de rota, permitindo que os nós afetados atualizem suas tabelas e iniciem nova descoberta, se necessário.

A mensagem HELLO é utilizada para manutenção da vizinhança local, permitindo identificar nós diretamente alcançáveis e atualizar informações associadas aos enlaces. Essas informações são importantes para a manutenção das rotas e para o cálculo da métrica híbrida baseada em número de saltos e qualidade do sinal.

As mensagens DATA e ACK estão associadas à comunicação da aplicação. A mensagem DATA transporta a carga útil entre origem e destino, utilizando as rotas válidas mantidas pelo protocolo. Quando necessário, a mensagem ACK confirma o recebimento dos dados em nível de aplicação, contribuindo para a avaliação da confiabilidade da entrega e para o controle de retransmissões.

5.5 Funcionamento do Algoritmo

O funcionamento do AODV-EN pode ser compreendido a partir de quatro processos principais: descoberta de rotas, encaminhamento de dados, manutenção de rotas e tratamento de falhas. Esses processos operam de forma integrada para permitir a comunicação multi-hop em redes baseadas em ESP-NOW, preservando a característica reativa do protocolo AODV original.

De modo geral, quando um nó precisa enviar dados para um destino e não possui rota válida, o protocolo inicia um processo de descoberta de rota. Após a definição do caminho, os pacotes de dados são encaminhados de salto a salto até o destino. Durante a operação da rede, mecanismos de manutenção verificam a validade das rotas e a conectividade entre vizinhos. Caso ocorra falha de enlace, as rotas afetadas são invalidadas e uma nova descoberta pode ser iniciada.

5.5.1 Descoberta de Rotas

A descoberta de rotas é iniciada quando um nó de origem precisa enviar dados para um destino e não possui uma rota válida em sua tabela de roteamento. Nessa situação, o nó originador emite uma mensagem RREQ contendo informações como endereço de origem, endereço de destino, identificador da requisição, número de sequência e limite de propagação.

Ao receber uma mensagem RREQ, cada nó intermediário verifica se a requisição já foi processada anteriormente. Caso seja uma requisição duplicada, ela é descartada. Caso contrário, o nó registra ou atualiza uma rota reversa para a origem, incrementa o contador de saltos e retransmite a requisição conforme a política de disseminação configurada.

Quando a mensagem RREQ alcança o destino, ou um nó que possua rota válida para ele, é gerada uma mensagem

RREP. Essa resposta retorna em unicast pelo caminho reverso previamente estabelecido, permitindo que os nós intermediários atualizem suas rotas diretas até o destino. Ao receber o RREP, a origem passa a possuir uma rota válida e pode iniciar o envio dos dados pendentes.

A Figura 5 apresenta o fluxo de descoberta de rotas no AODV-EN, destacando a propagação da mensagem RREQ e o retorno da mensagem RREP pelo caminho reverso.

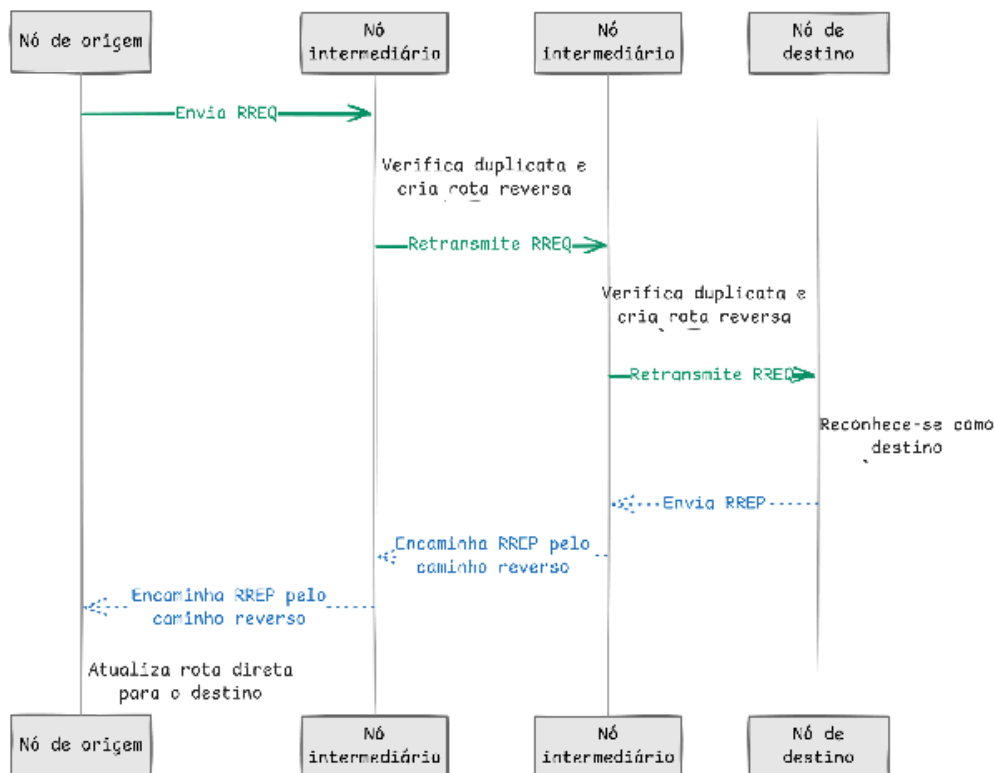


Figura 5 – Fluxo de descoberta de rotas no AODV-EN

5.5.2 Encaminhamento de Dados

Após a descoberta de uma rota válida, o encaminhamento dos dados ocorre de forma salto a salto. O nó de origem consulta sua tabela de rotas para identificar o próximo salto em direção ao destino e transmite a mensagem DATA ao vizinho correspondente. Cada nó intermediário repete o mesmo procedimento, consultando sua própria tabela de rotas até que o pacote alcance o destino final.

Caso a origem não possua rota válida no momento da tentativa de envio, o pacote é armazenado temporariamente em uma fila de dados pendentes. Em seguida, o protocolo inicia o processo de descoberta de rota por meio do envio de uma mensagem RREQ. Quando a rota é estabelecida, os dados pendentes são transmitidos conforme o caminho descoberto.

Esse mecanismo permite que a aplicação solicite o envio de dados mesmo quando a rota ainda não está disponível, preservando a característica reativa do AODV-EN e evitando o descarte imediato de pacotes em situações de descoberta inicial.

5.5.3 Manutenção de Rotas

A manutenção de rotas tem como objetivo preservar a validade das informações de roteamento durante a operação da rede. Para isso, o AODV-EN utiliza mecanismos de controle baseados em mensagens periódicas, temporizadores de expiração e atualização do estado dos vizinhos.

As mensagens HELLO são utilizadas para manter a conectividade local entre nós diretamente alcançáveis. A partir dessas mensagens, cada nó atualiza sua tabela de vizinhos, registrando informações como instante da última comunicação, estado do enlace e intensidade do sinal recebido. Essas informações contribuem para identificar vizinhos ativos e subsidiar o cálculo da métrica híbrida de roteamento.

Além da manutenção da vizinhança, cada rota possui um tempo de validade associado. Rotas que permanecem sem uso ou ultrapassam seu tempo de expiração deixam de ser consideradas válidas para encaminhamento. Esse mecanismo reduz o uso de informações obsoletas e contribui para evitar encaminhamentos por caminhos

indisponíveis.

A manutenção de rotas também está relacionada ao controle de estabilidade. Ao atualizar uma rota existente, o protocolo considera critérios como número de sequência, métrica, contagem de saltos e tempo de expiração, evitando substituições frequentes quando a nova rota não apresenta melhoria significativa.

5.5.4 Tratamento de Falhas

O tratamento de falhas é acionado quando o protocolo identifica indisponibilidade de um enlace ou impossibilidade de encaminhar dados por uma rota previamente considerada válida. Essa detecção pode ocorrer por ausência de resposta, falha de transmissão, expiração de vizinhança ou perda de conectividade com o próximo salto.

Ao detectar uma falha, o nó invalida as rotas que dependem do enlace afetado e gera uma mensagem RERR para notificar os nós impactados. Essa mensagem permite que os demais nós atualizem suas tabelas de roteamento, evitando o uso de caminhos inválidos. Quando ainda existem dados pendentes para o destino afetado, o protocolo pode iniciar uma nova descoberta de rota por meio de RREQ.

Esse mecanismo permite que o AODV-EN reaja dinamicamente a alterações na topologia da rede, buscando restabelecer a comunicação por rotas alternativas quando disponíveis. Dessa forma, o tratamento de falhas contribui para a capacidade de auto-recuperação da rede mesh.

A Figura 6 apresenta o fluxo de tratamento de falhas e reconvergência do AODV-EN, desde a detecção da falha até a tentativa de restabelecimento da comunicação.

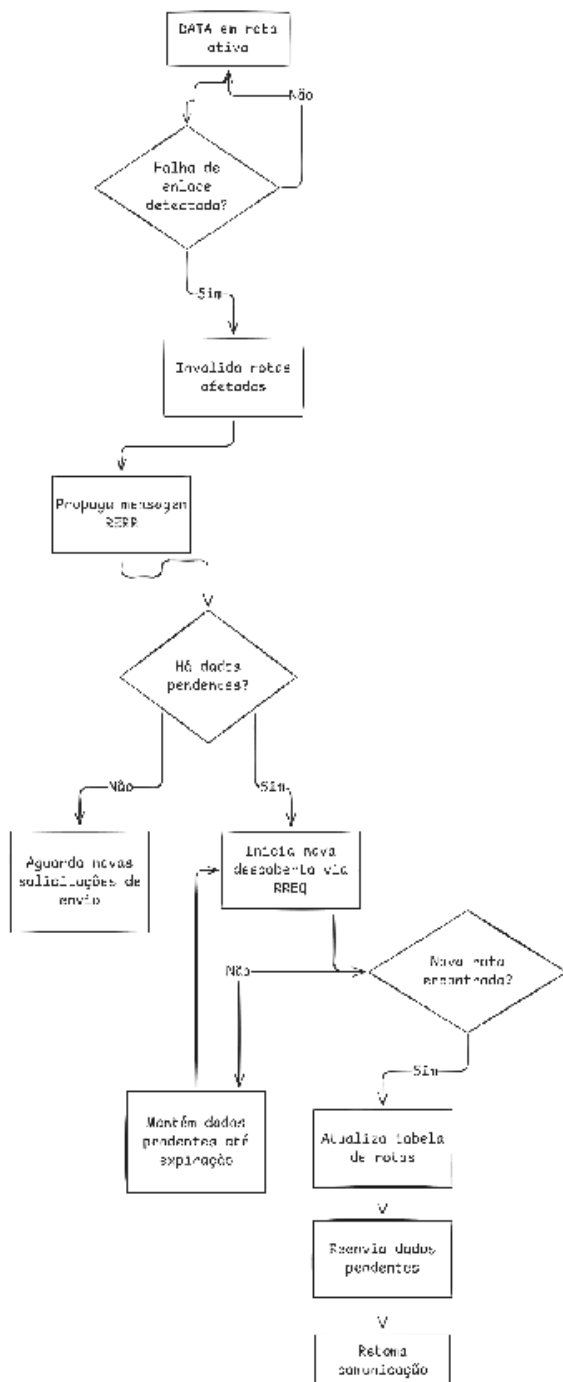


Figura 6 – Fluxo de tratamento de falhas e reconvergência no AODV-EN

5.6 Adaptações do AODV para ESP-NOW

A adaptação do AODV para operação sobre ESP-NOW exigiu modificações relacionadas ao endereçamento, à disseminação de mensagens, ao gerenciamento de peers e à métrica de seleção de rotas. Essas adaptações foram necessárias porque o ESP-NOW opera na camada de enlace, utiliza endereçamento baseado em MAC, possui limitação no número de peers registrados simultaneamente e não oferece, de forma nativa, mecanismos completos para roteamento multi-hop.

Dessa forma, o AODV-EN preserva os princípios fundamentais do AODV, como descoberta de rotas sob demanda, uso de mensagens de controle, manutenção de tabelas de roteamento e tratamento de falhas, mas incorpora mecanismos específicos para viabilizar sua execução em dispositivos ESP32 utilizando ESP-NOW.

O Quadro 14 sintetiza as principais adaptações realizadas no AODV-EN em relação ao AODV original.

Quadro 14 – Principais adaptações do AODV para o contexto do ESP-NOW

Aspecto	AODV original	Adaptação no AODV-EN
Endereçamento	Utiliza endereçamento IP	Utiliza endereços MAC dos dispositivos ESP32
Disseminação de RREQ	Baseada em broadcast	Pode utilizar flooding controlado por unicast sequencial
Gerenciamento de vizinhos	Associado à conectividade da rede ad-hoc	Mantém tabela de vizinhos com estado, último contato e RSSI
Gerenciamento de peers	Não aplicável diretamente ao AODV original	Usa cache de peers e política LRU para lidar com o limite do ESP-NOW
Métrica de rota	Baseada principalmente em número de saltos	Combina hop count e penalidade associada ao RSSI
Tratamento de falhas	Usa mensagens RERR e invalidação de rotas	Mantém RERR, invalidação de rotas e redescoberta adaptada ao ESP-NOW

Fonte: Elaborado pelos autores.

5.6.1 Endereçamento por MAC

No AODV original, o endereçamento dos nós é normalmente associado à camada de rede, utilizando endereços IP para identificar origem, destino e próximos saltos. No AODV-EN, essa lógica foi adaptada para o uso de endereços MAC, uma vez que o ESP-NOW opera diretamente sobre a camada de enlace do padrão IEEE 802.11.

Essa adaptação permite que cada dispositivo ESP32 seja identificado pelo seu endereço MAC, utilizado tanto nas mensagens de controle quanto nas mensagens de dados. Assim, campos como origem, destino, próximo salto e remetente do enlace são representados por endereços MAC, preservando a semântica de roteamento do AODV, mas adequando-a ao modelo de comunicação do ESP-NOW.

A utilização de endereçamento por MAC também simplifica a integração com o mecanismo de envio do ESP-NOW, uma vez que a transmissão unicast exige a indicação direta do endereço MAC do peer de destino.

5.6.2 Flooding Controlado por Unicast Sequencial

Uma das principais limitações para adaptar o AODV ao ESP-NOW está relacionada à disseminação das mensagens RREQ. No AODV original, a descoberta de rotas é realizada por meio da difusão da requisição de rota pela rede, geralmente utilizando broadcast. Entretanto, no ESP-NOW, o uso de broadcast apresenta limitações quanto à confiabilidade, confirmação de entrega e controle individual dos vizinhos alcançados.

Para contornar essa limitação, o AODV-EN adota a estratégia de flooding controlado por unicast sequencial. Nesse modelo, em vez de depender exclusivamente do broadcast, o nó percorre sua tabela de vizinhos ativos e envia a mensagem RREQ individualmente para cada vizinho, exceto aquele de quem recebeu a requisição. Essa abordagem aumenta o controle sobre a propagação das mensagens e permite melhor compatibilidade com o funcionamento do ESP-NOW.

Embora o unicast sequencial possa aumentar o número de transmissões individuais, ele oferece maior controle operacional sobre a disseminação da requisição de rota, além de permitir atualização de estatísticas, uso do mecanismo de peers e melhor integração com a lógica de vizinhança mantida pelo protocolo.

5.6.3 Gerenciamento Dinâmico de Peers

O ESP-NOW impõe restrições quanto ao número de peers que podem ser registrados simultaneamente para comunicação. Essa limitação afeta diretamente a construção de redes mesh, pois o número total de nós participantes pode ser superior ao número de peers mantidos fisicamente pelo dispositivo em determinado instante.

Para lidar com essa restrição, o AODV-EN separa a vizinhança lógica utilizada pelo protocolo da tabela física de peers mantida pelo ESP-NOW. A tabela de vizinhos representa os nós conhecidos ou alcançáveis no contexto do roteamento, enquanto o cache de peers controla quais dispositivos estão efetivamente registrados para transmissão no momento.

Quando a capacidade da tabela de peers é atingida, aplica-se uma política LRU (Least Recently Used), substituindo o peer menos recentemente utilizado por um novo peer necessário à comunicação. Essa estratégia permite que o protocolo opere dinamicamente em redes com mais nós do que o limite simultâneo de peers, preservando a compatibilidade com o ESP-NOW e reduzindo a necessidade de configuração manual da rede.

5.6.4 Métrica Híbrida com Hop Count e RSSI

O AODV original utiliza, de forma predominante, a contagem de saltos como critério de seleção de rotas. Essa abordagem favorece caminhos mais curtos, mas pode desconsiderar a qualidade dos enlaces sem fio. Em redes baseadas em ESP-NOW, essa limitação é relevante, pois enlaces com menor número de saltos podem apresentar baixa qualidade de sinal, maior instabilidade ou maior probabilidade de perda de pacotes.

Para considerar esse aspecto, o AODV-EN utiliza uma métrica híbrida que combina a distância lógica da rota, representada pelo número de saltos, com uma penalidade associada à qualidade do sinal recebido. A formulação geral da métrica pode ser expressa por:

$$\text{Custo(rota)} = \alpha \times \text{HopCount} + \beta \times \text{Penalidade(RSSI)}$$

Figura 7 - Fórmula do custo

Nessa formulação, α e β representam pesos configuráveis, HopCount corresponde ao número de saltos até o destino e Penalidade(RSSI) representa uma função de custo associada à qualidade média do enlace. Quanto menor o número de saltos e melhor a qualidade do sinal, menor tende a ser o custo da rota.

Essa adaptação permite que a seleção de rotas considere não apenas o caminho mais curto, mas também a robustez dos enlaces envolvidos, contribuindo para maior estabilidade da comunicação em ambientes sem fio sujeitos a interferências, obstáculos e variações de sinal.

5.7 Considerações sobre a Implementação

A implementação do AODV-EN buscou preservar os princípios fundamentais do protocolo AODV, especialmente a descoberta de rotas sob demanda, a manutenção de tabelas de roteamento, o uso de mensagens de controle e o tratamento de falhas de enlace. Entretanto, devido às particularidades do ESP-NOW, foram necessárias adaptações relacionadas ao endereçamento por MAC, ao gerenciamento dinâmico de peers, à disseminação controlada de requisições de rota e ao uso de métricas baseadas na qualidade do enlace.

Dessa forma, o algoritmo não corresponde a uma reprodução literal do AODV original, mas a uma adaptação orientada às restrições de comunicação, memória e operação do ESP32. A estrutura modular adotada permite isolar responsabilidades, facilitar futuras extensões e fornecer os dados necessários para a avaliação experimental.

6. RESULTADOS E DISCUSSÃO

Invariantes do AODV

AODV Base Invariantes

Estado

- alinhado com [aodv-en-spec-v1.md](#)
- ultima revisao: 2026-05-01

Objetivo

Este documento define o nucleo conceitual do AODV que o projeto `aodv-en` deve preservar.

Ele existe para evitar que o protocolo evolua de forma inconsistente ao longo da implementacao do TCC. Sempre que surgir uma nova ideia de arquitetura, mensagem, heuristica ou otimizacao, ela deve ser comparada com estas regras.

Se uma decisao violar um destes invariantes, o resultado deixa de ser uma adaptacao do AODV e passa a ser outro protocolo.

Escopo

O projeto `aodv-en` e uma adaptacao do AODV para uma rede mesh multi-hop baseada em ESP-NOW e ESP32.

As adaptacoes de plataforma sao permitidas, mas o comportamento essencial do algoritmo deve continuar reconhecivel como AODV.

Invariantes obrigatorios

1. Roteamento reativo

O protocolo deve descobrir rotas sob demanda.

Isso significa:

- a descoberta de rota ocorre quando um no precisa enviar dados e nao possui rota valida
- o protocolo nao deve manter um mapa global da rede atualizado continuamente
- o protocolo nao deve depender de anuncios periodicos de topologia para funcionar

2. Descoberta por RREQ e resposta por RREP

O mecanismo central de descoberta de rotas deve continuar sendo:

- `RREQ` para procurar uma rota
- `RREP` para responder a uma rota encontrada

Isso implica:

- a origem emite `RREQ` quando nao encontra rota valida

- o destino ou um nó com rota suficientemente válida responde com `RREP`
- a descoberta não pode ser substituída por flooding puro sem semântica de resposta

3. Roteamento hop-by-hop

As rotas devem ser mantidas localmente no formato:

- `destino -> next_hop`

O protocolo não deve carregar a rota completa dentro de cada pacote de dados, como em source routing.

4. Numeros de sequencia

O protocolo deve usar números de sequência para representar o frescor da informação de roteamento.

Esses números são obrigatórios para:

- comparar rotas
- preferir informação mais recente
- reduzir risco de loop
- manter coerência com o AODV

5. Rota reversa e rota direta

Durante o processo de descoberta:

- o `RREQ` deve construir rota reversa para a origem
- o `RREP` deve consolidar rota direta para o destino

Essas duas ideias são centrais no comportamento do AODV.

6. Supressao de duplicatas de RREQ

O protocolo deve ser capaz de identificar e descartar `RREQ`s duplicados.

No mínimo, isso exige o uso combinado de:

- `originator`
- `rreq_id`

Sem esse controle, a inundação se torna redundante e pouco controlada.

7. Invalidação de rotas

Rotas não podem ser consideradas válidas indefinidamente.

O protocolo deve possuir pelo menos um destes mecanismos:

- expiração por tempo
- sinalização explícita de erro via `RERR`
- ambos

8. Ausencia de loop como objetivo de projeto

Toda extensão proposta deve preservar a propriedade de roteamento sem loop, ou no mínimo não introduzir comportamento que contradiga as garantias buscadas pelo AODV.

Elementos que podem ser adaptados

Os itens abaixo podem ser adaptados ao ESP-NOW sem descaracterizar o AODV:

- endereçamento por MAC em vez de IP
- encapsulamento das mensagens em payload proprietário
- uso de HELLO, escuta passiva ou feedback de enlace para manter conectividade local
- adicao de metricas como RSSI, desde que hop_count e sequence_number continuem semanticamente coerentes
- politica de gerenciamento de peers do ESP-NOW
- mecanismos de ACK de aplicacao ou confirmacao fim a fim

Elementos que nos afastariam do AODV

As decisoes abaixo descaracterizam a proposta se virarem parte central do protocolo:

- remover RREQ e RREP do fluxo principal de descoberta
- remover numeros de sequencia
- usar source routing como mecanismo principal
- manter topologia global de forma proativa
- substituir tabela de rotas por flooding permanente
- usar apenas heurísticas locais sem semantica de rota reversa e direta

Interpretacao para o AODV-EN

O aodv-en deve ser entendido como:

- uma adaptacao do AODV ao ESP-NOW
- com tabelas locais de vizinhos e de rotas
- com descoberta reativa de rotas
- com manutencao hop-by-hop
- com numeros de sequencia
- com supressao de duplicatas
- com invalidacao temporal ou explicita de rotas
- com precursores por rota para emissao direcionada de RERR
- com fila local de DATA pendente durante descoberta

Compatibilidade com extensoes da v1

A v1 do AODV-EN adiciona dois mecanismos que precisam ser revisados contra os invariantes acima:

Fila de DATA pendente

A fila armazena um pacote da aplicacao enquanto o RREQ / RREP correspondente esta em curso. Ela:

- nao substitui a descoberta reativa (o RREQ continua sendo emitido)
- nao mantem topologia global, apenas pacotes pendentes do proprio no
- nao introduz mecanismo de roteamento alternativo

Por isso, e compativel com os invariantes 1 (reativo), 2 (descoberta por RREQ / RREP) e 8 (sem loop).

Precursores por rota

A v1 mantem, em cada rota direta valida, ate `MAX_PRECURSORS` MACs vizinhos que a usam. Esse conjunto e usado para:

- direcionar `RERR` apenas a quem precisa saber
- preservar a propriedade de `RERR` propagar de forma controlada para tras na arvore de descoberta

Precursores fazem parte da semantica original da RFC 3561 (secao 6.2). Adota-los aproxima o `AODV-EN` do AODV padrao em vez de afasta-lo, e nao introduz nem descoberta proativa nem topologia global.

Checklist de validacao

Antes de aceitar uma mudanca arquitetural, verificar:

- a descoberta continua sendo reativa?
- ainda existe `RREQ` ?
- ainda existe `RREP` ?
- as rotas continuam sendo `destino -> next_hop` ?
- os numeros de sequencia continuam presentes?
- `RREQ` duplicado continua sendo descartado?
- ainda existe rota reversa e rota direta?
- ainda existe expiracao ou erro de rota?

Se alguma resposta for "nao", a mudanca deve ser reavaliada.

Especificacao v1 (normativa)

AODV-EN Spec v1

Estado

- versao: 1.0
- estado: ATIVA
- supersede: [aodv-en-spec-v0.md](#)
- ultima revisao: 2026-05-01
- escopo: especificacao funcional fechada do protocolo AODV-EN para o TCC

Objetivo

Esta versao consolida o protocolo AODV-EN em um unico documento de referencia. Diferente da v0, que era um esboco de MVP, a v1 reflete o que esta realmente implementado em [firmware/components/aodv_en](#) e fixa as decisoes funcionais que estavam em aberto.

A intencao e que esta spec sirva como base direta para:

- o capitulo de proposta do TCC
- os criterios de aprovacao dos testes de bancada
- a definicao de o que e v1 e o que fica para versoes futuras

Tudo que nao estiver descrito aqui e considerado fora do escopo de v1.

Premissas

- todos os nos sao ESP32 com suporte a ESP-NOW v2
- todos os nos operam no mesmo canal Wi-Fi
- todos os nos compartilham o mesmo `network_id`
- a rede e ad hoc, sem coordenador central
- o protocolo de roteamento e reativo
- o transporte subjacente e ESP-NOW

Versao do protocolo no fio

- `protocol_version = 1`
- todo frame AODV-EN valido carrega esse valor no cabecalho
- frames com versao diferente devem ser descartados

Invariantes herdados

Esta spec preserva integralmente os invariantes definidos em [aodv-base-invariantes.md](#):

- roteamento reativo
- descoberta por `RREQ` e resposta por `RREP`
- roteamento hop-by-hop
- numeros de sequencia
- rota reversa e rota direta
- supressao de duplicatas de `RREQ`
- invalidacao de rotas
- ausencia de loop como objetivo de projeto

Qualquer mudanca futura que viole um desses invariantes deixa de ser `AODV-EN` e passa a ser outro protocolo.

Arquitetura em camadas

O `AODV-EN` e modelado em duas camadas claramente separadas, e isso e importante para o que e ou nao e parte do protocolo:

- camada de roteamento (`AODV-EN`)
- camada de aplicacao (qualquer payload da aplicacao final)

A camada de aplicacao do firmware de teste (com mensagens `HEALTH` , `TEXT` , `CMD` e CLI) e apenas um consumidor da camada de roteamento. Ela nao faz parte do `AODV-EN` e pode ser substituida por qualquer outra app sem afetar a spec.

Planos do protocolo

O `AODV-EN` opera em dois planos:

Plano de controle:

- `HELLO`
- `RREQ`
- `RREP`
- `RERR`
- `ACK`

Plano de dados:

- `DATA`

Tabelas locais

Cada no mantem seis estruturas residentes:

1. tabela de vizinhos
2. tabela de rotas (com precursores por rota)
3. cache de duplicatas de `RREQ`
4. cache de peers ESP-NOW
5. fila de `DATA` pendente
6. fila de `ACK` pendente

A definicao detalhada dos campos esta em [aodv-en-estruturas-dados.md](#). Esta spec descreve apenas o uso de cada estrutura no protocolo.

Mensagens

Todas as mensagens compartilham o mesmo cabeçalho comum.

Cabeçalho comum

- `protocol_version` (uint8)
- `message_type` (uint8)
- `flags` (uint8)
- `hop_count` (uint8)
- `network_id` (uint32)
- `sender_mac` (6 bytes)

`sender_mac` é o emissor do salto atual e é reescrito a cada repasse. Para identificar a origem fim a fim use `originator_mac` da carga específica.

Flags suportadas

- `ACK_REQUIRED` (`0x01`): aplicado em `DATA` quando o destino deve responder com `ACK`
- `ROUTE_REPAIR` (`0x02`): reservado para uso futuro de RREQ-repair; não consumido em v1

Tipos de mensagem

Tipo	Código	Plano	Direção	Estratégia
<code>HELLO</code>	0	controle	broadcast	periodico
<code>RREQ</code>	1	controle	broadcast	sob demanda
<code>RREP</code>	2	controle	unicast	resposta de descoberta
<code>RERR</code>	3	controle	unicast (precursores) ou broadcast (fallback)	sob falha
<code>DATA</code>	4	dados	unicast	sob demanda
<code>ACK</code>	5	controle	unicast	resposta de <code>DATA</code> com <code>ACK_REQUIRED</code>

HELLO

Campos além do cabeçalho:

- `node_mac`
- `node_seq_num`
- `timestamp_ms`

Regras:

- enviado periodicamente em broadcast por todos os nós
- intervalo padrão recomendado: `5000 ms`
- ao receber, o nó atualiza a tabela de vizinhos e instala uma rota direta de 1 salto válida para `node_mac`
- `HELLO` por si só não incrementa `self_seq_num` do nó emissor (o número divulgado é o atual)

RREQ

Campos além do cabeçalho:

- `originator_mac`

- `destination_mac`
- `originator_seq_num`
- `destination_seq_num`
- `rreq_id`
- `ttl`

Regras de emissao:

- a origem incrementa `self_seq_num` ao emitir um novo `RREQ`
- a origem incrementa `next_rreq_id` para gerar um identificador unico
- `destination_seq_num` e o ultimo conhecido pela origem; pode ser `0` se desconhecido
- `ttl` inicial usa `ttl_default` (16)

Regras de recepcao:

- se o par `(originator_mac, rreq_id)` estiver no cache de duplicatas, descartar e contabilizar `duplicate_rreq_drops`
- se nao estiver, registrar no cache e:
- instalar ou atualizar rota reversa para `originator_mac` no estado `REVERSE`
- se for o destino: emitir `RREP`
- caso contrario: repassar com `hop_count++` e `ttl--`, descartando se `ttl <= 1`

Politica de evicao do cache de duplicatas:

- entradas expiram naturalmente apos `rreq_cache_timeout_ms` (default `10000`)
- quando o cache esta cheio e uma entrada nova precisa ser inserida, a mais antiga por `created_at_ms` e substituida
- isso prefere descartar memoria de descobertas antigas em vez de bloquear novas, e evita deadlock em janelas de stress

RREP

Campos alem do cabecalho:

- `originator_mac` (origem da descoberta original)
- `destination_mac` (destino que esta respondendo)
- `destination_seq_num`
- `lifetime_ms`

Regras de emissao pelo destino (RFC 3561, secao 6.6.1):

- se `requested_dest_seq > self_seq_num` da do destino: `self_seq_num = requested_dest_seq`
- se `requested_dest_seq == self_seq_num`: `self_seq_num++`
- caso contrario: manter `self_seq_num`
- emitir `RREP` para o `next_hop` da rota reversa para `originator_mac`

Regras de repasse intermediario:

- so repassa quando existe rota reversa para `originator_mac`
- instala rota direta para `destination_mac` no estado `VALID`, com lifetime `lifetime_ms`
- atualiza precursores conforme RFC 3561, secao 6.6.2:
- vizinho de quem recebeu o `RREP` e precursor para `destination_mac`
- vizinho para quem repassa o `RREP` tambem e precursor para `destination_mac`

- vizinho de quem recebeu o `RREP` e precursor para `originator_mac` na rota reversa
- ao instalar rota direta valida, drena fila de `DATA` pendente para esse destino

RERR

Campos alem do cabecalho:

- `unreachable_destination_mac`
- `unreachable_dest_seq_num`

Regras de emissao:

- emitido quando ocorre falha de transmissao para um `next_hop` ou quando `DATA` chega sem rota valida
- contem exatamente um destino inalcançavel (decisao de v1; multiplos destinos ficam para versoes futuras)
- proximo passo logico:
- se a rota tem precursores: enviar `RERR` em unicast para cada precursor, exceto para o `next_hop` quebrado
- se a rota nao tem precursores: enviar `RERR` em broadcast como fallback

Regras de recepcao:

- invalida rota local para `unreachable_destination_mac`
- propaga `RERR` recursivamente para os precursores da rota invalidada, exceto para o emissor recebido

ACK

Campos alem do cabecalho:

- `originator_mac` (no que originou o `ACK`, ou seja, o destino do `DATA`)
- `destination_mac` (alvo do `ACK`, ou seja, a origem do `DATA`)
- `ack_for_sequence`

Regras:

- emitido pelo destino quando recebe `DATA` com flag `ACK_REQUIRED`
- segue a tabela de rotas como qualquer trafego
- a origem usa `ack_for_sequence` para casar com a entrada em `pending_ack` e cancelar retransmissao

DATA

Campos alem do cabecalho:

- `originator_mac`
- `destination_mac`
- `sequence_number`
- `ttl`
- `payload_length`
- `payload[payload_length]`

Regras:

- a origem aloca um `sequence_number` monotonicos
- consulta a tabela de rotas; se a rota nao for valida, segue o fluxo de fila pendente (ver secao "Fila de DATA pendente")
- intermediarios decrementam `ttl` e atualizam `sender_mac`
- se `ttl <= 1` no encaminhamento, descartar e emitir `RERR`

- se não houver rota válida no encaminhamento, descartar e emitir `RERR`
- o destino entrega o payload ao callback de aplicação e, se a flag `ACK_REQUIRED` estiver setada, responde com `ACK`

Numeros de sequencia

O protocolo mantém dois contadores monotonicos por nó:

- `self_seq_num`: número de sequência de roteamento do próprio nó (usado em `HELLO`, `RREQ` como originator e em `RREP` como destination)
- `next_rreq_id`: identificador único de `RREQ` emitidos pelo nó (usado para deduplicação)

Regras mínimas de v1:

- `self_seq_num` só cresce
- e incrementado:
- ao emitir um novo `RREQ` (a origem)
- ao responder a um `RREQ` cujo `destination_seq_num` já seja maior ou igual ao `self_seq_num` do destino, conforme RFC 3561 seção 6.6.1
- `next_rreq_id` só cresce e é usado para chave `(originator, rreq_id)` no cache de duplicatas

Comparação de frescor:

- a comparação em v1 é por `>` simples sobre `uint32_t`
- isso é suficiente para a janela de tempo dos cenários do TCC; tratamento de wrap-around (lógica `(a - b) < 2^31`) fica como ponto de evolução futura

Seleção e substituição de rotas

O upsert de rota usa um critério de "substituir somente se a candidata for melhor", com histerese para evitar flapping entre `next_hops`:

Ordem de prioridade:

1. estado: candidata `VALID` substitui `INVALID` ou `REVERSE`
2. histerese: se ambas são `VALID` mas o `next_hop` mudou, exigir ganho forte em pelo menos um destes: -
`seq_gain >= ROUTE_SWITCH_MIN_SEQ_GAIN` - `metric_gain >= ROUTE_SWITCH_MIN_METRIC_GAIN` - `hop_gain >= ROUTE_SWITCH_MIN_HOP_GAIN` - `lifetime_gain >= ROUTE_SWITCH_MIN_LIFETIME_GAIN_MS`
3. `dest_seq_num` maior vence
4. com `dest_seq_num` igual: `metric` menor vence
5. com `metric` igual: `hop_count` menor vence
6. com tudo igual: maior `expires_at_ms` vence

Quando o `next_hop` é o mesmo (refresh de rota), os precursores existentes são preservados. Quando o `next_hop` muda, os precursores são zerados.

Métrica de roteamento em v1

- `metric = hop_count` para todas as rotas instaladas em v1
- `RSSI` é mantido na tabela de vizinhos para diagnóstico e para critérios futuros de v2, mas não entra no cálculo de `metric`
- valor especial `A0DV_EN_ROUTE_METRIC_INFINITY` (`0xFFFF`) marca rotas invalidadas

Decisao explicita de v1: nao adotar metrica composta com `RSSI`. Esse experimento fica para uma versao futura para nao misturar duas variaveis quando comparar com o baseline de flooding.

Estados de rota

- `INVALID`: rota conhecida, mas nao usavel; nao gera trafego, podera ser substituida por candidata `VALID`
- `REVERSE`: rota construida durante recepcao de `RREQ`, valida para servir como caminho de volta de `RREP` e `ACK`, mas nao para `DATA` direto
- `VALID`: rota direta utilizavel para `DATA`

Regra de uso para repasse de `DATA`: somente rotas `VALID` e nao expiradas.

Expiracao

Sao expirados pelo tick periodico do no:

- vizinhos: por `neighbor_timeout_ms` (default `15000`) sem serem ouvidos
- rotas: por `expires_at_ms` ja passado, com remocao do registro
- entradas do cache de `RREQ`: por `rreq_cache_timeout_ms` (default `10000`)
- entradas pendentes de `DATA`: por `route_lifetime_ms` desde o `enqueued_at_ms`
- entradas pendentes de `ACK`: por `retries_left = 0`

Quando uma rota usada por uma fila pendente expira ou e invalidada, o no nao precisa esperar a janela total do retry: ao detectar quebra de `next_hop` o protocolo dispara descoberta nova para os destinos com pendencia (`aodv_en_node_trigger_discovery_for_pending_destinations`).

Fila de DATA pendente

A fila de pendentes e parte de v1 (ver tambem [features/enfileamento-dos-dados.md](#)).

Comportamento:

1. ao chamar `send_data` sem rota valida ou com rota expirada: - o `DATA` e enfileirado em `pending_data` - um `RREQ` e emitido para `destination_mac` - o status retornado e `AODV_EN_QUEUED`
2. ao instalar uma rota valida para um destino com pendencia: - todos os itens daquele destino sao drenados em ordem
3. no tick do no, para destinos com pendencia ainda sem rota: - o `RREQ` e reemitido respeitando backoff exponencial sobre `ack_timeout_ms`, com teto de `10000 ms`
4. expiracao: - se `now_ms - enqueued_at_ms >= route_lifetime_ms`, item descartado e contabilizado em `pending_data_dropped`

Backpressure quando a fila esta cheia:

- preferencia por reciclar a entrada mais antiga do mesmo destino (preserva diversidade entre destinos)
- caso nao exista, recicla a mais antiga global
- a entrada substituida e contabilizada como `pending_data_dropped`

Confiabilidade fim a fim

O `send_callback` do ESP-NOW so confirma entrega de 1 salto. Ele nao e suficiente para a aplicacao saber se o `DATA` chegou ao destino final.

Por isso o `AODV-EN` mantem `ACK` fim a fim no plano de controle:

- `ACK` e opt-in via flag `ACK_REQUIRED` em `DATA`
- a origem rastreia `pending_ack` por `(destination_mac, sequence_number)`
- timeout: `ack_timeout_ms` (default `1000 ms`)
- retries: limite compartilhado com `rreq_retry_count` (default `3`); apos esgotar, descarta com `ack_timeout_drops`
- se na hora do retry a rota voltou a ser invalida, o no dispara novo `RREQ` antes de retransmitir

Limitacao reconhecida: o orcamento de retentativas e o mesmo para `RREQ` e `ACK`. Separar essas duas politicas e um trabalho de evolucao.

Manutencao de vizinhanca

A vizinhanca e mantida por duas vias complementares:

1. observacao passiva: qualquer `on_recv` valido faz `neighbor_touch` no `link_src_mac` com `RSSI` real
2. anuncio ativo: `HELLO` periodico em broadcast

Em v1, ambos sao usados. `HELLO` ativo facilita reconvergencia em cadeia, especialmente em TC-002, TC-003 e TC-004.

Suavizacao de `RSSI`:

- `last_rssi` registra o ultimo valor visto
- `avg_rssi` aplica EMA com peso fixo: $avg = (3 * avg_anterior + rssi_atual) / 4$
- quando o vizinho vem de `INACTIVE`, o EMA e re-inicializado com o `rssi` atual em vez de continuar do valor antigo

Esses dois campos sao usados apenas para diagnostico em v1. Nenhuma decisao de roteamento depende de `RSSI` em v1 (ver "Metrica de roteamento em v1").

Quando uma transmissao unicast falha repetidamente para um vizinho:

- `link_fail_count` incrementa em `aodv_en_neighbor_note_link_failure`
- ao atingir `link_fail_threshold` (default `3`) o vizinho vai para `INACTIVE`
- todas as rotas que dependem desse `next_hop` sao invalidadas em massa
- precursores de cada rota invalidada recebem `RERR`

TTL e limites de tamanho

- `TTL` inicial: `16`
- `MAX_HOPS`: `16`
- payload de controle maximo: `128 bytes`
- payload de dados maximo: `1024 bytes`
- frames com tamanho incompativel com o tipo declarado sao descartados em `aodv_en_validate_message_size`

Adapter de transporte

A camada de roteamento nao conhece ESP-NOW diretamente. Ela depende de duas funcoes injetadas pelo adapter:

- `now_ms(user_ctx) -> uint32_t`: fonte de tempo monotonica em milissegundos
- `tx_frame(user_ctx, next_hop, frame, frame_len, broadcast) -> aodv_en_status_t`: envia um frame para 1 salto, broadcast ou unicast

Esse contrato esta declarado em [aodv_en.h](#) e e o que permite que a mesma logica rode na simulacao local em C e no firmware ESP-IDF sem alterar o nucleo.

Configuracao por no

Todos os timers, tamanhos de tabela e thresholds sao expostos em `aodv_en_config_t` ([firmware/components/aodv_en/include/aodv_en_types.h](#)). Os defaults dessa configuracao estao em [aodv_en_limits.h](#):

Constante	Default
<code>NEIGHBOR_TIMEOUT_MS</code>	15000
<code>ROUTE_LIFETIME_MS</code>	30000
<code>RREQ_CACHE_TIMEOUT_MS</code>	10000
<code>ACK_TIMEOUT_MS</code>	1000
<code>RREQ_RETRY_COUNT</code>	3
<code>LINK_FAIL_THRESHOLD</code>	3
<code>MAX_HOPS</code>	16
<code>TTL_DEFAULT</code>	16
<code>NEIGHBOR_TABLE_SIZE</code>	16
<code>ROUTE_TABLE_SIZE</code>	32
<code>RREQ_CACHE_SIZE</code>	64
<code>PEER_CACHE_SIZE</code>	8
<code>PENDING_DATA_QUEUE_SIZE</code>	4
<code>MAX_PRECURSORS</code> (por rota)	4
<code>CONTROL_PAYLOAD_MAX</code>	128
<code>DATA_PAYLOAD_MAX</code>	1024

Esses valores sao calibracao inicial. Cenarios de stress test podem reajusta-los, e a configuracao por no e independente.

Telemetria minima

A v1 expoe, via `aodv_en_stack_stats_t` em [aodv_en.h](#):

- `rx_frames`, `tx_frames`, `forwarded_frames`, `delivered_frames`
- `ack_received`, `ack_retry_sent`, `ack_timeout_drops`
- `route_discoveries`, `route_repairs`, `route_discovery_retries`
- `duplicate_rreq_drops`
- `pending_data_queued`, `pending_data_flushed`, `pending_data_dropped`
- `link_fail_events`, `route_invalidations_link_fail`

Esses contadores sao a base mensuravel para os experimentos da Fase 8 do plano de desenvolvimento.

Escopo de v1

Esta dentro de v1:

- HELLO, RREQ, RREP, RERR, DATA, ACK
- tabela de vizinhos com RSSI e estado ativo/inativo
- tabela de rotas com estados INVALID, REVERSE, VALID e precursores
- cache de duplicatas de RREQ
- cache de peers ESP-NOW
- fila de DATA pendente com retry de descoberta e expiracao
- fila de ACK pendente com retry e timeout
- regra de troca de rota com histerese
- propagacao de RERR para precursores
- adapter de transporte injetavel
- telemetria minima

Esta fora de v1:

- metrica composta com RSSI
- multiplos destinos por RERR
- detecao de articulation point (ver [features/articulation-point-planejado.md](#))
- separacao do orcamento de retry de RREQ e ACK
- tratamento explicito de wrap-around de numero de sequencia
- power saving e duty-cycle adaptativo
- fragmentacao de payload alem de DATA_PAYLOAD_MAX

Critérios de aprovacao da v1

Consideramos AODV-EN v1 aceito quando todos os criterios abaixo forem demonstrados em hardware e/ou simulador:

1. Em 2 nos com enlace direto: - HELLO descobre vizinhanca em ate 30 s - rota direta state=VALID, hops=1 instalada - DATA entregue e ACK recebido na origem
2. Em cadeia de 3 nos A-B-C sem enlace direto A<->C : - rota A->C com hops=2 apos descoberta - DATA chega em C - ACK chega em A
3. Reconvergencia em cadeia de 3 nos: - apos quebra do enlace intermediario, a rota VALID e invalidada por link_fail_threshold ou expiracao - apos retorno do enlace, nova descoberta restaura entrega em ate 30 s
4. Soak de 30 minutos com ciclos repetidos de degradacao e recuperacao: - sem reboot - cada ciclo termina com delivered_frames e ack_received voltando a crescer
5. Cache de duplicatas: - duplicate_rreq_drops > 0 em qualquer cenario com mais de 2 nos sob descoberta concorrente
6. Expiracao de rota: - rotas invalidadas saem da tabela apos route_lifetime_ms
7. RERR para precursores: - quando o caminho central cai, os precursores recebem RERR (verificavel por log)
8. Fila pendente: - DATA enviado antes da rota retorna AODV_EN_QUEUED e e drenado apos RREP

Os casos de teste correspondentes estao em [tests](#):

- TC-001 cobre 1
- TC-002 cobre 2 e 5
- TC-003 cobre 3 e 7

- `TC-004` cobre 4 e 6
- a fila pendente esta coberta pela simulacao em [sim/aodv_en_sim.c](#)

Caminho de evolucao para v2

Itens de v2 ja mapeados, sem compromisso de prazo:

- detecao e telemetria de articulation point
- metrica composta `hop_count + RSSI` com hysteresis explicita
- `RREP-ACK` opcional (RFC 3561 secao 5.7)
- repair local de rota (`RREQ` com flag `R`)
- compactacao de `RERR` com lista de destinos
- politica de retry separada para `RREQ` e `ACK`
- power saving cooperativo entre vizinhos

Referencias internas

- [aodv-en-funcionamento.md](#) - guia didatico com layout de bytes, traces e worked examples
- [aodv-base-invariantes.md](#) - regras que nao podem ser quebradas
- [aodv-en-estruturas-dados.md](#) - layout das tabelas e mensagens
- [features/enfileamento-dos-dados.md](#) - fila pendente
- [features/precursores.md](#) - precursores e RERR direcionado
- [features/articulation-point-planejado.md](#) - feature de v2
- [plano-desenvolvimento-completo.md](#) - roadmap geral
- [tests](#) - casos de teste de bancada

Funcionamento Completo do Protocolo

AODV-EN - Funcionamento Completo do Protocolo

Estado

- alinhado com [aodv-en-spec-v1.md](#)
- ultima revisao: 2026-05-01
- objetivo deste documento: explicar **como** o protocolo funciona na pratica, mostrando fluxos ponta a ponta, com layout de bytes, traces de execucao e exemplos completos para 2, 3 e 4 nos.

Este documento e um guia didatico. Para a definicao normativa do protocolo use a [spec v1](#). Para os invariantes do AODV preservados pelo projeto, ver [aodv-base-invariantes.md](#).

Sumario

1. [Visao em uma pagina](#)
 2. [Camadas e contratos](#)
 3. [Mensagens no fio](#)
 4. [Estado interno por no](#)
 5. [Numeros de sequencia](#)
 6. [Fluxos do protocolo \(com exemplos\)](#)
 7. [Selecao e substituicao de rotas](#)
 8. [Filas pendentes](#)
 9. [Falhas, RERR e precursores](#)
 10. [Worked example A-B-C-D](#)
 11. [Timers, limites e calibragem](#)
 12. [Telemetria e diagnostico](#)
 13. [Limites conhecidos](#)
 14. [Referencia rapida](#)
-

Visao em uma pagina

`AODV-EN` e uma adaptacao do `AODV` (RFC 3561) para redes mesh multi-hop sobre `ESP-NOW v2` em `ESP32`. Cada no:

- mantem **tabelas locais** (vizinhos, rotas, cache de `RREQ`, peers `ESP-NOW`, fila de `DATA` pendente, fila de `ACK` pendente);
- descobre rotas **sob demanda** (`RREQ` em broadcast, `RREP` em unicast pelo caminho reverso);
- entrega `DATA` **hop-by-hop** seguindo a tabela de rotas;
- responde `DATA` com `ACK` fim a fim quando a flag `ACK_REQUIRED` esta marcada;
- recupera de falhas com `RERR` direcionado a **precursores** (RFC 3561 secao 6.2).

Os componentes são desacoplados via um `adapter` injetável:

```
APP      +-----+      +-----+
----- on_data, on_ack --> |      | ESP-NOW |
|      AODV-EN      | <--> |      ou MOCK SIM      |
<-- send_data, send_hello -- |      |
+-----+      +-----+
      ^      ^
      |      |
      now_ms()  tx_frame(next_hop, frame, len, broadcast)
```

A mesma lógica roda no firmware ESP32 e no simulador local em [sim/aodv_en_sim.c](#).

Camadas e contratos

O código em [firmware/components/aodv_en](#) está dividido em três camadas claras:

1. Núcleo do protocolo (`aodv_en_node.*`)

Implementa a máquina de estado do `AODV-EN`. É **agnóstico de transporte**. Suas dependências externas são apenas duas funções que você passa por callback:

```
typedef aodv_en_status_t (*aodv_en_emit_frame_fn)(
    void *user_ctx,
    const uint8_t next_hop[AODV_EN_MAC_ADDR_LEN],
    const uint8_t *frame,
    size_t frame_len,
    bool broadcast);

typedef void (*aodv_en_deliver_data_fn)(
    void *user_ctx,
    const uint8_t originator_mac[AODV_EN_MAC_ADDR_LEN],
    const uint8_t *payload,
    uint16_t payload_len);
```

Mais um callback opcional `ack_received` para a aplicação saber quando um `ACK` chega.

2. Fachada (`aodv_en.*`)

Camada `aodv_en_stack.*` que a aplicação consome. Adiciona:

- alocação do `aodv_en_node_t` no heap (uma vez na init);
- contrato de adapter limpo (`now_ms`, `tx_frame`);
- versões "no_at" das funções que chamam `now_ms` automaticamente;
- snapshot de rotas e estatísticas em formato leitura-pública (`aodv_en_route_snapshot_t`).

```
aodv_en_status_t aodv_en_stack_init(
    aodv_en_stack_t *stack,
    const aodv_en_config_t *config,
    const uint8_t self_mac[AODV_EN_MAC_ADDR_LEN],
    const aodv_en_adapter_t *adapter,
    const aodv_en_app_callbacks_t *app_callbacks);

aodv_en_status_t aodv_en_stack_send_data(
    aodv_en_stack_t *stack,
    const uint8_t destination_mac[AODV_EN_MAC_ADDR_LEN],
    const uint8_t *payload,
    uint16_t payload_len,
    bool ack_required);
```

3. Adapter ESP-NOW (na app, atualmente em `app_proto_example.c` e `app_demo.c`)

Implementa as funcoes `tx_frame` e `now_ms` para `ESP-NOW / esp_timer_get_time`, processa o `recv_cb` e o `send_cb` do `ESP-NOW` e empurra para o nucleo.

A separacao entre app e adapter ainda nao esta 100% pronta - ver Fase 4 do [plano](#). Mas o contrato ja esta limpo o suficiente para portar para outros transportes (UART, BLE Mesh, simulacao em desktop, etc.).

Mensagens no fio

Todas as mensagens compartilham um cabecalho comum de **14 bytes** packed (`__attribute__((packed))`).

Cabecalho comum (14 bytes)

Offset	Tamanho	Campo	Tipo	Notas
0	1	<code>protocol_version</code>	uint8	sempre <code>1</code> em v1
1	1	<code>message_type</code>	uint8	<code>0=HELLO 1=RREQ 2=RREP 3=RERR 4=DATA 5=ACK</code>
2	1	<code>flags</code>	uint8	<code>0x01=ACK_REQUIRED</code> (em DATA), <code>0x02=ROUTE_REPAIR</code> (reservado)
3	1	<code>hop_count</code>	uint8	salto atual; incrementado a cada repasse
4-7	4	<code>network_id</code>	uint32 LE	identidade da malha
8-13	6	<code>sender_mac</code>	bytes	quem acabou de transmitir (1 hop atras)

`sender_mac` e reescrito a cada repasse. Para o emissor original do dado fim a fim, use `originator_mac` (presente em RREQ, RREP, DATA, ACK).

HELLO (28 bytes packed)

Anuncio periodico de presenca, em broadcast.

Offset	Tamanho	Campo
0-13	14	header (<code>message_type=0</code>)
14-19	6	<code>node_mac</code>
20-23	4	<code>node_seq_num</code>
24-27	4	<code>timestamp_ms</code>

Exemplo de bytes (NODE_A `28:05:A5:33:D6:1C`, network `0xA0DE0001`, seq `5`, ts `12345 ms`):

```
01 00 00 00 01 00 DE A0  protocol=1 type=0(HELLO) flags=0 hop=0 network=0xA0DE0001
28 05 A5 33 D6 1C      sender_mac
28 05 A5 33 D6 1C      node_mac (igual ao sender quando origem)
05 00 00 00            node_seq_num = 5
39 30 00 00            timestamp_ms = 12345
```

RREQ (39 bytes packed)

Pedido de descoberta de rota, em broadcast.

Offset	Tamanho	Campo
0-13	14	header (message_type=1)
14-19	6	originator_mac (origem da descoberta)
20-25	6	destination_mac (destino procurado)
26-29	4	originator_seq_num
30-33	4	destination_seq_num (ultimo conhecido pela origem; 0 se desconhecido)
34-37	4	rreq_id (deduplicacao com originator_mac)
38	1	ttl

A par chave de deduplicacao e (originator_mac, rreq_id) . Cada no incrementa hop_count no header e decrementa ttl antes de repassar.

RREP (34 bytes packed)

Resposta a RREQ , unicast pelo caminho reverso.

Offset	Tamanho	Campo
0-13	14	header (message_type=2)
14-19	6	originator_mac (quem fez o RREQ original)
20-25	6	destination_mac (quem responde)
26-29	4	destination_seq_num
30-33	4	lifetime_ms

RERR (24 bytes packed)

Sinalizacao de quebra de rota, unicast aos precursores ou broadcast como fallback.

Offset	Tamanho	Campo
0-13	14	header (message_type=3)
14-19	6	unreachable_destination_mac
20-23	4	unreachable_dest_seq_num

A v1 carrega exatamente um destino por RERR . Se varias rotas sao invalidadas pela mesma falha, varios RERR sao emitidos em sequencia.

ACK (30 bytes packed)

Confirmacao fim a fim de DATA com ACK_REQUIRED .

Offset	Tamanho	Campo
0-13	14	header (message_type=5)
14-19	6	originator_mac (no que origina o ACK = destino do DATA)
20-25	6	destination_mac (alvo do ACK = origem do DATA)
26-29	4	ack_for_sequence

DATA (33 bytes header + payload)

Carga de aplicacao, unicast pela rota instalada.

Offset	Tamanho	Campo
0-13	14	header (message_type=4)
14-19	6	originator_mac
20-25	6	destination_mac
26-29	4	sequence_number
30	1	ttl
31-32	2	payload_length (uint16 LE)
33+	n	payload (bytes da aplicacao)

`payload_length` permite que o receptor valide o tamanho. O parser do nucleo descarta frames com `payload_length + sizeof(header) != frame_len`.

Sobre limites: `AODV_EN_DATA_PAYLOAD_MAX = 1024` bytes. O ESP-NOW v2 suporta ate 1490 bytes de payload por frame, entao um `DATA` cheio ($33 + 1024 = 1057$ bytes) cabe sem problemas.

Estado interno por no

Cada no mantem seis estruturas residentes, todas alocadas estaticamente dentro de `aodv_en_node_t` ([firmware/components/aodv_en/include/aodv_en_node.h](#)):

1. Tabela de vizinhos (`aodv_en_neighbor_table_t`)

Quem foi visto a 1 salto, com qualidade de enlace.

```
typedef struct {
    uint8_t mac[6];
    int8_t avg_rssi;        // EMA: (3*old + new)/4
    int8_t last_rssi;       // ultimo valor
    uint8_t link_fail_count;
    uint8_t state;          // 0=INACTIVE, 1=ACTIVE
    uint32_t last_seen_ms;
    uint32_t last_used_ms;
} aodv_en_neighbor_entry_t;
```

Capacidade default: `16` vizinhos. Substituicao: nao tem - quando lota, novos vizinhos retornam `AODV_EN_ERR_FULL` ate que outro expire.

2. Tabela de rotas (`aodv_en_route_table_t`)

Caminhos `destination -> next_hop`, com **precursores** (RFC 3561 secao 6.2):

```
typedef struct {
    uint8_t destination[6];
    uint8_t next_hop[6];
    uint32_t dest_seq_num;
    uint32_t expires_at_ms;
    uint16_t metric;           // em v1, igual a hop_count; 0xFFFF = INFINITY
    uint8_t hop_count;
    uint8_t state;            // 0=INVALID, 1=REVERSE, 2=VALID
    uint8_t precursor_count;
    uint8_t precursors[4][6]; // ate 4 precursores por rota
} aadv_en_route_entry_t;
```

Estados: - **INVALID** : rota conhecida mas nao usavel; pode ser substituida. - **REVERSE** : instalada durante recepcao de **RREQ** ; valida apenas para **RREP** / **ACK** voltarem. - **VALID** : pronta para **DATA** .

Capacidade default: **32** rotas.

3. Cache de duplicatas de RREQ (`aadv_en_rreq_cache_t`)

Suprime processamento repetido de inundacoes:

```
typedef struct {
    uint8_t originator[6];
    uint32_t rreq_id;
    uint32_t created_at_ms;
    uint8_t hop_count;
    uint8_t used;
} aadv_en_rreq_cache_entry_t;
```

Capacidade default: **64** . **Politica de evicao**: se cheio e chega novo **RREQ** , o mais antigo por `created_at_ms` e substituido. Expira por `rreq_cache_timeout_ms` (default **10000 ms**).

4. Cache de peers ESP-NOW (`aadv_en_peer_cache_t`)

Liga o `next_hop` da tabela de rotas com a lista de peers registrados no driver **ESP-NOW** . Politica **LRU** que pula entradas com flag **PINNED** . Capacidade default: **8** .

5. Fila de DATA pendente (`aadv_en_pending_data_entry_t[4]`)

Buffer para **DATA** enquanto a rota nao chega:

```
typedef struct {
    uint8_t destination_mac[6];
    uint16_t payload_len;
    bool ack_required;
    bool used;
    uint32_t enqueued_at_ms;
    uint32_t last_rreq_at_ms;
    uint8_t discovery_attempts;
    uint8_t payload[1024];
} aadv_en_pending_data_entry_t;
```

Capacidade default: **4** itens. **Backpressure**: quando cheia, recicla a entrada mais antiga do mesmo destino primeiro (preserva diversidade entre destinos).

6. Fila de ACK pendente (`aadv_en_pending_ack_entry_t[4]`)

Rastreio de **DATA** com **ACK_REQUIRED** aguardando confirmacao para retransmissao:

```
typedef struct {
    uint8_t destination_mac[6];
    uint16_t payload_len;
    bool used;
    uint8_t retries_left;
    uint32_t sequence_number;
    uint32_t last_sent_at_ms;
    uint8_t payload[1024];
} aadv_en_pending_ack_entry_t;
```

`retries_left` herda `rreq_retry_count` (default 3). Timeout por entrada: `ack_timeout_ms` (default 1000 ms).

Configuracao e estatisticas

Toda a configuracao expostavel esta em `aadv_en_config_t`. Os contadores cumulativos para diagnostico/experimentos estao em `aadv_en_stack_stats_t`:

```
rx_frames, tx_frames, forwarded_frames, delivered_frames
ack_received, ack_retry_sent, ack_timeout_drops
route_discoveries, route_repairs, route_discovery_retries
duplicate_rreq_drops
pending_data_queued, pending_data_flushed, pending_data_dropped
link_fail_events, route_invalidations_link_fail
```

Numeros de sequencia

O protocolo usa numeros de sequencia para definir **frescor** da informacao de roteamento. Sao monotonicos crescentes por no.

Tres contadores

Contador	Onde fica	Quando incrementa
<code>self_seq_num</code>	<code>aadv_en_node_t.self_seq_num</code>	ao emitir <code>RREQ</code> (origem); ao responder <code>RREP</code> se <code>req_dest_seq_num</code> $\geq$ <code>self_seq_num</code>
<code>next_rreq_id</code>	<code>aadv_en_node_t.next_rreq_id</code>	a cada <code>RREQ</code> emitido
<code>next_data_seq</code>	<code>aadv_en_node_t.next_data_seq</code>	a cada <code>DATA</code> emitido

Regras RFC 3561 §6.6.1 (implementadas em `aadv_en_node_send_rrep`)

Quando um no e o destino e responde a um `RREQ`:

```
if (rreq_dest_seq_num > node->self_seq_num) {
    node->self_seq_num = rreq_dest_seq_num;
} else if (rreq_dest_seq_num == node->self_seq_num) {
    node->self_seq_num++;
}
// caso contrario: mantem self_seq_num
```

Isso garante que o `RREP` carregue um numero de sequencia **estritamente maior ou igual** ao que a origem ja conhece, alinhando com o invariante "rotas mais recentes vencem".

Comparacao de frescor

Em `aadv_en_route_should_replace`, comparamos `dest_seq_num` por `>` simples sobre `uint32_t`. Isso e suficiente para a janela de tempo dos cenarios do TCC. Tratamento explicito de wrap-around (logica `(a - b) < 2^31`) fica mapeado para v2.

Fluxos do protocolo (com exemplos)

A partir daqui, cada secao mostra:

1. **diagrama de sequencia ASCII**;
2. **estado das tabelas** antes e depois;
3. **trace** do que aparece nos logs reais.

Os exemplos usam MACs do TC-002:

```
NODE_A = 28:05:A5:33:D6:1C
NODE_B = 28:05:A5:33:EB:80
NODE_C = 28:05:A5:34:99:34
NODE_D = 28:05:A5:34:AA:AA (TC-005 apenas)
```

5.1 HELLO + descoberta passiva de vizinhos

`HELLO` e o anuncio mais simples. Vai em broadcast a cada `hello_interval_ms` (default `5000 ms`, configuravel pelo `Kconfig`).

```
t=0ms      NODE_A      NODE_B      NODE_C
           |           |           |
           |--- HELLO ---->|<--- alcance ---|
           |           |           |
           |           |   direto   |
           | (A nao ouve C)         |
           |           |           |
t=1000ms   on_rcv      on_rcv
           touch B (rssi=-50) touch B
                           touch nothing
```

Comportamento ao receber `HELLO` (`aadv_en_node_handle_hello`)

```
// Ao receber HELLO de mac=X:
1. neighbor_touch(X, rssi)           // EMA do RSSI, last_seen=now
2. route_upsert({
    destination = node_mac_no_HELLO, // emissor do HELLO
    next_hop    = link_src_mac,       // mesma coisa, pois e 1 hop
    dest_seq_num = node_seq_num,
    hop_count = 1, metric = 1,
    state = VALID
})
```

Isso cria uma rota direta de 1 salto sem precisar passar por `RREQ`.

Estado depois de `NODE_A` receber `HELLO` de `NODE_B`

Tabela de vizinhos de `NODE_A`:

mac	avg_rssi	last_seen_ms	state
28:05:A5:33:EB:80	-50	1000	ACTIVE

Tabela de rotas de NODE_A:

dest	via	hops	metric	state	expires
28:05:A5:33:EB:80	28:05:A5:33:EB:80	1	1	VALID	31000

Trace de log

```
I (1023) aadv_en_proto: routes=1 neighbors=1 tx=1 rx=1 delivered=0
I (1024) aadv_en_proto: route[0] dest=28:05:A5:33:EB:80 via=28:05:A5:33:EB:80 hops=1 metric=1 state=2 expires=31000
```

Alem do `HELLO` ativo, o nucleo tambem usa **observacao passiva**: qualquer `on_rcv` valido faz `neighbor_touch` no `link_src_mac` com o `RSSI` real. Entao mesmo um `RREQ` / `RREP` / `DATA` recebido atualiza a vizinhanca.

5.2 Descoberta de rota: RREQ -> RREP

Cenario classico: `NODE_A` quer enviar `DATA` para `NODE_C`, mas nao tem rota. `B` esta no meio.

```

NODE_A      NODE_B      NODE_C
|           |           |
| send_data()-->         |
| (sem rota)            |
|                       |
| enqueue pending       |
|                       |
| ---- RREQ ----> (broadcast)
|                   |
|                   | hop=0
|                   | rreq_id=1
|                   | orig_seq=1
|                   |
|                   | install rev_route
|                   | A via A, hop=1
|                   |
|                   | ---- RREQ ----> (broadcast forward)
|                   | hop=1
|                   |
|                   | install rev_route
|                   | A via B, hop=2
|                   |
|                   | C is dest!
|                   | self_seq=max(req_dst_seq, self_seq)+1
|                   |
|                   | <---- RREP ----> (unicast to B,
|                   | dest_seq=2       | via reverse_route)
|                   |
|                   | install dir_route
|                   | C via C, hop=1
|                   | add precursor A
|                   |
| <-- RREP ---->
| hop=1
|
| install dir_route
| C via B, hop=2
| flush_pending_data() --> envia DATA
```

Layout dos frames (resumo)

`RREQ` emitido por A (broadcast):

```
header:      protocol=1, type=1(RREQ), flags=0, hop_count=0, net=0xA0DE0001
sender_mac:  A
originator:  A
destination: C
orig_seq_num: 1
dest_seq_num: 0    (A nao conhecia C)
rreq_id:     1
ttl:         16
```

Ao chegar em B, ele: 1. consulta cache de **RREQ** : nao encontra **(A, 1)** , registra agora; 2. instala rota reversa para A: **next_hop=A, hops=1, state=REVERSE** ; 3. nao e o destino, entao reencaminha:

RREQ reencaminhado por B (broadcast):

```
header:      ... hop_count=1 ...
sender_mac:  B      <- reescrito por B
(originator, destination, orig_seq_num, dest_seq_num, rreq_id intactos)
ttl:         15      <- decrementado
```

C recebe o **RREQ** : 1. cache nao tem **(A, 1)** (C tambem nao viu antes); 2. instala rota reversa para A: **next_hop=B, hops=2, state=REVERSE** ; 3. **e o destino** -> emite **RREP** .

RREP emitido por C (unicast para B):

```
header:      protocol=1, type=2(RREP), flags=0, hop_count=0, net=...
sender_mac:  C
originator:  A      <- a origem da descoberta
destination: C      <- quem responde
dest_seq_num: 2      <- novo self_seq de C
lifetime_ms: 30000
```

C envia para **next_hop** da rota reversa para A, que e B.

B recebe o **RREP** : 1. instala rota direta para C: **next_hop=C, hops=1, state=VALID, dest_seq_num=2, lifetime=30000** ; 2. **adiciona precusores** (RFC 3561 secao 6.6.2): - vizinho de quem recebeu o RREP (= C) e precursor de C - na pratica, **route(C).precursors += [C]** ; - vizinho para quem repassa o RREP (= A) e precursor de C - **route(C).precursors += [A]** ; - vizinho de quem recebeu o RREP (= C) tambem e precursor da rota reversa para A - **route(A).precursors += [C]** ; 3. nao e a origem, repassa o RREP para A.

A recebe o **RREP** : 1. instala rota direta para C: **next_hop=B, hops=2, state=VALID** ; 2. atualiza precusores: **route(C).precursors += [B]** ; 3. **drena a fila pendente**: o **DATA** que ficou enfileirado quando **send_data** retornou **AODV_EN_QUEUED** agora vai pelo **next_hop=B** .

Estado depois da descoberta

NODE_A:

```
neighbors:  B (rssi)
routes:     C via B, hops=2, state=VALID, dest_seq=2
            B via B, hops=1, state=VALID (vinha de HELLO ou observacao passiva)
            (A->A nunca aparece, e self)
```

NODE_B:

```
neighbors:  A, C
routes:     A via A, hops=1, state=VALID, precursors=[]
            C via C, hops=1, state=VALID, precursors=[A]
            (rota reversa para A foi sobrescrita pela rota direta de HELLO/observacao)
```

NODE_C:

```
neighbors: B
routes:    A via B, hops=2, state=REVERSE, dest_seq=1
          B via B, hops=1, state=VALID
```

Observacao: a rota reversa em C para A continua como `REVERSE` ate haver trafego que justifique torna-la `VALID`. Quando A enviar `DATA`, os intermediarios podem usar `REVERSE` para encaminhar `RREP` / `ACK` mas o `DATA` em si exige rota `VALID`. Quando C precisar enviar `DATA` para A, isso disparara um novo `RREQ`.

5.3 Encaminhamento de DATA hop-by-hop

Logo apos a descoberta, o `DATA` pendente e drenado.

NODE_A	NODE_B	NODE_C
DATA seq=1		
flag=ACK_REQUIRED		
--unicast-->		
via=B, ttl=16		
	forward_data()	
	ttl=15	
	sender=B	
	add_precursor(A)	
	--unicast-->	
	via=C	
		deliver_data()
		callbacks.on_data(A, payload)
		ACK_REQUIRED set ->
		send_ack(A, seq=1)

Cada salto: - consulta `route_find_valid(destination)`; - decrementa `ttl`; - incrementa `header.hop_count`; - reescreve `sender_mac` para si mesmo; - adiciona o vizinho de quem recebeu como precursor da rota para o destino (RFC 3561); - chama `emit_frame(next_hop, ...)`.

Se em algum salto a rota nao existe ou nao e usavel, o no descarta o `DATA` e emite `RERR` para o `unreachable_destination_mac`.

5.4 ACK fim a fim

Quando o `DATA` chega em C com `ACK_REQUIRED`:

```
if (flags & AODV_EN_MSG_FLAG_ACK_REQUIRED) {
    aodv_en_node_send_ack(node, originator_mac=A, seq=1, now);
}
```

`send_ack` consulta a rota para A em C (que esta `REVERSE`, suficiente):

NODE_C	NODE_B	NODE_A
ACK seq=1		
originator=C		
destination=A		
--unicast-->		
via=B		
	handle_ack	
	nao e destino	
	repassa unicast	
	--unicast-->	
	via=A	
		handle_ack
		mac=A, e destino
		pending_ack.consume(C, 1)
		callbacks.on_ack(C, 1)

Em A: - a fila `pending_ack` tinha uma entrada `(C, seq=1, retries_left=3, last_sent_at)`; - `pending_ack.consume(C, 1)` remove ela e incrementa `stats.ack_received`; - a app recebe o callback.

Se o `ACK` nao chegar em `ack_timeout_ms` (default `1000 ms`): - `retries_left--`; - se rota ainda valida, retransmite o mesmo `sequence_number` (o que ja esta no buffer pendente); - se rota nao valida mais, dispara `RREQ` antes da proxima tentativa; - ao esgotar `retries_left` (= 0), remove e contabiliza em `ack_timeout_drops`.

Regra importante de v1

`rreq_retry_count` e `ack_retry_count` **compartilham o mesmo orcamento** (`config.rreq_retry_count`). Separar isso esta mapeado para v2.

5.5 Fila de DATA pendente

Quando `send_data` e chamado mas a rota nao existe ou esta expirada, o `DATA` e **enfileirado**, um `RREQ` e disparado, e o status retornado e `AODV_EN_QUEUED`.

Sequencia exata em `aodv_en_node_send_data`:

```
route = route_find_valid(destination);
if (route == NULL || !route_is_usable(route, now)) {
    queue_data(destination, payload, payload_len, ack_required, now);
    send_rreq(destination, now);
    pending_note_rreq_attempt(destination, now);
    return AODV_EN_QUEUED;
}
return send_data_via_route(route, ...);
```

Drain ao instalar rota nova

Quando `handle_rrep` instala uma rota direta valida:

```
route_upsert(C, via=B, state=VALID, ...);
flush_pending_data_for_destination(C, now); // <-- aqui
```

`flush_pending_data_for_destination` itera pelas `pending_data` e envia tudo que tem `destination_mac == C`, incrementando `pending_data_flushed`.

Retry de descoberta

A cada `tick`, o no checka se ha `pending_data` para destinos que ainda nao tem rota. Se sim, **reemite** `RREQ` com **backoff exponencial**:

```
interval = ack_timeout_ms;      // 1000 ms inicial
for each attempt > 1: interval *= 2 (cap em 10000 ms);
```

Sequencia de tentativas: `1000 ms, 2000 ms, 4000 ms, 8000 ms, 10000 ms, 10000 ms ...`

5.6 RERR e precursores

Quando uma rota cai, o no emite `RERR` para os **precursores** dela (vizinhos que estavam usando essa rota), excluindo o `next_hop` quebrado.

Disparo de RERR

`RERR` e emitido em tres situacoes:

1. **Falha de TX no link layer** (`aodv_en_node_on_link_tx_result(success=false)`): - `link_fail_count++` no vizinho; - se atinge `link_fail_threshold` (default 3), vizinho vai para `INACTIVE`; - todas as rotas com aquele `next_hop` sao invalidadas (`state = INVALID`); - para cada rota invalidada, `notify_precursors_of_break` e chamado.
2. **Recepcao de DATA sem rota valida** (em `forward_data`): - emite `RERR` para `unreachable_destination_mac` em broadcast.
3. **Recepcao de RERR** (em `handle_rerr`): - invalida a rota local; - propaga `RERR` para os precursores dessa rota, excluindo o emissor recebido.

Comportamento de `notify_precursors_of_break`

```
void notify_precursors_of_break(node, broken_next_hop, unreachable_mac, unreachable_seq, now):
    route = route_find(unreachable_mac);
    if (route == NULL) return;

    if (route->precursor_count == 0) {
        // sem precursores conhecidos, broadcast como fallback
        send_rerr(BROADCAST, unreachable_mac, unreachable_seq);
    } else {
        for each p in route->precursors:
            if (p == broken_next_hop) continue; // nao manda para o que ja caiu
            send_rerr(p, unreachable_mac, unreachable_seq);
    }
```

Cenario completo

Topologia `A-B-C`. A tem rota A->C via B. C cai.

NODE_A	NODE_B	NODE_C
DATA seq=2 ---->		(C ja desligado)
	forward_data ttl=15 ---->X (timeout em send_cb)	
	on_link_tx_result(C, success=false) link_fail_count(C)++ nao chegou em threshold ainda	
DATA seq=3 ---->		
	---->X link_fail_count(C) = 2	
DATA seq=4 ---->		
	---->X link_fail_count(C) = 3 = threshold! neighbor C -> INACTIVE invalidate routes via=C route(C) state=INVALID route(D) state=INVALID	
	notify_precursors_of_break(broken=C, dst=C): route(C).precursors = [A] send_rerr(A, dst=C, seq=...)	
<---RERR-----		
handle_rerr		
route_invalidate(C)		
route(C).precursors = []		
ja sem precursores, sem propagar		
(proximo send_data para C dispara novo RREQ que falhara enquanto C estiver fora, acumulando pending_data ate timeout)		

Quando C volta: - RREQ de A em broadcast alcanca B; - B encaminha; - C recebe, instala rota reversa, responde RREP; - A instala rota direta nova, drena pending_data.

Regra importante de hysteresis

Em uma rede onde A tem dois caminhos para C (via B e via D), route_should_replace exige que a candidata seja **estritamente melhor** para trocar o next_hop. Caso contrario, mantem a atual. Isso evita "flapping" entre caminhos similares. Os limiares sao:

- ROUTE_SWITCH_MIN_SEQ_GAIN = 2 (ganho de seq num)
- ROUTE_SWITCH_MIN_METRIC_GAIN = 1
- ROUTE_SWITCH_MIN_HOP_GAIN = 1
- ROUTE_SWITCH_MIN_LIFETIME_GAIN_MS = 5000

Quando o next_hop permanece o mesmo (refresh), os precursores existentes sao preservados. Quando o next_hop muda, os precursores sao zerados (vizinhos antigos nao necessariamente sabem do novo caminho).

Selecao e substituicao de rotas

Em route_should_replace(existing, candidate) a ordem e:

1. estado: `VALID` substitui `INVALID` ou `REVERSE` ;
2. histerese: se ambas `VALID` mas `next_hop` mudou, exigir ganho forte;
3. `dest_seq_num` maior vence;
4. com `dest_seq_num` igual: `metric` menor vence;
5. com `metric` igual: `hop_count` menor vence;
6. com tudo igual: maior `expires_at_ms` vence.

Exemplo simples

A tem rota `C via B, hops=2, dest_seq=5` . Recebe agora um `RREP` de B com `dest_seq=5, hops=2` . Sem ganho -> mantém.

Recebe outro `RREP` de D com `dest_seq=7, hops=2` . Ganho forte de seq (≥ 2) -> troca para `C via D, hops=2, dest_seq=7` .

Filas pendentes

Resumo do comportamento das duas filas (mais detalhe em [features/enfileamento-dos-dados.md](#)):

Fila	Tamanho	Trigger de inclusao	Trigger de remocao	Falha apos esgotamento
<code>pending_data</code>	4	<code>send_data</code> sem rota valida	<code>RREP</code> instalado + drain	descartado em <code>pending_data_dropped</code> apos <code>route_lifetime_ms</code>
<code>pending_ack</code>	4	<code>send_data</code> com <code>ACK_REQUIRED</code>	<code>ACK</code> correspondente recebido	descartado em <code>ack_timeout_drops</code> apos <code>retries_left=0</code>

Ambas tem capacidade fixa de 4. Quando `pending_data` enche, recicla a entrada mais antiga do mesmo destino primeiro.

Falhas, RERR e precursores

Quadro de eventos relacionados a falha:

Evento	O que dispara	Efeito
Vizinho silencioso	<code>last_seen_ms</code> excede <code>neighbor_timeout_ms</code> (15 s)	vizinho removido na proxima <code>tick</code>
TX unicast falha	adapter chama <code>on_link_tx_result(success=false)</code>	<code>link_fail_count++</code> ; ao atingir threshold, vizinho <code>INACTIVE</code> e rotas via ele invalidadas
Rota expira	<code>expires_at_ms</code> $\leq$ now	removida em <code>route_expire</code>
RERR recebido	<code>handle_rerr</code>	invalida rota local; propaga <code>RERR</code> para precursores excluindo o sender
DATA sem rota	<code>forward_data</code> nao acha rota valida	descarta + emite <code>RERR(BROADCAST, dst)</code>

Worked example A-B-C-D

Cenário `TC-005` : cadeia linear de 4 nos. A envia `DATA` para D periodicamente.

Fase 1 - boot e descoberta de vizinhos

HELLO periodico por todos. Tabelas iniciais (apos primeiros 5 segundos):

```
NODE_A.neighbors = { B }
NODE_A.routes     = { B via B hops=1 VALID }

NODE_B.neighbors = { A, C }
NODE_B.routes     = { A via A hops=1 VALID, C via C hops=1 VALID }

NODE_C.neighbors = { B, D }
NODE_C.routes     = { B via B hops=1 VALID, D via D hops=1 VALID }

NODE_D.neighbors = { C }
NODE_D.routes     = { C via C hops=1 VALID }
```

Fase 2 - primeira tentativa A -> D

A chama `send_data(D, "tc005 chain4", ack_required=true)`. A nao tem rota para D.

```
A: queue_data(D, ...)
A: send_rreq(D)
A: send_data returns AODV_EN_QUEUED

A --RREQ--> [broadcast] orig=A, dest=D, orig_seq=1, dest_seq=0, rreq_id=1
B handles RREQ:
  cache.remember(A, 1)
  route_upsert(A via A, hops=1, state=REVERSE) // ja era VALID, vira VALID que vence
  forward_rreq() hop=1, ttl=15

B --RREQ--> [broadcast] sender=B, hop=1
A receives own RREQ via broadcast - cache.contains(A,1) - drops, increments duplicate_rreq_drops
C handles RREQ:
  cache.remember(A, 1)
  route_upsert(A via B, hops=2, state=REVERSE)
  forward_rreq() hop=2, ttl=14

C --RREQ--> [broadcast] sender=C, hop=2
B sees, drops as duplicate.
D handles RREQ:
  cache.remember(A, 1)
  route_upsert(A via C, hops=3, state=REVERSE)
  D is dest!
  self_seq = max(0, self_seq) = ? D's self_seq starts at 0, so becomes 1
  send_rrep(orig=A, dest=D, dest_seq=1, lifetime=30000)
  target = next_hop of route to A = C
```

Continua na ida do `RREP`:

```

D --RREP--> [unicast to C] orig=A, dest=D, dest_seq=1, hop=0
C handle_rrep:
  route_upsert(D via D, hops=1, state=VALID, dest_seq=1)
  precursors of route(D): add C (de quem recebeu) and add B (para quem repassa = next_hop da rota reversa para A,
    ...na pratica, add link_src_mac=D e add reverse_route.next_hop=B)
  precursors of route(A): add D (de quem recebeu o RREP)
  forward_rrep() to next_hop of route to A = B

C --RREP--> [unicast to B] hop=1
B handle_rrep:
  route_upsert(D via C, hops=2, state=VALID, dest_seq=1)
  precursors of route(D): add C, add A
  precursors of route(A): add C
  forward_rrep() to next_hop of route to A = A

B --RREP--> [unicast to A] hop=2
A handle_rrep:
  route_upsert(D via B, hops=3, state=VALID, dest_seq=1, lifetime=30000)
  precursors of route(D): add B (de quem recebeu)
    (A nao tem reverse para si mesmo, segunda parte n/a)
  route(originator=A) is self, returns OK
  flush_pending_data_for_destination(D) <-- envia o DATA enfileirado!

```

Tabelas finais apos descoberta

NODE_A.routes:

```

B via B hops=1 VALID precursors=[]
D via B hops=3 VALID dest_seq=1 precursors=[B]

```

NODE_B.routes:

```

A via A hops=1 VALID precursors=[C] (C foi anexado durante RREP repasse)
C via C hops=1 VALID
D via C hops=2 VALID dest_seq=1 precursors=[C, A]

```

NODE_C.routes:

```

B via B hops=1 VALID
D via D hops=1 VALID dest_seq=1 precursors=[D, B]
A via B hops=2 REVERSE (foi REVERSE durante RREQ; nao foi promovida pelo RREP porque A nao envia RREP de volta ne
    (ela vira VALID se A e D fizerem comunicacao bidirecional)

```

NODE_D.routes:

```

C via C hops=1 VALID
A via C hops=3 REVERSE (instalada durante RREQ; permanece REVERSE ate haver trafego de D->A)

```

Fase 3 - DATA flow

```
A.flush_pending_data_for_destination(D):
    send_data_via_route(D, payload, ack_required=true)
    sequence_number = ++next_data_seq = 1
    pending_ack.track(D, payload, seq=1, retries_left=3)

A --DATA seq=1, ttl=16, ACK_REQ--> [unicast to B]
B handle_data:
    not destination
    forward_data:
        ttl=15, hop_count=1, sender=B
        route_to_D valid, next=C
        route(D).precursors += A (link_src_mac)
        stats.forwarded_frames++
B --DATA seq=1, ttl=15--> [unicast to C]
C handle_data:
    forward_data:
        ttl=14, hop=2, sender=C
        route_to_D valid, next=D
        route(D).precursors += B
C --DATA seq=1, ttl=14--> [unicast to D]
D handle_data:
    destination_mac == self_mac
    delivered_frames++
    callbacks.deliver_data(originator=A, payload, len)
    ACK_REQ flag set:
        send_ack(target=A, ack_for_sequence=1)
        find_usable_route(A) -> via=C
D --ACK seq=1--> [unicast to C]
C handle_ack:
    not destination
    forward via route_to_A.next_hop=B
C --ACK seq=1--> [unicast to B]
B handle_ack:
    forward via route_to_A.next_hop=A
B --ACK seq=1--> [unicast to A]
A handle_ack:
    destination==self, pending_ack.consume(D, 1)
    ack_received++
    callbacks.ack_received(D, 1)
```

Total: 1 DATA enviado + 2 forwards = 3 frames de DATA na malha; 1 ACK + 2 forwards = 3 frames de ACK. Para 3 saltos, **6 frames de TX no total** para entregar 1 DATA com confirmacao.

Fase 4 - falha de enlace B-C

Suponha que B e C percam contato. O proximo **DATA** de A:

```

A --DATA seq=2--> B (ok, B recebe)
B --DATA seq=2--> X (TX falha)
B.on_link_tx_result(C, false): link_fail_count(C) = 1
B.adapter retorna AODV_EN_ERR_STATE; A nao sabe ainda

(repete por 3 tentativas pois link_fail_threshold = 3)
B link_fail_count(C) = 3 -> C INACTIVE
B.invalidate_routes_by_next_hop(C):
    route(C) -> INVALID
    route(D) -> INVALID
B.notify_precursors:
    route(C).precursors = [] (foi resetada quando next_hop mudou)
    broadcast RERR(unreachable=C)
    route(D).precursors = [C, A]
    filter out C (broken)
    send_rerr(A, unreachable=D, seq=1)
B --RERR--> [broadcast/unicast A]
A.handle_rerr:
    route_invalidate(D)
    route(D).precursors = [B]
    send_rerr(B, unreachable=D) <-- mas B ja sabe, ele que invalidou
    (B handle_rerr: route(D) ja INVALID, no_op)

A continua tendo pending_data se a app continuou enviando.
Em ticks subsequentes:
    retry_route_discovery_for_pending(D):
        no route to D? send_rreq(D), backoff exponencial
        se C voltar fisicamente, B descobre por HELLO/RREQ e tabela e refeita.

```

Timers, limites e calibragem

Defaults em [aodv_en_limits.h](#):

Constante	Default	Faz o que
NEIGHBOR_TIMEOUT_MS	15000	janela sem <code>last_seen</code> antes de remover vizinho
ROUTE_LIFETIME_MS	30000	lifetime instalado em rota nova; tambem janela de validade da fila pendente
RREQ_CACHE_TIMEOUT_MS	10000	janela do cache de duplicatas
ACK_TIMEOUT_MS	1000	timeout para retransmitir DATA com <code>ACK_REQUIRED</code>
RREQ_RETRY_COUNT	3	retries de RREQ e ACK (compartilhado)
LINK_FAIL_THRESHOLD	3	falhas seguidas para marcar vizinho INACTIVE
MAX_HOPS	16	limite de saltos do protocolo
TTL_DEFAULT	16	TTL inicial em RREQ e DATA
NEIGHBOR_TABLE_SIZE	16	maximo de vizinhos
ROUTE_TABLE_SIZE	32	maximo de rotas
RREQ_CACHE_SIZE	64	maximo de duplicatas memorizadas
PEER_CACHE_SIZE	8	maximo de peers ESP-NOW registrados
PENDING_DATA_QUEUE_SIZE	4	maximo de DATA em fila
MAX_PRECURSORS	4	precursores por rota
CONTROL_PAYLOAD_MAX	128	maior frame de controle
DATA_PAYLOAD_MAX	1024	maior payload de DATA

Calibragem para diferentes cenários

- **Cadeias linear (TC-001 a TC-005):** defaults funcionam.
- **Malhas densas (>10 nos):** aumentar `NEIGHBOR_TABLE_SIZE` e `ROUTE_TABLE_SIZE`. Considerar também aumentar `RREQ_CACHE_SIZE` para que o aumento de RREQs concorrentes não cause descarte útil.
- **Cenários com flap:** aumentar `LINK_FAIL_THRESHOLD` para 5; aumentar `ROUTE_SWITCH_MIN_SEQ_GAIN` para 3; ajuda a estabilizar.
- **Cenários moveis (caminhada):** reduzir `NEIGHBOR_TIMEOUT_MS` para 8000; aumentar frequência de `HELLO`.

Telemetria e diagnostico

Cada no expoe via `aadv_en_stack_get_overview` os contadores em `aadv_en_stack_stats_t`:

Contador	Significado
<code>rx_frames</code>	total de frames recebidos validados
<code>tx_frames</code>	total de frames emitidos pelo <code>emit_frame</code> callback
<code>forwarded_frames</code>	frames repassados pelo no (RREQ, RREP, DATA, ACK)
<code>delivered_frames</code>	DATA entregue ao callback <code>on_data</code> (no como destino)
<code>ack_received</code>	ACK recebido pela aplicacao
<code>route_discoveries</code>	RREQ emitidos como origem
<code>route_repairs</code>	RERR emitidos
<code>route_discovery_retries</code>	RREQ reemitidos pelo retry da fila pendente
<code>duplicate_rreq_drops</code>	RREQ ignorado por estar no cache
<code>pending_data_queued</code>	DATA enviado para a fila pendente
<code>pending_data_flushed</code>	itens drenados da fila ao instalar rota
<code>pending_data_dropped</code>	itens descartados (timeout ou backpressure)
<code>ack_retry_sent</code>	retransmissoes de DATA com ACK_REQUIRED
<code>ack_timeout_drops</code>	DATA desistido apos esgotar retries
<code>link_fail_events</code>	callbacks <code>on_link_tx_result(false)</code>
<code>route_invalidations_link_fail</code>	rotas invalidadas pelo cascading de falha de vizinho

Metricas derivadas

- `PDR = delivered_frames / DATA_emitidos_pela_app`
- `overhead de controle = tx_frames - delivered_frames - forwarded_frames(DATA)`
- `latencia de descoberta = t(RREP recebido em A) - t(send_data com sem rota em A)` - capturada com `discovery_windows` em [tools/extract_monitor_metrics.py](#)

Limites conhecidos

Estas sao limitacoes da v1, ja mapeadas como itens de v2 na [spec v1](#):

1. **Orcamento de retry compartilhado:** `RREQ` e `ACK` retries usam o mesmo `rreq_retry_count`. Em cenarios onde a malha precisa varias descobertas mas o `ACK` e barato, esse acoplamento pode descartar `DATA` cedo demais. (resolver na v2 separando os dois)
2. **Wrap-around de sequence number:** comparacao por `>` simples. Em janelas longas (~50 dias com seq incrementando 1 por ms) ha risco teorico de overflow. Para o TCC nao e problema. (v2: comparacao serial)
3. **Multiplos destinos por RERR:** v1 carrega 1 destino por `RERR`. Se uma falha invalida `N` rotas, emite `N` `RERR`. RFC 3561 permite agregacao. (v2)
4. **Capacidade fixa de precursores:** `MAX_PRECURSORS = 4` por rota. Em malhas densas o array pode encher e novos precursores serao ignorados (`AODV_EN_ERR_FULL`). (v2: politica de eviction)
5. **Sem articulation point detection:** ainda nao implementado, ver [features/articulation-point-planejado.md](#).
6. **Metrica = hop_count:** RSSI nao entra no calculo da metrica. (v2: metrica composta)
7. **Sem RREP-ACK** (RFC 3561 secao 5.7): `RREP` enviado em unicast nao tem confirmacao explicita; perda silenciosa exige novo `RREQ`.
8. **App layer hardcoded broadcast HEALTH_REQ:** a app `app_proto_example` emite `HEALTH_REQ` periodico. Isso aparece como trafego no dashboard mas **nao e parte do AODV-EN**.

Referencia rapida

Funcoes-chave do nucleo

```
// inicializacao
aodv_en_status_t aodv_en_node_init(node, config, self_mac);
void aodv_en_node_set_callbacks(node, callbacks);

// loop periodico (chamar a cada ~50-100 ms)
void aodv_en_node_tick(node, now_ms);

// controle
aodv_en_status_t aodv_en_node_send_hello(node, now_ms);

// dados (com confirmacao opcional)
aodv_en_status_t aodv_en_node_send_data(
    node, destination_mac, payload, payload_len, ack_required, now_ms);

// recepcao do transporte
aodv_en_status_t aodv_en_node_on_recv(
    node, link_src_mac, frame, frame_len, rssi, now_ms);

// resultado de TX (callback do driver, opcional mas recomendado)
aodv_en_status_t aodv_en_node_on_link_tx_result(
    node, next_hop, success, now_ms, &invalidated_count);
```

Codigos de status

```
AODV_EN_OK           = 0
AODV_EN_NOOP         = 1  // operacao reconhecida mas sem efeito (e.g. RREQ duplicado)
AODV_EN_QUEUED       = 2  // DATA enfileirado, aguardando rota
AODV_EN_ERR_ARG      = -1
AODV_EN_ERR_FULL     = -2  // tabela ou fila lotada
AODV_EN_ERR_NOT_FOUND = -3
AODV_EN_ERR_EXISTS   = -4
AODV_EN_ERR_NO_ROUTE = -5
AODV_EN_ERR_SIZE     = -6
AODV_EN_ERR_PARSE    = -7
AODV_EN_ERR_STATE    = -8  // adapter ou estado interno inconsistente
```

Eventos esperados nos logs (TC-002)

```
boot:
I (xxxx) aadv_en_proto: node=NODE_A self_mac=... channel=6 network_id=0xA0DE0001
I (xxxx) aadv_en_proto: ESP-NOW version=2

trafego normal:
I (xxxx) aadv_en_proto: HELLO send status=0
I (xxxx) aadv_en_proto: DATA queued to route          <-- send_data com rota valida
I (xxxx) aadv_en_proto: ACK received from <C> for seq=N

descoberta:
I (xxxx) aadv_en_proto: DATA queued while route discovery is in progress

falha:
W (xxxx) aadv_en_proto: ESP-NOW send fail to <MAC>
W (xxxx) aadv_en_proto: invalidated 2 route(s) via <MAC> after link failures
W (xxxx) aadv_en_proto: DATA send status=-2          <-- AODV_EN_ERR_FULL

snapshot periodico:
I (xxxx) aadv_en_proto: routes=3 neighbors=2 tx=120 rx=95 delivered=0
I (xxxx) aadv_en_proto: route[0] dest=<C> via=<B> hops=2 metric=2 state=2 expires=470000
I (xxxx) aadv_en_proto: route[1] dest=<B> via=<B> hops=1 metric=1 state=2 expires=470000
```

Onde olhar quando algo da errado

Sintoma	Provavel causa	Onde investigar
<code>routes=0</code> por mais de 10s	HELLO nao esta chegando, canal/network_id divergente	comparar Kconfig dos nos, fisica do link
<code>duplicate_rreq_drops</code> cresce mas <code>route_discoveries</code> parado	RREQ nao retorna; destino unreachable ou sem RREP	logs de C/B, falha no caminho reverso
<code>pending_data_dropped</code> > 0	rota nao chegou a tempo; backpressure ou timeout	aumentar <code>route_lifetime_ms</code> ou <code>pending_data_queue_size</code>
<code>ack_timeout_drops</code> alto	enlace pessimo ou dest fora de alcance; route flap	investigar <code>link_fail_events</code> e <code>route_invalidations_link_fail</code>
flapping de rota	duas rotas com metric quase identica	considerar aumentar <code>ROUTE_SWITCH_MIN*_GAIN</code>
reset constante	watchdog ou stack overflow no task	<code>idf.py monitor</code> para ver guru meditation

Apendice: snippet de uso minimo

```
#include "aadv_en.h"

static aadv_en_stack_t stack;

static uint32_t my_now_ms(void *ctx) {
    (void)ctx;
    return (uint32_t)(esp_timer_get_time() / 1000ULL);
}

static aadv_en_status_t my_tx(void *ctx,
                             const uint8_t next_hop[AODV_EN_MAC_ADDR_LEN],
                             const uint8_t *frame, size_t frame_len, bool broadcast) {
    (void)ctx;
    const uint8_t bcast[6] = {0xff,0xff,0xff,0xff,0xff,0xff};
    const uint8_t *target = broadcast ? bcast : next_hop;
    return (esp_now_send(target, frame, frame_len) == ESP_OK)
        ? AODV_EN_OK : AODV_EN_ERR_STATE;
}

static void my_on_data(void *ctx, const uint8_t orig[6],
                      const uint8_t *payload, uint16_t len) {
    ESP_LOGI("APP", "got %u bytes from %02X:%02X:..", len, orig[0], orig[1]);
}

static void my_on_ack(void *ctx, const uint8_t from[6], uint32_t seq) {
    ESP_LOGI("APP", "ACK seq=%u", (unsigned)seq);
}

void app_main(void) {
    // ... esp_wifi + esp_now boilerplate ...

    aadv_en_config_t cfg;
    aadv_en_config_set_defaults(&cfg);
    cfg.network_id = 0xA0DE0001;

    uint8_t self_mac[6];
    esp_wifi_get_mac(WIFI_IF_STA, self_mac);

    aadv_en_adapter_t adapter = {
        .now_ms = my_now_ms, .tx_frame = my_tx,
    };
    aadv_en_app_callbacks_t cb = {
        .on_data = my_on_data, .on_ack = my_on_ack,
    };
    aadv_en_stack_init(&stack, &cfg, self_mac, &adapter, &cb);

    while (1) {
        aadv_en_stack_tick(&stack);
        // periodicamente: aadv_en_stack_send_hello(&stack);
        // sob demanda: aadv_en_stack_send_data(&stack, dest_mac, payload, len, true);
        vTaskDelay(pdMS_TO_TICKS(50));
    }
}
```

Referencias internas

- [aadv-en-spec-v1.md](#) - definicao normativa
- [aadv-base-invariantes.md](#) - invariantes herdados do AODV
- [aadv-en-estruturas-dados.md](#) - layout das tabelas e mensagens
- [features/enfilelamento-dos-dados.md](#) - fila pendente

- [features/precursores.md](#) - precursores e RERR direcionado
- [features/articulation-point-planejado.md](#) - feature de v2
- [tests](#) - casos de teste com criterios de aprovacao
- RFC 3561: <https://datatracker.ietf.org/doc/html/rfc3561>

Referencias de codigo

- [firmware/components/aodv_en/include/aodv_en.h](#) - API publica `aodv_en_stack_*`
- [firmware/components/aodv_en/include/aodv_en_node.h](#) - core API
- [firmware/components/aodv_en/include/aodv_en_messages.h](#) - layout no fio
- [firmware/components/aodv_en/include/aodv_en_limits.h](#) - constantes
- [firmware/components/aodv_en/src/aodv_en_node.c](#) - logica principal
- [firmware/components/aodv_en/src/aodv_en_routes.c](#) - tabela de rotas + precursores
- [firmware/components/aodv_en/src/aodv_en_neighbors.c](#) - vizinhos + EMA RSSI
- [firmware/components/aodv_en/src/aodv_en.c](#) - facade `stack_*`
- [sim/aodv_en_sim.c](#) - simulacao de referencia 3 nos

Estruturas de Dados

AODV-EN Estruturas de Dados

Estado

- alinhado com [aodv-en-spec-v1.md](#)
- ultima revisao: 2026-05-01
- fonte da verdade: cabecalhos em [firmware/components/aodv_en/include](#)

Objetivo

Detalhar as estruturas de dados que sustentam a implementacao do `AODV-EN v1`. Este documento traduz a spec funcional para o layout concreto dos cabecalhos C, sem repetir as regras semanticas ja descritas na spec.

Quando houver duvida entre este documento e a spec v1, a spec define o comportamento e este documento define o layout.

Visao geral

Cada no `AODV-EN` mantem seis estruturas residentes:

1. tabela de vizinhos
2. tabela de rotas (com precursores por rota)
3. cache de duplicatas de `RREQ`
4. cache de peers ESP-NOW
5. fila de `DATA` pendente
6. fila de `ACK` pendente

Mais a configuracao do no e o bloco de estatisticas. Todas sao alocadas estaticamente dentro de `aodv_en_node_t`, sem heap no caminho critico.

1. Tabela de vizinhos

Papel

Representa nos de 1 salto observados pelo radio.

Entrada

Definida em `aodv_en_neighbor_entry_t` ([aodv_en_types.h](#)):

- `mac[6]`
- `avg_rssi` (int8)
- `last_rssi` (int8)
- `link_fail_count` (uint8)

- `state` (uint8): `INACTIVE` ou `ACTIVE`
- `last_seen_ms` (uint32)
- `last_used_ms` (uint32)

Tabela

`aadv_en_neighbor_table_t` :

- `count` (uint16)
- `entries[AADV_EN_NEIGHBOR_TABLE_SIZE]`

Operacoes

API em `aadv_en_neighbors.h`:

- `aadv_en_neighbor_table_init`
- `aadv_en_neighbor_find` / `_const`
- `aadv_en_neighbor_touch` (passivo: cada `on_recv` valido toca o vizinho)
- `aadv_en_neighbor_mark_used` (apos uso de unicast)
- `aadv_en_neighbor_note_link_failure` (incrementa `link_fail_count` ; promove a `INACTIVE` ao atingir threshold)
- `aadv_en_neighbor_expire`
- `aadv_en_neighbor_count_active`

2. Tabela de rotas

Papel

Representa caminhos `destination -> next_hop` . Cada entrada tambem mantem precursores para emissao direcionada de `RERR` .

Entrada

`aadv_en_route_entry_t` :

- `destination[6]`
- `next_hop[6]`
- `dest_seq_num` (uint32)
- `expires_at_ms` (uint32)
- `metric` (uint16) — em v1, igual a `hop_count` ; valor `AADV_EN_ROUTE_METRIC_INFINITY` (`0xFFFF`) marca invalidada
- `hop_count` (uint8)
- `state` (uint8): `INVALID` , `REVERSE` , `VALID`
- `precursor_count` (uint8)
- `precursors[AADV_EN_MAX_PRECURSORS]` [6]

Tabela

`aadv_en_route_table_t` :

- `count` (uint16)
- `entries[AADV_EN_ROUTE_TABLE_SIZE]`

Operacoes

API em [aadv_en_routes.h](#):

- `aadv_en_route_table_init`
- `aadv_en_route_find` / `_const` / `_find_valid`
- `aadv_en_route_should_replace` — implementa a regra de selecao com histerese descrita na spec v1, secao "Selecao e substituicao de rotas"
- `aadv_en_route_upsert` — preserva precursores quando o `next_hop` nao muda; reseta quando muda
- `aadv_en_route_add_precursor`
- `aadv_en_route_invalidate_destination`
- `aadv_en_route_invalidate_by_next_hop` — usado quando um vizinho cai
- `aadv_en_route_expire`

3. Cache de duplicatas de RREQ

Papel

Suprimir reprocessamento de `RREQ` repetidos em janela curta.

Entrada

`aadv_en_rreq_cache_entry_t`:

- `originator[6]`
- `rreq_id` (uint32)
- `created_at_ms` (uint32)
- `hop_count` (uint8)
- `used` (uint8)

Tabela

`aadv_en_rreq_cache_t`:

- `count` (uint16)
- `entries[AADV_EN_RREQ_CACHE_SIZE]`

Operacoes

API em [aadv_en_rreq_cache.h](#):

- `aadv_en_rreq_cache_init`
- `aadv_en_rreq_cache_contains`
- `aadv_en_rreq_cache_remember`
- `aadv_en_rreq_cache_expire` (governado por `rreq_cache_timeout_ms`)

4. Cache de peers ESP-NOW

Papel

Limitar quantos peers ficam registrados no driver ESP-NOW de uma vez. Independente da tabela de vizinhos, mas alinhado a ela.

Entrada

`aadv_en_peer_cache_entry_t`:

- `mac[6]`
- `last_used_ms` (uint32)
- `flags` (uint8): `NONE`, `PINNED`, `REGISTERED`
- `reserved` (uint8)

Tabela

`aadv_en_peer_cache_t`:

- `count` (uint16)
- `entries[AADV_EN_PEER_CACHE_SIZE]`

Operacoes

API em [aadv_en_peers.h](#):

- `aadv_en_peer_cache_init`
- `aadv_en_peer_find` / `_const`
- `aadv_en_peer_touch` (LRU)
- `aadv_en_peer_set_registered`
- `aadv_en_peer_set_pinned`
- `aadv_en_peer_remove`

5. Fila de DATA pendente

Papel

Buffer temporario de `DATA` enquanto a descoberta de rota nao chega ao destino. Comportamento descrito em [features/enfileamento-dos-dados.md](#) e na spec v1, secao "Fila de DATA pendente".

Entrada

`aadv_en_pending_data_entry_t` ([aadv_en_node.h](#)):

- `destination_mac[6]`
- `payload_len` (uint16)
- `ack_required` (bool)
- `used` (bool)
- `enqueued_at_ms` (uint32)
- `last_rreq_at_ms` (uint32)
- `discovery_attempts` (uint8)
- `payload[AADV_EN_DATA_PAYLOAD_MAX]`

Operacoes

Encapsuladas dentro de `aadv_en_node_t` (nao expostas como API publica em v1):

- enfileirar com backpressure por reciclagem do mais antigo (preferencia pelo mesmo destino)
- drenar quando uma rota valida e instalada para o destino

- reemitir `RREQ` no tick com backoff exponencial sobre `ack_timeout_ms` ate `10000 ms`
- expirar quando `now - enqueued_at_ms >= route_lifetime_ms`

6. Fila de ACK pendente

Papel

Rastreia `DATA` enviados com `ACK_REQUIRED` para suportar retransmissao quando o `ACK` nao chega.

Entrada

`aodv_en_pending_ack_entry_t` :

- `destination_mac[6]`
- `payload_len` (uint16)
- `used` (bool)
- `retries_left` (uint8)
- `sequence_number` (uint32)
- `last_sent_at_ms` (uint32)
- `payload[AODV_EN_DATA_PAYLOAD_MAX]`

Operacoes

- alocar entrada ao enviar `DATA` com `ACK_REQUIRED`, herdando `retries_left = rreq_retry_count`
- consumir entrada ao receber `ACK` correspondente por `(destination, sequence_number)`
- no tick, retransmitir entradas cujo `now - last_sent_at_ms >= ack_timeout_ms`
- ao esgotar `retries_left`, descartar e contabilizar `ack_timeout_drops`
- se na hora do retry a rota for invalida, disparar `RREQ` antes da proxima tentativa

Cabecalho comum das mensagens

`aodv_en_header_t` ([aodv_en_messages.h](#)):

- `protocol_version` (uint8)
- `message_type` (uint8)
- `flags` (uint8)
- `hop_count` (uint8)
- `network_id` (uint32)
- `sender_mac[6]`

Todas as mensagens sao `__attribute__((packed))` para garantir layout determinista entre nos.

Mensagens

Tipo	Struct
HELLO	<code>aodv_en_hello_msg_t</code>
RREQ	<code>aodv_en_rreq_msg_t</code>
RREP	<code>aodv_en_rrep_msg_t</code>
RERR	<code>aodv_en_rerr_msg_t</code>
ACK	<code>aodv_en_ack_msg_t</code>
DATA	<code>aodv_en_data_msg_t</code> (header + metadados + <code>payload[]</code> flexível)

Os campos de cada uma estão detalhados na spec v1, seção "Mensagens", e materializados em [aodv_en_messages.h](#).

Configuração do no

`aodv_en_config_t`:

- `network_id` (`uint32`)
- `neighbor_timeout_ms` (`uint32`)
- `route_lifetime_ms` (`uint32`)
- `rreq_cache_timeout_ms` (`uint32`)
- `ack_timeout_ms` (`uint32`)
- `neighbor_table_size`, `route_table_size`, `rreq_cache_size`, `peer_cache_size` (`uint16`)
- `control_payload_max`, `data_payload_max` (`uint16`)
- `wifi_channel`, `max_hops`, `tll_default`, `rreq_retry_count`, `link_fail_threshold` (`uint8`)

`aodv_en_config_set_defaults` carrega os valores documentados em [aodv_en_limits.h](#).

Estatísticas

`aodv_en_stack_stats_t` ([aodv_en.h](#)):

- `rx_frames`, `tx_frames`, `forwarded_frames`, `delivered_frames`
- `ack_received`
- `route_discoveries`, `route_repairs`
- `duplicate_rreq_drops`
- `pending_data_queued`, `pending_data_flushed`, `pending_data_dropped`
- `route_discovery_retries`
- `ack_retry_sent`, `ack_timeout_drops`
- `link_fail_events`, `route_invalidations_link_fail`

Esses contadores são acumulativos e formam a base para os experimentos da Fase 8 do [plano](#).

Limites iniciais

Todos definidos em [aodv_en_limits.h](#):

Limite	Default
NEIGHBOR_TABLE_SIZE	16
ROUTE_TABLE_SIZE	32
RREQ_CACHE_SIZE	64
PEER_CACHE_SIZE	8
PENDING_DATA_QUEUE_SIZE	4
MAX_PRECURSORS	4
CONTROL_PAYLOAD_MAX	128
DATA_PAYLOAD_MAX	1024
MAX_HOPS / TTL	16

Estimativa de memoria

Com os defaults, o consumo agregado das tabelas e filas residentes em RAM por no e:

- vizinhos: 16 entradas x ~24 B = ~384 B
- rotas (com precursores e demais campos): 32 entradas x ~52 B = ~1.7 KB
- cache de RREQ : 64 entradas x ~16 B = ~1 KB
- cache de peers: 8 entradas x ~16 B = ~128 B
- pending_data : 4 entradas x (~24 B + 1024 B payload) = ~4.1 KB
- pending_ack : 4 entradas x (~24 B + 1024 B payload) = ~4.1 KB

Total residente da camada A0DV-EN : ~11.5 KB por no, dominado pelos buffers de payload das filas pendentes. E confortavel para um ESP32, mas e o item de memoria a observar caso DATA_PAYLOAD_MAX ou PENDING_DATA_QUEUE_SIZE sejam aumentados em experimentos futuros.

Principios de projeto

- tabelas pequenas, previsiveis e indexaveis sequencialmente
- nenhuma alocao dinamica no caminho critico
- separacao clara entre vizinhos e roteamento
- precursores como sub-array da rota, nao tabela separada (mantem cohesao)
- RREQ cache como estrutura de primeira classe
- cache de peers desacoplado da tabela de vizinhos para refletir a politica do driver ESP-NOW

Mapa doCodigo

AODV-EN - Mapa do Código

Estado

- alinhado com [aodv-en-spec-v1.md](#)
- complementar a [aodv-en-funcionamento.md](#)
- ultima revisao: 2026-05-01

Para quem este documento e

Voce ja entendeu **o que** o protocolo faz (em [aodv-en-funcionamento.md](#)) e quer agora saber **onde** cada peca esta implementada. Este documento e um guia de estudo do codigo, organizado por:

- arvore do projeto
- camadas e seus arquivos
- o que cada arquivo contem
- onde encontrar funcionalidades especificas (mapeamento por feature)

Quem precisa: voce mesmo daqui a 3 meses; coorientador; banca pedindo "me mostra onde isso acontece"; alguem que quer estender o protocolo.

Sumario

1. [Arvore do projeto](#)
 2. [Camada 1 - Nucleo do protocolo](#)
 3. [Camada 2 - Fachada da lib](#)
 4. [Camada 3 - Aplicacao firmware](#)
 5. [Camada 4 - Build system](#)
 6. [Camada 5 - Scripts](#)
 7. [Camada 6 - Simulacao](#)
 8. [Camada 7 - Ferramentas de analise](#)
 9. [Camada 8 - Testes funcionais](#)
 10. [Camada 9 - Documentacao](#)
 11. [Mapa por feature](#)
 12. [Cross-reference de constantes](#)
 13. [Como uma mensagem viaja pelo codigo](#)
-

Arvore do projeto

```
aadv-en/
├── README.md
├── docs/                                <- documentacao
│   ├── aadv-en-spec-v1.md
│   ├── aadv-en-funcionamento.md        <- "como funciona"
│   ├── aadv-en-mapa-do-codigo.md        <- voce esta aqui
│   ├── aadv-en-estruturas-dados.md
│   ├── aadv-base-invariantes.md
│   ├── aadv-en-spec-v0.md               (OBSOLETA)
│   ├── plano-desenvolvimento-completo.md
│   ├── runbook-bancada.md
│   └── features/
│       ├── enfileamento-dos-dados.md
│       ├── precursores.md
│       └── articulation-point-planejado.md
├── tests/                              <- casos de teste
│   ├── README.md
│   ├── tc-001-descoberta-e-entrega-direta.md
│   ├── tc-002-primeiro-multi-hop.md
│   ├── tc-003-reconvergencia-apos-falha.md
│   ├── tc-004-soak-estabilidade-e-reconvergencia.md
│   ├── tc-005-cadeia-4-nos.md
│   └── guia-leitura-graficos-monitor.md
├── firmware/                           <- ESP-IDF project
│   ├── CMakeLists.txt
│   ├── README.md
│   ├── sdkconfig.defaults
│   └── build.sh, flash_monitor.sh, monitor_log.sh, idf-env.sh
├── components/aadv_en/                  <- BIBLIOTECA DO PROTOCOLO
│   ├── CMakeLists.txt
│   ├── include/                         <- headers publicos
│   │   ├── aadv_en.h                   <- API publica (stack)
│   │   ├── aadv_en_node.h              <- API do nucleo
│   │   ├── aadv_en_messages.h          <- structs no fio
│   │   ├── aadv_en_types.h             <- tipos centrais + config
│   │   ├── aadv_en_status.h            <- codigos de retorno
│   │   ├── aadv_en_limits.h            <- constantes/defaults
│   │   ├── aadv_en_tables.h            <- structs das tabelas
│   │   ├── aadv_en_mac.h               <- utilitarios de MAC
│   │   ├── aadv_en_neighbors.h
│   │   ├── aadv_en_routes.h
│   │   ├── aadv_en_rreq_cache.h
│   │   └── aadv_en_peers.h
│   └── src/                             <- implementacao
│       ├── aadv_en.c                    (537 linhas) fachada
│       ├── aadv_en_node.c               (1658 linhas) nucleo
│       ├── aadv_en_routes.c              (352)  tabela de rotas + precursores
│       ├── aadv_en_neighbors.c           (203)  tabela de vizinhos
│       ├── aadv_en_peers.c               (209)  cache LRU de peers ESP-NOW
│       ├── aadv_en_rreq_cache.c          (150)  cache de duplicatas RREQ
│       └── aadv_en_mac.c                 (59)   utilitarios de MAC
├── main/                               <- APLICACAO ESP-IDF
│   ├── CMakeLists.txt
│   ├── Kconfig.projbuild               <- menuconfig
│   ├── main.c                           (10)  entry point com branch
│   ├── app_demo.c                       (555) app legado (HELLO + DATA)
│   ├── app_demo.h
│   ├── app_proto_example.c              (1282) app com CLI + HEALTH/TEXT/CMD
│   ├── app_proto_example.h
│   ├── aadv_en_app_proto.c              (670) protocolo app-layer
│   └── aadv_en_app_proto.h
└── tests/                              <- perfis por papel
```

```

├── tc001/
│   ├── build_flash.sh
│   ├── node_a.defaults
│   └── node_b.defaults
├── tc002/
│   ├── build_flash.sh
│   ├── monitor_log.sh
│   └── node_{a,b,c}.defaults
├── tc005/
│   ├── build_flash.sh
│   ├── monitor_log.sh
│   └── node_{a,b,c,d}.defaults
├── tools/
│   ├── extract_monitor_metrics.py    <- analise + dashboard
│   ├── plot_monitor_metrics.py      (661) parsing de log -> CSV/JSON
│   ├── plot_comparison_metrics.py   (213) graficos por run
│   ├── draw_topology.py             (222) compara runs
│   ├── live_monitor.py              (349) Mermaid/DOT/SVG
│   └── live_monitor_web/             (700+) dashboard real-time backend
│       ├── index.html
│       ├── app.css
│       └── app.js
├── (build/, logs/ - gerados, gitignored)
└── sim/                               <- SIMULACAO LOCAL
    ├── README.md
    ├── run_sim.sh                    <- aceita basic|large|100|1000
    ├── aadv_en_sim.c                (331) 3 nos A-B-C
    ├── aadv_en_sim_large.c          (265) 6 nos A-F
    ├── aadv_en_sim_100.c            (180) grade 10x10
    └── aadv_en_sim_1000.c           (165) grade 32x32

```

Linhas totais aproximadas: **~9.700** (sem contar build/, logs/).

Camada 1 - Nucleo do protocolo (`firmware/components/aadv_en/`)

A biblioteca `aadv_en` é onde mora o protocolo de fato. Ela é portável: o mesmo núcleo roda no firmware ESP32 e na simulação em desktop.

Headers publicos (`include/`)

`aadv_en.h` (184 linhas) - API pública da lib

A única fachada que a aplicação consome. Define:

Tipo / funcao	O que e
<code>aodv_en_now_ms_fn</code>	callback que retorna tempo monotonic em ms
<code>aodv_en_adapter_tx_frame_fn</code>	callback que envia um frame no transporte
<code>aodv_en_adapter_t</code>	par <code>now_ms + tx_frame + user_ctx</code>
<code>aodv_en_app_callbacks_t</code>	<code>on_data</code> e <code>on_ack</code> que a app implementa
<code>aodv_en_stack_t</code>	handle opaco <code>{ void *impl }</code>
<code>aodv_en_stack_stats_t</code>	bloco de contadores cumulativos (16 contadores)
<code>aodv_en_route_snapshot_t</code>	snapshot de rota (somente leitura)
<code>aodv_en_overview_t</code>	resumo (count rotas/vizinhos + stats)
<code>aodv_en_stack_init/deinit</code>	ciclo de vida da stack
<code>aodv_en_stack_tick(_at)</code>	loop periodico
<code>aodv_en_stack_send_hello(_at)</code>	emite HELLO
<code>aodv_en_stack_send_data(_at)</code>	envia DATA com ACK opcional
<code>aodv_en_stack_on_rcv(_at)</code>	entrega frame recebido
<code>aodv_en_stack_on_link_tx_result(_at)</code>	reporta resultado do TX
<code>aodv_en_stack_get_overview/get_route_count/get_route_at/get_stats</code>	leitura

Versoes `_at` recebem `now_ms` explicito; versoes sem `_at` chamam `now_ms()` do adapter automaticamente.

`aodv_en_node.h` (151 linhas) - API do nucleo (uso interno + sim)

Interface mais baixo nivel, usada pela fachada e diretamente pela simulacao.

Tipo / funcao	O que e
<code>aodv_en_emit_frame_fn</code>	callback de emissao (interno)
<code>aodv_en_deliver_data_fn</code> , <code>aodv_en_ack_received_fn</code>	callbacks pra app
<code>aodv_en_node_callbacks_t</code>	bundle dos 3 callbacks acima
<code>aodv_en_stats_t</code>	mesmos 16 contadores (espelhado em <code>stack_stats</code>)
<code>aodv_en_pending_data_entry_t</code>	item da fila de DATA pendente
<code>aodv_en_pending_ack_entry_t</code>	item da fila de ACK pendente
<code>aodv_en_node_t</code>	a struct gigante: config, self_mac, sequencias, todas as tabelas, ambas as filas, callbacks, stats
<code>aodv_en_node_init/set_callbacks/tick</code>	ciclo de vida e tick
<code>aodv_en_node_send_hello/send_data</code>	TX
<code>aodv_en_node_on_rcv/on_link_tx_result</code>	RX e feedback

`aodv_en_messages.h` (85 linhas) - Layout no fio

Define todas as structs de mensagem, todas com `__attribute__((packed))` :

```

aadv_en_header_t      14 bytes (cabecalho comum)
aadv_en_hello_msg_t   28 bytes
aadv_en_rreq_msg_t    39 bytes
aadv_en_rrep_msg_t    34 bytes
aadv_en_rerr_msg_t    24 bytes
aadv_en_ack_msg_t     30 bytes
aadv_en_data_msg_t    33 bytes header + payload[N]

```

Constantes de flags: `AADV_EN_MSG_FLAG_NONE/ACK_REQUIRED/ROUTE_REPAIR`.

Layout byte-a-byte de cada mensagem esta documentado em [aadv-en-funcionamento.md - Mensagens no fio](#).

`aadv_en_types.h` (112 linhas) - Tipos centrais e config

Tipo	Uso
<code>aadv_en_message_type_t</code>	enum HELLO/RREQ/RREP/RERR/DATA/ACK
<code>aadv_en_neighbor_state_t</code>	enum INACTIVE/ACTIVE
<code>aadv_en_route_state_t</code>	enum INVALID/REVERSE/VALID
<code>aadv_en_peer_flags_t</code>	enum bitmask NONE/PINNED/REGISTERED
<code>aadv_en_mac_addr_t</code>	wrapper com 6 bytes
<code>aadv_en_neighbor_entry_t</code>	linha da tabela de vizinhos
<code>aadv_en_route_entry_t</code>	linha da tabela de rotas (com precursores)
<code>aadv_en_rreq_cache_entry_t</code>	linha do cache de duplicatas
<code>aadv_en_peer_cache_entry_t</code>	linha do cache de peers
<code>aadv_en_config_t</code>	struct de configuracao do no

`aadv_en_status.h` (25 linhas) - Codigos de retorno

```

AADV_EN_OK           = 0
AADV_EN_NOOP         = 1  // operacao sem efeito (RREQ duplicado)
AADV_EN_QUEUED       = 2  // DATA enfileirado, aguardando rota
AADV_EN_ERR_ARG      = -1
AADV_EN_ERR_FULL     = -2  // tabela ou fila lotada
AADV_EN_ERR_NOT_FOUND = -3
AADV_EN_ERR_EXISTS   = -4
AADV_EN_ERR_NO_ROUTE = -5
AADV_EN_ERR_SIZE     = -6
AADV_EN_ERR_PARSE    = -7
AADV_EN_ERR_STATE    = -8  // adapter ou estado interno inconsistente

```

`aadv_en_limits.h` (86 linhas) - Constantes e defaults

Toda constante magica do protocolo nasce aqui. Todas tem versao `_DEFAULT` e versao "efetiva" (que pode ser sobrescrita por `-D` no compilador).

Categorias:

```

AODV_EN_PROTOCOL_VERSION      1
AODV_EN_MAC_ADDR_LEN          6

// tamanhos de tabela (compile-time, arrays estaticos)
AODV_EN_NEIGHBOR_TABLE_SIZE    16
AODV_EN_ROUTE_TABLE_SIZE      32
AODV_EN_RREQ_CACHE_SIZE       64
AODV_EN_PEER_CACHE_SIZE       8
AODV_EN_PENDING_DATA_QUEUE_SIZE 4
AODV_EN_MAX_PRECURSORS        4

// limites de payload
AODV_EN_CONTROL_PAYLOAD_MAX    128
AODV_EN_DATA_PAYLOAD_MAX      1024

// hops e ttl
AODV_EN_MAX_HOPS               16
AODV_EN_TTL_DEFAULT            16

// timers (ms)
AODV_EN_NEIGHBOR_TIMEOUT_MS    15000
AODV_EN_ROUTE_LIFETIME_MS      30000
AODV_EN_RREQ_CACHE_TIMEOUT_MS  10000
AODV_EN_ACK_TIMEOUT_MS         1000

// retries e thresholds
AODV_EN_RREQ_RETRY_COUNT       3
AODV_EN_LINK_FAIL_THRESHOLD    3

// hysteresis de troca de rota
AODV_EN_ROUTE_SWITCH_MIN_SEQ_GAIN  2
AODV_EN_ROUTE_SWITCH_MIN_METRIC_GAIN 1
AODV_EN_ROUTE_SWITCH_MIN_HOP_GAIN  1
AODV_EN_ROUTE_SWITCH_MIN_LIFETIME_GAIN_MS 5000

// metric INFINITY
AODV_EN_ROUTE_METRIC_INFINITY    0xFFFF

```

aodv_en_tables.h (38 linhas) - Wrappers das tabelas

Cada tabela e um array fixo + `count` :

```

aodv_en_neighbor_table_t { count, entries[NEIGHBOR_TABLE_SIZE] }
aodv_en_route_table_t   { count, entries[ROUTE_TABLE_SIZE] }
aodv_en_rreq_cache_t     { count, entries[RREQ_CACHE_SIZE] }
aodv_en_peer_cache_t     { count, entries[PEER_CACHE_SIZE] }

```

Os arrays sao alocados dentro de `aodv_en_node_t` (sem heap no caminho critico).

aodv_en_mac.h (23 linhas)

Funcao	O que faz
<code>aodv_en_mac_equal</code>	compara dois MACs
<code>aodv_en_mac_is_zero</code>	testa MAC <code>00:00:00:00:00:00</code>
<code>aodv_en_mac_is_broadcast</code>	testa MAC <code>FF:FF:FF:FF:FF:FF</code>
<code>aodv_en_mac_copy</code>	memcpy de 6 bytes
<code>aodv_en_mac_clear</code>	zera 6 bytes

`aadv_en_neighbors.h` (50 linhas)

API de operacoes na tabela de vizinhos. Detalhes na implementacao em `.c`.

`aadv_en_routes.h` (57 linhas)

API de operacoes na tabela de rotas + precursores.

`aadv_en_rreq_cache.h` (36 linhas)

API do cache de duplicatas de RREQ.

`aadv_en_peers.h` (45 linhas)

API do cache LRU de peers ESP-NOW.

Implementacao (`src/`)

`aadv_en_node.c` (1.658 linhas) - O coracao do protocolo

Este e o arquivo mais denso. Vou mapear por blocos de linha aproximada:

Linha	Bloco	O que faz
1-7	imports + constantes	inclui headers; define <code>BROADCAST_MAC</code>
9-17	forward declarations	<code>send_rreq</code> e <code>flush_pending_data_for_destination</code>
19-56	helpers de checagem	<code>is_self</code> , <code>route_is_usable</code> , <code>find_usable_route</code>
58-97	<code>aadv_en_node_emit</code>	wrapper de emissao via callback (atualiza peer cache + neighbor mark_used)
99-113	<code>aadv_en_fill_header</code>	escreve cabecalho comum em qualquer mensagem
115-204	helpers de pending_data	<code>clear</code> , <code>find_free</code> , <code>find_oldest</code> , <code>find_oldest_for_destination</code>
206-250	<code>send_data_with_sequence</code>	monta frame DATA com sequence_number e emite
252-311	helpers de pending_ack	<code>clear</code> , <code>find_free</code> , <code>find_oldest</code>
313-359	<code>track_pending_ack</code>	adiciona DATA na fila de ACK pendente
361-397	<code>pending_ack_consume</code>	consome (remove) entrada quando ACK chega
399-447	<code>send_data_via_route</code>	combina send + track_pending_ack
449-518	<code>queue_data</code>	enfileira DATA com backpressure (recicla mais antigo do mesmo destino)
520-547	<code>pending_note_rreq_attempt</code>	marca tentativa de descoberta numa entrada pendente
549-572	<code>pending_destination_seen</code>	dedup helper para evitar disparar 2 RREQs do mesmo destino
574-614	<code>discovery_retry_interval_ms</code>	backoff exponencial sobre <code>ack_timeout_ms</code> ate teto de 10s
616-660	<code>trigger_discovery_for_pending_destinations</code>	dispara RREQ pra todos os destinos pendentes (usado apos quebra)
662-714	<code>retry_route_discovery_for_pending</code>	reemite RREQ respeitando backoff (chamado no tick)
716-767	<code>flush_pending_data_for_destination</code>	drena fila quando rota e instalada
769-801	<code>expire_pending_data</code>	remove entradas que excedem <code>route_lifetime_ms</code>
803-878	<code>process_pending_ack_retries</code>	retransmite DATA pendente apos timeout, reduz <code>retries_left</code>
880-909	<code>send_rreq</code>	emite RREQ broadcast, incrementa <code>self_seq_num</code> e <code>next_rreq_id</code> , registra no cache
911-944	<code>send_rrep</code>	regras RFC 3561 §6.6.1 para <code>self_seq_num</code> + envia unicast pelo caminho reverso
946-966	<code>send_rerr</code>	emite RERR de 1 destino (unicast ou broadcast)

Linha	Bloco	O que faz
968-1000	<code>notify_precursors_of_break</code>	propaga RERR para precursores excluindo o <code>next_hop</code> quebrado; broadcast como fallback
1002-1023	<code>send_ack</code>	emite ACK pelo caminho reverso
1025-1042	<code>forward_rreq</code>	repassa RREQ com <code>hop_count++</code> e <code>ttl--</code>
1044-1062	<code>forward_rrep</code>	repassa RREP pelo caminho reverso
1064-1098	<code>forward_data</code>	repassa DATA, decrementa TTL, adiciona precursor (RFC 3561 §6.6.2)
1100-1118	<code>handle_hello</code>	instala vizinho + rota direta de 1 salto
1120-1162	<code>handle_rreq</code>	dedup por cache, instala rota reversa, responde se for o destino, senao reencaminha
1164-1211	<code>handle_rrep</code>	instala rota direta VALID, mantem precursores (3 atualizacoes RFC 3561 §6.6.2), drena pending data
1213-1239	<code>handle_rerr</code>	invalida rota local e propaga para precursores excluindo o sender
1241-1273	<code>handle_ack</code>	consome pending_ack ou repassa unicast
1275-1310	<code>handle_data</code>	entrega ao callback se for destino + envia ACK; senao repassa
1312-1356	<code>validate_header</code> , <code>validate_message_size</code>	sanity check antes de despachar
1358-1380	<code>aodv_en_config_set_defaults</code>	popula <code>aodv_en_config_t</code> com defaults
1382-1409	<code>aodv_en_node_init</code>	zera tabelas, copia config, copia self_mac
1411-1427	<code>aodv_en_node_set_callbacks</code>	injeta callbacks
1429-1444	<code>aodv_en_node_tick</code>	expiracao de vizinhos/rotas/cache + retry route discovery + retry ACK + expira pending data
1446-1533	<code>aodv_en_node_on_link_tx_result</code>	conta falhas de unicast, marca vizinho INACTIVE, invalida rotas em cascata, notifica precursores
1535-1552	<code>aodv_en_node_send_hello</code>	monta e emite HELLO
1554-1610	<code>aodv_en_node_send_data</code>	tenta enviar; se sem rota: enfileira + dispara RREQ + retorna <code>AODV_EN_QUEUED</code>
1612-1658	<code>aodv_en_node_on_recv</code>	valida frame, atualiza vizinho, dispatch para <code>handle_*</code> por tipo

aadv_en_routes.c (352 linhas) - Tabela de rotas + precursores

Linha	Funcao	O que faz
7-23	<code>route_remove_at</code> (static)	shift do array
25-35	<code>next_hop_changed</code>	helper para hysteresse
37-82	<code>candidate_is_strongly_better</code>	verifica ganho forte para evitar flap (4 criterios: seq, metric, hop, lifetime)
84-92	<code>aadv_en_route_table_init</code>	zera tabela
94-114	<code>aadv_en_route_find</code>	busca linear por destination
116-121	<code>find_const</code>	wrapper const
123-135	<code>find_valid</code>	retorna so se <code>state == VALID</code>
137-201	<code>should_replace</code>	decisao de upsert: estado, hysteresse, seq_num, metric, hop_count, expires_at_ms (ordem de prioridade)
203-251	<code>aadv_en_route_upsert</code>	insere ou substitui; preserva precursores quando next_hop e o mesmo , reseta quando muda
253-279	<code>aadv_en_route_add_precursor</code>	adiciona MAC ao array de precursores (idempotente, retorna FULL se cheio)
281-298	<code>aadv_en_route_invalidate_destination</code>	marca state=INVALID, metric=INFINITY
300-325	<code>aadv_en_route_invalidate_by_next_hop</code>	invalida em massa quando vizinho cai
327-352	<code>aadv_en_route_expire</code>	remove entradas com <code>expires_at_ms <= now_ms</code>

Hysteresse explicada: quando uma candidata `VALID` chega para um destino que ja tem rota `VALID` mas com `next_hop` diferente, `should_replace` exige que `candidate_is_strongly_better` retorne true. Isso evita "flapping" entre caminhos quase equivalentes.

aadv_en_neighbors.c (203 linhas) - Tabela de vizinhos

Funcao	O que faz
<code>neighbor_remove_at</code> (static)	shift do array
<code>neighbor_weighted_rssi</code> (static)	EMA: $(3 * \text{old} + \text{new}) / 4$
<code>neighbor_table_init</code>	zera
<code>aadv_en_neighbor_find</code> (+const)	busca por MAC
<code>aadv_en_neighbor_touch</code>	atualiza ou cria vizinho com EMA do RSSI; se vinha de INACTIVE, re-inicializa o EMA com o RSSI atual; sempre marca ACTIVE e zera <code>link_fail_count</code>
<code>aadv_en_neighbor_mark_used</code>	apenas atualiza <code>last_used_ms</code>
<code>aadv_en_neighbor_note_link_failure</code>	incrementa <code>link_fail_count</code> ; ao atingir threshold, marca INACTIVE
<code>aadv_en_neighbor_expire</code>	remove entradas com <code>last_seen_ms</code> antigo demais
<code>aadv_en_neighbor_count_active</code>	conta vizinhos ACTIVE

`aadv_en_peers.c` (209 linhas) - Cache LRU de peers ESP-NOW

Independente da tabela de vizinhos. Existe pra controlar quantos peers estao registrados no driver `ESP-NOW` simultaneamente (limite de hardware).

Funcao	O que faz
<code>peer_find_index</code> (static)	busca posicao por MAC
<code>peer_find_lru_index</code> (static)	encontra menos-recente, pulando entradas com <code>PINNED</code>
<code>peer_remove_at</code> (static)	shift
<code>aadv_en_peer_cache_init</code>	zera
<code>aadv_en_peer_find</code> (+const)	busca
<code>aadv_en_peer_touch</code>	atualiza <code>last_used_ms</code> ; se cache cheio, evicta LRU nao-pinned
<code>aadv_en_peer_set_registered/set_pinned</code>	flips de flags
<code>aadv_en_peer_remove</code>	delete explicito

`aadv_en_rreq_cache.c` (150 linhas) - Cache de duplicatas RREQ

Funcao	O que faz
<code>rreq_cache_find_index</code> (static)	busca por <code>(originator, rreq_id)</code>
<code>oldest_index</code> (static)	indice da entrada mais antiga por <code>created_at_ms</code>
<code>remove_at</code> (static)	shift
<code>aadv_en_rreq_cache_init</code>	zera
<code>aadv_en_rreq_cache_contains</code>	bool de existencia
<code>aadv_en_rreq_cache_remember</code>	insere; se cheio, substitui o mais antigo (politica de eviction documentada na spec v1)
<code>aadv_en_rreq_cache_expire</code>	remove entradas com idade <code>> rreq_cache_timeout_ms</code>

`aadv_en_mac.c` (59 linhas) - Utilitarios de MAC

5 funcoes simples (compare, is_zero, is_broadcast, copy, clear). Reutilizadas em todos os modulos.

Camada 2 - Fachada da lib (`aadv_en.c`)

`aadv_en.c` (537 linhas) - `aadv_en_stack_*` API

A camada que a aplicacao consome. Esconde o `aadv_en_node_t` por tras de um handle opaco.

```
typedef struct {
    aadv_en_node_t      node;
    aadv_en_adapter_t   adapter;      // now_ms + tx_frame
    aadv_en_app_callbacks_t app_callbacks; // on_data + on_ack
} aadv_en_stack_impl_t;
```

Bloco	O que faz
linhas 8-33	helpers para casting <code>stack_t* -> stack_impl_t*</code>
35-60	<code>stats_from_node</code> - copia 16 contadores do node para o snapshot
62-82	<code>stack_emit_frame</code> - bridge entre <code>aodv_en_emit_frame_fn</code> (interno) e <code>adapter.tx_frame</code> (externo)
84-120	<code>stack_deliver_data</code> e <code>stack_ack_received</code> - bridges para callbacks da app
122-138	<code>stack_now_ms</code> - helper que chama o <code>adapter.now_ms</code> com guard de NULL
140-194	<code>aodv_en_stack_init</code> - calloc() unico para <code>stack_impl_t</code> , copia adapter+app_callbacks, inicializa <code>node</code> interno e injeta os bridges
196-208	<code>stack_deinit</code> - <code>free</code>
210-228	<code>set_app_callbacks</code> - troca callbacks em runtime
230-264	<code>tick(_at)</code> - delega para <code>aodv_en_node_tick</code>
266-299	<code>send_hello(_at)</code>
301-354	<code>send_data(_at)</code>
356-409	<code>on_rcv(_at)</code>
411-460	<code>on_link_tx_result(_at)</code>
462-478	<code>get_overview</code> - retorna count + stats
480-491	<code>get_route_count</code>
493-521	<code>get_route_at</code> - copia para <code>aodv_en_route_snapshot_t</code> (so leitura)
523-537	<code>get_stats</code>

Heap usage: a fachada usa `calloc` apenas no `init` para alocar `stack_impl_t` (~600 bytes). Tudo mais e estatico dentro da struct. Caminho critico (`tick`, `on_rcv`) nao usa heap.

Camada 3 - Aplicacao firmware (`firmware/main/`)

`main.c` (10 linhas) - Entry point

```
#include "sdkconfig.h"
#include "app_demo.h"
#include "app_proto_example.h"

void app_main(void) {
#ifdef CONFIG_AODV_EN_APP_USE_APP_DEMO
    app_demo_run();
#else
    app_proto_example_run();
#endif
}
```

A escolha vem do `Kconfig.projbuild` (menuconfig), com default em `app_demo` (conservador, mais simples).

`app_demo.c` (555 linhas) - App legado simples

App de bancada minimal. Emite HELLO periodico e opcionalmente DATA periodico para um MAC alvo.

Bloco	O que faz
1-30	imports + macros (<code>APP_RX_QUEUE_LEN=8</code> , <code>APP_TX_RESULT_QUEUE_LEN=16</code> , <code>APP_LOOP_DELAY_MS=100</code>)
38-50	structs de eventos: <code>app_rx_event_t</code> (frame recebido), <code>app_tx_result_event_t</code> (resultado de TX)
52-72	<code>app_context_t</code> - guarda stack, queues, configs, agendamento de proximos eventos
75-104	helpers: <code>app_now_ms</code> (esp_timer), <code>app_format_mac</code> , <code>app_parse_mac</code> , <code>app_mac_is_broadcast</code>
153-169	<code>app_ensure_peer</code> - registra peer no driver ESP-NOW se ainda nao esta
171-205	<code>app_emit_frame</code> - implementacao do <code>tx_frame</code> do adapter; chama <code>esp_now_send</code>
207-230	<code>app_deliver_data</code> , <code>app_ack_received</code> - callbacks da app
232-262	<code>app_send_cb</code> - callback do ESP-NOW (resultado de tx); empurra para queue
264-288	<code>app_rcv_cb</code> - callback do ESP-NOW (frame recebido); empurra para queue
290-332	<code>app_log_routes</code> - dump periodico de stats e tabela de rotas no log
334-376	<code>process_rx_queue</code> , <code>process_tx_result_queue</code> - drena queues no contexto da task
378-435	<code>app_protocol_task</code> - loop principal: drain queues, tick, send hello, send data, log
437-477	<code>app_init_nvs</code> , <code>app_init_wifi</code> , <code>app_init_espnow</code> - boilerplate ESP-IDF
479-555	<code>app_demo_run</code> - entry point: setup tudo + cria task

`app_proto_example.c` (1.282 linhas) - App com CLI + protocolo de aplicacao

Mesmo esqueleto do `app_demo.c` , mais:

- camada de aplicacao tipada (`aadv_en_app_proto_t`) embutida no contexto
- task adicional `app_cli_task` que le `stdin` e despacha comandos
- handlers `cmd_ping` , `cmd_echo` , `cmd_info`
- broadcast periodico de `HEALTH_REQ`
- envio periodico opcional de `TEXT` e `CMD_REQ` para um target

Bloco	O que faz
1-65	imports + macros + flags Kconfig (<code>APP_PROTO_ENABLE_UNICAST</code> , <code>APP_PROTO_ENABLE_CLI</code>)
65-120	structs: <code>app_rx_event_t</code> , <code>app_tx_result_event_t</code> , <code>app_cli_event_t</code> , <code>app_context_t</code> (com <code>aodv_en_app_proto_t</code> embutido)
124-200	helpers de mac formatting e parsing
202-237	helpers de string e <code>app_log_cli_help</code>
239-401	<code>app_cli_parse_line</code> - parse <code>help</code> , <code>routes</code> , <code>health all <mac></code> , <code>text <mac> <msg></code> , <code>cmd <mac> <comando></code> [args]
403-455	<code>app_emit_frame</code> - mesmo do <code>app_demo</code>
457-479	bridges: <code>app_proto_send_data</code> (<code>send_data</code> do transport do <code>app_proto</code> chama <code>aodv_en_stack_send_data_at</code>) e <code>app_proto_now_ms</code>
481-561	callbacks <code>app_proto</code> : <code>on_health_req</code> , <code>on_health_rsp</code> , <code>on_text</code> , <code>on_cmd_req</code> , <code>on_cmd_rsp</code> (logging)
563-648	handlers de comandos: <code>cmd_ping</code> , <code>cmd_echo</code> , <code>cmd_info</code>
650-686	<code>app_stack_on_data</code> (passa pro <code>app_proto</code>), <code>app_stack_on_ack</code>
688-744	callbacks ESP-NOW (<code>send_cb</code> , <code>recv_cb</code>)
746-832	<code>app_log_routes</code> , <code>process_rx_queue</code> , <code>process_tx_result_queue</code>
834-940	<code>app_process_cli_queue</code> - dispatcher para os comandos digitados
942-1009	<code>app_cli_task</code> - le stdin caractere por caractere, monta linhas, parse e enfileira
1011-1122	<code>app_protocol_task</code> - loop principal: drain queues, tick, hello, <code>health_req</code> , <code>text/cmd</code> opcional, log
1124-1164	<code>init_nvs</code> , <code>init_wifi</code> , <code>init_espnw</code>
1166-1282	<code>app_proto_example_run</code> - setup + 2 tasks (<code>app_protocol_task</code> + <code>app_cli_task</code>)

`aodv_en_app_proto.h` (161 linhas) - API do protocolo de aplicacao

Define o protocolo app-layer que roda **acima** do AODV-EN. Mensagens:

```

HEALTH_REQ = 1 // ping de saude (broadcast ou unicast)
HEALTH_RSP = 2 // resposta automatica do node
TEXT       = 3 // mensagem de texto livre
CMD_REQ    = 4 // requisicao de comando
CMD_RSP    = 5 // resposta de comando

```


Tipo	Uso
<code>aadv_en_app_proto_t</code>	struct com <code>node_name</code> , <code>request_id</code> sequencial, <code>transport</code> , <code>callbacks</code> , comandos registrados
<code>aadv_en_app_proto_transport_t</code>	par <code>send_data</code> + <code>now_ms</code> (injetado pela aplicacao)
<code>aadv_en_app_proto_callbacks_t</code>	callbacks por tipo de mensagem
<code>aadv_en_app_proto_command_handler_fn</code>	assinatura de handler de comando custom
funcoes <code>_init</code> , <code>_set_callbacks</code> , <code>_register_command</code> , <code>_send_health_req</code> , <code>_send_text</code> , <code>_send_command</code> , <code>_on_mesh_payload</code>	API de uso

`aadv_en_app_proto.c` (670 linhas) - Implementacao do protocolo de aplicacao

Bloco	O que faz
7-43	helpers de leitura/escrita LE de uint16 e uint32
45-67	<code>next_request_id</code> (incremento idempotente que pula 0) e <code>now_ms</code>
69-113	<code>send_frame</code> - monta header de 8 bytes (<code>version type body_len request_id</code>) + body, e chama <code>transport.send_data</code>
115-149	<code>decode</code> - valida e parseia header de 8 bytes
151-178	<code>copy_text</code> - extrai string com size guard
180-205	<code>find_command</code> - busca handler por nome
207-237	<code>send_health_rsp</code> - resposta automatica com <code>node=NAME uptime_ms=N</code>
239-330	<code>handle_cmd_req</code> - parse <code>cmd_len cmd args_len args</code> , despacha handler ou retorna erro 404, monta <code>CMD_RSP</code>
332-380	<code>init</code> , <code>set_callbacks</code>
382-431	<code>register_command</code> - adiciona ao array de comandos (max 8)
433-500	<code>send_health_req</code> , <code>send_text</code>
502-563	<code>send_command</code> - encapsula <code>cmd args</code> no body
565-670	<code>on_mesh_payload</code> - dispatcher por tipo de mensagem (health_req/rsp, text, cmd_req, cmd_rsp)

Esta camada **nao faz parte do AODV-EN**. E uma aplicacao que consome a malha. A spec v1 afirma isso explicitamente. Pode ser substituida por qualquer outra app sem afetar o protocolo.

Camada 4 - Build system

`firmware/CMakeLists.txt` (4 linhas)

Project root. Inclui o `project.cmake` do ESP-IDF e declara `project(aadv_en_firmware)`.

firmware/main/CMakeLists.txt (9 linhas)

Registra o componente `main` com 4 sources (`main.c`, `app_demo.c`, `app_proto_example.c`, `aodv_en_app_proto.c`) e dependencia em `aodv_en` + componentes ESP-IDF (`esp_wifi`, `nvs_flash`, `esp_event`, `esp_netif`, `esp_timer`).

firmware/components/aodv_en/CMakeLists.txt (12 linhas)

Registra o componente `aodv_en` com 7 sources (todas as `.c` em `src/`) e `INCLUDE_DIRS "include"`. Sem dependencias de ESP-IDF (e portavel).

firmware/main/Kconfig.projbuild (99 linhas)

Define o menu `AODV-EN App` no `idf.py menuconfig`. Campos:

Config	Range/default
<code>AODV_EN_APP_EXAMPLE_MODE</code>	choice: <code>app_demo</code> (default) ou <code>app_proto_example</code>
<code>AODV_EN_APP_NODE_NAME</code>	string default <code>NODE_A</code>
<code>AODV_EN_APP_NETWORK_ID</code>	hex default <code>0xA0DE0001</code>
<code>AODV_EN_APP_WIFI_CHANNEL</code>	int 1-13 default 6
<code>AODV_EN_APP_HELLO_INTERVAL_MS</code>	int 1000-60000 default 5000
<code>AODV_EN_APP_SEND_INTERVAL_MS</code>	int 1000-60000 default 7000
<code>AODV_EN_APP_PRINT_INTERVAL_MS</code>	int 1000-60000 default 10000
<code>AODV_EN_APP_ENABLE_DATA</code>	bool default n
<code>AODV_EN_APP_TARGET_MAC</code> , <code>_PAYLOAD_TEXT</code>	string (so quando <code>ENABLE_DATA=y</code>)
<code>AODV_EN_APP_PROTO_HEALTH_INTERVAL_MS</code> , <code>_UNICAST_INTERVAL_MS</code> , <code>_ENABLE_UNICAST</code> , <code>_ENABLE_CLI</code> , <code>_TARGET_MAC</code> , <code>_TEXT</code> , <code>_COMMAND</code> , <code>_COMMAND_ARGS</code>	configs do <code>app_proto_example</code>

firmware/sdkconfig.defaults (2 linhas)

Configura `ESPT00LPY_FLASHSIZE=4MB`. So.

Camada 5 - Scripts

Build / flash / monitor (firmware/)

Script	O que faz
<code>build.sh</code> (11)	source <code>idf-env.sh</code> + <code>cd firmware</code> + <code>idf.py set-target esp32</code> + <code>idf.py build</code>
<code>flash_monitor.sh</code> (16)	recebe porta + flash + monitor numa task
<code>monitor_log.sh</code> (107)	versao mais elaborada: tee para arquivo em <code>firmware/logs/serial/<label>_<timestamp>.log</code>
<code>idf-env.sh</code> (59)	bootstrap ESP-IDF (busca em <code>ESP_IDF_EXPORT</code> , <code>IDF_PATH/export.sh</code> , <code>~/esp/esp-idf/export.sh</code> , ou <code>idf.py</code> no PATH)

Por TC (`firmware/tests/tcXXX/`)

Script	O que faz
<code>tc001/build_flash.sh</code>	recebe <code>node_a node_b <PORT></code> e (so para a) <code><TARGET_MAC></code> . Builda em <code>build/tc001_node_X/</code> e flasha.
<code>tc002/build_flash.sh</code>	mesma logica para 3 papeis
<code>tc002/monitor_log.sh</code>	wrapper que chama <code>firmware/monitor_log.sh -B build/tc002_node_X -l node_X -t <run_tag></code>
<code>tc005/build_flash.sh</code>	mesma logica para 4 papeis
<code>tc005/monitor_log.sh</code>	wrapper para os 4
<code>tcXXX/node_*.defaults</code>	overrides de Kconfig (network_id, channel, intervals, ENABLE_DATA, payload_text). Aplicados via <code>SDKCONFIG_DEFAULTS=...defaults;perfil</code>

Nota importante: `build_flash.sh` faz `cd "$FW_DIR"` antes de invocar `idf.py` (foi um fix recente). Sem isso, `idf.py` procura `CMakeLists.txt` no cwd do invocador.

Camada 6 - Simulacao (`sim/`)

A simulacao roda o **mesmo nucleo** do firmware (lib `aadv_en` em C) com mocks de transporte. Compila com `cc` em desktop, sem ESP-IDF.

`aadv_en_sim.c` (331 linhas) - Simulacao basica 3 nos

Cenario A-B-C linear. Validation completa do fluxo:

Fase	O que valida
route discovery	A envia DATA sem rota -> retorna <code>AADV_EN_QUEUED</code> ; verifica rotas instaladas em A e B
data	segundo <code>send_data</code> retorna OK; verifica <code>delivered_frames</code> em C e <code>ack_received</code> em A
ack retry	seta <code>drop_next_ack_to_a</code> ; verifica que <code>ack_retry_sent</code> cresce e ACK chega depois
late join retry discovery	quebra link B-C, invalida rotas, espera retries de RREQ; reativa link e verifica entrega

Estrutura:

```
typedef struct sim_network {
    uint32_t now_ms;
    bool links[3][3];           // matriz de conectividade
    bool drop_next_ack_to_a;    // injecao de falha
    uint8_t macs[3][6];
    aadv_en_node_t nodes[3];
    sim_endpoint_t endpoints[3];
} sim_network_t;
```

Funcoes:

- `sim_emit_frame` - simula radio: broadcast entrega para todos os links ativos; unicast verifica link
- `sim_deliver_data`, `sim_ack_received` - logging
- `sim_init_network` - configura 3 nos com MACs e callbacks
- `sim_tick_all` - chama `aadv_en_node_tick` em todos
- `main` - orquestra as 4 fases com asserts

`aodv_en_sim_large.c` (265 linhas)

6 nos em cadeia A-B-C-D-E-F. Foco em propagacao de RERR. Quebra link D-E e verifica que RERR propaga corretamente para A.

`aodv_en_sim_100.c` (180 linhas)

Grade 10x10 (100 nos). Tenta descobrir rota de (0,0) ate (9,9). Quebra coluna central (parede) e verifica reconvergencia em torno.

`aodv_en_sim_1000.c` (165 linhas)

Grade 32x32 (1024 nos) com 5% de falhas aleatorias. Cenario "smart city". Verifica robustez em escala.

`run_sim.sh` (60 linhas)

Aceita argumento opcional: `basic` (default), `large`, `100`, `1000`. Compila o `.c` correspondente com `cc -std=c11 -Wall -Wextra` em `/tmp/aodv_en_sim_<variant>` e executa.

Camada 7 - Ferramentas de analise (`firmware/tools/`)

`extract_monitor_metrics.py` (661 linhas) - Parsing de log -> CSV/JSON

Le um arquivo `.log` do `idf.py monitor` e produz um diretorio de analise com:

- `summary.json` - metricas agregadas
- `summary.txt` - mesma coisa em texto
- `events.csv` - todos os eventos parseados (timeline)
- `snapshots.csv` - snapshots periodicos com tx/rx/delivered deltas
- `routes.csv` - linhas de rota associadas a cada snapshot
- `target_route_series.csv` - estado da rota para o destino alvo ao longo do tempo
- `target_route_transitions.csv` - mudancas de estado (flap detection)
- `ack_events.csv`, `fail_events.csv`, `data_status_events.csv`
- `discovery_windows.csv` - intervalos abertos por `data_queued_discovery` e fechados por `ack_received`
- `minute_metrics.csv` - bucketizado por minuto (ack/min, fail/min, hops_1, hops_2, etc.)

Regexes principais (linhas 11-25):

```
LOG_RE      = r"^s*([IWE]) \((\d+)\) ([^:]+): (.*)$"
NODE_RE     = r"node=([A-Z0-9_])\s+self_mac=...\s+channel=(\d+)\s+network_id=0x(...)"
SNAPSHOT_RE = r"routes=(\d+)\s+neighbors=(\d+)\s+tx=(\d+)\s+rx=(\d+)\s+delivered=(\d+)"
ROUTE_RE    = r"route\[ (\d+) \]\s+dest=...\s+via=...\s+hops=(\d+)\s+metric=(\d+)\s+state=(\d+)\s+expires=(\d+)"
ACK_RE      = r"ACK received from ... for seq=(\d+)"
DELIVER_RE  = r"DATA deliver from ...:"
SEND_FAIL_RE = r"ESP-NOW send fail to ..."
DATA_STATUS_RE = r"DATA send status=(-?\d+)"
INVALIDATED_RE = r"invalidated (\d+) route\s(\s) via ..."
```

Estas mesmas regexes sao reusadas pelo `live_monitor.py`.

`plot_monitor_metrics.py` (213 linhas) - Graficos por run

Gera 7 PNGs em `<analysis_dir>/plots/`:

1. `01_ack_vs_fail_per_minute.png`

2. `02_queue_and_status_per_minute.png`
3. `03_route_state_timeline.png`
4. `04_route_state_distribution.png`
5. `05_neighbors_routes_per_minute.png`
6. `06_discovery_window_durations.png`
7. `07_tx_rx_delta_over_time.png`

`plot_comparison_metrics.py` (222 linhas) - Compara runs

Recebe varios `--run label::summary.json` e plota:

1. `01_core_metrics.png` - ACK/min, fail/min, status_neg2/min, queued_discovery/min
2. `02_stability_metrics.png` - flap/h, target absent ratio, max blackout
3. `03_route_profile.png` - distribuicao de hops_1/hops_2/absent

Suporta marcador `--change-index N` que desenha linha vertical entre runs "pre" e "pos" mudanca, util para A/B test antes/depois de fix.

`draw_topology.py` (349 linhas) - Mermaid + DOT + SVG

Recebe um ou mais diretorios de analise e gera:

- `topology.mmd` - Mermaid (renderizavel em GitHub)
- `topology.dot` - Graphviz DOT
- `topology.svg` - se `dot` estiver instalado
- `topology.json` - resumo estruturado (nos + edges + contextos)

Modo `--mode latest` usa o ultimo snapshot de cada nó. Modo `--mode observed` faz uniao de todas as edges observadas.

`live_monitor.py` (~700 linhas) - Dashboard real-time backend

Componentes:

Bloco	O que faz
<code>find_esptool</code> , <code>read_mac_via_esptool</code>	descobre MAC sem flashar (resolve alias->mac no startup)
Regexes (<code>LOG_RE</code> , <code>NODE_RE</code> , etc.)	mesmas do <code>extract_monitor_metrics.py</code>
<code>MeshState</code>	dicionario de nos + alias_to_mac + ring buffer de eventos
<code>Hub</code>	mantem subscribers WS + broadcast
<code>LineParser</code>	feed por linha; emite eventos (<code>node_seen</code> , <code>stats</code> , <code>routes</code> , <code>ack</code> , <code>deliver</code> , <code>send_fail</code> , <code>data_status</code> , <code>invalidated</code> , <code>queued_route</code> , <code>queued_discovery</code>); modo verbose imprime cada evento com cores ANSI
<code>SerialPump</code>	thread por porta; <code>pyserial.Serial.readline()</code> blocking; backoff exponencial em desconexao; tee opcional pra stdout em <code>-vv</code>
<code>serial_loop</code>	drena queue do <code>SerialPump</code> para <code>LineParser</code>
<code>demo_loop</code>	gera eventos sinteticos de uma malha A-B-C-D para teste sem hardware
<code>websocket_handler</code>	aiohttp WS - manda snapshot inicial + push de eventos
<code>index_handler</code> , <code>make_app</code>	aiohttp HTTP serve <code>live_monitor_web/</code>
<code>amain</code>	orquestra: pre-le MACs, abre serials, sobe HTTP+WS

`live_monitor_web/` - Dashboard frontend

Arquivo	O que faz
<code>index.html</code>	layout grid: graph card a esquerda + sidebar (resumo, lista de nos, eventos recentes)
<code>app.css</code>	estilos: paleta indigo/verde/laranja, console preto, animacoes pulse/flash
<code>app.js</code>	WebSocket client + Cytoscape.js: applyEvent dispatcha por tipo, atualiza state/sidebar/grafico, anima pulsePath em ACK, flash em send_fail/invalidated, ghost online quando rota e VALID

Camada 8 - Testes funcionais

Documentos (`docs/tests/`)

TC	Cenario	Cobre criterios da spec v1
<code>tc-001-descoberta-e-entrega-direta.md</code>	2 nos diretos	1
<code>tc-002-primeiro-multi-hop.md</code>	3 nos cadeia	2, 5
<code>tc-003-reconvergencia-apos-falha.md</code>	3 nos com falha intermediaria	3, 7
<code>tc-004-soak-estabilidade-e-reconvergencia.md</code>	3 nos sob ciclos por 30 min	4, 6
<code>tc-005-cadeia-4-nos.md</code>	4 nos cadeia (3 saltos)	2 estendido

Cada TC segue a estrutura definida em [docs/tests/README.md](#):

1. Objetivo
2. Topologia
3. Configuracao dos nos (com referencia ao perfil `.defaults`)
4. Procedimento (com comandos prontos)

5. Evidencias esperadas
6. Criterio de aprovacao
7. Resultado (preencher a cada execucao)

Perfis (`firmware/tests/`)

Veja Camada 5 - Scripts.

Camada 9 - Documentacao (`docs/`)

Arquivo	Tipo	Quem le
<code>aodv-en-spec-v1.md</code>	normativo	quem quer saber o que o protocolo deve fazer
<code>aodv-en-funcionamento.md</code>	tutorial	quem quer entender como funciona, com worked examples
<code>aodv-en-mapa-do-codigo.md</code>	guia de codigo	voce esta aqui - estudo do codigo
<code>aodv-en-estruturas-dados.md</code>	referencia	layout das tabelas e mensagens
<code>aodv-base-invariantes.md</code>	normativo	regras do AODV que nao podem ser quebradas
<code>aodv-en-spec-v0.md</code>	OBSOLETA	so registro historico
<code>plano-desenvolvimento-completo.md</code>	gestao	roadmap das fases do TCC
<code>runbook-bancada.md</code>	operacional	passo a passo de bancada
<code>features/enfilaremento-dos-dados.md</code>	feature integrada na v1	fila pendente de DATA
<code>features/precursores.md</code>	feature integrada na v1	precursores e RERR direcionado
<code>features/articulation-point-planejado.md</code>	feature de v2	detecao de no de corte
<code>tests/*</code>	casos de teste	bancada

Mapa por feature

A pergunta tipica: "**onde esta implementado X?**". Tabela cruzada:

Feature	Estado	Onde encontrar	Comentario
Cabecalho comum 14 bytes	implementado	aodv_en_messages.h struct <code>aodv_en_header_t</code>	preenchido por <code>aodv_en_fill_header</code> em <code>aodv_en_node.c</code>
HELLO emissao	implementado	aodv_en_node.c:1535 <code>aodv_en_node_send_hello</code>	chamado a cada <code>hello_interval_ms</code> pela app
HELLO recepcao	implementado	aodv_en_node.c:1100 <code>handle_hello</code>	instala vizinho + rota direta
Descoberta passiva de vizinhos	implementado	aodv_en_node.c:1612 <code>aodv_en_node_on_rcv</code> linha <code>aodv_en_neighbor_touch(...)</code>	qualquer frame valido toca o vizinho
RREQ emissao	implementado	aodv_en_node.c:880 <code>send_rreq</code>	incrementa <code>self_seq_num</code> e <code>next_rreq_id</code>
Dedup de RREQ	implementado	aodv_en_node.c:1120 <code>handle_rreq</code> checa <code>aodv_en_rreq_cache_contains</code>	cache em aodv_en_rreq_cache.c
RREP emissao (RFC 3561 §6.6.1)	implementado	aodv_en_node.c:911 <code>send_rrep</code>	regras de <code>self_seq_num</code> corretas
Precursores (RFC 3561 §6.6.2)	implementado	aodv_en_node.c:1164 <code>handle_rrep</code> ; storage em <code>aodv_en_route_entry_t.precursors</code>	3 atualizacoes: receive, forward target, reverse
RERR emissao	implementado	aodv_en_node.c:946 <code>send_rerr</code>	1 destino por RERR em v1
RERR direcionado a precursores	implementado	aodv_en_node.c:968 <code>notify_precursors_of_break</code>	broadcast como fallback se <code>precursor_count == 0</code>
Encaminhamento de DATA	implementado	aodv_en_node.c:1064 <code>forward_data</code>	adiciona <code>link_src_mac</code> como precursor (RFC 3561)
ACK fim a fim	implementado	aodv_en_node.c:1002, 1241 <code>send_ack</code> , <code>handle_ack</code>	dispara quando <code>flags & ACK_REQUIRED</code>
Fila de DATA pendente	implementado	aodv_en_node.c:449 <code>queue_data</code> (com backpressure)	drena em <code>flush_pending_data_for_destination</code> ao instalar rota VALID
Retry exponencial de RREQ	implementado	aodv_en_node.c:574 <code>discovery_retry_interval_ms</code>	backoff <code>ack_timeout_ms * 2^attempt</code> ate 10s
Fila de ACK pendente + retry	implementado	aodv_en_node.c:803 <code>process_pending_ack_retries</code>	<code>retries_left</code> compartilhado com <code>rreq_retry_count</code>
Hysteresse de troca de rota	implementado	aodv_en_routes.c:37 <code>candidate_is_strongly_better</code>	4 ganhos (seq, metric, hop, lifetime)
Selecao de rota	implementado	aodv_en_routes.c:137 <code>should_replace</code>	priority: state > hysteresse > seq > metric > hop > lifetime
Suavizacao EMA do RSSI	implementado	aodv_en_neighbors.c:25 <code>weighted_rssi</code>	$(3*old + new) / 4$; reset em INACTIVE->ACTIVE
Cascading de falha de vizinho	implementado	aodv_en_node.c:1446 <code>on_link_tx_result</code>	<code>link_fail_count >= threshold</code> -> INACTIVE -> invalida rotas + notifi precursores
Expiracao de rota por tempo	implementado	aodv_en_routes.c:327 <code>aodv_en_route_expire</code>	chamado em <code>aodv_en_node_tick</code>

Feature	Estado	Onde encontrar	Comentario
Expiracao de vizinho por tempo	implementado	aodv_en_neighbors.c:155 <code>aodv_en_neighbor_expire</code>	chamado em <code>aodv_en_node_tick</code>
Expiracao de RREQ cache	implementado	aodv_en_rreq_cache.c:123 <code>aodv_en_rreq_cache_expire</code>	chamado em <code>aodv_en_node_tick</code>
Cache LRU de peers ESP-NOW	implementado	aodv_en_peers.c:29 <code>peer_find_lru_index</code>	pula entradas PINNED
Validacao de header (version, network_id)	implementado	aodv_en_node.c:1312 <code>validate_header</code>	descarta cross-network/cross-version
Validacao de tamanho de mensagem	implementado	aodv_en_node.c:1329 <code>validate_message_size</code>	descarta DATA com <code>payload_length</code> inconsistente
Telemetria de stats	implementado	aodv_en_node.h <code>aodv_en_stats_t</code> (16 contadores); copia em aodv_en.c:35	exposta via <code>aodv_en_stack_get_overview/get_stats</code>
Adapter ESP-NOW	implementado	firmware/main/app_demo.c:171 <code>app_emit_frame</code> + callbacks <code>recv_cb/send_cb</code>	atualmente acoplado ao app, fase 4 vai extrair
Modo <code>app_demo</code>	implementado	firmware/main/app_demo.c	HELLO + DATA periodicos
Modo <code>app_proto_example</code>	implementado	firmware/main/app_proto_example.c	+ CLI + HEALTH/TEXT/CMD
Articulation point detection	planejada para v2	docs/features/articulation-point-planejado.md	nao implementada
Multi-destino por RERR	planejada para v2	spec v1 - secao "Caminho de evolucao para v2"	atualmente 1 destino por RERR
Wrap-around de seq number	parcial	comparacao por <code>></code> simples; suficiente para janelas curtas	v2: comparacao serial estilo RFC 1982
Metrica composta com RSSI	nao	<code>metric = hop_count</code> em v1; RSSI usado so para diagnostico	mapeado para v2

Cross-reference de constantes

Quando voce ver uma constante misteriosa no codigo, esta e a tabela:

Tamanhos de tabelas (compile-time)

Constante	Default	Onde define	Uso
<code>AODV_EN_NEIGHBOR_TABLE_SIZE</code>	16	aodv_en_limits.h:9	array de <code>aodv_en_neighbor_table_t.entries[]</code>
<code>AODV_EN_ROUTE_TABLE_SIZE</code>	32	aodv_en_limits.h:10	array de <code>aodv_en_route_table_t.entries[]</code>
<code>AODV_EN_RREQ_CACHE_SIZE</code>	64	aodv_en_limits.h:11	array de <code>aodv_en_rreq_cache_t.entries[]</code>
<code>AODV_EN_PEER_CACHE_SIZE</code>	8	aodv_en_limits.h:12	array de <code>aodv_en_peer_cache_t.entries[]</code>
<code>AODV_EN_PENDING_DATA_QUEUE_SIZE</code>	4	aodv_en_limits.h:13	filas <code>pending_data</code> e <code>pending_ack</code>
<code>AODV_EN_MAX_PRECURSORS</code>	4	aodv_en_limits.h:14	array de precursores por rota

Timers (runtime, configuraveis em `aadv_en_config_t`)

Constante	Default ms	Uso
<code>NEIGHBOR_TIMEOUT_MS</code>	15000	janela sem <code>last_seen</code> antes de expirar vizinho
<code>ROUTE_LIFETIME_MS</code>	30000	tempo de vida de rota nova; tambem janela de validade da fila pendente
<code>RREQ_CACHE_TIMEOUT_MS</code>	10000	janela do cache de duplicatas
<code>ACK_TIMEOUT_MS</code>	1000	timeout para retransmitir DATA com ACK_REQUIRED

Retries e thresholds

Constante	Default	Uso
<code>RREQ_RETRY_COUNT</code>	3	retries de RREQ e ACK (compartilhado em v1)
<code>LINK_FAIL_THRESHOLD</code>	3	falhas seguidas para marcar vizinho INACTIVE
<code>MAX_HOPS</code>	16	limite de saltos do protocolo
<code>TTL_DEFAULT</code>	16	TTL inicial em RREQ e DATA

Hysteresese de troca de rota

Constante	Default	Uso
<code>ROUTE_SWITCH_MIN_SEQ_GAIN</code>	2	ganho de <code>dest_seq_num</code> para mudar <code>next_hop</code>
<code>ROUTE_SWITCH_MIN_METRIC_GAIN</code>	1	ganho de <code>metric</code>
<code>ROUTE_SWITCH_MIN_HOP_GAIN</code>	1	reducao de <code>hop_count</code>
<code>ROUTE_SWITCH_MIN_LIFETIME_GAIN_MS</code>	5000	aumento de <code>expires_at_ms</code>

Tamanhos de payload

Constante	Default	Uso
<code>CONTROL_PAYLOAD_MAX</code>	128	maior frame de controle
<code>DATA_PAYLOAD_MAX</code>	1024	maior payload de DATA

Codigos especiais

Constante	Valor	Uso
<code>AADV_EN_PROTOCOL_VERSION</code>	1	escrito em todo header; recv descarta versao diferente
<code>AADV_EN_ROUTE_METRIC_INFINITY</code>	0xFFFF	marca rota invalidada (em metric)
<code>AADV_EN_MAC_ADDR_LEN</code>	6	bytes de MAC

Como uma mensagem viaja pelo codigo

Tracejando o caminho de um pacote DATA de A para C, com B intermediario:

1. App de A chama `send_data`

```
app_protocol_task() (firmware/main/app_demo.c:378)
→ aadv_en_stack_send_data_at() (aadv_en.c:301)
→ aadv_en_node_send_data() (aadv_en_node.c:1554)
  Sem rota valida:
  → aadv_en_node_queue_data() (aadv_en_node.c:449)
  → aadv_en_node_send_rreq() (aadv_en_node.c:880)
  → aadv_en_node_pending_note_rreq_attempt() (aadv_en_node.c:520)
  retorna AADV_EN_QUEUED
```

2. RREQ vai pro radio

```
aadv_en_node_send_rreq()
→ aadv_en_fill_header() (aadv_en_node.c:99)
→ aadv_en_node_emit() (aadv_en_node.c:58)
→ callbacks.emit_frame() // injetado pelo stack
→ stack_emit_frame() (aadv_en.c:62)
→ adapter.tx_frame()
→ app_emit_frame() (firmware/main/app_demo.c:171)
→ esp_now_send() // ESP-NOW driver
```

3. RREQ chega em B (no radio)

```
ESP-NOW driver
→ app_rcv_cb() (firmware/main/app_demo.c:264)
→ xQueueSend() // empurra para fila de RX
...
app_protocol_task() (loop)
→ app_process_rx_queue() (firmware/main/app_demo.c:334)
→ aadv_en_stack_on_rcv_at() (aadv_en.c:356)
→ aadv_en_node_on_rcv() (aadv_en_node.c:1612)
  → validate_header, validate_message_size
  → aadv_en_neighbor_touch() (aadv_en_neighbors.c:70) // observacao passiva
  → handle_rreq() (aadv_en_node.c:1120)
    → rreq_cache_remember() (aadv_en_rreq_cache.c:83)
    → route_upsert(rota REVERSE pra A) (aadv_en_routes.c:203)
    B nao e o destino:
    → forward_rreq() (aadv_en_node.c:1025)
      → emit (broadcast com hop_count++ e ttl--)
```

4. RREQ chega em C, C responde RREP

```
on_rcv -> handle_rreq:
- cache: nao encontrou (originator+rreq_id), insere
- install rota reversa pra A (via B, hops=2, REVERSE)
- C e o destino:
→ send_rrep() (aadv_en_node.c:911)
  - regras RFC 3561 §6.6.1 sobre self_seq_num
  - emit unicast pelo next_hop da rota reversa = B
```

5. RREP volta em B

```
on_rcv -> handle_rrep:
  → route_upsert(rota direta pra C, via=C, hops=1, VALID)
  → 3 chamadas a route_add_precursor (RFC 3561 §6.6.2):
    - rota(C).precursors += C (de quem recebeu)
    - rota(C).precursors += A (para quem repassa)
    - rota(A).precursors += C (precursor da rota reversa)
  B não é a origem:
  → forward_rrep() (aadv_en_node.c:1044)
```

6. RREP chega em A

```
on_rcv -> handle_rrep:
  → route_upsert(rota direta pra C, via=B, hops=2, VALID)
  → rota(C).precursors += B
  A é o originator, retorna OK
  → flush_pending_data_for_destination(C) (aadv_en_node.c:716)
    → loop: send_data_via_route() (aadv_en_node.c:399)
      → track_pending_ack() (aadv_en_node.c:313)
      → send_data_with_sequence() (aadv_en_node.c:206)
        → emit DATA (unicast para B)
    → stats.pending_data_flushed++
```

7. DATA chega em B, repassa para C

```
on_rcv -> handle_data:
  B não é destino:
  → forward_data() (aadv_en_node.c:1064)
    - ttl > 1
    - encontra rota_valida(C) -> via=C
    - aadv_en_route_add_precursor(rota_C, link_src=A) // RFC 3561
    - hop_count++, ttl--, sender_mac=B
    - stats.forwarded_frames++
    - emit DATA (unicast para C)
```

8. DATA chega em C, entrega + envia ACK

```
on_rcv -> handle_data:
  C é destino:
  - stats.delivered_frames++
  - callbacks.deliver_data(originator=A, payload, len)
    → app_deliver_data() (firmware/main/app_demo.c:207)
      → ESP_LOGI("DATA deliver from ...")
  - flag ACK_REQUIRED esta setada:
  → send_ack(target=A, seq) (aadv_en_node.c:1002)
    - encontra rota_usavel(A) -> via=B
    - emit ACK (unicast para B)
```

9. ACK volta para A

```
B: on_rcv -> handle_ack:
  B nao e destino:
  - stats.forwarded_frames++
  - encontra rota_usavel(A) -> via=A
  - hop_count++, sender_mac=B
  - emit ACK (unicast para A)

A: on_rcv -> handle_ack:
  A e destino:
  -> pending_ack_consume(C, seq) (aodv_en_node.c:361)
    - remove entrada da fila de ACK pendente
    - stats.ack_received++
    - callbacks.ack_received(C, seq)
      -> app_ack_received() (firmware/main/app_demo.c:220)
        -> ESP_LOGI("ACK received from ...")
```

Cada salto incrementa contadores de `tx_frames` e `rx_frames`. `forwarded_frames` cresce em B nas duas direcoes.

Apendice - Fluxo do tick periodico

`aodv_en_node_tick(now_ms)` ([aodv_en_node.c:1429](#)) e chamado a cada ~50-100ms pela app. Em ordem:

1. `aodv_en_neighbor_expire` - remove vizinhos inativos
2. `aodv_en_route_expire` - remove rotas vencidas
3. `aodv_en_rreq_cache_expire` - remove entradas antigas do cache
4. `aodv_en_node_retry_route_discovery_for_pending` - reemite RREQ pendentes com backoff
5. `aodv_en_node_process_pending_ack_retries` - retransmite DATA pendente
6. `aodv_en_node_expire_pending_data` - descarta DATA enfileirado por tempo demais

Tudo isso mantem o estado limpo sem precisar de threads adicionais. O proprio `tick` e single-threaded.

Como navegar este codigo

Se voce esta debugando uma feature (ex: "ACK nao chega"): 1. Olha o mapa por feature acima 2. Vai pra funcao indicada 3. Le os comentarios + asserts em volta 4. Coloca um `ESP_LOGI` extra se precisar 5. Re-flasha e usa `live_monitor.py -v` ou `idf.py monitor`

Se voce esta estendendo o protocolo (ex: adicionar v2 articulation point): 1. Le [features/articulation-point-planejado.md](#) 2. Decide onde no codigo encaixa (provavelmente novos handlers no `aodv_en_node.c` + novas estruturas no `aodv_en_types.h`) 3. Adiciona testes na simulacao (`sim/aodv_en_sim_*.c`) 4. Documenta na spec v1 ou cria spec v2

Se voce esta escrevendo o TCC: - "Como o protocolo funciona" -> copia de [aodv-en-funcionamento.md](#) - "Onde o codigo implementa cada feature" -> copia tabelas deste documento - "Implementacao em camadas" -> usa a estrutura "Camada 1-9" deste documento

Se voce e novo no projeto: 1. Le [aodv-en-spec-v1.md](#) primeiro (o que o protocolo e) 2. Depois [aodv-en-funcionamento.md](#) (como funciona) 3. Finalmente este documento (onde encontrar)

Referencia rapida - "preciso ver"

Quero ver	Va em
O que cada mensagem tem byte a byte	aodv-en-funcionamento.md - "Mensagens no fio"
Como <code>send_data</code> decide entre enviar e enfileirar	aodv_en_node.c:1554
Como o ACK encontra o caminho de volta	rota reversa instalada por <code>handle_rreq</code> (aodv_en_node.c:1120); <code>send_ack</code> consulta tabela (aodv_en_node.c:1002)
Onde o <code>link_fail_threshold</code> e checado	aodv_en_node.c:1467 <code>on_link_tx_result</code>
Por que rotas as vezes nao sao trocadas	hysteresis em aodv_en_routes.c:37 <code>candidate_is_strongly_better</code>
Onde a fila de DATA e drenada	aodv_en_node.c:716 <code>flush_pending_data_for_destination</code> ; chamada em <code>handle_rrep</code>
Onde o RSSI EMA e calculado	aodv_en_neighbors.c:25 <code>weighted_rssi</code>
Onde o snapshot do log e formatado	firmware/main/app_demo.c:290 e app_proto_example.c:746 <code>app_log_routes</code>
Onde o dashboard parseia o snapshot	firmware/tools/live_monitor.py - <code>LineParser.feed</code> + <code>SNAPSHOT_RE</code> + <code>ROUTE_RE</code>

Ultima linha de defesa: o codigo nao mente

Quando este documento estiver desatualizado em relacao ao codigo, **o codigo ganha**. O codigo tem: - testes da simulacao (`sim/aodv_en_sim*.c`) executando o nucleo - testes de bancada (TC-001..TC-005) - comentarios `RFC 3561 §X.Y.Z` em pontos chave

Se voce encontrar discrepancia, abra issue ou atualize aqui antes de seguir adiante.

Feature: Precursores

Feature: Precursores e RERR direcionado

Estado

- estado: `INTEGRADO_NA_V1`
- alinhado com [aodv-en-spec-v1.md](#)
- referencia: RFC 3561, secao 6.2 e secao 6.11

Contexto

A v0 do `AODV-EN` previa `RERR` como uma sinalizacao simples ao detectar quebra de enlace. Na pratica, isso forçava cada `RERR` a ser broadcast, o que gera trafego desnecessario em redes maiores.

A RFC 3561 resolve esse problema definindo precursores: para cada rota direta, um no mantem a lista dos vizinhos que sao "predecessores" no caminho de descoberta, ou seja, vizinhos que estao usando essa rota. Quando a rota cai, o no envia `RERR` apenas para esses precursores.

O que foi implementado

Estrutura

Cada `aodv_en_route_entry_t` carrega:

- `precursor_count` (uint8)
- `precursors[AODV_EN_MAX_PRECURSORS][6]` (default `MAX_PRECURSORS = 4`)

Quando precursores sao adicionados

No tratamento de `RREP` (RFC 3561 secao 6.6.2):

- vizinho de quem recebemos o `RREP` e adicionado como precursor da rota direta para `destination_mac`
- vizinho para quem repassamos o `RREP` tambem e adicionado como precursor da rota direta para `destination_mac`
- vizinho de quem recebemos o `RREP` e adicionado como precursor da rota reversa para `originator_mac`

No encaminhamento de `DATA`:

- vizinho de quem recebemos o `DATA` e adicionado como precursor da rota direta para `destination_mac`

Quando RERR usa precursores

No `RERR` direcionado (`aodv_en_node_notify_precursors_of_break`):

1. recupera a rota para `unreachable_mac`
2. se `precursor_count == 0`: emite `RERR` em broadcast como fallback

3. caso contrario: emite `RERR` em unicast para cada precursor, exceto o `next_hop` quebrado

Esse comportamento e disparado em duas situacoes:

- quando `aadv_en_node_handle_rerr` invalida uma rota local
- quando `aadv_en_node_on_link_tx_result` reporta falha de unicast e routes sao invalidados em massa por `next_hop`

Preservacao em refresh

`aadv_en_route_upsert` preserva os precursores existentes quando uma rota e atualizada com o mesmo `next_hop` (refresh). Quando o `next_hop` muda, os precursores sao reinicializados, porque vizinhos que conheciam o caminho antigo nao sao necessariamente parte do novo.

Arquivos impactados

- [firmware/components/aadv_en/include/aadv_en_types.h](#)
- [firmware/components/aadv_en/include/aadv_en_routes.h](#)
- [firmware/components/aadv_en/src/aadv_en_routes.c](#)
- [firmware/components/aadv_en/src/aadv_en_node.c](#)

Limitacoes atuais

- `MAX_PRECURSORS = 4` por rota; precursores extras sao ignorados (`AADV_EN_ERR_FULL`) sem politica de substituicao
- nao ha telemetria especifica para precursores (uso indireto via `route_repairs`)
- a remocao de precursores so ocorre quando o `next_hop` muda; nao ha decay temporal

Pontos de evolucao

- politica de substituicao quando o array de precursores enche (LRU? prioridade por uso?)
- contadores explicitos de `rerr_unicast` , `rerr_broadcast_fallback` e `precursors_added`
- considerar limpeza de precursores cuja rota reversa do precursor para nos ja expirou

Feature: Fila de DATA pendente

Feature: Pending DATA Queue

Estado

- estado: `INTEGRADO_NA_V1`
- alinhado com [aodv-en-spec-v1.md](#), secao "Fila de DATA pendente"
- ultima revisao: 2026-05-01

Objetivo

Evitar perda imediata de DATA quando a rota para o destino ainda nao existe no momento do envio.

Antes desta feature, a chamada de envio retornava erro de no route e o pacote precisava ser tratado fora do nucleo. Agora o pacote pode ser armazenado temporariamente enquanto a descoberta de rota ocorre.

Problema resolvido

Cenario comum:

- no A deseja enviar DATA para no C
- no A ainda nao possui rota valida para C
- descoberta via RREQ/RREP ainda nao terminou

Sem fila pendente, o primeiro DATA era perdido ou exigia retry da aplicacao.

Comportamento implementado

Envio com rota ausente

Quando a rota esta ausente ou expirada em `aodv_en_node_send_data`:

1. DATA e enfileirado na pending queue
2. RREQ e emitido para iniciar descoberta
3. retorno da funcao passa a ser `AODV_EN_QUEUED`

Quando a rota chega

Ao receber RREP e instalar rota valida para o destino:

1. o no tenta drenar a pending queue daquele destino
2. cada item e enviado usando a rota recém instalada
3. estatistica de flush e incrementada

Retry de descoberta para pendencias

No tick do no, quando ainda nao existe rota para um destino com pendencia:

- o no reemite RREQ em intervalo controlado
- as tentativas sao limitadas por rreq_retry_count
- retries sao contabilizados em route_discovery_retries

Esse comportamento cobre o caso em que o destino entra na rede depois da primeira tentativa.

ACK timeout e retry de DATA

Quando DATA exige ACK:

- o envio fica rastreado como ACK pendente
- se ACK nao chega em ack_timeout_ms, ocorre retransmissao do mesmo sequence_number
- apos esgotar retries, o envio e descartado e contabilizado

Expiracao da fila

No tick do no:

- itens pendentes expiram quando excedem route_lifetime_ms
- itens expirados sao removidos e contabilizados como dropped

Estruturas e constantes adicionadas

- AODV_EN_PENDING_DATA_QUEUE_SIZE_DEFAULT = 4
- AODV_EN_PENDING_DATA_QUEUE_SIZE
- novo status AODV_EN_QUEUED

Nova estrutura por item pendente:

- destination_mac
- payload_len
- ack_required
- used
- enqueued_at_ms
- last_rreq_at_ms
- discovery_attempts
- payload

Nova estrutura de ACK pendente:

- destination_mac
- payload_len
- used
- retries_left
- sequence_number
- last_sent_at_ms
- payload

Contadores novos em stats:

- pending_data_queued
- pending_data_flushed
- pending_data_dropped
- route_discovery_retries
- ack_retry_sent
- ack_timeout_drops

Arquivos impactados

- firmware/components/aodv_en/include/aodv_en_limits.h
- firmware/components/aodv_en/include/aodv_en_status.h
- firmware/components/aodv_en/include/aodv_en_node.h
- firmware/components/aodv_en/src/aodv_en_node.c
- firmware/main/main.c
- sim/aodv_en_sim.c

Evidencia de validacao

A simulacao local foi ajustada para validar o novo contrato:

- primeira tentativa para destino sem rota retorna AODV_EN_QUEUED
- apos descoberta, DATA e entregue e ACK e recebido

Resultado esperado no fluxo:

- route discovery phase inicia com queued
- data phase confirma entrega e ack
- ack retry phase confirma retransmissao apos perda de ACK
- late join retry discovery phase confirma entrega apos reemissao de RREQ
- simulation passed

Limitacoes atuais

- nao existe deduplicacao de payload pendente por destino
- expiracao usa route_lifetime_ms como janela da fila
- fila tem tamanho fixo e pode retornar AODV_EN_ERR_FULL
- retries de descoberta e de ACK compartilham rreq_retry_count

Proximos passos recomendados

1. separar configuracao de retry de ACK e retry de descoberta
2. adicionar telemetria de queue occupancy por destino
3. avaliar politica FIFO estrita e backpressure
4. criar cenarios de estresse com fila cheia no simulador

Feature planejada: Articulation Point (v2)

Feature Planejada: Articulation Point (No de Corte)

Status

- estado atual: `PLANEJADA_PARA_V2`
- implementacao em codigo: NAO
- decisao explicita em [aodv-en-spec-v1.md](#): fora do escopo de v1
- objetivo deste arquivo: registrar decisao tecnica, escopo e criterios de aceite antes da implementacao

Contexto

Em uma rede mesh, um articulation point (no de corte) e um no cuja falha separa a rede em dois ou mais segmentos.

Cenario tipico:

- Segmento A <-> No Ponte <-> Segmento B
- se o no ponte falha, A e B ficam particionados

Esse caso e critico para AODV-EN porque pode aparentar falha geral de entrega, quando na verdade e particionamento topologico.

Problema a resolver

Quando um no de corte falha:

1. rotas antigas podem continuar sendo tentadas por um intervalo
2. a origem pode repetir descoberta sem saber que houve particao
3. sem telemetria, diagnostico de campo fica dificil

Objetivos da feature

1. Detectar rapidamente indicios de particionamento por no de corte.
2. Melhorar recuperacao quando o no ponte retorna.
3. Produzir evidencias de rede particionada vs rede recuperada.

Escopo planejado

1. Simulador

Adicionar cenario dedicado no simulador com fases:

1. fase normal (entrega fim-a-fim)
2. falha do no ponte (particao)
3. retries durante particao

4. retorno do no ponte
5. reconvergencia e entrega restaurada

2. Firmware

Adicionar sinais operacionais para suspeita de no de corte:

- perda de next-hop com alto impacto em rotas
- repeticao de route discovery sem sucesso de entrega
- recuperacao apos retorno do caminho central

3. Telemetria minima

Registrar eventos para analise:

- neighbor_loss
- route_break_by_next_hop
- repeated_route_discovery
- partition_suspected
- partition_recovered

Fora de escopo agora

- algoritmo formal de deteccao de articulation point por teoria de grafos em runtime
- eleicao de no redundante automatica
- reconfiguracao autonoma de topologia

Critérios de aceite propostos

1. Em estado normal: PDR $\geq 95\%$.
2. Durante particao: PDR entre segmentos isolados = 0%.
3. Deteccao de particao suspeita em ate 5s.
4. Recuperacao apos retorno do no ponte em ate 10s.
5. Telemetria deve registrar transicao normal -> particao -> recuperacao.

Dependencias tecnicas

Esta feature depende do que ja foi entregue:

- pending DATA queue
- retry de descoberta para pendencias
- ACK timeout e retry

Esses mecanismos reduzem perda imediata e permitem observar comportamento real durante particao.

Plano de execucao (futuro)

1. etapa 1: ampliar simulador com cenario de no ponte (sem firmware)
2. etapa 2: definir eventos de telemetria no firmware
3. etapa 3: validar em hardware com 3-4 ESP32
4. etapa 4: calibrar thresholds de deteccao e reconvergencia

Observacao

Este documento registra apenas mapeamento e planejamento. Nenhuma implementacao de articulation point foi aplicada no codigo ate este momento.

Plano de Desenvolvimento Completo

Plano de Desenvolvimento Completo do AODV-EN

Estado

- alinhado com [aodv-en-spec-v1.md](#)
- ultima revisao: 2026-05-01

Objetivo

Este documento organiza o desenvolvimento do `AODV-EN` ate a conclusao do projeto tecnico e do TCC.

Ele serve como:

- roadmap de implementacao
- checklist de validacao
- guia de experimentacao
- trilha de escrita do trabalho final

Legenda de status

- `CONCLUIDO`
- `EM_ANDAMENTO`
- `PENDENTE`
- `OPCIONAL`

Estado atual resumido

Ja concluido

- `CONCLUIDO` invariantes do AODV em [aodv-base-invariantes.md](#)
- `CONCLUIDO` Spec v1 consolidada em [aodv-en-spec-v1.md](#), substituindo a v0
- `CONCLUIDO` modelagem das estruturas em [aodv-en-estruturas-dados.md](#)
- `CONCLUIDO` materializacao dos limites, tipos, mensagens e tabelas em `firmware/components/aodv_en/include`
- `CONCLUIDO` operacoes de vizinhos, rotas, cache de `RREQ` e cache de peers em `firmware/components/aodv_en/src`
- `CONCLUIDO` nucleo do no com `HELLO`, `RREQ`, `RREP`, `RERR`, `DATA`, `ACK`
- `CONCLUIDO` simulacao local do fluxo `RREQ -> RREP -> DATA -> ACK` em [sim/aodv_en_sim.c](#)
- `CONCLUIDO` fila pendente de `DATA` com retry de descoberta e expiracao ([features/enfileamento-dos-dados.md](#))
- `CONCLUIDO` precursores e `RERR` direcionado ([features/precursores.md](#))
- `CONCLUIDO` lib `AODV-EN` consumivel (adapter `aodv_en_stack_*`) em [aodv_en.h](#)

- **CONCLUIDO** app de bancada `app_proto_example` com CLI e mensagens `HEALTH/TEXT/CMD`
- **CONCLUIDO** perfis pre-prontos `node_a`, `node_b`, `node_c`, `node_d` em `firmware/tests/`
- **CONCLUIDO** casos `TC-001`, `TC-002`, `TC-003`, `TC-004`, `TC-005` documentados em [tests](#)
- **CONCLUIDO** `TC-002` validado em hardware em 2026-04-21 (**PASS** com evidencias indiretas)

Ainda nao concluido

- multi-hop em hardware com captura simultanea dos 3 nos (`TC-002` com captura completa)
- reconvergencia em hardware (`TC-003`)
- soak de 30 minutos em hardware (`TC-004`)
- baseline de flooding e experimentos comparativos
- escrita do TCC

Roadmap macro

Fase	Status	Objetivo principal	Entregavel
0. Fundacao conceitual	CONCLUIDO	Definir o que o protocolo e e o que ele nao pode violar	invariantes + spec v1
1. Base estrutural do firmware	CONCLUIDO	Materializar tipos, tabelas e mensagens	headers e modulos
2. Nucleo logico do protocolo	CONCLUIDO	Processar mensagens, manter rotas e filas	<code>aadv_en_node</code> + sim
3. Endurecimento do nucleo	CONCLUIDO	Fechar o nucleo conforme spec v1	<code>A0DV-EN core v1</code>
4. Adapter ESP-NOW / ESP-IDF	EM_ANDAMENTO	Conectar o nucleo ao transporte real	adapter desacoplado
5. Firmware de teste em ESP32	EM_ANDAMENTO	Rodar o protocolo em hardware real	app + perfis por papel
6. Testes funcionais de rede	EM_ANDAMENTO	Validar TC-001 a TC-004 com captura completa	relatorio de testes
7. Baseline de flooding	PENDENTE	Criar comparativo justo para o TCC	protocolo baseline
8. Experimentos e metricas	PENDENTE	Coletar dados para o trabalho	tabelas, logs, graficos
9. Escrita do TCC	PENDENTE	Transformar implementacao e resultados em texto academico	monografia
10. Defesa e demonstracao	PENDENTE	Preparar apresentacao final	slides e demo

Fase 0. Fundacao conceitual

Status: **CONCLUIDO**

Entregas

- invariantes do AODV
- spec v1 do `A0DV-EN` (substitui a v0 obsoleta)
- definicao das tabelas principais e mensagens
- decisoes explicitas sobre `HELLO`, precursores, fila pendente, metrica e escopo de v2

Critério de conclusao

- o projeto tem uma base teorica coerente
- a spec v1 fecha as decisoes funcionais que estavam em aberto

Fase 1. Base estrutural do firmware

Status: CONCLUIDO

Entregas

- limites e tipos centrais
- formatos de mensagem (`__attribute__((packed))`)
- wrappers das tabelas
- operacoes iniciais de vizinhos, rotas, cache de `RREQ` e peers

Fase 2. Nucleo logico do protocolo

Status: CONCLUIDO

Entregas

- `aadv_en_node_t` com filas pendentes, callbacks de emissao e entrega
- processamento de `HELLO` , `RREQ` , `RREP` , `RERR` , `DATA` , `ACK`
- precusores e `RERR` direcionado
- simulacao local do fluxo principal e dos cenarios de retry e late-join

Fase 3. Endurecimento do nucleo

Status: CONCLUIDO

Entregas que fecharam a fase

- fila pendente de `DATA` com retry de descoberta e expiracao
- timeout e retry de `ACK` com fila dedicada
- alinhamento das regras de `sequence number` em `RREP` com a RFC 3561 secao 6.6.1
- semantica de `RERR` direcionada a precusores
- helpers de validacao de header e tamanho de frame
- centralizacao de `TTL` e `hop_count` em `aadv_en_validate_*` e nos handlers
- regra de troca de rota com histerese para evitar flapping
- novos cenarios na simulacao local cobrindo descoberta, retry, late-join e ACK timeout

Itens reconhecidos como `OPCIONAL` para v1

- separar `rreq_retry_count` de `ack_retry_count` (hoje compartilham orcamento)
- tratamento explicito de wrap-around de `sequence number`
- politica de substituicao de precursor quando o array enche

Esses itens nao bloqueiam v1 e ficam mapeados na trilha de v2 da spec.

Fase 4. Adapter ESP-NOW / ESP-IDF

Status: `EM_ANDAMENTO`

Ja feito

- app ESP-IDF buildavel em `firmware/`
- integracao com `esp_wifi_start` e `esp_now_init`
- callback de recepcao conectado ao `aadv_en_stack_on_recv`
- callback de emissao conectado ao `esp_now_send`
- registro de peer broadcast
- processamento de frames fora do callback do Wi-Fi por fila local

Tarefas restantes

- extrair a camada de transporte do `main / app_proto_example` para um adapter dedicado `aadv_en_espnw`
- encapsular `esp_wifi_start`, `esp_now_init`, peers e callbacks no adapter
- integrar `peer_cache` ao driver com politica explicita de adicao/remocao
- padronizar logs estruturados por no
- tratar melhor falhas de `esp_now_send` (mapear para `aadv_en_stack_on_link_tx_result`)
- documentar configuracao de canal e interface

Entregavel

- adapter `aadv_en_espnw` desacoplado, com a app reduzida a glue code

Criterio de conclusao

- substituir o adapter por um mock nao exige mudar a app

Fase 5. Firmware de teste em ESP32

Status: `EM_ANDAMENTO`

Ja feito

- projeto `ESP-IDF` em `firmware/`
- `app_demo` (legacy) e `app_proto_example` (atual) com CLI
- envio periodico de `HELLO`
- envio opcional de `DATA` para MAC configurado
- logs de estatisticas e rotas
- perfis `NODE_A`, `NODE_B`, `NODE_C`, `NODE_D` prontos
- build validado com `aadv_en_firmware.bin`
- evidencia de funcionamento em hardware via `TC-002`

Tarefas restantes

- captura serial simultanea dos 3 nos no `TC-002`
- consolidar dumps periodicos (vizinhos, rotas, stats) em formato facil de parsear
- definir convencao de log para alimentar [tools/draw_topology.py](#)
- executar `TC-005` em hardware com 4 nos

Entregavel

- app de bancada com observabilidade boa o suficiente para reproduzir os criterios de aprovacao da spec v1

Fase 6. Testes funcionais de rede

Status: EM_ANDAMENTO

Cenarios obrigatorios para v1

Caso	Topologia	Cobre criterios da spec
TC-001	2 nos (A <--> B)	1
TC-002	3 nos (A <--> B <--> C)	2, 5
TC-003	3 nos com falha intermediaria	3, 7
TC-004	3 nos sob ciclos por 30 min	4, 6
TC-005	4 nos (A <--> B <--> C <--> D)	2 (estendido para 3 saltos)

Cenarios opcionais

- estresse com fila pendente cheia em simulador
- variantes de simulacao em escala (ver [../sim/README.md](#): large , 100 , 1000)

O que registrar

- logs por no
- tabelas antes e depois
- latencia aproximada de descoberta e de entrega
- numero de mensagens de controle
- contadores de stats em aadv_en_stack_stats_t
- sucesso ou falha da entrega

Entregavel

- relatorio funcional de bancada com PASS/FAIL por TC e evidencias

Criterio de conclusao

- os criterios de aprovacao da spec v1 estao demonstrados

Fase 7. Baseline de flooding

Status: PENDENTE

Tarefas

- implementar baseline simples de flooding usando o mesmo adapter aadv_en_espnw
- manter a mesma interface de app (mesmo send_data , mesmo callback de entrega)
- coletar as mesmas metricas de aadv_en_stack_stats_t (no que se aplica)

Entregavel

- modulo baseline `flooding` selecionavel por configuracao

Critério de conclusao

- existe uma referencia experimental comparavel ao `AODV-EN`

Fase 8. Experimentos e metricas

Status: `PENDENTE`

Metricas minimas

- `PDR` (`delivered_frames` / `DATA` enviados pela origem)
- latencia fim a fim (do envio ao `ACK`)
- overhead de controle (`tx_frames` excluindo `DATA`)
- numero total de transmissoes
- tempo para descoberta de rota
- tempo para recuperacao apos falha

Cenarios de experimento

- topologia linear (3 e 4 nos)
- topologia em arvore
- malha parcial com rota alternativa
- cenario com falha induzida

Tarefas

- definir formato de log exportavel
- repeticoes por cenario
- consolidar resultados em tabelas
- gerar graficos
- comparar `AODV-EN` vs `Flooding`

Observacao

Sem medicao real de energia, tratar consumo energetico como inferencia por numero de transmissoes ou limitacao explicitamente declarada.

Entregavel

- conjunto de resultados experimentais do TCC

Fase 9. Escrita do TCC

Status: `PENDENTE`

Estrutura sugerida

- introducao
- problema e motivacao

- objetivos geral e específicos
- fundamentacao teorica (redes ad hoc/mesh, ESP-NOW, AODV/RFC 3561)
- trabalhos relacionados
- proposta do AODV-EN (baseada na spec v1)
- arquitetura e design (baseada no documento de estruturas)
- implementacao (lib + adapter + app)
- metodologia experimental
- resultados
- discussao
- limitacoes
- conclusao
- referencias
- apendices

Tarefas

- transcrever a spec v1 e o documento de estruturas em texto academico
- detalhar metodologia de experimento
- consolidar resultados e interpretacoes
- revisar linguagem academica e citacoes

Entregavel

- versao final da monografia

Fase 10. Defesa e demonstracao

Status: PENDENTE

Tarefas

- preparar slides
- preparar roteiro de fala
- preparar demo com 3 a 4 nos
- preparar plano B em video
- preparar respostas para perguntas provaveis da banca

Perguntas que precisamos conseguir responder

- por que isso ainda e AODV?
- por que duas tabelas e duas filas?
- por que ESP-NOW v2?
- como o protocolo descobre e mantem rotas?
- onde ele simplifica e onde ele estende a RFC 3561?
- qual foi o ganho frente ao flooding?
- quais sao as limitacoes do projeto?

Caminho critico

Para fechar o TCC com folga:

1. fechar `TC-002` com captura simultanea dos 3 nos
2. executar e fechar `TC-003` e `TC-004`
3. extrair adapter ESP-NOW dedicado
4. implementar baseline de flooding com a mesma interface
5. executar a bateria de experimentos
6. escrever resultados e discussao
7. preparar defesa

Backlog imediato recomendado

1. fechar captura serial dos 3 nos no `TC-002`
2. extrair adapter ESP-NOW para um modulo dedicado
3. executar `TC-003` em hardware com log analisavel
4. iniciar baseline de flooding como segundo binario selecionavel por Kconfig
5. preencher campos de "Resultado" dos casos de teste a cada execucao

Definicao de pronto do projeto

O projeto esta completo quando:

- o `A0DV-EN v1` roda em ESP32 real
- os criterios de aprovacao da spec v1 estao demonstrados
- existe comparacao com baseline de flooding
- os resultados estao coletados e analisados
- o TCC esta escrito
- a apresentacao final esta preparada

Runbook de Bancada

Runbook de Bancada - AODV-EN

Para quem e este documento

Voce esta com `N` ESP32 na mesa, quer flashar o firmware `AODV-EN` em cada um, validar um caso de teste (`TC-001` / `TC-002` / `TC-005`) e ver a malha funcionando. Este runbook leva voce do "ESP plugado na USB" ate "rota e ACK aparecendo no log" sem precisar consultar outros documentos durante a sessao.

Para a teoria do protocolo, ver [aodv-en-funcionamento.md](#). Para os criterios formais de aprovacao, ver [aodv-en-spec-v1.md](#). Para os casos de teste documentados, ver [tests/](#).

Sumario

1. [Setup unico \(uma vez por maquina\)](#)
 2. [Bootstrap de cada sessao](#)
 3. [Identificar os ESP32 conectados](#)
 4. [Folha "minha bancada"](#)
 5. [Build do firmware](#)
 6. [Fluxo geral de um caso de teste](#)
 7. [TC-001 - 2 nos diretos \(passo a passo\)](#)
 8. [TC-002 - 3 nos em cadeia](#)
 9. [TC-003 - reconvergencia](#)
 10. [TC-004 - soak de 30 minutos](#)
 11. [TC-005 - cadeia de 4 nos](#)
 12. [Monitor: dashboard vs texto](#)
 13. [Analise pos-execucao](#)
 14. [Troubleshooting](#)
 15. [Cheatsheet de comandos](#)
-

1. Setup unico (uma vez por maquina)

Esses passos voce faz **uma vez** quando preparar a estacao de trabalho.

1.1 ESP-IDF

Esta maquina tem `ESP-IDF v6.0` em `/Users/huaksonlima/.espressif/v6.0/esp-idf` e a virtualenv Python em `/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env`.

Para **bootstrap automatizado** sem brigar com `pyenv`, foi criado um wrapper:

```
cat /tmp/aadv-en-idf-env.sh
```

Conteúdo:

```
export PATH="/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin:$PATH"
export ESP_IDF_EXPORT="/Users/huaksonlima/.espressif/v6.0/esp-idf/export.sh"
source "$ESP_IDF_EXPORT" >/dev/null 2>&1 || true
```

Persistencia: como `/tmp` é volátil, copie esse arquivo para `~/aadv-en-idf-env.sh` ou para o repo (`firmware/idf-env-local.sh`) se quiser que ele sobreviva a reinicialização do macOS.

1.2 Dependências do dashboard

O `live_monitor.py` precisa de `aiohttp` e `pyserial`. **Atenção:** quando você sourceia o `idf-env.sh`, o `python3` do PATH muda para o da venv do ESP-IDF (`idf6.0_py3.14_env`). Se você instalar com `pip install --user` no python global, a venv do IDF não enxerga.

Solução recomendada: instale **dentro da venv do ESP-IDF**:

```
/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin/pip install aiohttp pyserial
```

Verificação:

```
source /tmp/aadv-en-idf-env.sh
python3 -c "import aiohttp, serial; print(aiohttp.__version__, serial.__version__)"
# saída esperada: 3.13.5 3.5
```

Necessário apenas se você for usar o `live_monitor.py` (dashboard real-time).

1.3 Repo

Trabalhar em:

```
/Users/huaksonlima/Documents/tcc/aadv-en
```

Branch ativa: `develop`.

2. Bootstrap de cada sessão

Toda nova sessão de bancada começa com esses 3 comandos:

```
# 1) navegar pro repo
cd /Users/huaksonlima/Documents/tcc/aadv-en

# 2) checar branch e estado
git status
git branch --show-current

# 3) carregar ESP-IDF
source /tmp/aadv-en-idf-env.sh
idf.py --version # esperado: ESP-IDF v6.0
```

Se `idf.py --version` falhar, refaça o passo 1.1.

3. Identificar os ESP32 conectados

3.1 Listar portas seriais

```
ls /dev/cu.usbserial-*
```

Saida esperada (com 3 ESPs):

```
/dev/cu.usbserial-2110
/dev/cu.usbserial-2120
/dev/cu.usbserial-2130
```

Cada ESP32 com chip CP2102/CH340 aparece como `cu.usbserial-XXXX`. Os numeros mudam entre conexoes USB; sempre relista antes de uma sessao.

3.2 Ler MAC sem flashar

Para cada porta, leia o MAC. Use o `esptool` ja instalado:

```
ESPT00L=/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin/esptool

"$ESPT00L" --port /dev/cu.usbserial-2110 --before default_reset --after hard_reset read_mac 2>&1 | grep "MAC:" | h
"$ESPT00L" --port /dev/cu.usbserial-2120 --before default_reset --after hard_reset read_mac 2>&1 | grep "MAC:" | h
"$ESPT00L" --port /dev/cu.usbserial-2130 --before default_reset --after hard_reset read_mac 2>&1 | grep "MAC:" | h
```

Saida esperada (cada linha):

```
MAC:          28:05:a5:34:03:50
MAC:          28:05:a5:33:b9:ec
MAC:          28:05:a5:34:05:64
```

Por que ler MAC antes de flashar: para flashar `NODE_A` precisamos do MAC do destino (`NODE_B` em `TC-001`, `NODE_C` em `TC-002`, `NODE_D` em `TC-005`). Lendo todos os MACs antes, voce nao precisa do ciclo flash-anotar-reflash.

3.3 Atribuir papeis

Atribua cada porta a um papel (`NODE_A`, `NODE_B`, etc.) e **anote**. Convencao recomendada:

Papel	Comportamento
<code>NODE_A</code>	origem; envia <code>DATA</code> periodico para o destino
<code>NODE_B</code>	intermediario (relay) ou destino direto, depende do TC
<code>NODE_C</code>	intermediario ou destino, depende do TC
<code>NODE_D</code>	destino final em <code>TC-005</code>

Nao importa qual ESP fisico vira qual papel; apenas seja consistente dentro da sessao.

4. Folha "minha bancada"

Preencha esta tabela no inicio de cada sessao. Cole numa nota local; nao precisa ficar versionada.

data: _____
sessao: _____

papel	porta	MAC	observacoes
NODE_A	/dev/cu.usbserial-2110	28:05:A5:34:03:50	origem
NODE_B	/dev/cu.usbserial-2120	28:05:A5:33:B9:EC	relay
NODE_C	/dev/cu.usbserial-2130	28:05:A5:34:05:64	destino
NODE_D	_____	_____	_____

Esses MACs são os atuais. Os do TC-002 PASS de 2026-04-21 eram outros (33:D6:1C, 33:EB:80, 34:99:34); essa rotação de hardware é normal.

5. Build do firmware

O `build_flash.sh` de cada caso de teste já faz build implícito antes de flashar. Mas em muitos cenários você **deve** rodar um build limpo manualmente, especialmente:

- depois de pull/checkout que mexeu em código;
- depois de mudanças em headers (cache de dependências do ESP-IDF não percebe sempre);
- na primeira vez que o repo é clonado.

```
cd /Users/luaksonlima/Documents/tcc/aodv-en
source /tmp/aodv-en-idf-env.sh
rm -rf firmware/build
zsh firmware/build.sh
```

Tempo esperado: 60 a 120 segundos (primeira vez); 5 a 15 segundos (incremental).

Saída final esperada:

```
Generated /Users/luaksonlima/Documents/tcc/aodv-en/firmware/build/aodv_en_firmware.bin
aodv_en_firmware.bin binary size 0xbf190 bytes. Smallest app partition is 0x100000 bytes. ...
```

Importante: `build.sh` *builda* em `firmware/build/`. Os scripts `tc00X/build_flash.sh` *buildam* em `firmware/build/tc00X_node_X/` (separados por papel/teste para evitar conflito de Kconfig defaults).

6. Fluxo geral de um caso de teste

Independente do TC, a sequência é:

1. bootstrap da sessão (seção 2)
2. identificar ESPs e atribuir papéis (seção 3, 4)
3. flashar destino primeiro (NODE_B/C/D conforme TC)
4. flashar intermediários
5. flashar origem (NODE_A) com TARGET_MAC do destino
6. posicionar fisicamente os nós conforme topologia do TC
7. abrir monitor (dashboard ou texto) em todos os nós simultaneamente
8. observar logs por 30-180s
9. avaliar critérios de PASS
10. salvar logs e analisar (seção 13)

7. TC-001 - 2 nos diretos (passo a passo)

Objetivo: validar descoberta + entrega + ACK em 1 salto. Os dois nos ficam dentro do alcance direto um do outro.

Hardware necessario: 2 ESP32. Vamos usar `2110` (NODE_B, destino) e `2120` (NODE_A, origem). Anote os MACs no formulario da secao 4.

Passo 1 - Flash NODE_B (destino, sem target)

```
cd /Users/huaksonlima/Documents/tcc/aodv-en
source /tmp/aodv-en-idf-env.sh
zsh firmware/tests/tc001/build_flash.sh node_b /dev/cu.usbserial-2110
```

Tempo: ~30s a primeira vez, ~10s incremental.

Saida final esperada:

```
ok: node_b gravado em /dev/cu.usbserial-2110
monitor:
idf.py -B firmware/build/tc001_node_b -p /dev/cu.usbserial-2110 monitor
```

Passo 2 - Flash NODE_A com MAC do NODE_B

Use o MAC que voce leu no passo 3.2. Para esta bancada, o NODE_B e `28:05:A5:34:03:50`.

```
zsh firmware/tests/tc001/build_flash.sh node_a /dev/cu.usbserial-2120 28:05:A5:34:03:50
```

Passo 3 - Abrir monitor

Opcao A - dashboard real-time (recomendado para impressao visual):

```
python3 firmware/tools/live_monitor.py \
--port /dev/cu.usbserial-2110:NODE_B \
--port /dev/cu.usbserial-2120:NODE_A
# abrir http://localhost:8765/
```

Opcao B - monitor texto com captura de log (recomendado para analise pos-execucao):

Abra dois terminais:

```
# terminal 1
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-2110 -B build/tc001_node_b -t tc001_run -l node_b
# terminal 2
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-2120 -B build/tc001_node_a -t tc001_run -l node_a
```

Logs salvos em `firmware/logs/serial/`.

Passo 4 - Avaliar PASS

Logs esperados em **NODE_A**:

```
I (xxx) aadv_en_proto: node=NODE_A self_mac=28:05:A5:33:B9:EC channel=6 network_id=0xA0DE0001
I (xxx) aadv_en_proto: ESP-NOW version=2
I (xxx) aadv_en_proto: routes=1 neighbors=1 tx=N rx=N delivered=0
I (xxx) aadv_en_proto: route[0] dest=28:05:A5:34:03:50 via=28:05:A5:34:03:50 hops=1 metric=1 state=2 expires=...
I (xxx) aadv_en_proto: ACK received from 28:05:A5:34:03:50 for seq=1
I (xxx) aadv_en_proto: ACK received from 28:05:A5:34:03:50 for seq=2
...
```

Logs esperados em **NODE_B**:

```
I (xxx) aadv_en_proto: node=NODE_B self_mac=28:05:A5:34:03:50 ...
I (xxx) aadv_en_proto: DATA deliver from 28:05:A5:33:B9:EC: tc001 hello
I (xxx) aadv_en_proto: routes=1 neighbors=1 tx=N rx=N delivered=N
```

Critério formal em [tests/tc-001-descoberta-e-entrega-direta.md](#).

Passo 5 - Encerrar

- Ctrl+] para sair do `idf.py monitor`
- Ctrl+C para parar o `live_monitor.py`

Logs ficam em `firmware/logs/serial/`. Veja seção 13 para extrair métricas.

8. TC-002 - 3 nos em cadeia

Objetivo: multi-hop em 2 saltos `A-B-C`. `B` faz relay; `A` e `C` sem enlace direto.

Hardware: 3 ESP32.

Passo 1 - Flash dos 3 nos

```
# destino primeiro (sem target)
zsh firmware/tests/tc002/build_flash.sh node_c /dev/cu.usbserial-2130

# intermediario (sem target)
zsh firmware/tests/tc002/build_flash.sh node_b /dev/cu.usbserial-2120

# origem com MAC do destino (NODE_C = 28:05:A5:34:05:64)
zsh firmware/tests/tc002/build_flash.sh node_a /dev/cu.usbserial-2110 28:05:A5:34:05:64
```

Passo 2 - Posicionamento físico

```
sala 1          corredor          sala 2
[NODE_A]-----[NODE_B]-----[NODE_C]
(origem)        (relay)          (destino)
```

`NODE_A` e `NODE_C` devem ficar **fora do alcance direto** (parede entre eles, ou afastamento de 8-15m). `NODE_B` no meio.

Passo 3 - Monitor

Dashboard com 3 portas:

```
python3 firmware/tools/live_monitor.py \  
  --port /dev/cu.usbserial-2110:NODE_A \  
  --port /dev/cu.usbserial-2120:NODE_B \  
  --port /dev/cu.usbserial-2130:NODE_C
```

Ou texto, em 3 terminais (use o wrapper):

```
zsh firmware/tests/tc002/monitor_log.sh node_a /dev/cu.usbserial-2110 tc002_run  
zsh firmware/tests/tc002/monitor_log.sh node_b /dev/cu.usbserial-2120 tc002_run  
zsh firmware/tests/tc002/monitor_log.sh node_c /dev/cu.usbserial-2130 tc002_run
```

Passo 4 - Avaliar PASS

Aguarde 60-120s.

NODE_A deve mostrar:

```
route[0] dest=<MAC_C> via=<MAC_B> hops=2 metric=2 state=2 expires=...  
ACK received from <MAC_C> for seq=N
```

NODE_C deve mostrar:

```
DATA deliver from <MAC_A>: tc002 multihop
```

Critério formal em [tests/tc-002-primeiro-multi-hop.md](#).

9. TC-003 - reconvergencia

Objetivo: validar recuperacao apos falha de enlace intermediario.

Setup identico ao TC-002 (mesmos perfis `firmware/tests/tc002/*.defaults`).

Procedimento

1. Inicie o TC-002 normalmente (passos 1-3 da secao 8).
2. Aguarde 30s ate ver `ACK` chegando regularmente em `NODE_A`.
3. **Provoque a falha:** tire `NODE_B` da fonte de energia (USB) **OU** afaste fisicamente para perder o enlace `B<->C`.
4. Observe que `NODE_A` deve passar a logar `DATA queued while route discovery is in progress`, `ESP-NOW send fail`, e eventualmente `invalidated N route(s) via <MAC_B>`.
5. **Recupere:** religue ou aproxime `NODE_B`.
6. Aguarde nova descoberta. `NODE_A` deve voltar a logar `ACK received` em ate 30s apos a recuperacao.

Critério formal em [tests/tc-003-reconvergencia-apos-falha.md](#).

10. TC-004 - soak de 30 minutos

Objetivo: estabilidade sob ciclos repetidos de degradacao/recuperacao.

Setup identico ao TC-002.

Procedimento

Use `firmware/monitor_log.sh` para capturar logs longos:

```
# em 3 terminais, simultaneamente:
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-2110 -B build/tc002_node_a -t tc004_soak -l node_a
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-2120 -B build/tc002_node_b -t tc004_soak -l node_b
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-2130 -B build/tc002_node_c -t tc004_soak -l node_c
```

Cronograma sugerido (30 min):

Tempo	Acao
0-3 min	flash + boot
3-8 min	baseline estavel
8-10 min	ciclo 1: degradar (afastar B)
10-14 min	ciclo 1: recuperar
14-16 min	ciclo 2: degradar
16-20 min	ciclo 2: recuperar
20-22 min	ciclo 3: degradar
22-26 min	ciclo 3: recuperar
26-30 min	observacao final

Critério formal em [tests/tc-004-soak-estabilidade-e-reconvergencia.md](#).

11. TC-005 - cadeia de 4 nos

Objetivo: 3 saltos `A-B-C-D`. Estende `TC-002` para validar a malha alem de 2 hops.

Hardware: 4 ESP32 (voce esta com 3 hoje, entao este caso fica para quando o quarto chegar).

Passo 1 - Flash dos 4 nos

```
# destino primeiro
zsh firmware/tests/tc005/build_flash.sh node_d /dev/cu.usbserial-XXxD

# intermediarios
zsh firmware/tests/tc005/build_flash.sh node_c /dev/cu.usbserial-XXxC
zsh firmware/tests/tc005/build_flash.sh node_b /dev/cu.usbserial-XXxB

# origem com MAC do NODE_D
zsh firmware/tests/tc005/build_flash.sh node_a /dev/cu.usbserial-XXxA <MAC_D>
```

Passo 2 - Posicionamento

```
[A]---d1---[B]---d2---[C]---d3---[D]
```

`A` nao deve alcancar `C` nem `D` diretamente. `B` nao deve alcancar `D` diretamente.

Passo 3 - Monitor

```
python3 firmware/tools/live_monitor.py \  
  --port /dev/cu.usbserial-XXXX:NODE_A \  
  --port /dev/cu.usbserial-XXXX:NODE_B \  
  --port /dev/cu.usbserial-XXXX:NODE_C \  
  --port /dev/cu.usbserial-XXXX:NODE_D
```

Passo 4 - PASS

NODE_A :

```
route[0] dest=<MAC_D> via=<MAC_B> hops=3 metric=3 state=2  
ACK received from <MAC_D> for seq=N
```

NODE_D :

```
DATA deliver from <MAC_A>: tc005 chain4
```

Critério formal em [tests/tc-005-cadeia-4-nos.md](#).

12. Monitor: dashboard vs texto

Voce **so pode usar um deles por porta**. Eles competem pelo mesmo `/dev/cu.usbserial-XXXX`.

Caracteristica	<code>live_monitor.py</code> (dashboard)	<code>idf.py monitor</code> / <code>monitor_log.sh</code> (texto)
Visualizacao	grafo animado, painel, eventos	linhas de log brutas
Captura em arquivo	nao automatica	sim em <code>firmware/logs/serial/</code>
Ideal para	demo, defesa, ver topologia	extracao de metricas, soak test
Resets/panic	mostra mas nao detalha	mostra stack trace completo
Multi-no	sim, todos no mesmo browser	um terminal por no

Quando usar dashboard

- Demonstracao para banca
- Sanity check de "esta funcionando?"
- Visualizar fluxo de `DATA` e `ACK`
- Detectar quebra de link visualmente

Quando usar texto + captura

- Coletar dados para analise (`extract_monitor_metrics.py`)
- Soak test de 30+ minutos
- Debug de panic/guru meditation
- Validacao formal dos criterios de PASS

Como rodar dashboard com hardware real

```
cd /Users/luaksonlima/Documents/tcc/aodv-en
python3 firmware/tools/live_monitor.py \
  --port /dev/cu.usbserial-2110:NODE_A \
  --port /dev/cu.usbserial-2120:NODE_B \
  --port /dev/cu.usbserial-2130:NODE_C
# abrir http://localhost:8765/
```

Encerrar com Ctrl+C no terminal do `live_monitor.py`.

Como rodar dashboard sem hardware (demo)

```
python3 firmware/tools/live_monitor.py --demo
```

Util para mostrar a interface antes de plugar os ESPs.

13. Analise pos-execucao

Apos uma sessao com captura de log via `monitor_log.sh`, os arquivos estao em `firmware/logs/serial/`.

13.1 Extrair metricas

Para cada arquivo `.log`:

```
python3 firmware/tools/extract_monitor_metrics.py firmware/logs/serial/node_a_tc002_run_20260501-101533.log
```

Saida em `firmware/logs/analysis/<basename>/`: - `summary.json` e `summary.txt` - CSVs: `events`, `snapshots`, `routes`, `target_route_series`, `ack_events`, `discovery_windows`, `minute_metrics` ...

13.2 Gerar graficos

```
python3 firmware/tools/plot_monitor_metrics.py firmware/logs/analysis/node_a_tc002_run_20260501-101533
```

Gera 7 PNGs em `plots/`: ACK vs Fail por minuto, fila por minuto, timeline da rota, distribuicao de hops, etc.

13.3 Comparar runs (uso tipico TC-004)

```
python3 firmware/tools/plot_comparison_metrics.py \
  --run baseline::firmware/logs/analysis/node_a_run1/summary.json \
  --run pos-fix::firmware/logs/analysis/node_a_run2/summary.json \
  --change-index 1
```

13.4 Desenhar topologia observada

```
python3 firmware/tools/draw_topology.py \
  firmware/logs/analysis/node_a_tc002_run_20260501-101533 \
  firmware/logs/analysis/node_b_tc002_run_20260501-101533 \
  firmware/logs/analysis/node_c_tc002_run_20260501-101533 \
  --mode latest
```

Gera Mermaid + DOT + SVG (se `dot` instalado) em `topology/`.

14. Troubleshooting

"command not found: idf.py" depois de source

Voce sourceou em uma sub-shell. Sourceie de novo no terminal atual:

```
source /tmp/aodv-en-idf-env.sh
```

"CMakeLists.txt not found in project directory"

Voce esta chamando `idf.py` de fora da pasta `firmware/`. Os scripts `tc00X/build_flash.sh` ja fazem `cd "$FW_DIR"` antes de invocar `idf.py`. Se voce estiver chamando `idf.py` direto, garanta:

```
cd /Users/huaksonlima/Documents/tcc/aodv-en/firmware
idf.py -B build/algo build
```

"ModuleNotFoundError: No module named 'aiohttp'" no live_monitor

Quando o `idf-env.sh` esta sourceado, `python3` aponta para a venv do ESP-IDF, nao para o python do `pyenv`. Instale `aiohttp` direto na venv do IDF:

```
/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin/pip install aiohttp pyserial
```

"Permission denied" na porta serial

Outro processo esta usando a porta. Verifique se `live_monitor.py` ou `idf.py monitor` ainda esta aberto em outro terminal:

```
lsof /dev/cu.usbserial-2110
```

Mate o processo culpado ou feche o terminal.

"Failed to connect to ESP32: No serial data received"

- Verifique cabos USB (alguns sao "charge only" e nao tem dados)
- Pressione e segure o botao `B00T` do ESP32 enquanto inicia o flash
- Tente um baud menor: `--baud 115200` em vez do `460800` default

Build falha com `aodv_en_node.h: No such file`

Cache de build velho referenciando os stubs deletados em uma versao anterior. Solucao:

```
rm -rf firmware/build firmware/build/tc00*_*
zsh firmware/build.sh
```

`routes=0 neighbors=0` por mais de 30s

Os nos nao estao se vendo. Possiveis causas:

- canal Wi-Fi diferente entre os nos: confira `CONFIG_AODV_EN_APP_WIFI_CHANNEL` em cada `.defaults`
- `network_id` diferente: idem
- distancia fisica grande ou obstaculo metalico
- `app_proto_example` vs `app_demo`: confira que os 2 estao no mesmo modo

"ESP-NOW send fail" toda hora

- Vizinho fora de alcance ou desligado
- Buffer do ESP-NOW lotado: investigue se ha nos com `tx` muito alto e fica reciclando peers do cache
- Interferencia (usar canal diferente do Wi-Fi de casa, idealmente canal 1 ou 11 limpo)

`DATA send status==2 (= AODV_EN_ERR_FULL)`

Fila pendente de `DATA` cheia. Significa que o no esta tentando enviar mais rapido do que a malha consegue descobrir rotas. Comportamento esperado durante quebras prolongadas. Para o TCC, contar como evidencia de que o backpressure funcionou.

Reset constante / guru meditation

Use `idf.py monitor` (nao o dashboard) para ver o stack trace completo. Ferramenta `idf.py monitor` decodifica enderecos automaticamente.

Esquecimento - "qual MAC era qual no?"

Releia direto via `esptool` na porta:

```
ESPT00L=/Users/huaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin/esptool
"$ESPT00L" --port /dev/cu.usbserial-XXXX read_mac
```

Ou olhe a primeira linha de log do no apos boot - tem `node=NODE_X self_mac=...`.

15. Cheatsheet de comandos

```
# === SETUP ===
cd /Users/luaksonlima/Documents/tcc/aadv-en
source /tmp/aadv-en-idf-env.sh

# === DESCOBRIR HARDWARE ===
ls /dev/cu.usbserial-*
ESPT00L=/Users/luaksonlima/.espressif/python_env/idf6.0_py3.14_env/bin/esptool
"$ESPT00L" --port /dev/cu.usbserial-XXXX read_mac

# === BUILD LIMPO ===
rm -rf firmware/build && zsh firmware/build.sh

# === FLASH POR TC ===
# TC-001 (2 nos)
zsh firmware/tests/tc001/build_flash.sh node_b /dev/cu.usbserial-2110
zsh firmware/tests/tc001/build_flash.sh node_a /dev/cu.usbserial-2120 <MAC_B>

# TC-002/003/004 (3 nos)
zsh firmware/tests/tc002/build_flash.sh node_c /dev/cu.usbserial-2130
zsh firmware/tests/tc002/build_flash.sh node_b /dev/cu.usbserial-2120
zsh firmware/tests/tc002/build_flash.sh node_a /dev/cu.usbserial-2110 <MAC_C>

# TC-005 (4 nos)
zsh firmware/tests/tc005/build_flash.sh node_d /dev/cu.usbserial-XXXX
zsh firmware/tests/tc005/build_flash.sh node_c /dev/cu.usbserial-XXXC
zsh firmware/tests/tc005/build_flash.sh node_b /dev/cu.usbserial-XXXB
zsh firmware/tests/tc005/build_flash.sh node_a /dev/cu.usbserial-XXXA <MAC_D>

# === MONITOR DASHBOARD ===
python3 firmware/tools/live_monitor.py \
  --port /dev/cu.usbserial-2110:NODE_A \
  --port /dev/cu.usbserial-2120:NODE_B \
  --port /dev/cu.usbserial-2130:NODE_C
# http://localhost:8765/

# === MONITOR TEXTO + LOG ===
zsh firmware/monitor_log.sh -p /dev/cu.usbserial-XXXX -B build/tc002_node_a -t run01 -l node_a

# === ANALISE ===
python3 firmware/tools/extract_monitor_metrics.py firmware/logs/serial/<arquivo>.log
python3 firmware/tools/plot_monitor_metrics.py firmware/logs/analysis/<basename>
python3 firmware/tools/draw_topology.py firmware/logs/analysis/<basename> --mode latest

# === LIMPEZA ===
rm -rf firmware/build firmware/logs/serial firmware/logs/analysis
```

Anexo - mapping rapido sessao 2026-05-01

3 ESPs detectados nesta sessao:

Porta	MAC	Sugestao de papel
/dev/cu.usbserial-2110	28:05:A5:34:03:50	NODE_A (origem)
/dev/cu.usbserial-2120	28:05:A5:33:B9:EC	NODE_B (relay)
/dev/cu.usbserial-2130	28:05:A5:34:05:64	NODE_C (destino)

Comandos prontos para colar no terminal:

```
cd /Users/huaksonlima/Documents/tcc/aodv-en
source /tmp/aodv-en-idf-env.sh

# TC-002 completo - copia e cola:
zsh firmware/tests/tc002/build_flash.sh node_c /dev/cu.usbserial-2130
zsh firmware/tests/tc002/build_flash.sh node_b /dev/cu.usbserial-2120
zsh firmware/tests/tc002/build_flash.sh node_a /dev/cu.usbserial-2110 28:05:A5:34:05:64

# Dashboard:
python3 firmware/tools/live_monitor.py \
  --port /dev/cu.usbserial-2110:NODE_A \
  --port /dev/cu.usbserial-2120:NODE_B \
  --port /dev/cu.usbserial-2130:NODE_C
```

Casos de Teste - Indice

Testes de Hardware - Padrao

Este directorio guarda casos de teste funcionais de bancada para o `aodv-en`.

Contexto

Os casos de teste deste directorio sao a evidencia operacional dos criterios de aprovacao definidos em [aodv-en-spec-v1.md](#), secao "Criterios de aprovacao da v1".

Mapeamento:

Caso	Criterios da spec v1	Status
TC-001	1 (descoberta e entrega direta)	ATIVO
TC-002	2 (multi-hop em cadeia) e 5 (<code>duplicate_rreq_drops</code>)	ATIVO (<code>PASS</code> em 2026-04-21 com evidencia indireta)
TC-003	3 (reconvergencia) e 7 (<code>RERR</code> para precursores)	ATIVO
TC-004	4 (soak 30 min) e 6 (expiracao de rota)	ATIVO
TC-005	cadeia de 4 nos (3 saltos)	ATIVO

Convencao de nomes

- Arquivo: `tc-XXX-descricao-curta.md`
- Exemplo: `tc-001-descoberta-e-entrega-direta.md`

Estrutura minima obrigatoria

Cada caso deve conter:

- Objetivo
- Topologia
- Configuracao dos nos
- Procedimento
- Evidencias esperadas
- Criterio de aprovacao
- Resultado (preencher a cada execucao)

Casos disponiveis

- TC-001 : [Descoberta e entrega direta](#)
- Script de execucao: `firmware/tests/tc001/build_flash.sh`
- TC-002 : [Primeiro multi-hop em cadeia](#)
- Script de execucao: `firmware/tests/tc002/build_flash.sh`

- Captura de log (global): `firmware/monitor_log.sh`
- Wrapper por papel (TC-002): `firmware/tests/tc002/monitor_log.sh`
- TC-003 : [Reconvergencia apos falha de enlace](#)
- Script de execucao: `firmware/tests/tc002/build_flash.sh` (mesmo setup de 3 nos)
- Captura de log (global): `firmware/monitor_log.sh`
- Wrapper por papel (TC-003): `firmware/tests/tc002/monitor_log.sh`
- TC-004 : [Soak test de estabilidade com reconvergencia ciclica](#)
- Script de execucao: `firmware/tests/tc002/build_flash.sh` (mesmo setup de 3 nos)
- Captura de log (global): `firmware/monitor_log.sh`
- TC-005 : [Cadeia de 4 nos \(3 saltos\)](#)
- Script de execucao: `firmware/tests/tc005/build_flash.sh`
- Captura de log (global): `firmware/monitor_log.sh`
- Wrapper por papel (TC-005): `firmware/tests/tc005/monitor_log.sh`

Guia auxiliar: [Guia de leitura dos graficos de monitor.](#)

TC-001 - Descoberta e Entrega Direta

id: TC-001 title: Descoberta de vizinho e entrega direta (1 salto) status: ATIVO version: 1.0 last_updated: 2026-04-21 owner: firmware scope: hardware applies_to: [esp32, esp-now, aodv-en]

TC-001 - Descoberta de vizinho e entrega direta (1 salto)

1. Objetivo

Validar, em hardware real, que dois nos:

- se descobrem via `HELLO` ;
- instalam rota valida (`state=2`) com `hops=1` ;
- entregam `DATA` fim a fim com `ACK` .

2. Topologia

- `NODE_A <-> NODE_B` (alcance direto)
- ambos no mesmo canal Wi-Fi e mesmo `network_id`

3. Configuracao dos nos

NODE_A (origem de DATA)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_A"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=y`
- `CONFIG_AODV_EN_APP_TARGET_MAC="<MAC do NODE_B>"`
- `CONFIG_AODV_EN_APP_PAYLOAD_TEXT="tc001 hello"`
- perfil pronto: `firmware/tests/tc001/node_a.defaults`

NODE_B (destino de DATA)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_B"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc001/node_b.defaults`

Script de build/flash do caso

- `firmware/tests/tc001/build_flash.sh`

4. Procedimento

1. Gravar `NODE_A`: - `zsh firmware/tests/tc001/build_flash.sh node_a <PORTA_NODE_A> <MAC_NODE_B>`
2. Gravar `NODE_B`: - `zsh firmware/tests/tc001/build_flash.sh node_b <PORTA_NODE_B>`
3. Abrir monitor serial em ambos os nos: - `idf.py -B firmware/build/tc001_node_a -p <PORTA_NODE_A> monitor`
- `idf.py -B firmware/build/tc001_node_b -p <PORTA_NODE_B> monitor`
4. Aguardar de 30 a 60 segundos com os dois ligados.
5. Salvar log serial de cada no.

5. Evidencias esperadas

NODE_A

- `neighbors >= 1`
- `routes >= 1`
- rota para MAC do `NODE_B` com `state=2` e `hops=1`
- eventos de `ACK received from ...`

NODE_B

- `neighbors >= 1`
- `routes >= 1`
- logs `DATA deliver from ...`
- `delivered > 0`

6. Criterio de aprovacao

`PASS` quando todos os itens abaixo forem verdadeiros:

- os dois nos mantem rota valida entre si por pelo menos 3 janelas de log consecutivas;
- `NODE_B` registra entrega de `DATA` (`delivered > 0`);
- `NODE_A` recebe `ACK` correspondente ao menos uma vez;
- nao ha reboot/restart durante a janela de observacao.

`FAIL` quando qualquer item acima nao for atendido.

7. Resultado (preencher a cada execucao)

- Data: `2026-04-21`
- Firmware SHA: `40c43ba-dirty`
- MAC `NODE_A`: `28:05:A5:33:D6:1C`
- MAC `NODE_B`: `28:05:A5:33:EB:80`
- Resultado: `PASS`
- Observacoes:
- `NODE_A` enviou `DATA` periodico para `NODE_B` e recebeu `ACK` sequenciais (`seq=1..5` no trecho observado).

- `NODE_B` registrou entregas repetidas de payload (`DATA deliver ... tc001 hello`) e `delivered` evoluiu de `1` para `17` no trecho observado.
- Rota permaneceu valida em ambos os nos (`state=2` , `hops=1` , `neighbors=1` , `routes=1`) sem reboot durante a janela de observacao.

TC-002 - Primeiro Multi-hop

id: TC-002 title: Primeiro multi-hop em cadeia (2 saltos) status: ATIVO version: 1.0 last_updated: 2026-04-21 owner: firmware scope: hardware applies_to: [esp32, esp-now, aodv-en]

TC-002 - Primeiro multi-hop em cadeia (2 saltos)

1. Objetivo

Validar, em hardware real, que a malha faz entrega fim a fim em dois saltos:

- `NODE_A` envia `DATA` para `NODE_C` ;
- `NODE_B` atua como intermediario;
- `NODE_A` recebe `ACK` de `NODE_C` .

2. Topologia

- `NODE_A <--> NODE_B <--> NODE_C` (cadeia linear)
- `NODE_A` e `NODE_C` devem ficar sem enlace direto
- todos no mesmo canal Wi-Fi e mesmo `network_id`

3. Configuracao dos nos

NODE_A (origem de DATA para C)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_A"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=y`
- `CONFIG_AODV_EN_APP_TARGET_MAC="<MAC do NODE_C>"`
- `CONFIG_AODV_EN_APP_PAYLOAD_TEXT="tc002 multihop"`
- perfil pronto: `firmware/tests/tc002/node_a.defaults`

NODE_B (intermediario)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_B"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc002/node_b.defaults`

NODE_C (destino de DATA)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_C"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc002/node_c.defaults`

Script de build/flash do caso

- `firmware/tests/tc002/build_flash.sh`
- `firmware/monitor_log.sh` (captura global em `firmware/logs/serial/`)
- `firmware/tests/tc002/monitor_log.sh` (wrapper por papel para `NODE_A/B/C`)

4. Procedimento

1. Gravar `NODE_A` : - `zsh firmware/tests/tc002/build_flash.sh node_a <PORTA_NODE_A> <MAC_NODE_C>`
2. Gravar `NODE_B` : - `zsh firmware/tests/tc002/build_flash.sh node_b <PORTA_NODE_B>`
3. Gravar `NODE_C` : - `zsh firmware/tests/tc002/build_flash.sh node_c <PORTA_NODE_C>`
4. Posicionar fisicamente os nos em cadeia (`A-B-C`) sem enlace direto entre `A` e `C` .
5. Abrir monitor serial nos tres nos (preferencialmente com captura de log): - opcao global:
 - `zsh firmware/monitor_log.sh -p <PORTA_NODE_A> -B build/tc002_node_a -t tc002_run -l node_a`
 - `zsh firmware/monitor_log.sh -p <PORTA_NODE_B> -B build/tc002_node_b -t tc002_run -l node_b`
 - `zsh firmware/monitor_log.sh -p <PORTA_NODE_C> -B build/tc002_node_c -t tc002_run -l node_c`
 - opcao wrapper por papel:
 - `zsh firmware/tests/tc002/monitor_log.sh node_a <PORTA_NODE_A> tc002_run`
 - `zsh firmware/tests/tc002/monitor_log.sh node_b <PORTA_NODE_B> tc002_run`
 - `zsh firmware/tests/tc002/monitor_log.sh node_c <PORTA_NODE_C> tc002_run`
 - (alternativa sem arquivo: `idf.py -B firmware/build/tc002_node_a -p <PORTA_NODE_A> monitor` etc.)
6. Aguardar de 60 a 120 segundos e salvar logs dos tres nos.

5. Evidencias esperadas

NODE_A

- rota para `NODE_C` com `state=2` e `hops=2`
- `ACK received from <MAC do NODE_C> ...`

NODE_B

- `neighbors >= 2`
- rotas validas para `NODE_A` e `NODE_C` (tipicamente `hops=1`)

NODE_C

- `DATA deliver from <MAC do NODE_A> ...`
- `delivered > 0`

6. Criterio de aprovacao

PASS quando todos os itens abaixo forem verdadeiros:

- **NODE_A** estabelece rota para **NODE_C** com **hops=2** por pelo menos 3 janelas de log;
- **NODE_C** registra entrega de **DATA** (**delivered > 0**);
- **NODE_A** recebe **ACK** de **NODE_C** ;
- nao ha reboot/restart durante a janela de observacao.

FAIL quando qualquer item acima nao for atendido.

7. Resultado (preencher a cada execucao)

- Data:
- Firmware SHA:
- MAC **NODE_A** :
- MAC **NODE_B** :
- MAC **NODE_C** :
- Resultado: **PASS** ou **FAIL**
- Observacoes:

Execucao 2026-04-21 (PASS com evidencia indireta via NODE_A)

- Firmware SHA: 40c43ba-dirty
- MAC **NODE_A** : 28:05:A5:33:D6:1C
- MAC **NODE_B** : 28:05:A5:33:EB:80
- MAC **NODE_C** : 28:05:A5:34:99:34
- Resultado desta execucao: **PASS** (com evidencia indireta)
- Evidencias confirmadas:
- fase de indisponibilidade seguida de reconvergencia:
 - inicio com **routes=0 neighbors=0** e **DATA queued while route discovery is in progress**
 - evento de fila cheia observado em janela de descoberta prolongada: **DATA send status=-2** (**AODV_EN_ERR_FULL**)
 - recuperacao posterior com ACKs em sequencia (**seq=1..4**) e retorno de rota valida
- rota ativa para **NODE_C** com 2 saltos via **NODE_B** :
 - **dest=28:05:A5:34:99:34 via=28:05:A5:33:EB:80 hops=2 metric=2 state=2**
- ACKs fim a fim recebidos no **NODE_A** :
 - **ACK received from 28:05:A5:34:99:34 for seq=1**
 - **ACK received from 28:05:A5:34:99:34 for seq=2**
 - **ACK received from 28:05:A5:34:99:34 for seq=3**
 - **ACK received from 28:05:A5:34:99:34 for seq=4**
 - **ACK received from 28:05:A5:34:99:34 for seq=5**
- Observacao de bancada:
- sem captura serial simultanea de **NODE_B** e **NODE_C** nesta execucao por restricao de alimentacao/infra da bancada
- fechamento aceito com evidencias indiretas consistentes do **NODE_A** (rota **hops=2** via **NODE_B** + ACKs do **NODE_C**)

TC-003 - Reconvergencia apos Falha

id: TC-003 title: Reconvergencia apos falha de enlace intermediario status: ATIVO version: 1.0 last_updated: 2026-04-21 owner: firmware scope: hardware applies_to: [esp32, esp-now, aodv-en]

TC-003 - Reconvergencia apos falha de enlace intermediario

1. Objetivo

Validar, em hardware real, que a malha reconverge apos perda temporaria do enlace intermediario:

- `NODE_A` envia `DATA` para `NODE_C` pela cadeia `A-B-C` ;
- durante indisponibilidade de rota, `NODE_A` entra em descoberta/queue;
- apos retorno do caminho, a rota volta e os `ACK` retornam.

2. Topologia

- `NODE_A <-> NODE_B <-> NODE_C` (cadeia linear)
- `NODE_A` e `NODE_C` sem enlace direto estavel
- todos no mesmo canal Wi-Fi e mesmo `network_id`

3. Configuracao dos nos

NODE_A (origem de DATA para C)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_A"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=y`
- `CONFIG_AODV_EN_APP_TARGET_MAC="<MAC do NODE_C>"`
- `CONFIG_AODV_EN_APP_PAYLOAD_TEXT="tc002 multihop"`
- perfil usado: `firmware/tests/tc002/node_a.defaults`

NODE_B (intermediario)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_B"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`

- perfil usado: `firmware/tests/tc002/node_b.defaults`

NODE_C (destino de DATA)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_C"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil usado: `firmware/tests/tc002/node_c.defaults`

Script de build/flash do caso

- `firmware/tests/tc002/build_flash.sh`
- `firmware/monitor_log.sh` (captura global em `firmware/logs/serial/`)
- `firmware/tests/tc002/monitor_log.sh` (wrapper por papel para `NODE_A/B/C`)

4. Procedimento

1. Gravar os nós com o mesmo fluxo do `TC-002` .
2. Iniciar captura de log, no mínimo do `NODE_A` : - opção global:
 - `zsh firmware/monitor_log.sh -p <PORTA_NODE_A> -B build/tc002_node_a -t tc003_reconvergencia -l node_a`
 - opção wrapper por papel:
 - `zsh firmware/tests/tc002/monitor_log.sh node_a <PORTA_NODE_A> tc003_reconvergencia`
 - opcional: capturar também `NODE_B` e `NODE_C` quando houver serial disponível.
3. Aguardar fase estável inicial da cadeia `A-B-C` .
4. Provocar indisponibilidade temporária do caminho intermediário (ex.: queda de enlace/interferência/posicionamento de bancada).
5. Observar no `NODE_A` fase de descoberta com `DATA` enfileirada e ausência temporária de rota.
6. Restabelecer o caminho intermediário.
7. Confirmar reconvergência com rota válida e retorno de `ACK` .

5. Evidências esperadas

NODE_A

- durante falha:
 - `DATA queued while route discovery is in progress`
- janela com `routes=0 neighbors=0` (quando o caminho some)
- após recuperação:
 - rota válida para `NODE_C` via `NODE_B` com `hops=2`
 - `ACK received from <MAC do NODE_C> ...`

NODE_B e NODE_C

- quando disponível monitor serial:
- `NODE_B` deve voltar a encaminhar após reconvergência

- `NODE_C` deve voltar a receber `DATA` do `NODE_A`

6. Criterio de aprovacao

`PASS` quando todos os itens abaixo forem verdadeiros:

- houve fase de indisponibilidade observavel (descoberta/queue sem rota estavel);
- houve recuperacao automatica da rota para `NODE_C` via `NODE_B` (`hops=2`);
- `ACK` ao `NODE_A` voltou apos a recuperacao;
- nao houve reboot/reset durante a janela observada.

`FAIL` quando qualquer item acima nao for atendido.

7. Resultado (preencher a cada execucao)

- Data:
- Firmware SHA:
- MAC `NODE_A` :
- MAC `NODE_B` :
- MAC `NODE_C` :
- Resultado: `PASS` ou `FAIL`
- Observacoes:

Execucao 2026-04-21 (PASS com evidencia indireta via NODE_A)

- Firmware SHA: `40c43ba-dirty`
- MAC `NODE_A` : `28:05:A5:33:D6:1C`
- MAC `NODE_B` : `28:05:A5:33:EB:80`
- MAC `NODE_C` : `28:05:A5:34:99:34`
- Resultado desta execucao: `PASS` (com evidencia indireta)
- Evidencias confirmadas:
- fase de indisponibilidade com descoberta em progresso e sem vizinhos/rotas:
 - `DATA queued while route discovery is in progress`
 - `routes=0 neighbors=0 tx=... rx=0 delivered=0`
- saturacao de fila durante indisponibilidade prolongada:
 - `DATA send status=-2 ( AODV_EN_ERR_FULL )`
- recuperacao do caminho com ACKs retornando:
 - `ACK received from 28:05:A5:34:99:34 for seq=1`
 - `ACK received from 28:05:A5:34:99:34 for seq=2`
 - `ACK received from 28:05:A5:34:99:34 for seq=3`
 - `ACK received from 28:05:A5:34:99:34 for seq=4`
- rota restabelecida via intermediario com 2 saltos:
 - `dest=28:05:A5:34:99:34 via=28:05:A5:33:EB:80 hops=2 metric=2 state=2`
- confirmacao de continuidade apos reconvergencia:
 - `ACK received from 28:05:A5:34:99:34 for seq=5`
 - `ACK received from 28:05:A5:34:99:34 for seq=6`

- Observacao de bancada:
- sem captura serial simultanea de `NODE_B` e `NODE_C` nesta execucao por restricao de alimentacao/infra da bancada
- fechamento aceito com evidencias indiretas consistentes do `NODE_A`

Caso complementar 2026-04-21 (flapping de rota observado)

- Contexto:
- mesma topologia `A-B-C` e mesmo perfil de envio periodico do `NODE_A`
- Comportamento observado no `NODE_A` :
- alternancia de rota para `NODE_C` entre:
 - multi-hop via intermediario: `dest=28:05:A5:34:99:34 via=28:05:A5:33:EB:80 hops=2`
 - rota direta temporaria: `dest=28:05:A5:34:99:34 via=28:05:A5:34:99:34 hops=1`
- janelas sem rota ativa para `NODE_C` (`routes=1` , apenas rota para `NODE_B`) com retorno de `DATA queued while route discovery is in progress`
- falhas intermitentes de enlace:
 - `ESP-NOW send fail to 28:05:A5:33:EB:80`
 - `ESP-NOW send fail to 28:05:A5:34:99:34`
 - `DATA send status=-2` em janelas prolongadas de instabilidade (`A0DV_EN_ERR_FULL`)
- recuperacao recorrente com retomada de `ACK` e restabelecimento de rota
- Interpretacao tecnica:
- evidencia de flapping de rota por variacao de qualidade de enlace em bancada (RF dinamico)
- mecanismo de reconvergencia permaneceu funcional, com retorno automatico de entrega/ACK apos oscilacoes
- Uso no TCC:
- este caso documenta comportamento realista de malha sem fio em ambiente nao controlado: alternancia de caminho, perda temporaria e autorrecuperacao

TC-004 - Soak de Estabilidade

id: TC-004 title: Soak test de estabilidade com reconvergencya ciclica status: ATIVO version: 1.0 last_updated: 2026-04-21 owner: firmware scope: hardware applies_to: [esp32, esp-now, aadv-en]

TC-004 - Soak test de estabilidade com reconvergencya ciclica

1. Objetivo

Validar estabilidade operacional em janela estendida, com ciclos repetidos de degradacao e recuperacao de rota:

- medir continuidade de entrega/ACK sob oscilacao de enlace;
- medir reconvergencya automatica em cada ciclo;
- quantificar eventos de falha (`ESP-NOW send fail` , `DATA send status=-2`) sem reboot.
- `status=-2` deve ser interpretado como `AADV_EN_ERR_FULL` (fila/buffer de pending DATA cheio).

2. Topologia

- `NODE_A <=> NODE_B <=> NODE_C` (cadeia principal)
- preferencialmente com baixa conectividade direta `A <=> C` (para privilegiar caminho multi-hop)
- todos no mesmo canal Wi-Fi e mesmo `network_id`

3. Configuracao dos nos

Usar os mesmos perfis do `TC-002` / `TC-003` :

- `NODE_A` : `firmware/tests/tc002/node_a.defaults`
- `NODE_B` : `firmware/tests/tc002/node_b.defaults`
- `NODE_C` : `firmware/tests/tc002/node_c.defaults`
- build/flash: `firmware/tests/tc002/build_flash.sh`

4. Ferramentas de captura

- monitor global: `firmware/monitor_log.sh`
- monitor por papel (wrapper): `firmware/tests/tc002/monitor_log.sh`
- logs salvos em: `firmware/logs/serial/`

5. Procedimento (passo a passo com tempo)

Duracao total sugerida: `30 minutos` .

1. T-00:00 a T-03:00 : - gravar os 3 nos e iniciar monitor do `NODE_A` com arquivo: - `zsh firmware/monitor_log.sh -p <PORTA_NODE_A> -B build/tc002_node_a -t tc004_soak -l node_a` - opcional: iniciar tambem `NODE_B` e `NODE_C` .
2. T-03:00 a T-08:00 (baseline estavel): - manter topologia estavel; - confirmar `DATA queued to route` e `ACK received ...` .
3. T-08:00 a T-10:00 (ciclo 1 - degradacao): - provocar degradacao do caminho intermediario (ex.: afastar/interferir `NODE_B`); - observar descoberta/fila (`DATA queued while route discovery is in progress`).
4. T-10:00 a T-14:00 (ciclo 1 - recuperacao): - restabelecer caminho; - observar retorno de rota e ACK.
5. T-14:00 a T-16:00 (ciclo 2 - degradacao): - repetir degradacao.
6. T-16:00 a T-20:00 (ciclo 2 - recuperacao): - restabelecer e observar reconvergencia.
7. T-20:00 a T-22:00 (ciclo 3 - degradacao): - repetir degradacao.
8. T-22:00 a T-26:00 (ciclo 3 - recuperacao): - restabelecer e observar reconvergencia.
9. T-26:00 a T-30:00 (fechamento): - manter estavel e encerrar captura.

6. Evidencias esperadas

No `NODE_A` :

- baseline com `ACK received ...` recorrente;
- em degradacao: aumento de `DATA queued while route discovery is in progress` ;
- em recuperacao: retorno de `ACK` e rota para `NODE_C` (hops=2 via `NODE_B` ou hops=1 quando houver enlace direto temporario);
- ausencia de reboot durante toda a janela.

7. Criterio de aprovacao

`PASS` quando todos os itens abaixo forem verdadeiros:

- sem reboot/reset durante os 30 minutos;
- reconvergencia observada nos 3 ciclos apos restabelecimento;
- retorno de `ACK` em todos os ciclos de recuperacao;
- `DATA send status=-2` (fila/buffer cheio) ocorre, no maximo, de forma esporadica e sem travar recuperacao.

`FAIL` quando qualquer item acima nao for atendido.

8. Extracao rapida de metricas (log do NODE_A)

Assumindo log em `firmware/logs/serial/node_a_tc004_soak_<timestamp>.log` :

```
python3 firmware/tools/extract_monitor_metrics.py \
firmware/logs/serial/node_a_tc004_soak_<timestamp>.log
```

Arquivos gerados automaticamente:

- `firmware/logs/analysis/node_a_tc004_soak_<timestamp>/summary.json`
- `firmware/logs/analysis/node_a_tc004_soak_<timestamp>/summary.txt`
- `firmware/logs/analysis/node_a_tc004_soak_<timestamp>/*.csv`

Geracao automatica de graficos (PNG):

```
python3 firmware/tools/plot_monitor_metrics.py \
firmware/logs/analysis/node_a_tc004_soak_<timestamp>
```

PNG em:

- `firmware/logs/analysis/node_a_tc004_soak_<timestamp>/plots/`
- guia de leitura: `docs/tests/guia-leitura-graficos-monitor.md`
- formulas das metricas: secar `Formulas e como chegar nos resultados` em `docs/tests/guia-leitura-graficos-monitor.md`

Comparativo entre multiplas execucoes (com marcador pre/pós mudanca de codigo):

```
python3 firmware/tools/plot_comparison_metrics.py \
--run "exec_1::firmware/logs/analysis/node_a_tc004_soak_<ts1>/summary.json" \
--run "exec_2::firmware/logs/analysis/node_a_tc004_soak_<ts2>/summary.json" \
--run "exec_pos_fix::firmware/logs/analysis/node_a_tc004_soak_v2_<ts3>/summary.json" \
--change-index 2 \
--out-dir firmware/logs/analysis/comparisons/tc004_exec1_exec2_posfix
```

Arquivos comparativos:

- `firmware/logs/analysis/comparisons/<nome>/comparison_metrics.csv`
- `firmware/logs/analysis/comparisons/<nome>/01_core_metrics.png`
- `firmware/logs/analysis/comparisons/<nome>/02_stability_metrics.png`
- `firmware/logs/analysis/comparisons/<nome>/03_route_profile.png`

Extracao manual complementar:

```
LOG="firmware/logs/serial/node_a_tc004_soak_<timestamp>.log"
rg -c "ACK received" "$LOG"
rg -c "DATA queued while route discovery is in progress" "$LOG"
rg -c "DATA send status=-2" "$LOG"
rg -c "ESP-NOW send fail to" "$LOG"
rg -c "hops=2" "$LOG"
rg -c "hops=1" "$LOG"
```

9. Resultado (preencher a cada execucao)

- Data:
- Firmware SHA:
- MAC `NODE_A` :
- MAC `NODE_B` :
- MAC `NODE_C` :
- Duracao real:
- Resultado: `PASS` ou `FAIL`
- Observacoes:

TC-005 - Cadeia de 4 Nos

id: TC-005 title: Cadeia de 4 nos (3 saltos) status: ATIVO version: 1.0 last_updated: 2026-05-01 owner: firmware scope: hardware applies_to: [esp32, esp-now, aodv-en]

TC-005 - Cadeia de 4 nos (3 saltos)

1. Objetivo

Validar, em hardware real, que a malha faz entrega fim a fim em tres saltos:

- `NODE_A` envia `DATA` para `NODE_D` ;
- `NODE_B` e `NODE_C` atuam como intermediarios;
- `NODE_A` recebe `ACK` de `NODE_D` .

Este caso estende `TC-002` para cadeias mais longas e ajuda a evidenciar para o TCC que o `AODV-EN` opera alem de 2 saltos.

2. Topologia

- `NODE_A <--> NODE_B <--> NODE_C <--> NODE_D` (cadeia linear)
- `NODE_A` e `NODE_D` devem ficar sem enlace direto
- `NODE_A` e `NODE_C` tambem devem ficar sem enlace direto
- `NODE_B` e `NODE_D` tambem devem ficar sem enlace direto
- todos no mesmo canal Wi-Fi e mesmo `network_id`

3. Configuracao dos nos

NODE_A (origem de DATA para D)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_A"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=y`
- `CONFIG_AODV_EN_APP_TARGET_MAC="<MAC do NODE_D>"`
- `CONFIG_AODV_EN_APP_PAYLOAD_TEXT="tc005 chain4"`
- perfil pronto: `firmware/tests/tc005/node_a.defaults`

NODE_B (intermediario)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_B"`

- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc005/node_b.defaults`

NODE_C (intermediario)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_C"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc005/node_c.defaults`

NODE_D (destino de DATA)

- `CONFIG_AODV_EN_APP_NODE_NAME="NODE_D"`
- `CONFIG_AODV_EN_APP_NETWORK_ID=0xA0DE0001`
- `CONFIG_AODV_EN_APP_WIFI_CHANNEL=6`
- `CONFIG_AODV_EN_APP_ENABLE_DATA=n`
- perfil pronto: `firmware/tests/tc005/node_d.defaults`

Script de build/flash do caso

- `firmware/tests/tc005/build_flash.sh`
- `firmware/monitor_log.sh` (captura global em `firmware/logs/serial/`)
- `firmware/tests/tc005/monitor_log.sh` (wrapper por papel para `NODE_A/B/C/D`)

4. Procedimento

1. Gravar `NODE_A` : - `zsh firmware/tests/tc005/build_flash.sh node_a <PORTA_NODE_A> <MAC_NODE_D>`
2. Gravar `NODE_B` : - `zsh firmware/tests/tc005/build_flash.sh node_b <PORTA_NODE_B>`
3. Gravar `NODE_C` : - `zsh firmware/tests/tc005/build_flash.sh node_c <PORTA_NODE_C>`
4. Gravar `NODE_D` : - `zsh firmware/tests/tc005/build_flash.sh node_d <PORTA_NODE_D>`
5. Posicionar fisicamente os nos em cadeia (`A-B-C-D`) sem enlace direto fora dos vizinhos imediatos.
6. Abrir monitor serial nos quatro nos: - `zsh firmware/tests/tc005/monitor_log.sh node_a <PORTA_NODE_A>`
`tc005_run` - `zsh firmware/tests/tc005/monitor_log.sh node_b <PORTA_NODE_B> tc005_run` - `zsh`
`firmware/tests/tc005/monitor_log.sh node_c <PORTA_NODE_C> tc005_run` - `zsh`
`firmware/tests/tc005/monitor_log.sh node_d <PORTA_NODE_D> tc005_run`
7. Aguardar de 60 a 180 segundos e salvar logs dos quatro nos.

5. Evidencias esperadas

NODE_A

- rota para `NODE_D` com `state=2` e `hops=3`
- `ACK received from <MAC do NODE_D> ...`

NODE_B

- rotas validas para `NODE_A ( hops=1 )` e `NODE_D ( hops=2 )`
- contagem positiva de `forwarded_frames`

NODE_C

- rotas validas para `NODE_A ( hops=2 )` e `NODE_D ( hops=1 )`
- contagem positiva de `forwarded_frames`

NODE_D

- `DATA deliver from <MAC do NODE_A> ...`
- `delivered > 0`

6. Critério de aprovacao

`PASS` quando todos os itens abaixo forem verdadeiros:

- `NODE_A` estabelece rota para `NODE_D` com `hops=3` por pelo menos 3 janelas de log;
- `NODE_D` registra entrega de `DATA ( delivered > 0 )`;
- `NODE_A` recebe `ACK` de `NODE_D` ;
- nao ha reboot/restart durante a janela de observacao.

`FAIL` quando qualquer item acima nao for atendido.

7. Resultado (preencher a cada execucao)

- Data:
- Firmware SHA:
- MAC `NODE_A` :
- MAC `NODE_B` :
- MAC `NODE_C` :
- MAC `NODE_D` :
- Resultado: `PASS` ou `FAIL`
- Observacoes:

Guia de leitura dos graficos do monitor

Guia de Leitura dos Graficos de Monitor

Este guia documenta como interpretar os PNG gerados por:

```
python3 firmware/tools/plot_monitor_metrics.py <analysis_dir>
```

Exemplo de pasta:

- `firmware/logs/analysis/node_a_tc004_soak_<timestamp>/plots/`

Escopo

- Visao principal focada no `NODE_A` (origem do log serial analisado).
- Os graficos sao derivados dos CSV em `firmware/logs/analysis/<run>/`.
- O mapeamento oficial dos arquivos gerados fica em `plots/manifest.json`.

Desenho da topologia (Mermaid/DOT)

Para gerar um desenho da topologia observada nos logs analisados:

```
python3 firmware/tools/draw_topology.py <analysis_dir> --mode latest
```

Arquivos gerados em `<analysis_dir>/topology/`:

- `topology.mmd` (Mermaid)
- `topology.dot` (Graphviz DOT)
- `topology.svg` (quando o comando `dot` estiver instalado)
- `topology.json` (resumo estruturado de nos, enlaces e snapshot usado)

Modos:

- `--mode latest`: desenha apenas o ultimo snapshot (estado final).
- `--mode observed`: desenha a uniao dos enlaces vistos ao longo da execucao.

01 - ACK vs Fail por Minuto

Arquivo:

- `01_ack_vs_fail_per_minute.png`

Pergunta que responde:

- O enlace esta entregando com regularidade ou esta degradando?

Eixos:

- X: minuto da execucao.

- Y: quantidade de eventos no minuto.
- Curvas: `ACK/min` e `ESP-NOW fail/min`.

Leitura pratica:

- Bom: `ACK` consistente e `fail` baixo.
- Alerta: queda de `ACK` junto de pico de `fail`.
- Critico: longos periodos com `ACK=0` e `fail` alto.

CSV de apoio:

- `minute_metrics.csv` (`ack_count` , `send_fail_count`).

02 - Fila de DATA e status=-2 por Minuto

Arquivo:

- `02_queue_and_status_per_minute.png`

Pergunta que responde:

- O no esta pressionando fila por falta de rota?

Eixos:

- X: minuto.
- Y esquerdo: eventos de fila (`queued_to_route` , `queued_discovery`).
- Y direito: `status=-2` por minuto.

Leitura pratica:

- Bom: fila oscila, mas `status=-2` e raro.
- Alerta: `queued_discovery` alto por varios minutos.
- Critico: `status=-2` recorrente + baixa taxa de ACK.

Definicao importante:

- `status=-2` corresponde a `A0DV_EN_ERR_FULL` , ou seja, **fila/buffer interno de pending DATA cheio**.
- Interpretacao recomendada para relatorio: evento de **pressao de memoria de fila** (capacidade interna do componente), e nao erro generico.

CSV de apoio:

- `minute_metrics.csv` (`queued_route_count` , `queued_discovery_count` , `status_neg2_count`).

03 - Timeline do Estado de Rota Alvo

Arquivo:

- `03_route_state_timeline.png`

Pergunta que responde:

- A rota para o destino alvo esta estavel, ausente, ou flapeando?

Eixos:

- X: tempo em segundos.
- Y: estado (`absent` , `hops=1` , `hops=2` , `other`).

Leitura pratica:

- Bom: blocos longos no mesmo estado.
- Alerta: alternancia frequente entre `hops=1` e `hops=2`.
- Critico: trechos longos em `absent`.

CSV de apoio:

- `target_route_series.csv`.
- `target_route_transitions.csv` (quantifica transicoes/flaps).

04 - Distribuicao de Estado da Rota Alvo

Arquivo:

- `04_route_state_distribution.png`

Pergunta que responde:

- Qual estado predominou no teste completo?

Eixos:

- X: categorias (`hops=2` , `hops=1` , `absent`).
- Y: numero de snapshots.

Leitura pratica:

- Bom: alta proporcao de rota presente (`hops=1` ou `hops=2`).
- Alerta: proporcao alta de `absent`.
- Uso tipico em relatorio: percentual de tempo com rota disponivel.

CSV de apoio:

- `target_route_series.csv` (`state_label`).

05 - Media de Neighbors e Rotas por Minuto

Arquivo:

- `05_neighbors_routes_per_minute.png`

Pergunta que responde:

- O no esta enxergando vizinhos/rotas de forma continua?

Eixos:

- X: minuto.
- Y: media por snapshot no minuto.
- Curvas: `avg_neighbors` e `avg_routes`.

Leitura pratica:

- Bom: curvas acima de zero e sem quedas longas.
- Alerta: quedas repetidas para valores baixos.
- Critico: varios minutos proximos de zero (no "cego").

CSV de apoio:

- `minute_metrics.csv` (`avg_neighbors` , `avg_routes`).

06 - Duracao das Janelas de Discovery

Arquivo:

- `06_discovery_window_durations.png`

Pergunta que responde:

- Quanto tempo a reconvergencia leva em cada janela?

Eixos:

- X: indice da janela de discovery.
- Y: duracao em segundos.
- Linhas: media e maximo.

Leitura pratica:

- Bom: duracoes curtas e distribuidas de forma estavel.
- Alerta: cauda longa (algumas janelas muito maiores que a media).
- Critico: maximos altos repetidos (reconvergencia lenta/intermitente).

CSV de apoio:

- `discovery_windows.csv` (`duration_ms`).

07 - Evolucao de TX/RX (Delta no Tempo)

Arquivo:

- `07_tx_rx_delta_over_time.png`

Pergunta que responde:

- O volume transmitido cresce com recepcao proporcional?

Eixos:

- X: tempo em segundos.
- Y: delta acumulado desde o primeiro snapshot.
- Curvas: `tx_delta` e `rx_delta` .

Leitura pratica:

- Bom: crescimento de TX e RX com distancia moderada.
- Alerta: gap crescente de TX >> RX.
- Critico: TX sobe forte com RX quase estagnado (overhead/retries).

CSV de apoio:

- `snapshots.csv` (`tx_delta` , `rx_delta`).

Checklist rapido de leitura

1. Verificar `01` (saude de entrega: ACK vs fail).
2. Verificar `02` (pressao de fila e `status=-2`).

3. Verificar 03 + 04 (estabilidade e disponibilidade de rota alvo).
4. Verificar 06 (tempo de reconvergencia).
5. Fechar com 07 (eficiencia TX/RX no periodo total).

Observacao metodologica

- Em bancada real com oscilacao RF, variacao entre hops=1 e hops=2 pode ocorrer.
- O objetivo nao e eliminar toda variacao, e sim manter entrega/ACK e reconvergencia dentro de limites aceitaveis.

Formulas e como chegar nos resultados

Esta secao mostra exatamente como os indicadores sao calculados a partir dos artefatos de analise.

Fontes de verdade (arquivos)

- `summary.json` : agregados finais da execucao.
- `minute_metrics.csv` : contagens por minuto.
- `target_route_series.csv` : estado da rota alvo por snapshot.
- `target_route_transitions.csv` : transicoes de estado (flaps).
- `discovery_windows.csv` : duracao de janelas de descoberta.

Notacao usada

- `D_min` : duracao total em minutos (`summary.timeline.duration_min`).
- `D_h` : duracao total em horas (`D_min / 60`).
- `ACK` : total de ACK (`summary.counts.acks`).
- `FAIL` : total de falhas ESP-NOW (`summary.counts.espnw_send_fail`).
- `Q_disc` : total de `data_queued_discovery` (`summary.counts.data_queued_discovery`).
- `FLAP` : total de transicoes de estado da rota alvo (`summary.route_target_analysis.flap_count`).
- `FULL_-2` : total de eventos `status=-2` (`summary.data_send_status_by_code["-2"]`).
- `S_abs` : snapshots sem rota alvo (`summary.route_target_analysis.snapshots_without_target`).
- `S_pres` : snapshots com rota alvo (`summary.route_target_analysis.snapshots_with_target`).
- `S_total = S_abs + S_pres` .

Formulas principais (comparativo entre execucoes)

1. `ack_per_min`

```
ack_per_min = ACK / D_min
```

2. `send_fail_per_min`

```
send_fail_per_min = FAIL / D_min
```

3. `status_neg2_per_min`

```
status_neg2_per_min = FULL_-2 / D_min
```

Onde `status=-2` significa `A0DV_EN_ERR_FULL` (fila/buffer interno cheio).

4. `queued_discovery_per_min`

```
queued_discovery_per_min = Q_disc / D_min
```

5. `flap_per_hour`

```
flap_per_hour = FLAP / D_h
```

6. `target_absent_ratio`

```
target_absent_ratio = S_abs / S_total
```

7. `hops1_ratio` e `hops2_ratio`

```
hops1_ratio = hops_1_count / S_total  
hops2_ratio = hops_2_count / S_total
```

8. `absent_window_max_s`

```
absent_window_max_s = absent_window_max_ms / 1000
```

(`absent_window_max_ms` vem de `summary.route_target_analysis.absent_window_max_ms`).

Como o flap e contado

Um flap e cada troca de estado consecutiva em `target_route_series.csv`, por exemplo:

- `hops_2 -> hops_1`
- `hops_1 -> absent`
- `absent -> hops_2`

Cada troca soma `1` em `flap_count`.

Validacao rapida (manual)

1. Abrir `summary.json`.
2. Copiar `duration_min`, `counts` e `route_target_analysis`.
3. Aplicar as formulas acima.
4. Conferir se bate com: - `comparison_metrics.csv` (quando houver comparativo), - ou com os valores exibidos no `summary.txt`.

Exemplo de reproducao por script

```
python3 - <<'PY'
import json
from pathlib import Path

p = Path("firmware/logs/analysis/node_a_tc004_soak_<timestamp>/summary.json")
s = json.loads(p.read_text())

D_min = s["timeline"]["duration_min"]
D_h = D_min / 60.0
ACK = s["counts"]["acks"]
FAIL = s["counts"]["espnow_send_fail"]
Q_disc = s["counts"]["data_queued_discovery"]
FLAP = s["route_target_analysis"]["flap_count"]
S_abs = s["route_target_analysis"]["snapshots_without_target"]
S_pres = s["route_target_analysis"]["snapshots_with_target"]
S_total = S_abs + S_pres
status_neg2 = int(s.get("data_send_status_by_code", {}).get("-2", 0))

print("ack_per_min =", ACK / D_min)
print("send_fail_per_min =", FAIL / D_min)
print("status_neg2_per_min =", status_neg2 / D_min)
print("queued_discovery_per_min =", Q_disc / D_min)
print("flap_per_hour =", FLAP / D_h)
print("target_absent_ratio =", S_abs / S_total if S_total else 0)
PY
```

Firmware - README

Firmware ESP32 do AODV-EN

App ESP-IDF de bancada que consome a lib `AODV-EN` (`components/aodv_en`) sobre ESP-NOW v2.

A spec funcional do protocolo esta em `../docs/aodv-en-spec-v1.md`. Os criterios de aprovacao em hardware estao mapeados em `../docs/tests`.

O que ele faz hoje

- inicia Wi-Fi em modo `STA`
- inicia `ESP-NOW`
- cria um no `AODV-EN`
- envia `HELLO` periodico
- processa frames recebidos fora do callback do Wi-Fi
- opcionalmente tenta enviar `DATA` periodico para um MAC configurado
- imprime logs, estatisticas e estado geral no serial

Modos de aplicacao

Selecionavel por `menuconfig` -> `AODV-EN App` -> `Application example mode`:

- `app_proto_example` (padrao): camada de aplicacao com `HEALTH/TEXT/CMD` e CLI serial
- `app_demo` (legado): apenas `HELLO` e `DATA` periodicos

`main.c` faz o branch via `CONFIG_AODV_EN_APP_USE_APP_DEMO` / `CONFIG_AODV_EN_APP_USE_PROTO_EXAMPLE`.

API da lib (para outras aplicacoes)

A fachada da lib esta em `components/aodv_en/include/aodv_en.h`.

Padrao de uso recomendado:

1. Definir um `adapter` com duas funcoes: - `now_ms(user_ctx)` para fornecer tempo em ms. - `tx_frame(user_ctx, next_hop, frame, len, broadcast)` para enviar frame no radio/transporte.
2. Definir callbacks de app: - `on_data(...)` para entrega de payload. - `on_ack(...)` para confirmacoes.
3. Inicializar stack: - `aodv_en_stack_init(...)`.
4. No loop da aplicacao: - chamar `aodv_en_stack_tick_at(..., now_ms)` periodicamente; - encaminhar RX por `aodv_en_stack_on_rcv_at(...)`; - encaminhar resultado TX por `aodv_en_stack_on_link_tx_result_at(...)`.
5. Envio de mensagens: - `aodv_en_stack_send_hello_at(...)` - `aodv_en_stack_send_data_at(...)`

Esse modelo separa o protocolo (core) da plataforma (adapter), o que permite portar para outros ambientes sem alterar o nucleo do `AODV-EN`. A simulacao em `../sim` usa exatamente esse contrato com mocks.

Como buildar

```
zsh firmware/build.sh
```

Como gravar e abrir monitor

```
zsh firmware/flash_monitor.sh /dev/cu.usbserial-XXXX
```

Como gravar e salvar log em arquivo

```
zsh firmware/monitor_log.sh -p /dev/ttyUSB0 -B build/tc002_node_a -t tc004_soak -l node_a
```

Arquivos salvos em `firmware/logs/serial/`.

Casos de teste por papel

- `tests/tc001` - 2 nos diretos (`TC-001`)
- `tests/tc002` - 3 nos em cadeia (`TC-002` , `TC-003` , `TC-004`)
- `tests/tc005` - 4 nos em cadeia (`TC-005`)

Cada diretorio tem `build_flash.sh` , `monitor_log.sh` (wrapper) e `node_X.defaults` .

Como configurar

```
source firmware/idf-env.sh
idf.py menuconfig
```

`idf-env.sh` resolve o ESP-IDF nesta ordem:

1. `ESP_IDF_EXPORT` (quando definido)
2. `IDF_PATH/export.sh`
3. `$HOME/esp/esp-idf/export.sh`
4. `idf.py` ja disponivel no `PATH`

Menu: `A0DV-EN App` . Campos principais:

- modo de exemplo (`app_proto_example` ou `app_demo`)
- nome do no
- `network_id`
- canal Wi-Fi
- intervalo de `HELLO`
- habilitacao de `DATA`
- MAC alvo
- texto do payload

Para `app_proto_example` , ficam disponiveis campos extras:

- `PROTO health interval (ms)`
- `PROTO unicast interval (ms)`
- `PROTO enable periodic unicast text/cmd`

- `PROTO enable serial CLI commands`
- `PROTO target MAC (unicast)`
- `PROTO text payload`
- `PROTO command name`
- `PROTO command args`

CLI serial (modo `app_proto_example`)

Quando `PROTO enable serial CLI commands` estiver habilitado:

- `help`
- `health all`
- `health <AA:BB:CC:DD:EE:FF>`
- `text <AA:BB:CC:DD:EE:FF> <mensagem>`
- `cmd <AA:BB:CC:DD:EE:FF> <comando> [args]`
- `routes`

Comandos embutidos: `ping`, `echo`, `info`.

Dashboard ao vivo (real-time)

`firmware/tools/live_monitor.py` levanta um dashboard web em `http://localhost:8765/` que le multiplas portas seriais em paralelo, parseia os logs e empurra eventos via WebSocket para uma topologia animada (Cytoscape.js) com painel de metricas e timeline.

```
# instalar dependencias uma vez
pip install aiohttp pyserial

# modo demo (sem hardware) - util para validar o dashboard
python3 firmware/tools/live_monitor.py --demo

# com hardware: uma --port DEV:ALIAS por no
python3 firmware/tools/live_monitor.py \
  --port /dev/ttyUSB0:NODE_A \
  --port /dev/ttyUSB1:NODE_B \
  --port /dev/ttyUSB2:NODE_C
```

Recursos visuais: - topologia ao vivo com cores por papel (gateway, relay, leaf) e estado online/offline - pulse verde animando o caminho `A -> B -> C` quando o `ACK` chega na origem - flash vermelho em `ESP-NOW send fail` e `invalidated` (link quebrado) - card de alerta para fila cheia (`status=-2`) e falhas de enlace - timeline scrollavel com 60 eventos recentes - painel "Resumo da malha" com nos online, hops max, rotas validas, ACKs e deliveries

A interface esta em `firmware/tools/live_monitor_web/` (HTML + CSS + JS, sem build system).

Ferramentas de analise

A partir dos logs em `firmware/logs/serial/`:

```
python3 firmware/tools/extract_monitor_metrics.py firmware/logs/serial/<arquivo>.log
```

Saida em `firmware/logs/analysis/<basename>/`: - `summary.json` e `summary.txt` - CSVs: `events`, `snapshots`, `routes`, `target_route_series`, `ack_events`, `discovery_windows`, `minute_metrics` ...

Geracao de graficos:


```
python3 firmware/tools/plot_monitor_metrics.py firmware/logs/analysis/<basename>  
python3 firmware/tools/plot_comparison_metrics.py --run label1::path1/summary.json --run label2::path2/summary.jsd
```

Geracao de topologia:

```
python3 firmware/tools/draw_topology.py firmware/logs/analysis/<basename> --mode latest
```

Saidas em `topology/`: `topology.mmd` (Mermaid), `topology.dot` (Graphviz), `topology.svg` (quando `dot` estiver instalado), `topology.json` (resumo).

Observacao sobre permissoes

Se os scripts ainda nao estiverem com permissao de execucao no seu ambiente, rode-os com `zsh`, como mostrado acima.

Simulacao - README

Simulacao Local do AODV-EN

Esta pasta contem simulacoes locais do nucleo do protocolo, sem ESP32 real.

Objetivo

Validar localmente o fluxo:

- RREQ
- RREP
- DATA
- ACK
- RERR

reusando o mesmo nucleo do firmware via mock do adapter de transporte.

Variantes disponiveis

Variante	Arquivo	Topologia	Foco
basic (padrao)	aodv_en_sim.c	3 nos A-B-C	descoberta, retry de ACK, late-join
large	aodv_en_sim_large.c	6 nos A-B-C-D-E-F	RERR propagando na cadeia + reconvergencia
100	aodv_en_sim_100.c	grade 10x10	parede central, descoberta em escala
1000	aodv_en_sim_1000.c	grade 32x32	smart city com falhas aleatorias (lento)

Como rodar

```
bash sim/run_sim.sh          # basic (padrao)
bash sim/run_sim.sh basic
bash sim/run_sim.sh large
bash sim/run_sim.sh 100
bash sim/run_sim.sh 1000
bash sim/run_sim.sh help
```

O que esperar do basic

- na primeira tentativa de envio, A nao tem rota para C
- A emite RREQ
- B encaminha
- C responde com RREP
- a rota A -> C via B e instalada
- na segunda tentativa, DATA chega em C

- `C` responde com `ACK`
- `A` registra a confirmacao
- depois exercita perda de `ACK` (com retry) e late-join (descoberta apos retorno do enlace)

Observacao sobre escala

Os simuladores `100` e `1000` setam `config.route_table_size` e `config.neighbor_table_size` na inicializacao, mas esses campos nao redimensionam os arrays em runtime. Os tamanhos reais sao constantes de compilacao em [firmware/components/aodv_en/include/aodv_en_limits.h](https://github.com/iperf/iperf-ng/blob/master/firmware/components/aodv_en/include/aodv_en_limits.h):

- `AODV_EN_ROUTE_TABLE_SIZE` (default `32`)
- `AODV_EN_NEIGHBOR_TABLE_SIZE` (default `16`)

Para experimentos com tabelas maiores, recompile passando `-DAODV_EN_ROUTE_TABLE_SIZE=N` (e similares) na linha de compilacao. Caso contrario, parte das rotas/vizinhos sera descartada com `AODV_EN_ERR_FULL` .

Apendice: Especificacao v0 (OBSOLETA)

AODV-EN Spec v0

Estado

- estado: OBSOLETA
- substituida por: [aodv-en-spec-v1.md](#)

Esta versao foi mantida apenas para registro historico do escopo inicial de MVP. Toda referencia normativa do projeto deve apontar para a `v1`. Em caso de conflito, a `v1` prevalece.

Mudancas mais relevantes da `v0` para a `v1`:

- `HELLO` deixa de ser opcional e passa a ser parte do MVP
- `RERR` ganha semantica de precursores (RFC 3561 secao 6.2)
- fila de `DATA` pendente passa a ser parte do nucleo
- regras de `sequence number` em `RREP` alinhadas com RFC 3561 secao 6.6.1
- `metric = hop_count` fixado para `v1`; `RSSI` fica como ponto de evolucao
- adicao explicita de criterios de aprovacao de `v1` baseados nos casos de teste de bancada

Objetivo

Este documento define a versao inicial do protocolo `AODV-EN`, uma adaptacao do AODV para redes mesh multi-hop construidas sobre ESP-NOW e ESP32.

Esta especificacao serve como base de implementacao do TCC. Ela nao pretende ser definitiva, mas precisa ser suficientemente completa para orientar o desenvolvimento do firmware e dos testes.

Premissas

- todos os nos do testbed sao ESP32 com suporte a ESP-NOW v2
- todos os nos operam no mesmo canal Wi-Fi
- todos os nos compartilham o mesmo `network_id`
- a rede e ad hoc, sem coordenador central
- o protocolo de roteamento e reativo
- o transporte subjacente e ESP-NOW

Decisoes de alto nivel

Uso de ESP-NOW v2

O protocolo sera implementado sobre ESP-NOW v2.

Entretanto:

- mensagens de controle devem permanecer compactas
- o maior payload disponível não deve ser usado sem necessidade
- o protocolo não deve depender de pacotes gigantes para funcionar

Separacao de planos

O protocolo será modelado em dois planos:

- plano de controle
- plano de dados

Plano de controle:

- HELLO
- RREQ
- RREP
- RERR
- ACK

Plano de dados:

- DATA

Tabelas locais

Cada nó manterá quatro estruturas principais:

- tabela de vizinhos
- tabela de rotas
- cache de duplicatas de RREQ
- cache de peers ativos ESP-NOW

Estruturas de dados

1. Tabela de vizinhos

Responsável por manter informação de nós de um salto.

Campos mínimos:

- mac[6]
- last_seen_ms
- avg_rssi
- link_fail_count
- peer_cached
- active

Responsabilidades:

- registrar nós ouvidos diretamente
- permitir flooding controlado para vizinhos ativos
- oferecer informação local de qualidade de enlace
- apoiar política de cache de peers

2. Tabela de rotas

Responsavel por manter informacao de encaminhamento.

Campos minimos:

- `destination[6]`
- `next_hop[6]`
- `hop_count`
- `dest_seq_num`
- `lifetime_ms`
- `valid`
- `metric`

Responsabilidades:

- decidir o proximo salto para um destino
- armazenar rotas reversas e diretas
- invalidar rotas expiradas ou quebradas

3. Cache de duplicatas de RREQ

Responsavel por suprimir mensagens repetidas.

Campos minimos:

- `originator[6]`
- `rreq_id`
- `created_at_ms`

4. Cache de peers ESP-NOW

Responsavel por limitar quantos peers ficam registrados no driver.

Campos minimos:

- `mac[6]`
- `last_used_ms`
- `pinned`

Politica inicial:

- LRU simples
- numero pequeno de peers ativos
- adicao sob demanda

Tamanhos iniciais recomendados

Para o testbed do TCC:

- `neighbor_table_size = 16`
- `route_table_size = 32`
- `rreq_cache_size = 64`
- `peer_cache_size = 8`

Esses tamanhos sao suficientes para o ambiente inicial de testes e podem ser ajustados por configuracao.

Tipos de mensagem

1. HELLO

Objetivo:

- manter conhecimento minimo de vizinhanca
- atualizar `last_seen`
- atualizar `RSSI`

Uso:

- opcional no MVP
- pode ser substituido por observacao passiva no inicio

Campos:

- `node_mac`
- `node_seq_num`
- `timestamp_ms`

2. RREQ

Objetivo:

- descobrir rota para um destino

Campos obrigatorios:

- `protocol_version`
- `message_type`
- `network_id`
- `originator_mac`
- `destination_mac`
- `sender_mac`
- `originator_seq_num`
- `destination_seq_num`
- `rreq_id`
- `hop_count`
- `ttl`

Regras:

- a origem emite `RREQ` quando nao possui rota valida
- cada no cria ou atualiza rota reversa para a origem
- cada no incrementa `hop_count`
- cada no descarta duplicatas
- o destino responde com `RREP`

3. RREP

Objetivo:

- responder a uma descoberta de rota

Campos obrigatorios:

- `protocol_version`
- `message_type`
- `network_id`
- `originator_mac`
- `destination_mac`
- `sender_mac`
- `destination_seq_num`
- `hop_count`
- `lifetime_ms`

Regras:

- volta pelo caminho reverso
- cada intermediario cria ou atualiza rota direta para o destino
- a origem instala a rota final ao receber o `RREP`

4. RERR

Objetivo:

- sinalizar rotas quebradas

Campos obrigatorios:

- `protocol_version`
- `message_type`
- `network_id`
- `sender_mac`
- `unreachable_destination_mac`
- `unreachable_dest_seq_num`

Regras:

- emitido quando um `next_hop` falha
- invalida rotas dependentes daquele enlace
- pode disparar nova descoberta se necessario

5. DATA

Objetivo:

- transportar dados da aplicacao

Campos obrigatorios:

- `protocol_version`
- `message_type`
- `network_id`
- `originator_mac`
- `destination_mac`

- sender_mac
- sequence_number
- hop_count
- ttl
- payload_length
- payload

Regras:

- segue a tabela de rotas
- cada salto atualiza sender_mac
- destino entrega ao handler da aplicacao

6. ACK

Objetivo:

- confirmar entrega fim a fim no nível da aplicacao

Campos obrigatorios:

- protocol_version
- message_type
- network_id
- originator_mac
- destination_mac
- ack_for_sequence

Regras:

- o destino envia ACK quando recebe DATA
- a origem usa timeout para decidir retransmissao ou falha

Cabecalho comum

Todas as mensagens devem compartilhar um cabecalho comum.

Campos iniciais recomendados:

- protocol_version
- message_type
- flags
- network_id
- sender_mac

Campos especificos entram apos o cabecalho comum.

Regras operacionais

Descoberta de vizinhos

MVP:

- descoberta passiva por observacao de trafego recebido

Fase seguinte:

- HELLO opcional e leve

Descoberta de rota

Fluxo:

1. aplicacao solicita envio para um destino
2. no consulta tabela de rotas
3. se existir rota valida, envia DATA
4. se nao existir, emite RREQ
5. intermediarios constroem rota reversa
6. destino retorna RREP
7. intermediarios constroem rota direta
8. origem envia DATA

Encaminhamento

Cada no usa:

- tabela de rotas para encontrar next_hop
- tabela de vizinhos para decidir se o enlace direto ainda e plausivel

Duplicatas

RREQ duplicado deve ser descartado pelo par:

- originator_mac
- rreq_id

Expiracao

Rotas e entradas auxiliares devem expirar por tempo.

Politica inicial:

- vizinho expira se ficar tempo demais sem ser visto
- rota expira apos lifetime_ms
- entrada de duplicata expira apos janela curta

Timers iniciais

Valores iniciais de projeto, sujeitos a calibracao:

- neighbor_timeout_ms = 15000
- route_lifetime_ms = 30000
- rreq_cache_timeout_ms = 10000
- ack_timeout_ms = 1000
- rreq_retry_count = 3

Metricas

MVP:

- `hop_count` como critério principal

Fase seguinte:

- métrica composta com `RSSI`

Diretriz:

- `sequence_number` define frescor
- `hop_count` define preferência principal no MVP
- `RSSI` entra como extensão controlada, não como substituto da lógica do AODV

Confiabilidade

O `send callback` do ESP-NOW não é suficiente para garantir entrega na aplicação.

Por isso:

- `ACK` fim a fim permanece no protocolo
- timeout e retransmissão ficam na camada do AODV-EN

Escopo do MVP

O MVP deve implementar:

- tabela de vizinhos
- tabela de rotas
- cache de duplicatas
- `RREQ`
- `RREP`
- `DATA`
- `ACK`
- expiração básica

O MVP ainda não precisa implementar:

- `RERR` sofisticado
- métrica híbrida completa
- fragmentação de payload
- power saving adaptativo

Critérios de aceite da v0

Consideraremos a implementação inicial coerente com esta especificação quando:

- dois nós conseguirem descobrir rota e trocar `DATA`
- três nós conseguirem fazer relay multi-hop
- a origem instalar rota após `RREP`
- o destino responder com `ACK`
- pacotes duplicados de `RREQ` forem descartados
- rotas expiradas deixarem de ser usadas